

IP 설계 이론

혼성신호·디지털 IC 설계 — 대학원 이론서 · 한국어판
옮긴 장 23 / 23

차례

- [T1. 네 가지 기계로서의 MOSFET](#)
- [T2. 지연 — RC 하나에서 타이밍 경로 전체까지](#)
- [T3. 준안정 — 설계로 없앨 수 없는 단 하나의 실패](#)
- [T4. 잡음 — 그리고 그것이 왜 모든 것의 크기를 정하나](#)
- [T5. 표본화 — 연속 신호가 수가 되는 자리](#)
- [T6. 데이터 변환기 — 구조마다 같은 한계를 어디에 쓰는가](#)
- [T7. 위상 고정 루프 — 출력이 시간인 되먹임 시스템](#)
- [T8. 직렬 링크 — 켜진 것, 안 되는 자리까지](#)
- [T9. 채널 — 손실이 어디서 오는가, 항 하나씩](#)
- [T10. 스위치드 커패시터 — 시간으로 산 정확도](#)
- [T11. 바이어스와 기준 — 움직이지 않는 수 하나 만들기](#)
- [T12. 정합 — 똑같은 소자 둘은 똑같지 않다](#)
- [T13. 전원 무결성 — 나머지 전부가 올라왔은 회로망](#)
- [T14. 신뢰성 — 십 년 뒤 회로는 어떤 꼴인가](#)
- [T15. 레이아웃 — 옳은 회로도도 그래도 질 수 있는 곳](#)
- [T16. ESD — 칩에 전원이 들어오기도 전에 나는 실패](#)
- [T17. 구현 흐름 — 단계마다 무엇을 정하나](#)
- [T18. 플로어플랜과 전원 계획 — 나머지 전부가 올라왔는 기하](#)
- [T19. 배치, 클럭 트리, 배선](#)
- [T20. 사인오프 — 지어도 되는지를 정하는 검사들](#)
- [T21. 검증 환경 — 무엇을 증명하나](#)
- [T22. DFT 에서 테스터까지 — 시간, 값, 그리고 새어 나가는 것](#)
- [T23. 흐름을 끝에서 끝까지 돌리기 — 블록 하나, 아홉 단계, 진짜 수](#)

T1. 네 가지 기계로서의 MOSFET

소자는 하나인데 동작 영역은 넷이다 — 그리고 영역마다 다른 회로를 설계하게 된다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 전기장 · 전하 · 전자 · 지수함수 · 로그 · 미분 · 옴의법칙

이 장에서 새로 나오는 것

문턱전압 · 차단영역 · 선형영역 · 포화영역 · 아문턱영역 · 온저항 · 아문턱기울기 · 누설전류 · 트랜스컨덕턴스 · 채널 길이변조 · 출력저항 · 바디효과 · 게이트용량

MOSFET 은 보통 스위치로 소개되고, 디지털 첫 수업에는 그것으로 충분하다. 그러나 양산 IP 를 설계하기에는 충분하지 않다. 같은 네 단자 소자가 걸어 준 전압에 따라 질적으로 다른 네 가지 기계로 동작하고, 일하는 설계자는 그 넷 사이를 오간다 — 흔히 한 블록 안에서 그러다. 표본화 전단은 이 소자를 저항으로 쓰고, 그 뒤의 비교기는 증폭기로 쓰며, 다시 그 뒤의 디지털 논리는 스위치로 쓴다. 그리고 그 전부가 놓고 있는 동안 전지를 비우는 누설은, 아무도 설계하지 않은 영역에서 소자가 아주 나쁜 스위치로 동작하는 것이다.

이 장은 네 영역을 세운 다음, 영역마다 어디에 쓰는지, 언제 꺼내 쓰는지, 어떻게 적용하는지, 그리고 실제 IP 납품 물에 팔려 나가는 코드가 어떻게 생겼는지를 적는다. 이 책의 뒤에서 쓰이지 않는 것은 여기에 넣지 않았다.

T1.1 게이트가 실제로 하는 일

영역으로 들어가기 전에 물리 사실 하나를 먼저 본다. 뒤의 모든 식이 이것 위에 서 있기 때문이다. 게이트는 축전기의 한쪽 판이다. 채널 — 산화막 아래의 얇은 실리콘 층 — 이 다른 쪽 판이다. 그 사이에 원자 몇 개 두께의 절연체가 있다.

단위 면적당 게이트 산화막 용량, $C_{ox} = \epsilon_{ox}/t_{ox}$. 물리적 산화막 두께가 1.2 nm 이고 $\epsilon_{ox} \approx 3.9 \epsilon_0$ 이면, 이 값은 약 16 mF/m² — 제곱 마이크로미터당 대략 1.60×10^{19} fF 다. 트랜지스터가 하는 모든 일이 이 값에 비례한다.

유도 — 게이트 전압이 왜 전류를 제어하는가

	이렇게 된다	왜냐하면
1	NMOS의 게이트에 양의 전압을 건다. 게이트는 축전기의 한 쪽 판이므로 단위 면적당 전하 $Q = C_{ox} \cdot V$ 를 쌓는다.	그것이 용량의 정의다. 아직 트랜지스터 물리는 하나도 안 썼다.
2	반대 부호의 전하가 다른 쪽 판 — 산화막 바로 아래의 실리콘 — 에 나타나야 한다.	축전기는 두 판에 크기가 같고 부호가 반대인 전하를 갖는다.
3	p형 실리콘에서 움직일 수 있는 음전하는 전자인데, 전자는 귀하다. 게이트 전압이 어느 값 아래면 실리콘은 필요한 전하를 정공을 밀어내서 대신 마련하고, 그 자리에 고정된 음의 반개 이온이 남는다.	다수 운반자를 비우는 쪽이 소수 운반자를 만드는 쪽보다 싸다.
4	게이트 전압이 어느 값을 넘으면 공핍이 더 자라기를 멈추고, 실리콘이 대신 움직이는 전자를 내놓기 시작한다. 그 전자 층이 채널이다.	표면이 충분히 끌려가고 나면, 전자를 끌어오는 것이 공핍층을 더 넓히는 것보다 에너지가 덜 든다.
5	그 값이 일어나는 게이트 전압이 문턱전압 V_t다.	소자에 설계해 넣은 스위치 동작점이 아니다. 실리콘이 어떤 종류의 전하를 내놓을지를 바꾸는 전압이다.
6	V_t 위에서 채널의 움직이는 전하는 단위 면적당 $Q_{ch} = C_{ox}(V_{GS} - V_t)$ 다.	처음 V_t 만큼의 게이트 전압은 공핍에 쓰였다. 그것을 넘은 몫만 움직이는 전하를 만든다.
7	전류는 전하 공급기 속도이고, 속도는 이동도 곱하기 전기장이다: $I = W \cdot Q_{ch} \cdot \mu \cdot E$.	표류 전류다. 저항에서 옴의 법칙을 주는 바로 그 관계다.

마지막 줄이 이 소자의 전부다. 아래의 모든 식은 이 관계에, 전기장 E 가 채널을 따라 어떻게 변하는지에 대한 서로 다른 가정을 얹은 것이다. $V_{GS} - V_t$ 라는 양은 너무 자주 나와서 이름이 있다. **과구동(overdrive)** 이고 V_{ov} 로 적는다. 설계자가 “그 소자를 과구동 150 밀리볼트로 돌렸다”고 말할 때 그 수가 이것이다.

V_t 는 하나의 수가 아니다. 온도에 따라 움직이고 (대략 $-1 \text{ mV}/^\circ\text{C}$), 바디 바이어스에 따라, 채널 길이에 따라, 이웃한 폴리실리콘이 얼마나 가까운지에 따라 움직인다. 그리고 다이마다 다르고 같은 다이 안의 트랜지스터마다 다르다. 공칭 V_t 에서만 도는 설계는 도는 설계가 아니다. T12 장이 이것을 정량화한다. 지금은 이 장의 모든 V_t 를 분포의 중심이라고 여기라.

T1.2 영역 1 — 차단: 완전히 꺼지지 않는 스위치

차단과 아문턱 누설

문턱 아래에서 채널은 비어 있지 않다 — 지수적으로 희박할 뿐이다. 전류는 멈추지 않는다. V_{GS} 가 V_t 아래로 **85 mV** 내려갈 때마다 열 배씩 준다. 그 기울기가 **아문턱 기울기 S** 이고, 저전력 설계에서 가장 중요한 수 하나다.

$$I_{leak} = I_0 \cdot 10^{(V_{GS} - V_t)/S}$$

S 에는 바닥이 있다. 채널 전하는 볼츠만 인자를 거쳐 표면 전위에 반응하므로 상온에서 $S \geq (kT/q) \cdot \ln 10 = \mathbf{59.5 \text{ mV/decade}}$ 이고, 실제 소자는 게이트 전압의 일부가 공핍 용량에 새기 때문에 그보다 20-40 % 위에 앉는다. **이 바닥이, 전원 전압이 줄어드는 동안 누설이 전력의 지배항이 된 까닭이다**: 속도를 지키려고 V_t 를 낮추면 누설을 지수로 물어야 하고, 그 지수를 더 가파르게 만들 수는 없다.

어디에 쓰나	내보내는 모든 블록의 대기 전력. 보존 레지스터. 메모리 배열(누설이 셀 수백만 개로 곱해지는 자리). 전원 게이팅과 헤더/푸터 스위치의 치수 결정. 대기 전류를 명시해서 파는 모든 IP — 대개가 그렇다.
언제 쓰나	데이터시트에 누설 수치를 적을 때마다. 합성에서 소자 종류를 고를 때마다. 고객이 보존 모드를 요구할 때마다. 그리고 블록이 대부분 놓고 있을 때마다 — 시간의 3 % 만 도는 등화기는 동적 전력 문제가 아니라 누설 문제다.
어떻게 쓰나	파운드리 특성화에서 최악 코너(최고 온도, 최저 V_t , 최고 전원 — 누설은 속도와 반대 방향으로 나빠진다)의 폭 마이크로미터당 I_{off} 를 받는다. 블록의 총 폭을 곱한다. 온도 인자를 다시 곱한다 — 경험칙은 10 °C 마다 2 배이고, S 자체가 T 에 비례하므로 S 식에서 바로 따라 나온다. 그리고 나서 정한다: 답이 예산을 넘으면 임계가 아닌 경로를 고 V_t 셀로 옮기고 타이밍을 다시 잡는다.

실제 코드는 이렇게 생겼다

```
// Liberty (.lib) — what the foundry actually ships you.
// This is the leakage the synthesiser reads. Note that it is
// state-dependent: a NAND2 leaks differently per input pattern,
// because the stacked device changes the internal node voltage.

cell (NAND2_X1) {
    area : 1.596 ;
    cell_leakage_power : 4.2851e+03 ;           // nW, the default
    leakage_power () {
        when      : "!A1 & !A2" ;
        value      : 1.8032e+03 ;           // both off: stack effect helps
    }
    leakage_power () {
        when      : "A1 & !A2" ;
        value      : 4.9210e+03 ;
    }
    leakage_power () {
        when      : "A1 & A2" ;
        value      : 6.1104e+03 ;           // both on: worst
    }
    ...
}

# Synthesis — trade leakage against timing by device flavour.
# The tool will use LVT only where it must to close timing.
set_db library [list sc9_base_svt.lib sc9_base_hvt.lib sc9_base_lvt.lib ]
set_db leakage_power_effort      high
set_db dynamic_power_effort      medium
# Forbid the leaky flavour outside the critical paths:
set_dont_use [get_lib_cells */*_LVT*]
# ... place and route, then selectively re-enable on failing paths:
set_dont_use -false [get_lib_cells */BUF_X*_LVT]
```

여기서 틀린다. 공칭 온도에서 누설을 인용하는 것. 누설은 10 °C 마다 대략 두 배가 되므로, 25 °C 수치는 125 °C 차량용 코너에 대고 보면 **8-16 배 낙관적**이다. 온도와 전원과 코너를 말하지 않고 “대기 12 μA ” 라고 적은 데이터시트는 규격이 아니다 — 그리고 자기 열 한계에서 진짜 수를 알게 된 고객은 다시 사지 않는다.

예제 6. 데이터시트가 요구하는 방식으로 낸 블록의 누설

주어진 것. 게이트 40,000 개, 게이트당 평균 소자 폭 0.8 μm (n 과 p 를 합쳐), 25 °C 전형 코너에서 $V_{GS} = 0$ 일 때 $I_{off} = 0.01 \text{ nA}/\mu\text{m}$ 인 블록. 고객은 105 °C 에서 돌리며 최악 대기 전류를 묻는다.

푸는 법. 총 폭 곱하기 마이크로미터당 I_{off} , 곱하기 온도 인자(10 °C 마다 2 배), 곱하기 fast-fast 실리콘의 코너 인자 (누설에서 보통 2-3 배).

답. 폭 = $40,000 \times 0.8 = 32,000 \mu\text{m}$. 25 °C 전형에서 $32,000 \times 0.01 \text{ nA} = \mathbf{0.24 \mu\text{A}}$. 온도: $105 - 25 = 80$ °C 이므로 $2^8 = 256$ 배. 코너: $\times 2.5$. 합하면 약 **0.16 mA**.

10 °C 마다 2 배라는 규칙은 지수를 어림한 것이다. 진짜 납품물에서는 외삽하지 않고 파운드리 코너 표에서 수를 읽는다. 이 어림의 쓸모는, 예산의 두 배 안에 있는지 십 배 밖에 있는지를 **이른 시점에** 아는 것이다.

여기서 틀린다. 여기서 25 °C 전형 수치와 105 °C fast 수치는 **640 배** 차이가 난다. 기술자가 앞의 것을 인용하는 까닭은 도구가 기본으로 찍어 주는 수가 그것이기 때문이다. 고객이 재는 수는 뒤의 것이다. 수 옆에 늘 세 가지 — 온도, 전원, 공정 코너 — 를 같이 적으라. 그것이 없는 누설 수치는 아무 뜻이 없다.

T1.3 영역 2 — 선형: 저항으로서의 트랜지스터

선형(삼극관) 영역과 온저항

게이트를 V_t 보다 한참 위로 올리고 드레인을 소스 가까이 붙들어 둔다. 이제 채널이 길이 전체에 걸쳐 존재하고 그 위의 전기장이 거의 고르므로, 소자는 게이트가 값을 제어하는 저항처럼 동작한다:

$$I_D = \mu C_{ox}(W/L)[(V_{GS} - V_t)V_{DS} - V_{DS}^2/2]$$

V_{DS} 가 작으면 제곱 항이 사라지고 소자는 그냥 이것이다

$$R_{on} = 1 / [\mu C_{ox}(W/L)(V_{GS} - V_t)]$$

저 분모를 읽으라. R_{on} 은 과구동에 반비례한다. 지나보내는 신호가 V_S 를 움직이면 V_{GS} 도 따라 움직이고, **저항이 신호에 따라 변한다.** 표본화 경로에서 신호에 따라 변하는 저항은 왜곡이고, 부트스트랩 스위치가 존재하는 까닭이 그것이다.

어디에 쓰나

모든 전송 게이트와 패스 게이트. 모든 ADC 와 모든 트래킹앰플리퍼 앞단의 표본화 스위치. 전원 게이팅 헤더와 푸터. CTLE 안의 가변 저항. SRAM 의 쓰기 드라이버. 디지털 신호를 다시 만들지 않고 지나보내야 하는 모든 자리.

언제 쓰나

한 마디를 구동하는 것이 아니라 두 마디를 잇는 것이 필요할 때. “신호를 버퍼로 받는다” 가 아니라 “신호를 지나보낸다” 라고 쓰고 있는 자신을 발견하는 순간, 선형 영역에 있고 R_{on} 이 문제가 된다.

어떻게 쓰나

정착을 먼저 예산 잡고 거기서 치수를 낸다. 저항 R 인 스위치가 용량 C 를 채우면 시간상수 $\tau = RC$ 로 정착한다. N 비트까지 정착하려면 $t_{settle} \geq RC \cdot \ln(2^{N+1})$ 이 필요하다. R 로 풀고, $R_{on} = 1/[\mu C_{ox}(W/L)V_{ov}]$ 로 W/L 을 낸다. 그다음 **최악 신호 준위에서 R_{on} 을 확인한다** — 중간 눈금이 아니다. 올라가는 신호를 지나보내는 NMOS 는 신호가 오를수록 V_{GS} 가 줄고, $V_{DD} - V_t$ 근처에서 R_{on} 이 터진다.

실제 코드는 이렇게 생겼다

```

* SPICE — the measurement you actually run on a sampling switch.
* You are not checking that it works; you are checking how Ron varies
* with the signal, because that variation is the distortion.

.param vin_val=0.45
M1 drain gate source bulk nch W=2u L=30n
Vg gate 0 DC 0.9
Vs source 0 DC 'vin_val'
Vd drain 0 DC 'vin_val+1m' $ 1 mV across the switch: stay in triode
.dc vin_val 0 0.9 0.01
.measure dc ron PARAM='1m/I(Vd)' $ R = V/I at each input level
.alter
.param boot=1 $ repeat with a bootstrapped gate
.end

// Verilog-AMS behavioural model used in mixed-signal top-level sims.
// A digital verification engineer meets this, not the SPICE deck.
module sw_nmos (inout d, inout s, input g);
    parameter real ron_nom = 250.0; // ohms at mid-scale
    parameter real roff = 1e12;
    analog begin
        // Ron rises as (Vgs - Vt) falls: this is the term that matters.
        real vgs, ron;
        vgs = V(g) - V(s);
        ron = (vgs > 0.35) ? ron_nom * 0.55 / (vgs - 0.35) : roff;
        I(d,s) <+ V(d,s) / ron;
    end
endmodule

```

여기서 틀린다. 중간 눈금에서 스위치 치수를 정하고 끝났다고 하는 것. 입력 범위 꼭대기에서의 R_{on} 은 중간 눈금 값의 몇 배가 될 수 있고, 그것이 만드는 정착 오차는 신호에 따라 달라진다 — 즉 잡음이 아니라 고조파 왜곡이고, 평균을 내도 없어지지 않는다. 입력 범위 전체에 대해 R_{on} 을 살펴 보고 가장 나쁜 점에 맞춰 치수를 정하라.

예제 7. 정착 요구에서 표본화 스위치의 치수를 낸다

주어진 것. 10 비트 변환기가 $C = 1$ pF 에 2 ns 안에 표본화한다. 전원 0.9 V, $V_t = 0.3$ V, 최악 입력이 0.5 V 이므로 $V_{ov} = 0.9 - 0.5 - 0.35 = 0.1$ V.

푸는 법. 10 비트까지 정착하려면 오차가 $\frac{1}{2}$ LSB 아래여야 하므로 $e^{-t/\tau} < 2^{-11}$, 즉 $t/\tau > 11 \cdot \ln 2 = 7.62$ 다. 경로의 나머지를 위해 τ 에 2 배 여유를 두고, $\tau = RC$ 에서 R 을 풀고, R_{on} 에서 W/L 을 낸다.

답. $\tau \leq 2$ ns / 7.62 = 262 ps. 2 배 여유를 두면 $\tau = 131$ ps. $R = \tau/C = 131$ Ω . 그러면 $W/L = 1/(\mu C_{ox} \cdot R \cdot V_{ov}) = 317$.

여기서 틀린다. 최악 입력이 아니라 중간 눈금 입력(0.45 V)에서 V_{ov} 를 잡으면 $V_{ov} = 0.1$ V 가 나오고 W/L 이 약 2 배 작아진다 — 그러면 스위치는 신호가 가장 큰 바로 그 자리에서 정착 목표를 놓치고, 왜곡이 가장 잘 보이는 자리가 거기다. 스위치는 평균 신호가 아니라 최악 신호 준위에서 치수를 정하라.

V_{DS} 를 과구동보다 높이면 드레인 근처에서 채널이 핀치오프 된다. 그 자리의 국소 V_{GS} 가 V_t 까지 떨어져 더 이상 움직이는 전하가 없다. 그 점을 넘으면 드레인 전압을 올려도 전류가 별로 안 는다. 소자가 게이트로 제어 되는 전류원이 된 것이다:

$$I_D = \frac{1}{2} \mu C_{ox} (W/L) (V_{GS} - V_t)^2 \cdot (1 + \lambda V_{DS})$$

여기서 나오는 두 양이 아날로그 설계의 일을 다 한다:

트랜스컨덕턴스 $g_m = \partial I_D / \partial V_{GS} = 2I_D / V_{ov}$. 입력 1 볼트당 얼마나 많은 전류가 나오나 — 즉 이득을 만드는 항이다.

출력저항 $r_o = \partial V_{DS} / \partial I_D \approx 1/(\lambda I_D)$. 이상적인 전류원에 얼마나 가까운가. λ 라는 항이 있는 까닭은 핀치오프 지점이 드레인 전압에 따라 움직이기 때문이고 — **채널 길이 변조** — 짧은 채널일수록 나쁘다. 아날로그 블록 이 최소 길이의 몇 배 되는 소자를 쓰는 까닭이 그것이다.

그 곱 $g_m r_o = 2/(\lambda V_{ov})$ 가 고유 이득이다: **트랜지스터 하나가 줄 수 있는 전압 이득의 최대치**. 공정이 작아질 수록 이 값이 줄고, 그래서 요즘 아날로그 설계는 날 이득보다 디지털의 도움에 기댄다.

어디에 쓰나	비교기와 감지증폭기 — 그 앞단 증폭기. 모든 전류 거울과 바이어스 회로. CTLE 와 SerDes 의 아날로그 전단. 천이 중의 레벨 시프터. PLL 의 링 발진기 지연 셀. 회로가 스위칭 이 아니라 증폭 을 해야 하는 모든 자리.
언제 쓰나	물음이 “이득이 얼마인가” 또는 “부하와 무관하게 전류가 얼마인가” 일 때. 그리고 뜻밖에도 디지털 분석에서도 쓴다: 논리 천이 동안 두 소자가 모두 포화를 지나가고, 그때 흐르는 단락 전류가 T4 가 느린 입력 예지에 왜 전력이 드는지 설명할 때 쓰는 항이다.
어떻게 쓰나	W/L 이 아니라 g_m/I_D 로 설계한다. 필요한 g_m 을 이득이나 잡음 요구에서 고르고, g_m/I_D 를 속도와 전력의 맞바꿈에서 고른다 (g_m/I_D 가 크면 $\approx$ 과구동이 작고 $\approx$ 효율적이지만 느리다. g_m/I_D 가 작으면 $\approx$ 과구동이 크고 $\approx$ 빠르지만 전력을 먹는다). 그러면 I_D 가 정해지고, W/L 은 파운드리가 특성화해 준 g_m/I_D 곡선에서 읽는다. 이 방법은 공정이 바뀌어도 통하고, 100 nm 아래에서 정확하기를 그만두는 제품 법칙 식에 기대지 않는다.

실제 코드는 이렇게 생겼다

- * The gm/ID sweep every analog team runs once per process,
- * then reuses for every block. You are not simulating a circuit here;
- * you are characterising the device so design becomes look-up.

```
.param L_val=100n
M1 d g 0 0 nch W=10u L='L_val'
Vg g 0 DC 0.5
Vd d 0 DC 0.45          $ hold Vds at VDD/2, in saturation
.dc Vg 0.2 0.9 0.005
.measure dc gm_id PARAM='deriv(I(Vd),V(g))/I(Vd)'
.measure dc ft PARAM='deriv(I(Vd),V(g))/(2*3.14159*cgg)'
.step param L_val list 30n 60n 100n 200n 500n
.end

// What a digital designer sees of all this: a characterised cell.
// The analog block arrives as a .lib timing/power view plus a
// behavioural model. You must still know what is inside, because
// the constraints you write depend on it.
cell (CMP_STRONGARM) {
    pin (CLK) { direction : input; clock : true; capacitance : 4.8; }
    pin (VINP) { direction : input; capacitance : 12.1; }
    pin (OUTP) { direction : output;
        timing () {
            related_pin : "CLK" ;
            timing_type : rising_edge ;
            cell_rise (delay_template) { ... }
        } }
    // The number a digital designer must not ignore:
    // the comparator has a metastability window like any bistable.
    // T3 derives it; the vendor states it here.
}
```

여기서 틀린다. 요즘 공정에서 제곱 법칙 식으로 설계하는 것. 대략 100 nm 아래에서는 운반자 속도가 포화하고 지수가 2 에서 1 쪽으로 밀려서, 식이 수십 퍼센트 틀릴 수 있다. 따져 보는 데는 여전히 훌륭하다 — 방향과 민감도를 바르게 알려 준다 — 그러나 수로는 쓸모가 없다. 식으로 따지고, 치수는 특성화 곡선에서 내라.

아문턱 동작

누설을 만드는 그 지수가 쓸 만한 전달 특성이기도 하다. V_t 아래로 바이어스하면 소자는 g_m/I_D 를 최대로 준다 — 약 27.1 V^{-1} , 오로지 S 가 정하는 값 — 이것이 어떤 MOSFET 도 넘지 못하는 전류 효율의 한계다. 값은 속도다. f_T 가 자릿수로 떨어지고, V_t 에 대한 지수 의존 때문에 소자 사이의 V_t 불일치가 수십 퍼센트의 전류 불일치로 바뀐다.

그러므로 아문턱은 연산당 에너지가 지배하고 속도는 그렇지 않을 때 고르는 영역이고, 전원이 꺼질 때 떠밀려 들어가는 영역이다. 또한 **보존** 회로가 사는 영역이다: 겨우 켜져 있는 소자가 상태를 붙들고 있다.

어디에 쓰나	상시 전원 영역 — 깨우기 논리, 워치독 타이머, 보존 레지스터. 센서 전단. 아주 작은 전류에서도 살아남아야 하는 바이어스 발생기. 그리고 반대 방향으로, 내보내는 모든 블록의 모든 누설 경로.
언제 쓰나	동작률이 아주 낮고 에너지 예산이 초당이 아니라 연산당일 때. 고객이 마이크로암페어 수치를 명시한 보존 모드를 요구할 때. V_t 쪽으로 무너지는 전원에서 살아남으라고 할 때.
어떻게 쓰나	속도가 아니라 정합이 면적을 정한다는 것을 받아들인다. 강반전에서는 전류 불일치가 g_m/I_D 를 거쳐 V_t 불일치를 따라간다. 아문턱에서는 g_m/I_D 가 최대이므로 같은 V_t 불일치가 가장 큰 전류 불일치를 만든다. 정합 요구에서 소자 치수를 내고(T12 의 펄그롬 관계), 그다음에 속도를 확인한다 — 반대 순서가 아니다.

실제 코드는 이렇게 생겼다

```
# UPF (Unified Power Format) — how the always-on domain is
# declared to the tools. A digital IP deliverable that supports
# retention ships this file, and it is as much a part of the
# product as the RTL.

create_power_domain PD_ALWAYS_ON -elements {u_wake u_timer}
create_power_domain PD_CORE -elements {u_dsp u_eq} -scope .

create_supply_net VDD_AON -domain PD_ALWAYS_ON
create_supply_net VDD_CORE -domain PD_CORE
set_domain_supply_net PD_CORE -primary_power_net VDD_CORE \
    -primary_ground_net VSS

# Retention: state survives while VDD_CORE collapses.
set_retention RET_CORE -domain PD_CORE \
    -retention_power_net VDD_AON -retention_ground_net VSS \
    -save_signal {u_dsp/ret_save high} \
    -restore_signal {u_dsp/ret_restore low}

# Isolation: outputs of a collapsed domain must not float into
# the always-on domain, or the always-on logic burns crowbar current.
set_isolation ISO_CORE -domain PD_CORE \
    -isolation_power_net VDD_AON -clamp_value 0 \
    -applies_to outputs
set_isolation_control ISO_CORE -domain PD_CORE \
    -isolation_signal u_pmc/iso_en -isolation_sense high
```

여기서 틀린다. 아이솔레이션을 잊는 것. 전원 영역이 무너지면 그 출력은 중간 전압으로 떠간다. 그것을 상시 전원 게이트에 넣으면 두 소자가 모두 도통한다 — 상시 전원 영역이 마이크로암페어가 아니라 밀리암페어를 먹게 되고, 팔았

던 보존 모드는 존재하지 않게 된다. 저전력 IP 에서 가장 흔한 통합 실패 가운데 하나이고, RTL 시뮬레이션에는 중간 전압이 없으므로 RTL 에서는 안 보인다.

T1.6 무시할 수 없는 이차 효과 둘

바디 효과

V_t 는 소스를 벌크에 묶은 상태로 적힌 값이다. 소스를 벌크보다 올리면 채널 아래 공핢층이 넓어지고, 반전이 일어나기까지 게이트 전압이 더 필요해진다. 즉 V_t 가 오른다. 의존 관계는 $V_t = V_{t0} + \gamma(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F})$ 이고, γ 는 0.2-0.5 $\sqrt{V}$ 정도의 공정 상수다.

실제로 따라오는 결과는 이것이다: **소스가 레일에 있지 않은 소자는 라이브러리가 말하는 것보다 약하다.**

어디에 쓰나	NAND/NOR 안의 쌓인 소자 — 위쪽 NMOS 는 소스가 접지보다 위에 있다. 패스 게이트 사슬. 캐스코드 전류원. 전송 게이트로 지은 다중화기 나무. 트랜지스터가 직렬로 놓이는 모든 자리.
언제 쓰나	쌓을 때마다. 4 입력 NAND 는 인버터보다 그냥 4 배 느린 것이 아니다 — 그보다 나쁘다. 위쪽 소자들이 바디 효과를 받기 때문이다. 넓은 게이트를 평평하게 짓지 않고 나무로 쪼개는 까닭 가운데 하나가 이것이다.
어떻게 쓰나	손으로 따질 때는 V_t 이동을 더하고 위쪽 소자의 과구동을 다시 계산한다. 실제 치수는 특성화된 라이브러리가 처리하게 둔다 — .lib 의 지연표에 이미 이 효과가 들어 있다 — 그러나 그것이 거기 있다는 것은 알고 있어야 , 새로 지어낸 쌓기 구조가 선형으로 비례할 것이라고 넘겨짚지 않는다. 쌓기가 깊으면 대신 버퍼를 끼운다.

실제 코드는 이렇게 생겼다

```
// Why you will not find `body effect` in an RTL file, but will
// feel it: the synthesiser's cell choice reflects it.
// These two are logically identical and time very differently.

assign y = ~(a & b & c & d);           // maps to NAND4 — deep stack

wire ab = ~(a & b);                   // maps to NAND2 + NAND2 + NOR2
wire cd = ~(c & d);                   // shallower stacks, more cells
assign y = ~(~ab | ~cd) ? 1'b0 : 1'b1;

// Liberty shows the penalty directly. Compare the intrinsic delays:
//   NAND2_X1   cell_rise (A1) ~ 0.021 ns
//   NAND3_X1   cell_rise (A1) ~ 0.031 ns
//   NAND4_X1   cell_rise (A1) ~ 0.045 ns    <- worse than linear
```

여기서 틀린다. 소자 n 개를 쌓으면 n 배 느려진다고 여기는 것. 바디 효과와 내부 마디 용량 때문에 초선형이고, 서너를 넘으면 벌점이 빠르게 커진다. 합성된 넷리스트의 임계 경로에 NAND4 가 있다면, 대개 가장 먼저 구조를 뜯어고칠 자리가 거기다.

게이트 용량 — 당신이 몰고 있는 부하

당신이 모는 게이트는 축전기다: $C_g \approx C_{ox} \cdot W \cdot L$, 여기에 L 에 비례하지 않는 겹침 용량이 더해진다. 1.2 nm 산화막이면 게이트 면적당 $1.60 \times 10^7 \text{ fF}/\mu\text{m}^2$ 정도다.

이 사실 하나가 디지털 설계 전체를 묶는다. 한 게이트의 출력이 다음 게이트의 입력 용량을 채워야 하므로, **지연은 부하 용량 대 구동 세기의 비에 비례한다**. 그 비가 T_2 의 전기적 노력이고, 치수 정하기가 게이트마다의 문제가 아니라 사술의 문제인 까닭이 그것이다.

아날로그와 고속 디지털에서 무슨 향이 하나 더 있다. 게이트-드레인 겹침 용량 C_{gd} 가 출력 쪽에 그 단의 이득만큼 곱해져서 나타난다(밀러 효과). 비교기 앞단 증폭기에서는 이것이 부하를 지배할 수 있고, 디지털 게이트에서는 천이 동안 보이는 입력 용량이 정적인 값보다 큰 까닭이 된다.

어디에 쓰나	모든 지연 계산. 클럭 트리 전력(클럭은 엄청난 총 게이트 용량을 모는데, 클럭 게이팅이 첫 번째 전력 최적화인 까닭이 그것이다). 고객이 자기 드라이버를 예산 잡을 수 있도록 데이터시트에 적는 입력 용량.
언제 쓰나	디지털에서는 늘. 특히 IP 데이터시트의 입력 용량 줄을 적을 때, 그리고 셀 하나가 부하를 몇 개까지 몰아도 되는지 정할 때.
어떻게 쓰나	부하를 최소 인버터 입력 용량의 배수로 적으면 셈이 손으로 할 만해진다 — 논리적 노력(T_2)이 형식화한 것이 정확히 이것이다. 납품물 쪽에서는 핀마다의 입력 용량을 .lib 와 데이터시트에 적는다. 통합하는 쪽이 그것으로 자기 드라이버 치수를 정하고 클럭 트리를 짓는다.

실제 코드는 이렇게 생겼다

```
// The datasheet line that comes from this, and the .lib it matches.
// If these two disagree, the customer's timing closure fails and
// they will find out before you do.

pin (D) {
    direction      : input ;
    capacitance     : 1.412 ;      // fF — what the integrator budgets
    rise_capacitance : 1.455 ;
    fall_capacitance : 1.369 ;
    max_transition  : 0.300 ;      // ns — beyond this, delay tables
                                // are extrapolated, not valid
}

// SDC the customer writes against your block:
set_load      0.0014 [get_ports eq_out*] ;# pF
set_driving_cell -lib_cell BUF_X4 [get_ports adc_data*]
set_max_transition 0.300 [current_design]
```

여기서 틀린다. 넷이 max_transition 을 넘게 두는 것. 그 점을 넘으면 라이브러리의 지연표는 외삽되고 있는 것이고, 타이밍 보고서는 더 이상 특성화된 자료가 아니다 — 자신만만한 수가 옆에 붙은 짐작이다. max-transition 위반은 경고가 아니라 확실한 오류로 다루라.

T1.7 어느 일에 어느 영역인가

표 46. 네 영역은 네 가지 설계 연장이다

영역	조건	소자가 이렇게 동작한다	이런 데 쓴다	중요한 수
차단	$V_{GS} < V_t$	새는 열린 스위치	대기, 보존, 전원 게이팅	I_{off} 와 아문턱 기울기 S
선형	$V_{GS} > V_t, V_{DS} < V_{ov}$	게이트로 제어되는 저항	패스 게이트, 표본화 스위치, 전원 스위치	R_{on} , 그리고 그것이 신호에 따라 얼마나 변하나
포화	$V_{GS} > V_t, V_{DS} > V_{ov}$	전압으로 제어되는 전류원	증폭기, 비교기, 전류 거울, CTLE	$g_m, r_o, g_m/I_D$
아문턱	V_{GS} 가 V_t 보다 조금 < 작다	지수적 트랜스컨덕터	상시 전원 블록, 초저전력, 보존	S , 그리고 V_t 불일치

디지털 셀은 거의 모든 시간을 앞의 두 줄에서 보내고, 천이할 때마다 세 번째 줄을 잠깐 지나간다. 그 잠깐이 사소한 것이 아니다. 그것이 T2 의 단락 전류이고 T3 의 재생이며, 이 책에서 가장 어려운 문제 둘 — 준안정과 비교기의 판별 — 은 전부 소자가 선형이나 차단으로 정착하지 않고 포화에 머무는 동안 무슨 일이 일어나는가에 대한 논의다.

실제로 난 일

양산에 들어간 표본화 스위치를 중간 눈금 R_{on} 에서 치수 잡았고, 모든 코너의 시뮬레이션에서 정착 목표를 맞췄다. 그런데 실리콘에서 2 차 고조파 왜곡이 시뮬레이션보다 12 dB 나쁘게 나왔다. 까닭은 검증 쓸기가 정착을 볼 때는 입력을 중간 눈금에 붙들어 두고, 입력을 쓸 때는 이득만 봤다는 것이다 — 그래서 정착 중요한 조합, 큰 입력과 빠듯한 정착 이 한 번도 시뮬레이션되지 않았다. 설계에는 틀린 것이 없었다. **검증 격자**가 틀렸다. T21 이 이것을 규칙으로 만든다: 축을 하나씩 쓸지 말고 같이 쓸라.

표 47. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
아문턱 기울기 S	모든 블록의 대기 규격	데이터시트의 누설 수치, 그리고 보존이 가능한가
R_{on} 과 그 신호 의존	표본화 스위치, 패스 게이트, 전원 스위치	정착 시간, 왜곡, 전원 스위치 양단의 IR 강하
g_m/I_D	비교기 앞단, CTLE, 바이어스	전류 한 단위당 이득과 잡음 — 즉 전력 예산
채널 길이 변조 λ	전류 거울, 캐스코드	바이어스 전류가 출력 전압에 따라 얼마나 움직이나
바디 효과 γ	쌓인 논리, 전송 게이트 나무	깊은 쌓기가 왜 초선형으로 느린가
게이트 용량 C_g	모든 지연 계산과 클럭 전력 계산	데이터시트의 입력 용량 줄

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

소자 수준의 발명은 팹리스 IP 회사에는 대체로 닫혀 있다 — 소자는 파운드리인 것이다. **열려 있는 것은 주어진 소자들 가운데 무엇을 어떻게 고르는가다.** 실제로 팔리는 IP 포트폴리오의 청구항은 흔히 선택과 배치 에 걸린다: 상태를 붙드는 마디에만 고 V_t 소자를 쓰고 읽기 경로는 표준 V_t 두는 보존 래치. 코너 전체에서 경합 전류가 묶이도록 풀업 쌓기의 치수를 정한 레벨 시프터. V_{GS} 를 일정하게 붙들어 R_{on} 이 신호와 무관해지게 만드는 부스트스트랩 회로를 가진 표본화 스위치. 셋 다 소자 영역에 대한 논의이고, 셋 다 회로로서 특허가 된다. 나는 지금 어느 영역에 있고, 그 영역이 내게 무엇을 주는가를 물으며 이 장을 읽으라 — 청구항은 그 물음에서 나온다.

T2. 지연 — RC 하나에서 타이밍 경로 전체까지

도구가 찍어 주는 지연 수치는 넷 중 하나의 모형이다. 어느 것인지 알라. 모르면 당신 회로와 무관했던 수를 믿게 된다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 미분 · 지수함수 · 옴의법칙 · 문턱전압 · 선형영역 · 포화영역 · 온저항 · 게이트용량 · 출력저항

이 장에서 새로 나오는 것

RC지연 · 엘모어지연 · 논리적노력 · 전기적노력 · 기생지연 · FO4 · 슬루 · 단락전류 · 배선지연 · 리피터 · 도착시각 · 요구시각 · 슬랙 · OCV · NLDM · 비선형지연모델

기술자 넷에게 인버터의 지연이 얼마냐고 물으면 답이 넷 나오고, 넷 다 맞다. 서로 다른 네 모형으로 답하고 있기 때문이다. SPICE 를 쓰는 사람은 미분방정식으로 답한다. 물리 설계 기술자는 Liberty 파일의 2 차원 찾아보기 표로 답한다. 마이크로아키텍트는 FO4 로 답한다. 합성 기술자는 피코초가 하나도 안 들어간 무차원 수로 답한다. 이것들은 하나의 진실을 두고 겨루는 근사가 아니다. 서로 다른 일을 맡은 네 개의 연장이고, 엉뚱한 것을 쓰는 것이 바로 **실패한 경로와 무관했던 수**로 설계를 사인오프하게 되는 길이다.

이 장은 그 넷을 차례로, 앞의 것에서 뒤의 것을 지어 나가고, 걸음마다 그 모형이 옳은 연장이 되는 영역을 말한다. 여기 나오는 수는 전부 이 장의 소스 맨 위에 선언된 공정 상수 한 벌 — 0.9 V 의 28 nm 급 저전력 노드 — 에서 계산한 것이고, 아래의 모든 표는 손으로 타이핑한 것이 아니라 그것에서 계산된다.

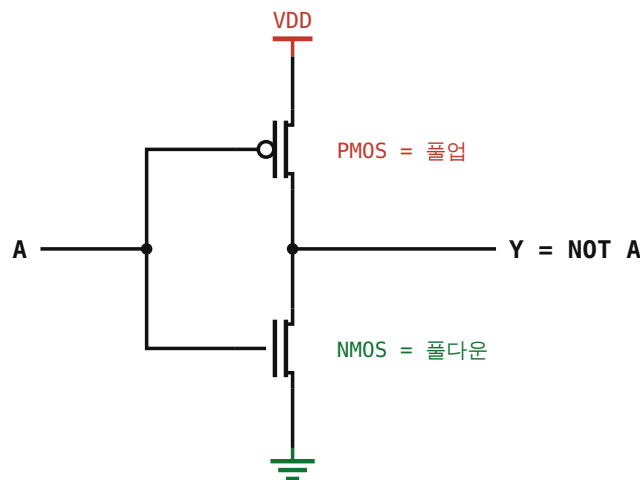
T2.1 머릿속으로 할 수 있는 단 하나의 모형: 스위치 달린 RC

CMOS 게이트는 저항처럼 동작하는 트랜지스터를 거쳐 축전기를 채우고 비운다. 그것이 전부다. 이 장의 나머지는 전부 저 한 문장을 다듬은 것이고, 다듬을 때마다 저 문장 안의 어떤 가정이 깨졌기 때문에 다듬은 것이다.

RC지연 (RC delay) — 게이트의 출력 마디는 등가 저항 R 을 거쳐 구동되는 용량 C 다. 그 마디의 전압은 감쇠하는 지수함수이고, 지연은 받는 쪽의 스위칭 문턱을 지나가는 데 걸리는 시간 — 관례로 V_{DD} 의 50 % — 이다.

유도 — 트랜지스터에서 0.69 RC 까지

	이렇게 된다	왜냐하면
1	스위칭하는 게이트는 출력 마디를 PMOS 쌍기를 거쳐 V_{DD} 에 잇거나 NMOS 쌍기를 거쳐 접지에 잇는다.	정적 CMOS 게이트가 바로 그것이다. 서로 보완하는 두 회로망이 있고, 정상상태에서는 정확히 하나만 도통한다.
2	스위칭하는 동안 도통하는 트랜지스터는 저항이 아니다 — 포화에서 시작해 선형으로 끝난다.	V_{DS} 가 V_{DD} 에서 시작해 0 으로 떨어지므로, 소자가 천이 중에 영역을 건너간다 (T_1).
3	그래도 우리는 그것을 등가 저항 R_{eq} 하나로 같음한다. 같은 50 % 통과 시간을 주는 값으로 정의한다.	이것은 쯤 저항이 아니라 맞춰 넣은 양이다 — 책마다 값이 다른 까닭이고, 출처가 다른 R_{eq} 를 서로 견주면 안 되는 까닭이다.
4	그러면 떨어지는 출력에 대해 $V(t) = V_{DD} \cdot e^{-t/RC}$ 다.	계단 입력을 받는 1 차 선형 상미분방정식의 표준해다.
5	$V = V_{DD}/2$ 로 놓으면 $t = RC \cdot \ln 2 = 0.69 RC$ 다.	$\ln 2 = 0.693$. 모든 지연 식에 나오는 0.69 가 여기서 온다 — 그것은 50 % 지점일 뿐 그 이상이 아니다.
6	10-90 % 상승/하강 시간은 같은 해에서 $t = RC \cdot \ln 9 = 2.2 RC$ 로 나온다.	같은 지수함수, 다른 두 통과점. 0.69 와 2.2 는 같은 파형을 말한다 — 지연과 슬루는 한 곡선을 두 번 읽은 것이다.



입력이 0 -> 위 (PMOS)가 켜짐 -> Y 를 VDD 로 당겨 올린다
 입력이 1 -> 아래 (NMOS)가 켜짐 -> Y 를 GND 로 당겨 내린다
 둘이 동시에 켜지지 않는다 -> 정상상태 전류가 0 이다

그림 25. 단위 인버터. 풀업과 풀다운 회로망이 RC 의 R 이고, 다음 단의 게이트 용량과 배선이 C 다.

$\tau = R_{eq} \cdot C_g$ — 이 공정의 시간 단위

기준 소자를 하나 고른다 — 최소 크기 인버터 — 그리고 수 둘을 준다: 등가 출력 저항 $R_{eq} = 8\text{ k}\Omega$ 과 입력 용량 $C_g = 1\text{ fF}$. 그 곱 $\tau = 8\text{ ps}$ 가 이 장의 나머지에서 유일한 절대 시간이다. 다른 모든 지연은 그것의 배수로 적힌다.

이것은 꾸밈이 아니다. 게이트 사이의 비는 위상이 정하고 공정·전압·온도가 바뀌어도 몇 퍼센트 안에서 안정적이다. 절대값 τ 는 코너를 건너가면 **2 배 넘게** 움직인다. 그 둘을 갈라 두어야 종이 위에서 아키텍처를 하고 피코초는 도구가 대게 할 수 있다.

어디에 쓰나

임계 경로를 손으로 어림하는 모든 자리, 이른 아키텍처 맞바꿈 전부, RTL 이 생기기 전에 '이게 한 주기에 들어가나' 를 묻는 모든 자리.

언제 쓰나

합성 전에 — STA 를 걸 넷리스트가 아직 없을 때. 그리고 STA 뒤에, 나온 수가 그럴듯하기라도 한지 확인할 때.

어떻게 쓰나

경로를 τ 로 적고 마지막에 한 번만 곱한다. 쓸 만한 기준점이 **FO4** 다 — 인버터가 자기 복사본 넷을 모는 것 — 이 공정에서는 40 ps 다. 요즘의 고성능 파이프라인 한 단은 대략 12-20 FO4 이고, 저전력이면 25-40 이다.

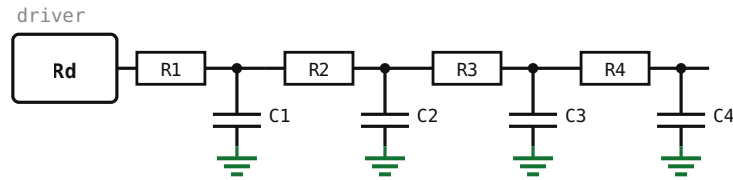
실제 코드는 이렇게 생겼다

```
// A 'timing shell' exists so that the number you measure is about
// your logic, not about the pads. Synthesise the DUT inside registers:
module fir_shell (input clk, input rst, input signed [7:0] din, output signed [15:0] dout);
    reg signed [7:0] din_q;
    reg signed [15:0] dout_q;
    wire signed [15:0] y;
    always @(posedge clk) begin din_q <= din; dout_q <= y; end
    fir4 u_dut (.clk(clk), .rst(rst), .x(din_q), .y(y));
    assign dout = dout_q;
endmodule
// Measured on this repository's own FIR: 455 MHz without the shell, 47 MHz with it.
// The 455 MHz was the tool timing a path that started and ended at a pin.
```

여기서 틀린다. 입력과 출력이 핀인 설계에서 F_{max} 를 인용하는 것. IP 데이터시트에서 가장 흔한 거짓 수치다. 경로에 띄우는 레지스터도 받는 레지스터도 없으면 도구는 제약할 것이 없어서 찾아낸 조합 지연을 그대로 찍는다 — 흔히 10 배 낙관적이다.

T2.2 엘모어 지연 — RC 하나가 사슬이 될 때

단일 RC 모형은 저항이 한쪽 끝에, 용량이 다른 쪽 끝에 있다고 본다. 배선은 그 둘을 다 어긴다. 저항도 용량도 배선을 따라 퍼져 있다. 그것을 N 토막으로 잘라 더한 것이 엘모어 지연이고, 그 합의 모양은 외워 둘 값어치가 있다. 긴 배선이 왜 비례해서 줄지 않는지의 까닭이 거기 있기 때문이다.



조각 i 의 저항은 뒤에 달린 용량 전부를 민다 -- 항이 $1+2+\dots+N$ 로 는다
 그래서 배선 지연은 길이에 비례하지 않고 길이의 제곱으로 크다

그림 26. 배선을 RC 네 토막으로 자른 것. 1 번 토막의 저항은 1-4 번 토막의 용량을 전부 밀어야 하고, 4 번 토막의 저항은 제 것만 민다.

유도 — 배선 지연이 왜 길이의 제곱으로 자라나

	이렇게 된다	왜냐하면
1	길이 L 인 배선을 같은 크기 N 토막으로 자른다. 토막마다 저항 R/N , 용량 C/N .	$R = r \cdot L$, $C = c \cdot L$ 이고, r 은 $\Omega/\mu\text{m}$, c 는 $\text{F}/\mu\text{m}$ 로 기술 파일에서 온다.
2	i 번 토막보다 뒤에 있는 모든 용량의 전하는 i 번 토막의 저항을 지나가야 한다.	전하가 거기 닿는 길이 하나뿐이다 — 엘모어 모형의 내용이 그것뿐이다.
3	그래서 지연은 $\sum_i (R/N) \cdot (N-i+1) (C/N) = RC \cdot (N+1)/(2N)$ 이다.	등차급수 $1+2+\dots+N = N(N+1)/2$. 대입하면 닫힌 꼴이 나온다.
4	$N \rightarrow \infty$ 이면 $RC/2$ 로 가고, R 과 C 는 각각 L 에 비례한다.	따라서 $t \propto r \cdot c \cdot L^2$ — 배선 길이를 두 배로 하면 지연이 네 배가 된다. 게이트 지연은 전혀 안 변하는데도 그렇다.
5	이 공정에서 $1, \mu\text{m}$ 배선은 $RC/2 = 90 \text{ ps}$ 를 주고, 네 토막 근사는 112 ps 를 준다.	거친 토막내기는 1.25 배 과대평가한다. 퍼져 있는 경우에는 닫힌 꼴을 쓰고, 토막은 그 토막이 실제로 있을 때만(비아, 가지, 탭) 쓴다.

표 70. 퍼진 배선을 토막내기: 몇 토막이면 되나

토막 수 N	엘모어 지연	퍼진 $RC/2$ 에 대한 오차
1	18 ps	1 %
2	135 ps	50 %
4	112 ps	25 %
8	101 ps	12.5 %
16	95.6 ps	6.25 %
32	92.8 ps	3.12 %

저 표에 사실 둘이 숨어 있다. 첫째, **한 토막은 100 % 틀린다** — 그리고 한 토막이야말로 순진한 스프레드시트가 하는 짓이다. 둘째, 수렴이 느리다. 8 토막이어도 여전히 6 % 높다. 추출 도구가 아예 토막을 안 내고 퍼진 회로망을 푸는 까닭이 이것이다. 손 여림은 $RC/2$ 를 쓰되, 그것이 추출기가 낼 값의 하한이라는 것을 기억하라. 추출기는 이웃과의 결합 용량까지 보기 때문이다.

엘모어 지연

RC 나무의 임펄스 응답의 1 차 모멘트다. 마디 k 에 대해 $t_k = \sum_i R_{ik} \cdot C_i$ 이고, R_{ik} 는 마디 i 로 가는 길과 마디 k 로 가는 길이 함께 쓰는 저항이다. 단일 RC 에서는 정확하고, 계단 입력을 받는 RC 나무의 50 % 지연에 대해서는 늘 **상한**이며, 인덕턴스는 전혀 안 본다.

어디에 쓰나	클럭 트리 합성(요의 균형 맞추기), 버퍼와 리피터 끼우기, 그리고 모든 배치기의 증분 타이밍 엔진 안에 든 지연 모형 — 배치기는 한 번 옮길 때마다 전체 추출을 할 여유가 없다.
언제 쓰나	나무 구조에 대해 단조롭고 값싸게 갱신되는 지연이 필요할 때마다. 절댓값으로는 10-30 % 틀리고 순서로는 맞는데, 최적화기가 필요한 것이 바로 그 순서다.
어떻게 쓰나	구동점에서 나무를 따라 내려가며 앞마다 함께 쓰는 저항을 누적한다. 절대로 사인오프에 쓰지 마라 — 이것은 최적화 목적함수이지 사인오프 모형이 아니다.

실제 코드는 이렇게 생겼다

```
# Extraction produces this; you read it when a net is suspect.
# SPEF, standard parasitic exchange format, one net:
*D_NET net_dout_3 1.84e-01          $ total capacitance, pF
*CONN
*I u_fir/Z 0 *L 0 *D INVX4          $ driver
*I u_reg/D I *L 1.02e-03            $ receiver pin cap, pF
*CAP
1 net_dout_3:1 2.31e-02
2 net_dout_3:2 net_clk:7 4.10e-03    $ COUPLING to an aggressor -- not in your hand estimate
*RES
1 net_dout_3:1 net_dout_3:2 1.79e+01 $ ohms
```

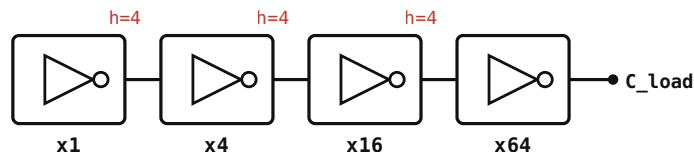
여기서 틀린다. 엘모어는 계단 입력에 대한 상한이다. 진짜 입력은 슬루가 유한하고, 빠른 입력이 긴 배선으로 들어가면 실제 50 % 지연이 그 상한을 넘을 수 있다. 슬루가 크면 당신이 기대는 그 상한은 배운 그 상한이 아니다 — 그것이 다 음절이다.

T2.3 논리적 노력 — 피코초를 덜어 낸 지연

게이트 사슬의 치수를 RC 셈으로 정하는 것은 되기는 되는데 괴롭다. 논리적 노력은 문제에서 공정을 통째로 걷어내고, 머릿속에서 최적화할 수 있는 무차원 식만 남긴다. 그리고 실제로 부딪히는 두 물음에 답한다: 단을 몇 개로 할 것인가, 그리고 단마다 얼마나 커져야 하는가.

유도 — 한 단의 정규화된 지연

	이렇게 된다	왜냐하면
1	게이트의 지연을 $d = (R_{\text{gate}} \cdot C_{\text{out}}) + (R_{\text{gate}} \cdot C_{\text{self}})$ 로 적는다.	출력 마디는 부하 와 그 게이트 자신의 드레인 용량을 같이 진다. 둘 다 같은 저항을 거쳐 채워진다.
2	전체를 $\tau = R_{\text{inv}} C_g$ 로 나눈다.	모든 것을 최소 인버터에 견주어 재기로 고른 것이다. 잃는 것은 없다. 단위만 없어진다.
3	첫 항이 $g \cdot h$ 가 된다. 논리적 노력 g 는 이 위상이 전류를 모는데 인버터보다 얼마나 나쁜가이고, 전기적 노력 $h = C_{\text{out}}/C_{\text{in}}$ 이다.	g 는 오직 위상에만 달려 있고(NAND2 = 4/3, NOR2 = 5/3, 인버터 = 1), h 는 부하 대 입력의 비에만 달려 있으므로 절대 크기와 무관하다.
4	둘째 항이 p , 기생 지연 이 된다 — 아무것도 안 모는 게이트의 지연.	그 게이트 자신의 확산과 함께 자라는데, 확산은 구 동력이 자라는 것과 똑같이 폭을 따라 자란다 — 그래서 p 는 크기와 무관하다. $p_{\text{inv}} \approx 1$.
5	$d = g \cdot h + p$.	곱 두 번과 덧셈 한 번, 그리고 공정 상수는 어디에도 없다.
6	N 단 경로에서 합은 모든 단이 같은 단 노력 $f = (G \cdot B \cdot H)^{1/N}$ 을 질 때 최소가 된다.	단마다의 크기로 합을 미분해 0 으로 놓으면 대칭해가 떨어진다. $G = \prod g$, $H = C_{\text{load}}/C_{\text{in}}$, B 는 경로에서 갈라져 나가는 가지를 센다.



한 칸의 지연 = $g \cdot h + p$. 사실 전체는 칸마다 같은 h 일 때 가장 빠르다
 그 h 가 대략 **3.6** (전기적 노력의 N 제곱근) -- **4** 로 잡고 시작한다

그림 27. 네 단, 단마다 앞의 4 배. 합을 최소로 만드는 것은 단 노력이 같은 것이지, 크기가 같은 것도 마지막 단 하나가 거대한 것도 아니다.

예제 11. 64 배 부하를 몬다: 단을 몇 개로?

주어진 것. 1 배 인버터가 $64 \cdot C_g$ 부하(받는 쪽이 여럿인 칩 내부 버스)를 몰아야 한다. 인버터 사슬의 논리적 노력은 $G = 1$, 갈래가 없으므로 $B = 1$, $H = 64$ 이고 경로 노력 $F = 64$ 다.

푸는 법. N 마다 $D = N \cdot F^{1/N} + N \cdot p_{inv}$ 를 계산해 가장 작은 것을 고른다. 아래 표는 외운 것이 아니라 계산한 것이다.

답. $N = 3$ 단, 단 노력 $f = 4$, 총 지연 $15 \cdot \tau = 12$ ps. 한 단으로 하면 52 ps 가 걸린다 — **4.33 배 느리다.**

표 71. $F = 64$ 일 때 단 수에 따른 총 지연

단 수 N	단 노력 f	지연 (τ)	지연
1	64	65	52 ps
2	8	18	144 ps
3	4	15	12 ps
4	2.83	15.31	123 ps
5	2.3	16.49	132 ps
6	2	18	144 ps

여기서 틀린다. 표의 최솟값을 날카로운 최적점으로 읽는 것. $N = 3$ 과 $N = 4$ 는 2.1 % 차이다 — 어떤 실제 p 의 값에 안에 들어가는 차이다. 이름뿐인 빠른 쪽이 아니라 싼 쪽을 고르라. 단 하나를 더하면 면적과 누설과, 그리고 그다음에 몰아야 할 슬루가 따라온다.

표 72. 실제로 인스턴스화하게 될 셀들의 논리적 노력

게이트	g	p (τ)	g 가 왜 그 값인가
인버터	1	1	기준
NAND2	4/3	2	NMOS 둘이 직렬: 같은 풀다운 전류를 내려면 각각 2 배 넓어야 하므로 입력 용량이 입력당 4/3 로 오른다
NAND3	5/3	3	셋이 직렬 — g 는 쌓기 깊이에 선형으로 자란다
NOR2	5/3	2	PMOS 가 직렬인데 PMOS 는 약한 소자다. 빠른 논리에서 NOR 를 피하고 NAND 는 안 피하 는 까닭이 이것이다
NOR3	7/3	3	같은 까닭, 더 나쁘다
XOR2	4	4	두 입력의 두 극성이 다 내부에 필요하다
MUX2	2	4	전달 경로 둘에 선택 구동까지
삼상태 인버터	2	2	인에이블 소자가 출력과 직렬로 붙는다 — 전형적인 버스 드라이버

설계 연장으로 쓰는 논리적 노력

$d = g \cdot h + p$. 경로 전체는 $D = \sum(g_i \cdot h_i) + \sum p_i$ 이고, 단 노력이 $f = F^{1/N}$ 으로 같을 때 최소다. 실무 규칙은 $f \approx 4$ ($p_{inv} = 1$ 일 때의 참 최적은 3.6)이고, 단을 하나 더하는 것은 그것이 p 보다 많은 지연을 견어 낼 때만 값어치가 있다.

어디에 쓰나	합성 전에 파이프라인 한 단을 얼마나 깊게 할 수 있는지 정할 때. NAND-NAND 로 할지 NOR-NOR 로 할지 고를 때. 클럭이나 리셋 분배를 손으로 치수 잡을 때. 이상해 보이는 합성 결과와 다룰 때.
언제 쓰나	아키텍처 단계에서, 그리고 도구의 답이 온전한지 확인해야 할 때마다. 합성은 속으로 이것을 하지만 면적과 전력까지 저울에 다는 비용함수를 쓰므로 답이 다르게 나온다 — 이상값을 알아야 그 차이가 맞바꿈인지 실수인지 가를 수 있다.
어떻게 쓰나	경로를 따라 G 를 세고, 실제 부하에서 H 를 내고, $F = G \cdot B \cdot H$ 를 잡는다. 그러면 $N \approx \log_4 F$ 이고 $f = F^{1/N}$ 이다. 부하에서 거꾸로 올라가며 단마다 치수를 낸다: $C_{in,i} = g_i \cdot C_{out,i} / f$.

실제 코드는 이렇게 생겼다

```
# The synthesis constraint that makes the tool solve the same problem.
# Without a real load and a real driver, the tool optimises a fiction.
set_load      0.064      [get_ports dout*]    ;# pF -- from the receiver's .lib
set_driving_cell -lib_cell INVX4 [get_ports din*]    ;# not an ideal source
set_max_transition 0.150 [current_design]          ;# ns -- bounds slew degradation
set_max_fanout 16      [current_design]
# And the one that is almost always missing on a first pass:
set_max_capacitance 0.200 [current_design]
```

여기서 틀린다. g 는 위상의 성질이지 라이브러리에서 고른 셀의 성질이 아니다. 라이브러리의 NAND2_X4 와 NAND2_X1 은 g 가 같다. 다른 것은 C_{in} 뿐이다. 단 수를 안 바꾸고 더 큰 셀로 갈아 끼워 '타이밍을 고쳤다' 고 하는 기술자는 앞 단의 h 를 바꾼 것이다 — 문제를 상류로 옮긴 것이고, 사슬이 길면 그것이 돌아온다.

T2.4 슬루, 그리고 지연에 인자가 둘인 까닭

지금까지의 두 모형은 입력을 하나만 받는다: 부하. 실제 타이밍 도구는 둘을 받는다. 게이트의 지연이 그 입력이 얼마나 빨리 움직였는지에 달려 있기 때문이다. 느린 입력은 두 트랜지스터를 어정쩡하게 켜 둔 채로 두어 쌓기를 통해 전류를 버리고, 출력을 늦게 띄운다. 그것이 위의 모든 것에 대한 가장 큰 보정 하나이고, Liberty 파일이 식이 아니라 표를 들고 있는 까닭이다.

슬루 (slew / transition time) — 신호가 V_{DD} 의 정해진 두 비율 사이를 지나는 데 걸리는 시간. 관례로 10 %-90 % 또는 30 %-70 % 다. 어느 쌍인지는 라이브러리의 성질이고, 서로 다른 쌍을 쓰는 라이브러리를 섞으면 설계의 모든 지연이 조용히 바뀐다.

단락전류 (short-circuit current) — 천이하는 동안 풀업과 풀다운 회로망이 동시에 도통해서 쌓기를 통해 V_{DD} 를 접지에 단락시킨다. 이렇게 잃는 전하는 입력 천이 시간에 비례한다. 보통의 슬루에서는 스위칭 에너지의 5-15 % 이고, 슬루가 망가지면 쓸모 있는 스위칭 에너지를 넘을 수도 있다.

표 73. 같은 인버터, 서로 다른 '지연' 넷 — NLDM 의 한 조각

입력 슬루 (ps)	부하 4 fF	부하 16 fF	부하 64 fF
20	16 ps	40.4 ps	137 ps
60	24 ps	49.2 ps	147 ps
120	36 ps	62.4 ps	162 ps
240	60 ps	88.8 ps	192 ps

저 표는 모형이지 켜 것이 아니다 — 슬루 계수는 보이기 위한 값이다. **그렇다고 말하라.** 진짜 NLDM 표는 코너 전체에 걸쳐 격자점마다 SPICE 를 돌려 특성화한 것이고, 보통 이음마다 · 셀마다 · 코너마다 7×7 점이다. 이 표에서 진짜인 것은 모양이다. 지연은 부하에 대체로 선형으로, 입력 슬루에 대체로 선형으로 오르고, **슬루 항은 작지 않다** — 가벼운 부하에서는 그것이 지배 항이다.

NLDM — 비선형 지연 모형, 그리고 도구가 실제로 읽는 것

Liberty (.lib) 파일은 셀마다, 타이밍 이음마다 2 차원 표 둘을 준다 — 하나는 지연, 하나는 출력 슬루 — 그리고 입력 천이 시간과 총 출력 용량으로 색인한다. 도구는 쌍선형으로 보간해서(가장자리에서는 **외삽해서**) 수를 낸다. 그 뒤의 모든 것 — STA, 합성 비용, 배치 타이밍 — 이 이 표 위에 서 있다.

그 후계인 CCS(composite current source)는 스칼라가 아니라 전류 파형을 저장한다. 받는 쪽의 입력 용량 자체가 전압에 따라 변하거나, 넷의 저항이 커서 드라이버와 넷이 서로 간섭할 때 그 차이가 문제가 된다. IP 납품물에는 **당신의 수가 어느 모형에서 나왔는지 적으라**. 5 % 수준에서 둘은 바꿔 쓸 수 있는 것이 아니다.

어디에 쓰나	합성 · STA · 배치배선의 모든 지연 수치. 그리고 간접적으로 IP 데이터시트에도: 당신의 $F_{\max}$ 는 라이브러리와 코너를 같이 적어야 뜻이 생긴다.
언제 쓰나	.lib 를 직접 읽게 되는 경우는 딱 둘이다. 어떤 경로의 지연이 말이 안 될 때 도구가 표 밖으로 외삽했는지 보려고, 그리고 한 번도 본 적 없는 라이브러리를 지원하라고 할 때.
어떻게 쓰나	이름을 찾아 index_1(입력 슬루)과 index_2(부하)를 읽고, 당신의 동작점이 둘 다의 안쪽인지 확인한다. 표 밖 외삽이 '도구는 통과라는데 실리콘은 아니다' 의 표준적인 까닭이다.

실제 코드는 이렇게 생겼다

```
/* Liberty, one arc of one cell -- this is what the tool reads. */
cell (INVX1) {
  area : 1.596 ;
  pin (A) { direction : input ; capacitance : 1.02 ; }      /* fF */
  pin (Z) {
    direction : output ;
    function : "!A" ;
    max_capacitance : 64.0 ;
    timing () {
      related_pin : "A" ;
      timing_sense : negative_unate ;
      cell_fall (delay_7x7) {
        index_1 ("0.004, 0.012, 0.028, 0.060, 0.124, 0.252, 0.508"); /* input slew, ns */
        index_2 ("0.4, 1.6, 4.0, 8.8, 18.4, 37.6, 76.0");           /* load, fF */
        values ( "0.0091, 0.0142, 0.0241, 0.0431, 0.0812, 0.1574, 0.3098", \
                  "0.0113, 0.0164, 0.0263, 0.0453, 0.0834, 0.1596, 0.3120", \
                  /* ... five more rows, one per input slew ... */ );
      }
      fall_transition (delay_7x7) { /* the OUTPUT slew -- this feeds the next stage */ }
    }
  }
}
```

여기서 틀린다. 지연과 출력 슬루는 서로 다른 표 둘이고, 출력 슬루는 다음 단의 입력 슬루다. 잘못 몰린 넷 하나가 경로 전체를 이렇게 오염시킨다. 단마다 더 나쁜 슬루를 받아 더 나쁜 슬루를 내놓고, 그 악화가 누적된다.

set_max_transition 이 있는 까닭이 그 누적을 막는 것이고, max_transition 제약이 없는 설계는 보고서에서는 셋업을 맞추고 실리콘에서는 실패할 수 있다.

T2.5 배선, 그리고 그것을 구해 주는 리피터

게이트 지연은 공정 노드마다 줄어든다. 배선 지연은 안 줄어든다. 배선이 가늘어지면서 r 은 오르고, 마이크로미터 당 c 는 대체로 그대로이며, 다이는 트랜지스터만큼 빨리 안 줄어든다. 노드마다 짧아져 온 어떤 길이를 넘으면 배선이 경로를 지배하고, 해답은 더 큰 드라이버가 아니다 — 배선이 길지 않게 만드는 것이다.

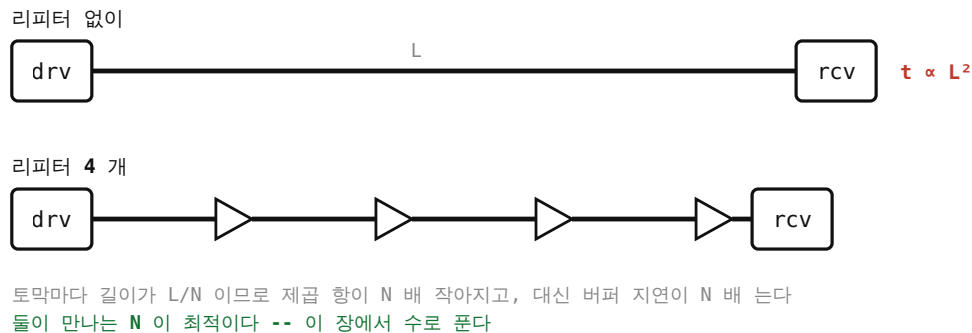


그림 28. 긴 배선, 그리고 리피터로 토막 낸 같은 배선. 제곱 항은 N 분의 1 로 줄고 버퍼 지연은 N 배로 늘어난다.

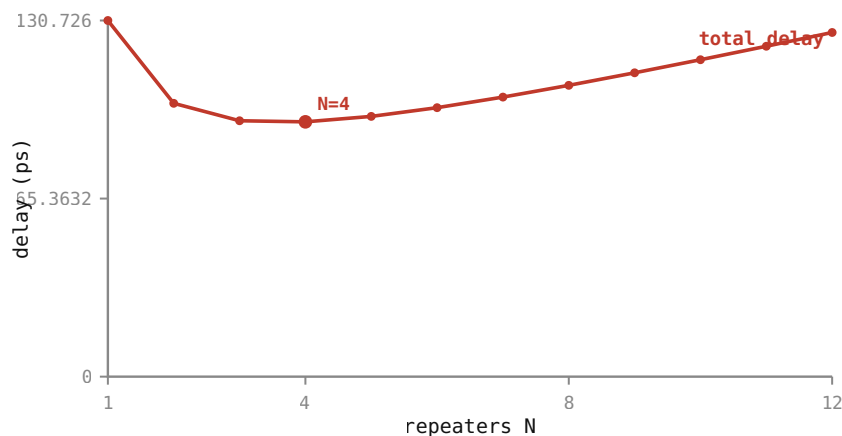


그림 29. 1, μm 배선에 대한 전체 쓰기. 점마다 그 점의 최적 리피터 크기를 썼다. **최솟값이 같다** — $N = 3, 4, 5$ 가 서로 2.1 % 안에 있다 — 그러니 배선이 되는 쪽을 고르라.

유도 — 최적의 어디서 나오나, 그리고 그것을 대조한다

	이렇게 된다	왜냐하면
1	배선을 k 토막으로 자르고 토막마다 크기 h 인 인버터로 만든다.	토막마다 리피터 하나. 마지막 것이 받는 쪽을 본다.
2	토막마다 드라이버가 $0.7 \cdot (R_{\text{inv}}/h)(C_W/k + h \cdot C_g)$, 퍼진 배선이 $0.4 \cdot R_S C_S$, 먼 쪽 부하가 $0.7 \cdot R_S \cdot h C_g$ 를 치른다.	0.4 와 0.7 은 표준적인 분포 RC 계수다 — 자기 용량을 모는 배선에는 0.4, 뭉친 부하에는 0.69.
3	k 와 h 를 수치로 쓰면 $k = 4$, $h = 38$ 에서 93.5 ps 가 나온다.	격자 전체를 무시하게 뒀던 것이다 — 최적의 모양에 대해 아무것도 가정하지 않았다.
4	달힌 꼴 $k = \sqrt{(0.4 R_W C_W / 0.7 R_{\text{inv}} C_g)}$ 과 $h = \sqrt{(R_{\text{inv}} C_W / R_W C_g)}$ 은 $k = 3.59$, $h = 37.9$ 를 준다.	독립 대조다. 같은 답으로 가는 두 길이 크기에서 0.1 % 안에서 맞는다. 아무도 대조하지 않은 유도 하나가 틀린 상수를 몇 해씩 살려 두는 길이다.
5	리피터 없이 같은 배선을 그 배선의 최적 드라이버로 물면 131 ps 다.	그러니 여기서 리피터가 사 주는 것은 1.4 배다 — 그리고 그것이 정직한 표제다. 쓰기에서 가장 유리한 구석에서 집어 온 수가 아니다.

리피터 끼우기

지연이 길이의 제곱인 배선을, 제곱 항이 버퍼 지연과 견줄 만해질 만큼 짧은 토막으로 끊는다. 최적 토막 길이는 **토막마다의 배선 RC가 버퍼 자신의 지연과 같아지는** 길이다. 그것을 넘으면 아무것도 못 아끼면서 버퍼 값만 치르는 것이다.

이 공정에서 갈림점은 배선 25 μm 언저리다 — 그보다 짧으면 리피터가 아끼는 것보다 더 든다.

어디에 쓰나	블록을 벗어나는 모든 넷: 블록 사이 데이터패스, 클럭 척추, 리셋 분배, 그리고 당신의 IP 포트가 닿는 SoC 최상위 조립.
언제 쓰나	물리 설계에서 정해지지만 당신의 납품물에서 제약된다 : 당신 IP의 출력에 200 ps 예산이 있는데 통합하는 쪽의 배선이 300을 요구하면, RTL에는 아무 잘못이 없는데 블록이 실패한다.
어떻게 쓰나	통합하는 쪽에 바람이 아니라 수를 주라. 출력 지연, 구동 세기, 최대 부하를 데이터시트에 적고 양쪽 끝을 <code>set_output_delay / set_load</code> 로 제약한다. 그러면 리피터 문제는 그쪽 것이 되고, 풀 수 있는 문제가 된다.

실제 코드는 이렇게 생겼다

```
# What the integrator must be told, and what you must constrain.
# In your IP's SDC (shipped with the deliverable):
create_clock -name clk -period 4.000 [get_ports clk]
set_output_delay -max 1.200 -clock clk [get_ports {dout[*] dvalid}]
set_input_delay -max 1.000 -clock clk [get_ports {din[*] dvalid_i}]
set_load 0.064 [get_ports dout[*]] ;# what we assumed -- say it in the datasheet
# And in DATASHEET.md, the line that prevents the support call:
# Output timing assumes <= 64 fF load and <= 150 ps transition at the port.
# Beyond that, insert a repeater; every 100 fF adds approximately 25 ps.
```

여기서 틀린다. 리피터는 반전한다. 홀수 개면 넷의 극성이 뒤집히고, 그 해결책 — 인버터 대신 버퍼(인버터 둘)를 쓰는 것 — 은 인버터 사슬보다 지연이 더 든다. 도구가 이것을 처리하지만, **리셋이나 스캔 인에이블 경로에 손으로 놓은 리피터는 전형적인 브링업 버그다**: 한 코너에서만 블록이 틀린 극성으로 리셋된다. 끼워 넣은 한 쌍이 바로 그 코너에서 최적으로 지워졌기 때문이다.

T2.6 셀에서 경로로: STA가 실제로 더하는 것

정적 타이밍 분석은 시뮬레이션하지 않는다. 그래프를 따라 수 둘을 민다 — 신호가 도착할 수 있는 가장 이른 시각과 가장 늦은 시각 — 그리고 끝점마다 요구하는 것과 견준다. 위의 모든 것이 그 그래프의 이름 하나를 댄다. 이 절은 그 이름들이 어떻게 판정이 되는가다.

유도 — 셋업과 홀드를 각각 부등식 하나로

	이렇게 된다	왜냐하면
1	띄우는 플롭이 $t_{\text{launch}} = T_{\text{clk,launch}} + t_{\text{cq}}$ 에 데이터를 내놓는다.	클럭은 띄우는 플롭에 그 플롭 나옴의 시각에 닿는다 — 클럭 지연은 경로의 양 끝에서 같지 않다.
2	그것이 받는 플롭에 $t_{\text{arrival}} = t_{\text{launch}} + t_{\text{logic}}$ 에 닿는다.	t_{logic} 이 이 장이 지금까지 계산해 온 전부다.
3	받는 예지는 $T_{\text{clk,capture}} + T_{\text{period}}$ 에 오고, 데이터는 그보다 t_{setup} 앞서 안정돼 있어야 한다.	그 안정 요구는 플롭의 성질이고, 다른 이음과 똑같은 방식으로 특성화된다.
4	셋업: $t_{\text{arrival}} \leq T_{\text{capture}} + T - t_{\text{setup}} - t_{\text{uncertainty}}$ 슬랙은 그 차다.	클럭 불확실도가 지터와, (CTS 전이라면) 도구가 아직 알 수 없는 스큐를 흡수한다. 슬랙이 양이면 통과, 음이면 실패다.
5	홀드: 가장 빠른 경로가 같은 받는 예지의 홀드 창보다 먼저 닿으면 안 된다: $t_{\text{arrival,min}} \geq T_{\text{capture}} + t_{\text{hold}}$	홀드에는 T가 없다. 클럭을 느리게 해도 홀드 위반은 안 고쳐진다 — 처음 설계하는 사람에게 이 장에서 가장 쓸모 있는 사실 하나다.

표 74. 경로의 수가 당신 어림보다 나빠지는 네 가지 길

효과	무엇을 하나	대략 크기	어디서 정하나
OCV / AOCV / POCV	띄우는 클럭 경로와 받는 클럭 경로의 셀·넷 지연을 서로 다르게 디레이트한다. 같은 다이 위의 똑같은 셀 둘이 똑같지 않기 때문이다	클럭 경로의 3-10 %	set_timing_derate
클럭 불확실도	지터를, 그리고 CTS 전이면 스큐까지 대신하는 평평한 뽀렘	50-150 ps	set_clock_uncertainty
CPPR / CRPR	띄우는 클럭 경로와 받는 클럭 경로가 함께 쓰는 부분에 두 번 걸린 디레이트를 돌려준다	10-50 ps 를 되찾는다	사인오프 STA 에서는 기본으로 꺼짐. 합성에서는 흔히 꺼짐
SI / 누화 지연	반대 방향으로 스위칭하는 가해자가 당신의 천이를 늘린다	길게 나란한 넷에서 5-15 %	SPEF 의 결합을 넣은 SI 인지 STA

report_timing 을 속지 않고 읽기

타이밍 보고서는 장부다. 띄우는 플롭까지의 클럭 경로, 데이터 경로를 따르는 셀과 넷의 지연, 받는 플롭까지의 클럭 경로, 라이브러리 셋업, 그리고 디레이트. **그 보고서가 뜻이 있는지를 정하는 줄은 그것이 어느 코너와 어느 모드에서 돌았는가 하나다** — 이상 클럭으로 전형 코너에서 나온 보고서는 아무것에 대한 증거도 아니다.

어디에 쓰나	모든 타이밍 사인오프, 모든 '이게 왜 실패하지' 조사, 그리고 데이터시트에 적는 모든 F_{max} 수치.
언제 쓰나	슬랙이 음이면 경로 전체를 읽는다. 양이면 시작점과 끝점을 먼저 읽어서, 도구가 당신 생각의 그 경로를 켜진 것이 맞는지 확인한다. 거짓 경로와 제약 없는 끝점은 여기 말고 다른 데서는 안 드러난다.
어떻게 쓰나	확인할 것: (1) 끝점이 포트가 아니라 레지스터인가, (2) 클럭이 진짜(전파된 것)인가 이상 클럭인가, (3) 가장 나쁜 셀에서 입력 천이가 얼마인가 — .lib 색인 범위의 끝이면 그 수는 외삽된 것이다, (4) 경로에서 넷 지연이 얼마인가. 넷 지연이 40 % 를 넘으면 고칠 것은 논리가 아니라 배치다.

실제 코드는 이렇게 생겼다

```
# Startpoint: u_fir/acc_reg[7] (rising edge-triggered flip-flop clocked by clk)
# Endpoint:   u_fir/y_reg[15] (rising edge-triggered flip-flop clocked by clk)
# Corner: ss_0p81v_125c    Check: setup    Path Group: clk
# Point      Incr      Path
# clock clk (rise edge)      0.000      0.000
# clock network delay (propagated) 0.412      0.412    <- real CTS, not ideal
# u_fir/acc_reg[7]/CK (DFFX1) 0.000      0.412 r
# u_fir/acc_reg[7]/Q (DFFX1)  0.138      0.550 f
# u_fir/U231/Z (NAND2X2)      0.061      0.611 r
# u_fir/U248/Z (INVX4)        0.029      0.640 f
# net (fanout=6, length=214um) 0.087      0.727    <- 12% of the path is WIRE
# u_fir/y_reg[15]/D (DFFX1)   0.000      0.727 f
# data arrival time          0.727
# clock clk (rise edge)      4.000      4.000
# clock network delay (propagated) 0.438      4.438
# clock reconvergence pessimism 0.021      4.459    <- CPPR gives this back
# library setup time         -0.096      4.363
# data required time          4.363
# slack (MET)                 3.636
```

여기서 틀린다. 보고서에 clock network delay (ideal) 이라고 적혀 있고 (propagated) 가 아니라면, 클럭 트리가 아직 없다는 뜻이고 스큐는 짐작이다. 이상 클럭 보고서의 수는 사인오프에서 흔히 200-400 ps 움직인다 — 나쁜 쪽으로. 이상 클럭 F_{max} 를 절대 데이터시트에 적지 마라.

T2.7 월요일에 무엇을 할 것인가

표 75. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
RC지연 · τ · FO4	RTL 이 생기기 전의 모든 블록	목표 주파수에서 그 기능이 한 주기에 들어가나
엘모어지연	클럭 트리, 리셋 트리, 긴 데이터패스 넷	버퍼를 어디에 놓아야 하나, 어느 길이 늦은 것인가
논리적노력	임계 경로 재구성, 파이프라인 깊이	단 수, 그리고 단 사이의 크기 비
슬루 · NLDM	.lib 읽기, 말 안 되는 지연 디버깅	도구가 당신 동작점을 보간했나 외삽했나
배선지연 · 리피터	블록 경계와 최상위 조립	데이터시트에 내놓는 출력 부하와 천이 한계
도착시각 · 요구시각 · 슬랙	모든 사인오프	지금 팔려는 그 수가 진짜인가

실제로 난 일

이 저장소는 자기 FIR 필터를 **455 MHz** 로 켜다가 다시 **47.4 MHz** 로 켜다 — 9.6 배 차이 — 같은 RTL 에서. 다른 것은 타이밍 꺾이기 하나였다. 경계에 레지스터가 없으면 도구는 핀에서 핀까지 가는 경로의 지연을 찍는데, 그것은 칩 안에 존재하지 않는 경로다. 이 장의 모든 수는 그것을 켜 경로만큼만 뜻이 있고, 10 배 틀리는 가장 값싼 방법은 엉뚱한 경로를 재는 것이다.

다음 장은 이 장이 다룰 수 없는 딱 하나의 경우를 말한다. 셋업 부등식이 깨지는 까닭이 경로가 느려서가 아니라 데이터가 애초에 그 클럭과 동기가 아니었기 때문일 때 무슨 일이 일어나는가. 답은 더 긴 지연이 아니라 확률이고, 이 책에서 결코 0 으로 만들 수 없는 유일한 타이밍 양이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

게이트의 지연은 물리이고 특허가 안 된다. **지연이 문제가 되지 않도록 블록이 스스로를 배치하는 방식은 특허가 된다**. 실제로 팔리는 IP 의 청구항이 무엇을 덮는지 보라. 되먹임 굵셈-덧셈을 임계 고리에서 빼내 미리 계산한 다중화기로 갈음하는 투기적(언롤링) 결정 되먹임 등화기 — 산술은 그대로인데 고리 지연은 그대로가 아니다. 두 자리올림을 모두 계산해서 면적으로 지연을 사는 캐리 셀렉트 가산기. 느린 경로가 빠른 경로에서 시간을 빌리도록 두 띄우기 지점을 일부러 어긋나게 하는 클럭 트리 방식. 어느 경우든 발명은 지연을 어디로 옮겼는가 이지 트랜지스터가 얼마나 빠른가가 아니다. 이 장이 식만큼이나 어느 향이 지배하는가 에 시간을 쓰는 까닭이 그것이다: 짚어 내지 못한 향은 옮길 수도 없다.

T3. 준안정 — 설계로 없앨 수 없는 단 하나의 실패

다른 모든 타이밍 양은 통과하게 만들 수 있다. 이것만은 드물게 만들 수 있을 뿐이고, 얼마나 드문지를 당신이 말해야 한다.

읽기 전에 알아야 하는 것

전압 · 전류 · 미분 · 지수함수 · 로그 · 확률 · 평균 · 트랜스 컨덕턴스 · 출력저항 · 게이트용량 · RC지연 · 도착시각 · 요구시각 · 슬랙 · FO4

이 장에서 새로 나오는 것

준안정 · 결정시간상수 · 해결시간 · 준안정창 · MTBF · 동기화기깊이 · 다중비트문제 · 재수렴 · 그레이코드 · 핸드셰이크 · 비동기FIFO · 리셋동기화기 · 리커버리 · 리무벌 · CDC검증 · 준정적신호

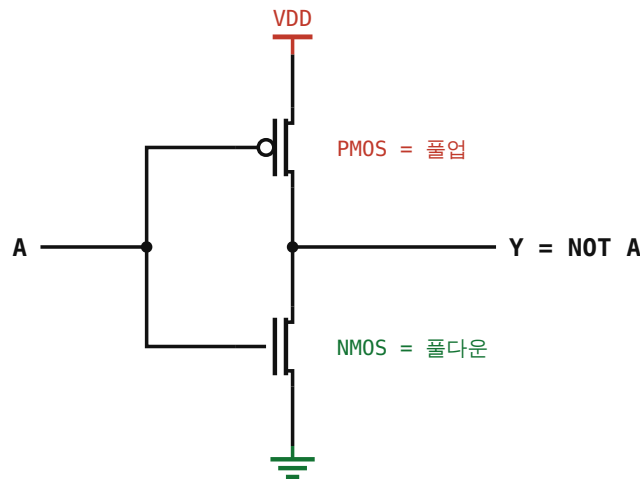
셋업 위반은 일정 문제다. 데이터가 늦었고, 클럭을 늦추거나 논리를 줄이면 고쳐진다. 준안정은 그것이 아니다. 플립플롭이 클럭 에지 바로 그 순간에 변하고 있는 입력을 표본화하면, 출력이 두 합법적인 준위 사이에 무한정 머물 수 있다. 아무리 느리게 해도 없어지지 않는다 — 표본화 순간과 데이터 천이는 서로 무관하므로 그 나쁜 우연은 또 일어난다. **물음은 얼마나 자주냐, 그리고 그것을 예산에 넣었느냐뿐이다.**

이 장은 그 물음에 수로 답한 다음, 나머지 분량을 더 흔한 실패에 쓴다. 수를 바르게 계산하고 2 단 동기화를 넣 고도 멈추는 칩을 내보내는 기술자들 — 경계를 건넌 것이 한 비트가 아니었기 때문이다.

T3.1 결정이 왜 무한정 걸릴 수 있나

유도 — 교차 결합 쌍에서 지수함수까지

	이렇게 된다	왜냐하면
1	플립플롭의 저장 소자는 고리를 이룬 인버터 둘이다. 각각의 출력이 다른 쪽의 입력을 몬다.	그 고리가 클럭 없이 상태를 붙드는 것이다 — 축전기를 따로 이름 붙이지 않고도 기억을 갖는 디지털 설계의 유일한 구조다.
2	그 고리에는 평형점이 셋 있다. 단단한 0, 단단한 1, 그리고 두 인버터가 각자의 스위칭 문턱에 앉는 가운데 한 점.	V_{out1} 대 V_{in1} 과 두 번째 인버터의 거울 곡선을 그리면 세 번 만난다. 가운데 교점이 준안정점이다.
3	가운데 평형은 불안정 하다. 거기서 고리 이득이 $A^2 > 1$ 이므로 어떤 벗어남이든 자란다.	인버터마다 문턱에서 소신호 이득 $A = -g_m(r_{o,n} \parallel r_{o,p})$ 를 갖고, T1 이 두 항을 다 줬다.
4	그 둘레에서 선형화하면 $C \cdot dv/dt = (A - 1) \cdot v/R$, 즉 $v(t) = v_0 \cdot e^{t/\tau}$ 이고 $\tau = RC/(A - 1)$ 이다.	여기서 τ , 결정 시간상수가 나온다 . 그것은 회로의 성질이고 — 대략 고리의 RC 를 그 남는 이득으로 나눈 것 — 당신이 고를 수 있는 규격이 아니다.
5	올바른 준위로 결판내려면 고리가 벗어남을 v_0 에서 대략 $V_{DD}/2$ 까지 키워야 하고, 그것이 $t = \tau \cdot \ln(V_{DD}/2v_0)$ 걸린다.	시간은 표본이 가운데에 얼마나 가까이 떨어졌는지의 로그 다 — 그리고 v_0 는 얼마든지 작을 수 있으므로 t 에 상한이 없다. 대비할 최악의 경우가 없고 분포만 있다.
6	결판나는 데 t_r 보다 오래 걸리는 창 안에 표본이 떨어질 확률은 $e^{-t_r/\tau}$ 로 준다.	클럭 주기 안에서 도착이 고르고 v_0 가 에지에 얼마나 가까웠는지에 비례하므로, 지수가 곧바로 나온다.



입력이 0 -> 위(PMOS)가 켜짐 -> Y 를 VDD 로 당겨 올린다
 입력이 1 -> 아래(NMOS)가 켜짐 -> Y 를 GND 로 당겨 내린다
 둘이 동시에 켜지지 않는다 -> 정상상태 전류가 0 이다

그림 30. 고리를 이룬 두 인버터 가운데 하나. 준안정점에서 그것은 자기 스위칭 문턱에 앉아 있고 두 소자가 모두 포화에 있다 — 그 인버터가 가질 수 있는 가장 이득이 크고, 가장 전류가 크고, 가장 불안정한 동작점이다.

저 유도에서 결론 둘이 떨어지고 둘 다 실무에서 중요하다. 첫째, τ 는 소신호 양이다. 그래서 이득이 나빠지는 바로 그 자리에서 나빠진다 — 느린 코너, 낮은 전압, 높은 온도. 둘째, 준안정점은 플롭이 전류를 가장 많이 태우는 자리다 — 두 소자가 다 켜져 있다. 자주 준안정에 빠지는 설계는 논리적으로만 실패하는 것이 아니라, 예산에 없던 전류까지 끌어간다.

T3.2 MTBF, 그리고 그 안의 기호 하나하나가 실제로 무엇인가

MTBF (mean time between failures) — 동기화기가 주어진 시간 안에 결판내지 못하는 사건 둘 사이의 평균 시간. **보장이 아니라 비율이다** — MTBF 1000 년은 첫 실패가 1000 년째에 난다는 뜻이 아니라, 실패가 그 평균을 갖는 포아송이라는 뜻이다.

MTBF 식

$$MTBF = e^{t_r/\tau} / (T_0 \cdot f_c \cdot f_d)$$

t_r — 해결 시간: 무엇인가가 읽기 전에 동기화기 플롭에게 정착하라고 준 시간. 한 단 건넘이라면 T_{clk} 에서 그 플롭 자신의 t_{cq} 를 빼고, 그것을 읽는 쪽의 셋업을 빼고, 사이의 논리를 뺀 것이다. τ — 결정 시간상수, T3.1 에서 왔고 여기서는 25 ps. T_0 — 표본이 불안정에 빠질 수 있는 구경의 실효 폭, 여기서는 1 ps. 둘 다 유도된 것이 아니라 셀마다 코너마다 댄 것이다. f_c — 표본화(목적지) 클럭 주파수. f_d — 비동기 입력이 변하는 비율.

지수에 들어 있는 것은 t_r 뿐이고, 이 식의 공학적 내용은 전부 그것이다: **당신이 할 수 있는 것은 전부 선형이고, 당신이 제어할 수 없는 것이 지수다.** f_d 를 두 배로 하면 MTBF 가 반이 된다. 동기화기 단을 하나 더하면 지수가 곱해진다.

어디에 쓰나	설계의 모든 비동기 경계: 바깥 인터럽트, 리셋 해제, 누름 버튼, 클럭 도메인을 건너가는 FIFO 포인터, PHY 의 복원 클럭 도메인이 코어와 만나는 자리.
언제 쓰나	아키텍처 단계에서 깊이를 고르고, 사인오프에서 수를 적는다. MTBF 와 그것을 계산한 조건 없이 'CDC 안전' 이라고 적은 IP 데이터시트는 아무것도 주장하지 않은 것이다.
어떻게 쓰나	τ 와 T_0 를 라이브러리의 동기화기 셀 특성화에서 받는다(코너마다 공개돼 있다). t_r 은 이상적인 클럭 주기가 아니라 실제 경로에서 낸다. 그리고 MTBF 를 느린 코너에서 적는다. τ 가 거기서 가장 나쁘기 때문이다.

실제 코드는 이렇게 생겼다

```
// The synchroniser, written so the tools recognise it.
// The attribute is not decoration: it tells the tool to keep the two flops
// adjacent and out of retiming, and it tells CDC checkers this is intentional.
(* ASYNC_REG = "TRUE" *) reg [1:0] sync_q; // Xilinx; Intel: altera_attribute
always_ff @(posedge clk_dst or negedge rst_n) begin
    if (!rst_n) sync_q <= 2'b00;
    else sync_q <= {sync_q[0], async_in}; // 2-stage
end
wire safe_in = sync_q[1];

// And the constraint, without which STA tries to time an impossible path:
// set_max_delay -datapath_only 2.0 -from [get_pins u_src/*/Q] -to [get_pins u_sync/sync_q_reg[0]/D]
// NOT set_false_path -- a false path lets the tool put unbounded wire delay there,
// and then the first stage's own setup window is no longer what you assumed.
```

여기서 틀린다. 건넘에 `set_false_path` 를 거는 것이 가장 흔한 CDC 제약 잘못이다. 타이밍 보고서를 조용하게 만들지만, 동기화기로 들어가는 배선 지연의 한계도 같이 없앤다 — 그래서 원천이 목적지 예지에 얼마든지 가까이 변할 수 있게 되고 게다가 버스의 두 비트 사이 스큐에 상한이 없어진다. `set_max_delay -datapath_only` 를 쓰라.

표 76. 클럭 주파수와 동기화기 깊이에 따른 MTBF ($\tau = 25$ ps, $T_0 = 1$ ps, $f_d = f_c/10$, 부대비용 15 ps)

f_clk	단마다 t _r	1 단	2 단	3 단
0.5 GHz	1,85 ps	1.7e+18 yr	2.4e+50 yr	3.3e+82 yr
1 GHz	85 ps	1.85 년	1.1e+15 yr	6.3e+29 yr
2 GHz	35 ps	9.53×10 ⁻¹⁰ 년	0 년	1,378 년

예제 12. 왜 2 단이 기본이고, 어디서부터 그것으로 모자라나

주어진 것. 목적지 클럭 1 GHz, 100 MHz 로 변하는 입력, 단마다 부대비용 15 ps 이므로 t_r = 85 ps.

푸는 법. 깊이마다 수를 식에 한 번씩 넣는다. 단마다 자기 몫의 t_r 을 통째로 받으므로 지수가 단 수에 비례해 커진다.

답. 1 단: 1.85 년 — 건넌 하나당 몇 해에 한 번 실패한다. 2 단: 1.1e+15 yr. 3 단: 6.3e+29 yr.

표 77. 1 GHz 에서 같은 2 단 건넌, τ 가 나빠질 때

τ	MTBF
1×	1.08e+15 yr
1.2×	1.34e+09 yr
1.5×	1.54e+05 yr
2×	1.85 yr

여기서 틀린다. 2 단 수치를 코너의 성질이 아니라 설계의 성질로 읽는 것. τ 가 겨우 2 배 나빠진 값으로 — 느린 코너, 낮은 전압에서 충분히 있을 수 있는 열화다 — 다시 계산하면 같은 2 단 건넌이 **1.85 yr** 을 준다. 지수가 무너진 것이다. **MTBF 는 당신이 제어하지 못하는 바로 그 하나의 매개변수에 지수적으로 민감하다.** 그 수를 느린 코너에서 적어야 하는 까닭이고, 세 번째 단이 있는 까닭이다.

2 GHz 에서 그 표가 멈춰 볼 만한 것을 말한다. **1 단** 건넌의 MTBF 가 0.0301 초다. 해가 아니라 초다. 그 주파수에서 1 단으로 100 년에 닿으려면 t_r = 984.4 ps 가 있어야 하는데, 그것은 클럭 주기 자체(500 ps)보다 길다. **한 주기로는 요구를 아예 못 맞춘다** — 깊이가 안전 여유이기를 그만두고 회로가 도는 유일한 방법이 되는 영역이다.

T3.3 동기화기는 CDC 의 해답이 아니다

지금까지는 전부 한 비트 이야기다. 실제로 실리콘까지 닿는 실패는 MTBF 를 잘못 잡아서 나는 일이 거의 없다. 버스의 비트마다 제대로 동기화해 놓고도 진 설계자들에게서 난다.

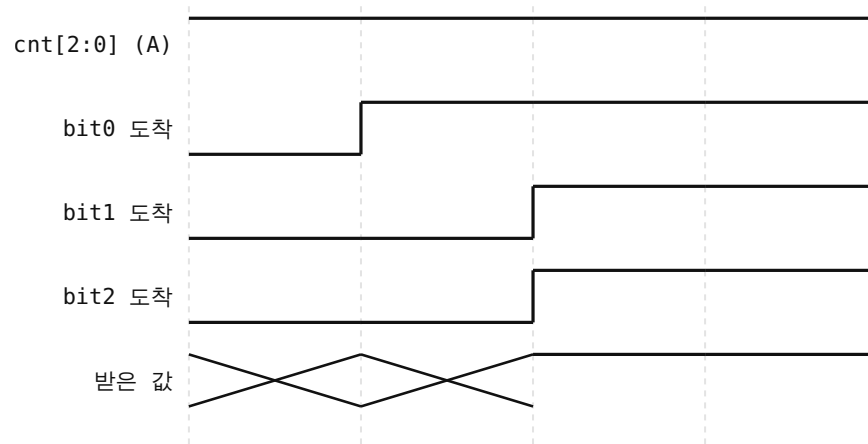


그림 31. 각자의 동기화기를 거쳐 건너는 두 비트. 비트 하나 하나는 안전하다. 그 쌍은 안전하지 않다. 하나는 이번 주기에 결판나고 다른 하나는 다음 주기에 결판나서, 받는 쪽은 한 번도 보내진 적 없는 값을 본다.

다중 비트 문제, 그리고 빠져나가는 세 길

서로 독립인 동기화기는 서로 독립으로 결정한다. 같은 원천 주기에 두 비트가 변하면 각각이 경계의 어느 쪽으로든 결판날 수 있으므로, 목적지는 중간값을 볼 수 있다 — **원천이 한 번도 몰지 않은 코드**. 이진 계수기가 0111 → 1000 으로 건너가는 것을 1111 로도 0000 으로도, 그 사이 아무것으로도 볼 수 있다. 어떤 MTBF 계산도 이것을 못 잡는다. 플롭은 하나하나 다 옳게 동작했다.

어디에 쓰나	클럭 경계를 건너는 모든 버스, 계수기, 포인터, 상태 워드, 설정 필드. 실제로는 FIFO 포인터, 인터럽트 상태 레지스터, 그리고 느린 설정 도메인이 쓰고 빠른 데이터패스가 읽는 '모드' 필드.
언제 쓰나	비트가 하나를 넘는 순간부터. 3 비트라고 32 비트보다 안전하지 않다 — 중요한 것은 폭이 아니라 동시성이다.
어떻게 쓰나	하나를 고르라. (1) 그레이 코드 — 하나 셀 때마다 한 비트만 변하므로 중간에 잡힌 표본은 옛 값 아니면 새 값이고 둘 다 올바르다. 계수기와 포인터에는 되고 임의의 데이터에는 안 된다. (2) 핸드셰이크 — 데이터를 안정하게 붙들고 요청 비트 하나만 건넨다. 받는 쪽이 응답한다. 왕복 지연이 들지만 무엇에든 된다. (3) 비동기 FIFO — 그레이 코드 포인터를 갖는 이중 포트 메모리. (1) 과 (2) 를 합친 것이고, 흐르는 데이터에는 이것을 내보낸다.

실제 코드는 이렇게 생겼다

```
// Gray-coded pointer crossing -- the standard async FIFO core.
// The pointer is converted to gray BEFORE crossing and back AFTER.
function automatic [PW:0] bin2gray(input [PW:0] b); return b ^ (b >> 1); endfunction

always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wbin, wgray} <= '0;
    else          {wbin, wgray} <= {wbin_next, bin2gray(wbin_next)};

// crossing: gray value only, two stages, one synchroniser per bit is now SAFE
// because at most one bit differs between consecutive gray values.
(* ASYNC_REG = "TRUE" *) reg [PW:0] wq1_rgray, wq2_rgray;
always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wq2_rgray, wq1_rgray} <= '0;
    else          {wq2_rgray, wq1_rgray} <= {wq1_rgray, rgray};

// FULL is computed from the SYNCHRONISED read pointer, so it is CONSERVATIVE:
// it can be asserted when the FIFO has already been drained, never the reverse.
assign wfull = (wgray_next == {~wq2_rgray[PW:PW-1], wq2_rgray[PW-2:0]});
```

여기서 틀린다. 그레이 코드는 포인터를 지키지 데이터를 지키지 않는다. 비동기 FIFO 의 데이터 경로는 이중 포트 RAM 이고, 그것이 안전한 까닭은 오직 읽기 주소가 쓰기 쪽이 아직 다 쓰지 못한 자리에 닿는 것이 허용되지 않기 때문이다 — 보수적인 full/empty 비교가 사 주는 것이 그것이다. 그레이 코드 수법을 데이터 버스에 베풀 놓아도 아무 소용이 없다. 이어지는 데이터 워드는 여러 비트가 한꺼번에 다르다.

실제로 난 일

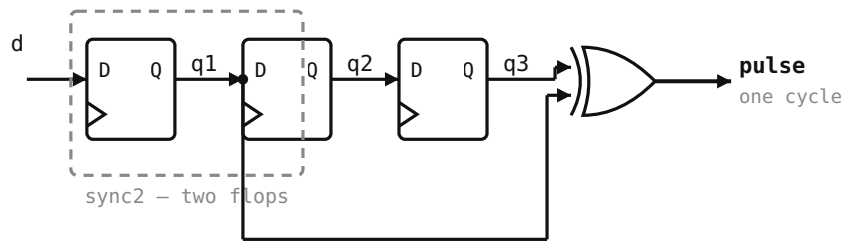
건넌 건 준안정 사건이 **하나도** 없이도 실패할 수 있다. 원천 펄스가 목적지 클럭 주기 하나보다 짧으면 목적지는 그것을 영영 표본화하지 못할 수 있다 — 동기화기는 매번 깨끗하게 옛 값으로 결판난다. 이것은 확률적 실패가 아니라 확

정적 실패이고, MTBF 분석에는 안 보인다. 그것을 막는 규칙: **느린 도메인으로 건너가는 신호는 목적지 주기의 1.5 배 이상 붙들어야 한다.** 아니면 레벨 토글로 바꾸고 돌아오는 핸드셰이크를 둔다.

표 78. 건넌 방식마다 값과 지키는 것

방식	지연 (목적지 주기)	처리량	지키는 것	실패하는 때
2 플롭 동기화기	2	주기당 1 비트	레벨이 안정한 한 비트	한 비트를 넘게 같이 변할 때. 1.5 주기보다 짧은 펄스
펄스 동기화기 (토글)	2-3	3 주기당 사건 1	어떤 비율에서든 단일 사건	사건이 왕복보다 빨리 올 때
그레이 포인터	2	주기당 1 증가	단조 계수기와 포인터	임의의 데이터에 쓸 때
req/ack 핸드셰이크	왕복 4-6	왕복당 1 워드	폭에 관계없이 임의의 데이터	지연 예산이 왕복을 못 삼킬 때
비동기 FIFO	2-3 + 채움	주기당 1 워드 지속	흐르는 데이터, 속도 불일치	깊이가 그 버스트 모양에 비해 작을 때
준정적 (설정 필드)	0	한 번 쓴다	쓰는 쪽이 놓고 있거나 리셋 중일 때만 변하는 필드	데이터패스가 도는 동안 누가 그것을 쓸 때

준정적신호 (quasi-static signal) — 목적지가 그것을 쓰지 않는 동안에만 변하도록 **규약으로 보장된** 건넌 신호 — 보통 블록이 켜지기 전에 쓰는 설정 필드다. 동기화기는 필요 없지만, **데이터시트에 적힌 규칙**과 그렇다고 말하는 CDC 예외는 필요하다. 적어 두지 않은 준정적 가정이야말로 IP 가 실험실에서는 되고 고객 시스템에서는 안 되는 길이다.



Different input-output levels across one flop → exactly one cycle of dif

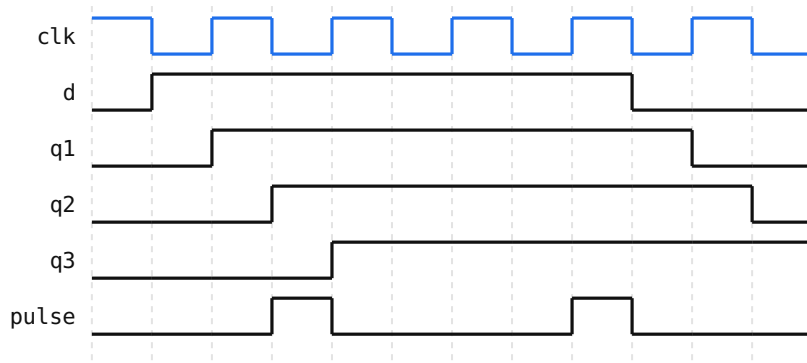
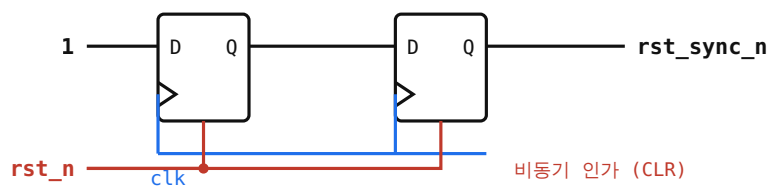


그림 32. 한 주기짜리 사건을 클럭 경계 너머로 나르는 표준적인 방법. 플롭 둘이 준안정을 걸어 내고, 세 번째가 앞의 값을 붙든다. 그 둘의 XOR 가 어떤 레벨 변화든 목적지 도메인의 정확히 한 펄스로 바꾼다. 파형이 그 증거다. 넓은 펄스가 들어가고 한 주기짜리 펄스가 나오는데, 양쪽 에지에서 다 그렇다.

T3.4 리셋: 모두가 잊는 건넌

리셋은 본디 비동기다 — 어떤 클럭도 돌기 전에 동작해야 한다 — 그리고 어쩔 수 없이 동기이기도 하다. 엉뚱한 순간에 풀면 플롭의 리커버리 시간을 어겨서 상태 기계를 첫 주기에 준안정 상태로 만들기 때문이다. 해법은 모든 설계에서 같고, 한 문장으로 외워 둘 값어치가 있다: **비동기로 걸고, 동기로 푼다.**



인가는 즉시 (클럭 없이도 리셋이 걸린다)
해제는 클럭에 맞춰 (두 클럭 뒤) -- recovery/removal 위반을 막는다

그림 33. 리셋 동기화기. 비동기로 거는 경로에는 클럭이 하나도 없다. 푸는 경로는 목적지 도메인으로 클럭되는 플롭 둘을 지난다.

리커버리 · 리무벌 (recovery and removal)

리커버리 시간은 리셋 **해제**에 걸리는 셋업 제약이다. 리셋은 활성 클럭 에지보다 최소한 t_{recovery} 앞서 풀려야 한다. 리무벌 시간은 그에 대응하는 홀드 제약이다. 다른 타이밍 이음과 똑같이 특성화되어 .lib 에 들어 있다 — 그 말은 STA 가 그것을 **그 플롭에서 리셋이 타이밍이 걸린 동기 신호일 때만** 검사한다는 뜻이다.

동기화기 없이 풀리는 비동기 리셋은 클럭과 그런 관계가 없으므로 STA 는 아무것도 보고하지 않고, 실패는 전원을 올린 뒤 상태 기계가 가끔 불법 상태에서 출발하는 꼴로 나타난다 — 대개 한 코너에서, 어떤 보드에서만, 어떤 전원 상승 모양에서만.

어디에 쓰나	모든 클럭 도메인. 클럭이 셋인 설계는 리셋 동기화기가 셋 필요하고, 각각 자기 도메인으로 클럭되며 모두 같은 비동기 원천을 받는다.
언제 쓰나	늘. 값은 도메인마다 플롭 돌리고, 그것이 막아 주는 실패는 재현하기 가장 어려운 부류다.
어떻게 쓰나	거는 것은 바로 통과시키고(클럭 없이), 푸는 것은 플롭 돌을 거치게 한다. 그러고 나서 제약한다. 해제는 이제 동기 신호이므로 STA 가 그것이 닿는 모든 플롭에서 리커버리/리무벌을 검사한다.

실제 코드는 이렇게 생겼다

```
// Reset synchroniser -- one per clock domain.
module reset_sync (input clk, input arst_n, output rst_n);
    (* ASYNC_REG = "TRUE" *) reg [1:0] q;
    always_ff @(posedge clk or negedge arst_n)
        if (!arst_n) q <= 2'b00;          // ASSERT: asynchronous, no clock needed
        else        q <= {q[0], 1'b1};    // RELEASE: two clocked stages
    assign rst_n = q[1];
endmodule

// SDC: the asynchronous assertion is not a timed path; the release is.
set_false_path -from [get_ports arst_n] -to [get_pins u_rsync/q_reg[0]/CLR]
// and every flop fed by rst_n gets a normal recovery/removal check -- verify with:
// report_timing -delay_type max -check_type recovery -from [get_pins u_rsync/q_reg[1]/Q]
```

여기서 틀린다. 동기화기 자신의 플롭을 동기화된 리셋으로 리셋하는 것은 빠져나갈 길 없는 고리다. 해제 에지를 놓치면 블록이 영영 리셋에서 못 나온다. 동기화기는 클리어 핀에 **날것의 비동기** 리셋을 받아야 한다. 리셋 동기화기의 두 플롭이 0 으로 리셋되고 1 을 밀어 넣는 것이지 그 반대가 아닌 까닭도 이것이다.

T3.5 STA 가 못 보는 것, 그리고 CDC 도구가 대신 하는 것

표 79. 도구 둘, 물음 둘

물음	STA	CDC 분석
주어진 클럭 관계에서 이 경로가 셋업을 맞추나?	본다	안 본다
클럭 관계라는 것이 있기는 한가?	있다고 가정한다	이것이 그 일이다
모든 건넌에 동기화기가 있나?	안 본다	본다 (구조적으로)
여러 비트가 같이 건너나?	안 본다	본다 — 재수렴과 버스 일관성 검사
원천이 충분히 오래 안정한가?	안 본다	본다 — 규약 검사, 보통 시뮬레이션의 생성된 어서션으로
이 건넌은 일부러 동기화를 안 한 것인가?	안 본다	당신이 이름과 까닭을 적어 예외로 두었을 때만

예외 파일이야말로 CDC 사인오프에서 그 분석에 뜻이 있었는지를 정하는 부분이다. 예외 이백 개를 달고 돌려서 나온 초록 요약은 증거가 아니다. 그것은 기술자 한 사람의 주장 이백 개다. **IP 납품물은 CDC 예외를 읽을 수 있는 문서로 같이 내보내야 한다.** 하나마다 까닭 한 줄씩. 통합하는 쪽이 그 전부를 물려받는데, 넷리스트에서 그것을 되살려 낼 수는 없기 때문이다.

구조뿐 아니라 시뮬레이션으로도 건넌을 증명하기

구조적 CDC 분석은 동기화기가 있다는 것을 증명한다. 원천이 그 동기화기가 가정하는 규약을 지키는지는 증명하지 못한다 — 요청이 높은 동안 데이터가 안정한지, 펄스가 충분히 긴지, 준정적 필드가 데이터패스가 도는 동안 쓰이지 않는지. 그것들은 **시간적 성질**이고, 손으로 쓰거나 CDC 도구가 생성한 어서션으로 검사한다.

쓸 수 있는 가장 강한 검사는 준안정 주입까지 더한다. 시뮬레이터가 동기화기 출력을 무작위로 한 주기 늦춰서, 건넌이 즉시 결판날 때만 도는 설계가 실리콘이 아니라 회귀에서 실패하게 만든다.

어디에 쓰나

클럭이 하나를 넘는 모든 것의 블록 수준 검증, 그리고 남의 클럭 구조 안으로 팔 생각인 모든 IP.

언제 쓰나

RTL 을 얼릴 때마다 구조적 CDC 를 돌리고, 야간 회귀에서 주입을 돌린다. 둘은 서로 다른 버그를 찾고 어느 쪽도 다른 쪽을 대신하지 못한다.

어떻게 쓰나

어서션을 건넌 모듈에 묶고, 주입을 켜 채 회귀를 돌리고, 결과를 납품물의 검증 절차 건넌 수와 예외 수와 함께 적는다.

실제 코드는 이렇게 생겼다

```
// The protocol the handshake crossing assumes, stated as a checkable property.
property data_stable_while_req;
    @(posedge clk_src) disable iff (!rst_n)
        req |-> ##0 $stable(data) throughout (req [*1:$] ##1 ack_sync);
endproperty
assert property (data_stable_while_req)
    else $error("CDC: data changed while req was asserted -- the receiver may latch a mix");

// Metastability injection (simulation only): the synchroniser may take one extra cycle.
`ifdef CDC_INJECT
    always_ff @(posedge clk_dst)
        if ($urandom_range(0,99) < 5) sync_q[1] <= sync_q[1]; // hold: resolve later
`endif
```

여기서 틀린다. 원천 도메인 신호에 대해 목적지 도메인에서 쓴 어서션은 그 자체가 클럭 도메인 건넌이고, 테스트벤치가 만들어 낸 가짜 실패를 보고할 수 있다. 원천 신호는 원천 도메인에서 표본화하고 (@(posedge clk_src)) 불리언 결과만 건네라.

T3.6 월요일에 무엇을 할 것인가

표 80. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
MTBF · 결정시간상수	모든 비동기 경계	동기화기 깊이, 그리고 데이터시트에 적는 수
동기화기깊이	아키텍처, 건넘마다	기본은 둘. 1 GHz 위이거나 τ 특성화가 시원치 않으면 셋
다중비트문제 · 그레이코드	FIFO 포인터, 계수기, 상태 워드	그 건넘에 부호화가 필요한가, 핸드셰이크인가, FIFO 인가
핸드셰이크 · 비동기FIFO	도메인 사이로 흐르는 데이터	지연과 깊이, 따라서 면적
리셋동기화기 · 리커버리	모든 클럭 도메인	모든 코너에서 블록이 합법 상태로 출발하는가
CDC검증 · 준정적신호	사인오프와 납품물	통합하는 쪽이 물려받는 예외 목록

이 장을 끝내는 수는 둘이고, 들고 갈 것이 그 둘이다. 1 GHz 에서 1 단 건넘은 대략 1.8 년에 한 번 실패하고, τ 가 두 배인 같은 건넘은 2 단으로도 대략 1.8 년에 한 번 실패한다. 첫 번째 수가 동기화기가 존재하는 까닭이고, 두 번째 수가 답을 늘 그 코너와 함께 적어야 하는 까닭이다.

다음 장은 이 장이 내내 가정하고 있던 것 — 표본은 맞거나 틀리거나 둘 중 하나라는 것 — 을 분포로 바꾼다. 잡음은 혼성신호 IP 의 모든 아날로그 측정 아래에 바닥을 깔고, 준안정과 달리 전력·면적과 곧바로 맞바꿀 수 있다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

준안정 자체는 물리이고, 2 플롭 동기화기는 이 글을 읽는 누구보다도 오래됐다. **지금도 청구되고 있는 것은 그 둘레의 배치다:** 다중 비트 건넘을 N 개가 아니라 동기화기 하나로 안전하게 만드는 포인터 부호화(그레이 코드와 그 후에 들). 동작 모드에 따라 동기화기 깊이를 바꾸는 지연 대 MTBF 방식. 자격 신호 하나만 동기화하고 데이터는 경계 너머로 안정하게 붙들어서 데이터를 아예 동기화하지 않는 데이터 경로 건넘. 이 하나하나가 플롭에 대한 논의가 아니라 무엇이 건넘야 하는가에 대한 논의다. 이 장을 이렇게 물으며 읽으라: 내 신호 가운데 정말로 건넘야 하는 것은 무엇이고, 나머지는 무엇이 있으면 한쪽에 머물 수 있는가?

T4. 잡음 — 그리고 그것이 왜 모든 것의 크기를 정하나

잡음은 영리함만으로는 개선할 수 없는 단 하나의 규격이다. 6 dB 마다 용량이 4 배이고, 따라서 전력도 4 배다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 온도 · 확률 · 분산 · 표준편차 · 적분 · 로그 · 주파수 · 트랜스컨덕턴스 · 게이트용량 · 출력저항 · RC지연 · 준안정

이 장에서 새로 나오는 것

열잡음 · 산탄잡음 · 플리커잡음 · 잡음스펙트럼밀도 · 잡음 대역폭 · kTC잡음 · 입력환산잡음 · 플리커코너 · 잡음지수 · 정적잡음여유 · 나비곡선 · 읽기방해 · 최소동작전압 · SNR · SNDR · ENOB

회로에서 재는 모든 것에는 잡음이 얹혀 온다. 그 잡음은 서툰 공학에서 오는 것이 아니다 — 전하의 열운동과 전자가 알갱이라는 사실에서 온다. 그래서 가장 다루기 어려운 종류의 규격이다. 없앨 수는 없고 면적과 전력을 써서 평균으로 깎아 내릴 수만 있는데, 그 환율은 물리가 **비트 하나당 4 배**로 고정해 두었다.

이 장은 실제로 만나게 될 잡음원 셋을 세우고, 혼성신호 IP 의 아날로그 블록 절반의 크기를 정하는 수 하나(kT/C)를 유도한 다음, 잡음 장에서는 드문 일을 한다. 디지털 쪽으로 건너간다. 거기서는 같은 낱말이 전혀 다른 것을 뜻하고, 그 결과 — SRAM 을 읽을 수 없게 되는 전압 — 를 주장하지 않고 ngspice 로 여기서 켜다.

T4.1 세 가지 원천, 그리고 그중 무엇에 손댈 수 있나

표 81. IC 안의 잡음원을 출처로 가른다

원천	밀도	출처	바꿀 수 있나?
열잡음 (존슨)	$S_V = 4kTR [V^2 / \text{Hz}]$	저항성 요소 안 운반자의 열운동	R 이나 T, 또는 내다보는 대역폭을 바꿔서만
채널 열잡음	$S_i = 4kT\gamma g_m [A^2 / \text{Hz}]$	MOSFET 채널 안의 같은 기구. $\gamma \approx 2/3$ (장채널), 1-2 (단채널)	g_m 을 올리거나(전류가 든다) 거른다
플리커 (1/f)	$S_V = K_f / (C_{ox}WL \cdot f)$	산화막 계면에서의 포획과 방출	있다 — 소자를 크게, PMOS 입력, 초핑, 자동영점
산탄	$S_i = 2qI [A^2 / \text{Hz}]$	장벽을 건너는 전하가 알갱이라는 것(채널이 아니라 접합)	전류를 바꿔서만

유도 — 답이 왜 늘 kT/C 로 끝나나

	이렇게 된다	왜냐하면
1	저항의 열잡음 밀도는 $4kTR$ 이고 주파수와 무관하다.	평형 열역학에서 나온 나이퀴스트의 결과다 — R 말고는 소자 매개변수가 하나도 안 들어 있고, 그래서 이것은 설계 선택이 아니라 바닥이다.
2	그 저항을 축전기 C 와 직렬로 놓는다 — 모든 표본화 스위치가 바로 그것이다.	스위치의 온저항($T1$: 선형 영역)과 표본화 축전기가 직렬이다. 세 번째 요소는 없다.
3	RC 저역통과가 그것을 모양 지우고, 극 하나의 잡음 대역폭은 $\pi/2 \times f_{-3dB} = 1/(4RC)$ 이다.	$1/(1+(f/f_0)^2)$ 를 모든 주파수에 걸쳐 적분하면 f_0 가 아니라 $\pi/2 \cdot f_0$ 가 나온다 — 사람들이 빠뜨리고는 왜 측정이 예측보다 1.6 배 나쁜지 의아해하는 그 인자다.
4	$v_n^2 = 4kTR \times 1/(4RC) = kT/C$.	R 이 사라진다. 표본화된 축전기의 잡음은 스위치와 전혀 무관하다 — C 와 온도에만 달려 있다.
5	3 K 에서 $C = 1$ pF 이면 $v_n = 64.4 \mu V$ rms 다.	외워 둘 값어치가 있는 수 하나. 나머지는 전부 여기서 $1/\sqrt{C}$ 로 비례한다.

kT/C — 축전기의 크기를, 따라서 전력을 정하는 규칙

표본화된 마디는 어떻게 표본화했든 kT/C 의 잡음 전력을 진다. 그것을 N 비트 변환기의 양자화 잡음($LSB/\sqrt{12}$) 아래로 두려면 축전기가 $C \geq 12kT \cdot 2^{2N}/V_{FS}^2$ 를 만족해야 한다. **비트 하나마다 C 가 네 배가 되고**, 정해진 시간 안에 정착하려면 $g_m \propto C$ 이므로 전류도 네 배가 된다.

이 관계 하나가, 16 비트 변환기가 12 비트보다 "조금 더 어려운" 것이 아닌 까닭이다. 용량이 256 배다.

어디에 쓰나

모든 스위치드 커패시터 회로: SAR 과 파이프라인 ADC 의 표본화 회로망, 샘플앤홀드, 스위치드 커패시터 필터와 이득단, 그리고 그것들을 몰아야 하는 기준 버퍼.

언제 쓰나

아날로그 블록 설계의 맨 처음, 트랜지스터 치수를 정하기 전에. 축전기가 먼저다. 그것이 전력 예산을 고정하기 때문이다.

어떻게 쓰나

V_{FS} 와 N 을 정해 C 를 내고, 표본화 구간 안에 정착하도록 드라이버 치수를 정하고($T1$ 의 스위치 R_{on} 을 쓴 $T2$ 의 RC 셀), 기준이 주기마다 그 전하를 댈 수 있는지 확인한다.

실제 코드는 이렇게 생겼다

```
* The measurement that separates kT/C from everything else you did wrong.
* Transient noise in ngspice: sample the same node 1000 times and take the std dev.
.tran 1n 10u
.noise v(out) VIN dec 20 1 1G      $ small-signal, for the spectrum
.measure noise inoise_total_rms param='sqrt(inoise_total)'
* And the closed form to check it against -- ALWAYS have the hand calculation:
*   v_n = sqrt(kT/C) = sqrt(1.38e-23*300/1e-12) = 64.4 uV for C = 1 pF
```

여기서 들린다. kT/C 는 **표본화된** 값에 적용된다. 트래킹 구간에서는 잡음 대역폭을 표본화가 아니라 회로가 정한다 — 그래서 트래킹 중에 켜 트래킹앤홀드는 같은 회로를 홀드 뒤에 켜 것과 다른(보통 더 큰) 수를 보인다. 규격이 뜻하는 쪽이 아닌 쪽을 보고하는 것은 혼하고 비싼 잘못이다.

표 82. 비트당 축전기와 전력 값 ($V_{FS} = 1\text{ V}$, 300 K , 잡음 = 양자화 잡음)

비트 N	LSB	양자화 rms	필요한 C	8 비트 대비
8	$3,906\text{ }\mu\text{V}$	$1,128\text{ }\mu\text{V}$	3.26 fF	$1\times$
10	$977\text{ }\mu\text{V}$	$282\text{ }\mu\text{V}$	52.1 fF	$16\times$
12	$244\text{ }\mu\text{V}$	$70.5\text{ }\mu\text{V}$	834 fF	$256\times$
14	$61\text{ }\mu\text{V}$	$17.6\text{ }\mu\text{V}$	13.3 pF	$4,096\times$
16	$15.3\text{ }\mu\text{V}$	$4.4\text{ }\mu\text{V}$	213 pF	$65,536\times$

T4.2 플리커 잡음, 그리고 아키텍처를 정하는 코너

플리커코너 (flicker corner, f_c) — $1/f$ 밀도가 열잡음 밀도와 같아지는 주파수. 그 아래는 플리커가 지배하고 그 위는 열잡음이 지배한다. 요즘 CMOS 증폭기에서는 NMOS 입력이면 $\sim 100\text{ kHz}$ 에서 몇 MHz 사이 어디든 있고, PMOS 면 보통 한 자릿수 낮다.

신호가 그 코너의 어느 쪽에 있나

플리커 잡음 밀도는 $1/f$ 로 오르므로, f_1 에서 f_2 까지의 **적분된** 전력은 $\ln(f_2/f_1)$ 에 비례한다 — 폭이 아니라 비에 달려 있다. DC 결합 측정은 $f_1 \rightarrow 0$ 이고 적분이 로그로 발산한다: **오래 평균할수록 나빠진다**. 열잡음과 정반대다.

그 사실 하나가 아키텍처 결정을 이끈다. 신호가 DC 근처에 있으면 (온도 센서, 밴드갭 기준, DC 오프셋) 평균은 도움이 안 되고 신호를 옮겨야 한다 — 코너 위 주파수로 잘게 썰어 올려 거기서 증폭하고 다시 내린다. 신호가 이미 코너 위에 있으면(RF 나 고속 링크 전단) 플리커는 상관없고 거기에 아무것도 안 쓴다.

어디에 쓰나

센서 전단, 기준과 바이어스 회로, 비교기 오프셋, DC 정확도가 필요한 모든 측정. 그리고 발진기의 위상 잡음 — 소자 플리커가 가까운 쪽 위상 잡음으로 옮겨 실려 간다.

언제 쓰나

관심 대역에 ~ 1 MHz 아래가 들어가면 언제든. 아키텍처 단계에서 그 물음을 명시적으로 던지라. 다 만든 증폭기에 초핑을 나중에 끼워 넣는 것은 재설계다.

어떻게 쓰나

면적으로 줄이거나($S \propto 1/WL$ — 6 dB 에 면적 4 배), PMOS 입력을 고르거나, 변조로 줄인다: **초핑**(들어갈 때와 나올 때 구형파로 변조한다. 초핑 주파수에 잔여 리플이 남는다) 또는 **상관 이중 표본화**(오프셋을 표본화해 빼낸다. 잡음 접힘과 대역폭을 치른다).

실제 코드는 이렇게 생겼다

```
* Flicker in the model card is one parameter -- and simulators differ.
.model NM NMOS (LEVEL=54 ... KF=1e-25 AF=1.0 EF=1.0)
* KF alone means nothing: the density is KF/(Cox*W*L*f^EF), so a KF taken from
* another PDK, or from a model with a different EF convention, is a WRONG number
* that still simulates. Always sanity-check the corner against measured silicon:
.noise v(out) VIN dec 10 1 100MEG
* then read the point where the 1/f asymptote crosses the flat part.
```

여기서 틀린다. 초핑은 잡음을 줄이지 않는다. **신호를 잡음에서 멀리 옮기는 것**이다. 열잡음 바닥은 그대로이고, 초퍼는 초핑 주파수와 그 고조파에 자기 나름의 스위칭 부산물을 더한다. 저주파 잡음 수치는 아름답게 나오고 f_{chop} 에 스퍼가 있는 설계는, 그 스퍼가 대역 안에 떨어지면 아무것도 개선한 것이 없다.

T4.3 또 다른 잡음: 정적 잡음 여유를 잔다

디지털 쪽에서 '잡음'이라는 낱말은 다른 것을 뜻하고, 그 양은 스펙트럼 밀도가 아니라 **전압**이다. 저장 마디가 뒤집히기 전에 얼마만큼의 DC 교란을 삼킬 수 있는가. 구조는 T3 에서 준안정을 낳았던 그 교차 결합 쌍 그대로이고, 동적으로가 아니라 정적으로 볼 뿐이다.

정적잡음여유 (SNM, static noise margin) — 교차 결합 쌍의 두 저장 마디에 직렬로 끼워 넣어도 쌍안정을 무너뜨리지 않는 가장 큰 DC 전압. 기하로는 두 인버터 전달 곡선이 이루는 나비 곡선 안에 들어가는 가장 큰 정사각형의 한 변이다.

실제로 난 일

아래에서 쓴 추출기는 맞기 전에 **두 번 틀렸고**, 두 번 다 수가 그럴듯해 보였다. 첫 판은 내접 정사각형이 아니라 가로간격을 재서 $\text{hold SNM} \approx 0.5 \cdot V_{\text{DD}}$ 를 냈다 — 너무 높다. 둘째 판은 액세스 트랜지스터의 게이트가 워드선이 아니라 비트선에 물려 있었다. 그랬더니 셀비가 오를수록 read SNM 이 떨어졌다. 거꾸로다. **버그를 잡은 것은 값이 아니**

라 경향이였다. 이 저장소가 굴러가는 규칙이 그것이다: 측정을 믿기 전에, 내가 이해하는 것을 바꿨을 때 그것이 어느 쪽으로 움직이는지 보라.

그리고 나서 추출기를 두 극한으로 자해검사했다. 이득이 아주 큰 인버터는 $SNM \rightarrow V_{DD}/2$ 를 내야 하고(0.4825 를 냈다), 이득이 1 인 '인버터' 는 0 을 내야 한다(0.0 을 냈다). 그 둘을 다 지나고 나서야 아래 셀 수치를 적었다.

표 83. 6T 셀의 측정된 SNM (ngspice, LEVEL=1 모형, $V_{DD} = 1.0\text{ V}$, $V_{th} = 0.35\text{ V}$) — β 는 셀비 $W_{pulldown} / W_{access}$

β	Hold SNM	Read SNM	Read / Hold
1	0.421 V	0.194 V	0.46
1.5	0.417 V	0.207 V	0.5
2	0.414 V	0.226 V	0.55
2.5	0.411 V	0.241 V	0.59
3	0.409 V	0.253 V	0.62

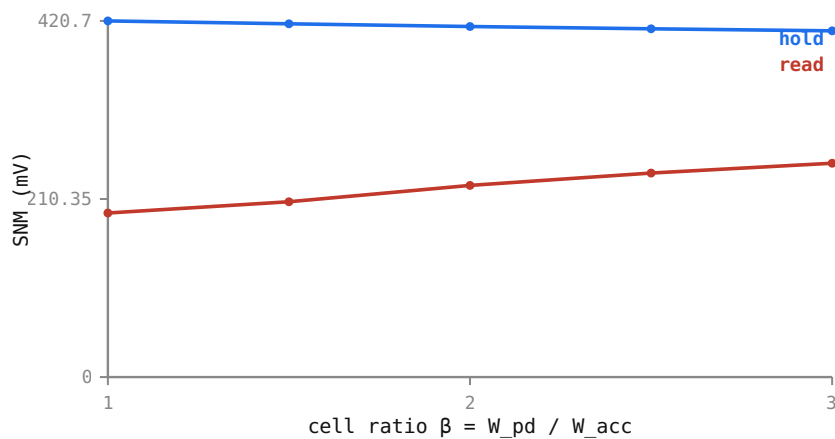


그림 34. 읽기는 셀 여유의 절반 넘게를 먹고, 그것을 되사는 구조적인 길은 액세스 소자에 견준 풀다운을 강하게 하는 것뿐이다 — 6T 셀이 최소 소자로 만들어지지 않는 까닭이다.

유도 — 셀을 읽는 것이 왜 그 셀을 거의 부수는 일인가

	이렇게 된다	왜냐하면
1	읽으려면 두 비트선이 높게 미리 충전된 상태에서 워드선이 액세스 트랜지스터 둘을 켜다.	읽기 포트를 따로 두지 않고 셀을 감지하는 길이 그것뿐이다.
2	이제 '0' 쪽의 액세스 트랜지스터가 그 마디를 위로 끌어올린다. 셀 자신의 풀다운 NMOS 와 이루는 분압기를 거쳐서.	V_{DD} 와 접지 사이에 NMOS 둘이 직렬이고 저장 마디가 가운데 있다 — 그 마디가 0 V 에 머물 수가 없다.
3	저항 극한에서 '0' 마디가 대략 $V_{DD} \cdot (1/(1+\beta))$ 까지 오른다. 그러니 β 가 2 여도 벌써 3 분의 1 만큼 올라간다.	그 시점에 두 소자가 다 선형 영역에 있고, 둘의 세기 비가 곧 분압비다. 셀비가 설계 매개변수로 존재하는 까닭은 오직 이것이다.
4	그 상승이 여유를 곧바로 먹는다. $\beta = 2$ 에서 측정된 read SNM 은 0.226 V 이고 hold SNM 은 0.414 V 다 — 워드선이 오르는 순간 여유의 45.3 % 가 사라진다.	주장이 아니라 쟁 것이다 — 텍과 추출기가 이 장의 소스에 있다.
5	β 를 올리면 read SNM 이 오르고 hold SNM 은 조금 내려간다 .	쥬다: 같은 쓸기에서 hold 가 0.421 V 에서 0.409 V 로 떨어진다. 풀다운이 강해지면 인버터마디의 전이점이 $V_{DD}/2$ 에서 밀려나기 때문이다. 공짜인 방향은 없다 . 그래서 이것이 규칙이 아니라 설계점인 것이다.

V_{min} — 논리가 아니라 메모리가 칩을 멈추는 전압

Read SNM 은 전원을 내릴 때 점잖게 나빠지지 않는다. $\beta = 2$ 인 같은 셀에서 재면 1.0 V 에서 0.7 V 까지는 0.22 V 언저리를 지키다가 **무너진다**. 0.6 V 에서 0.145 V, 0.5 V 에서 0.087 V. 그 무릎은 V_{DD} 가 $2V_{th}$ (여기서는 0.7 V)에 가까워지는 자리에 앉는다 — 그 아래에서는 소자가 약반전에 있고 분압비가 지수 전류에 달리게 되며, 불일치(T12)가 아슬아슬한 셀을 실패하는 셀로 바꾼다.

그래서 **SRAM 이 칩의 최소 동작 전압을 정한다**. DVFS 방식이 배열의 V_{min} 위에서 멈추거나 배열에 자기 전원을 따로 주는 까닭이다.

어디에 쓰나	칩 안에 SRAM 이 있고 전압점이 하나를 넘는 모든 SoC: DVFS 범위 전체, 잠 모드의 보존 전압, 그리고 사인오프의 저전압 코너.
언제 쓰나	전력 모드가 제안되는 즉시. '0.6 V 까지 내려도 되나' 는 논리의 물음이기 전에 SRAM 의 물음이다.
어떻게 쓰나	보조 회로로 V_{min} 을 올리거나(읽기에는 워드선 언더드라이브, 쓰기에는 음의 비트선), 배열의 전원을 가르거나, 읽기 경로가 저장 마디에 닿지 않는 8T 셀을 쓴다.

실제 코드는 이렇게 생겼다

```
* The sweep that produces the number above -- run it at every corner,
* not just typical, because Vmin is a MISMATCH-limited quantity (see T12).
.param beta=2.0
.dc VIN 0 VDD 0.004 VDD 1.0 0.5 -0.1      $ nested sweep: input, then supply
.print dc v(vr)
* Extraction is NOT a .measure one-liner: you need the butterfly and the largest
* inscribed square. Script it, and SELF-CHECK the extractor on two limits
* (very high gain -> VDD/2, gain 1 -> 0) before trusting any cell number.
```

여기서 틀린다. 전형 코너에서 전 SNM 은 V_{min} 에 거의 쓸모가 없다. 배열은 수백만 개 가운데 **가장 나쁜** 셀에서 실패하므로, 중요한 양은 꼬리다. V_{min} 은 대략 -5σ 에서 -6σ 셀이 정하고, 그 말은 분석이 공칭 DC 쓸기가 아니라 중요도 표본화를 쓴 몬테카를로라는 뜻이다. 위의 쓸기는 모양을 알려 줄 뿐 수율을 알려 주지 않는다.

T4.4 볼트에서 데이터시트의 수로: SNR, SNDR, ENOB

표 84. 세 수, 그리고 각각이 감추는 것

양	정의	감추는 것
SNR	신호 전력 / 잡음 전력, 왜곡은 뺀다	왜곡 . 선형성이 끔찍한 변환기도 SNR 은 훌륭할 수 있다
SNDR (SINAD)	신호 / (잡음 + 모든 왜곡)	둘 중 무엇이 지배하는지 — 스펙트럼도 늘 같이 적으라
ENOB	$(\text{SNDR}_{\text{dB}} - 1.76) / 6.02$	그것이 전 것이 아니라 유도된 것이라는 사실. ENOB 은 단위만 같아입은 SNDR 이다. 입력 주파수와 진폭 없이 적힌 ENOB 은 측정이 아니다
SFDR	신호 / 가장 큰 스퍼 하나	광대역 잡음을 통째로 — 깨끗해 보이는 SFDR 에 나쁜 잡음 바닥은 흔하다

예제 13. 잡음 64 μV 는 몇 비트를 허락하나?

주어진 것. 전체 눈금 1 V 인 변환기의 표본화 회로망에 $C = 1 \text{ pF}$ 이 있어 표본화된 잡음이 $64.4 \mu\text{V rms}$ 이고, 나머지는 전부 완벽하다고 하자.

푸는 법. 전체 눈금 사인파의 rms 진폭은 $V_{FS}/(2\sqrt{2})$ 다. $\text{SNR} = 20 \cdot \log_{10}(\text{신호 rms} / \text{잡음 rms})$ 이고, $\text{ENOB} = (\text{SNR} - 1.76)/6.02$ 다.

답. $\text{SNR} = 74.8 \text{ dB}$ 이고 $\text{ENOB} = 12.1$ 비트다. **표본화 축전기 1 pF 은 ~12 비트짜리 부품이다.** 나머지 설계가 아무리 좋아도 그렇다.

여기서 틀린다. 표본화 회로망이 보통 두 번 표본화된다는 것(입력과 기준)을 잊는 것, 그리고 축전기를 차동 쌍으로 두면 잡음 전력이 두 배가 되면서 신호 진폭도 두 배가 된다는 것 — 둘이 일부 상쇄하고, 나오는 인자는 위상에 달려 있다. 그 구체적인 회로망을 적어라. 외워 둔 인자를 재활용하지 마라.

T4.5 월요일에 무엇을 할 것인가

표 85. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
kTC잡음	트랜지스터 치수를 정하기 전의 모든 스위치드 커패시터 블록	표본화 축전기, 따라서 전류 예산
열잡음 · 잡음대역폭	증폭기와 기준 설계	입력단에 g_m 이 얼마나 필요한가
플리커코너	DC 근처 신호 경로의 아키텍처	이 설계에 초핑이나 자동영점이 아예 필요한가
정적잡음여유 · 나비 곡선	SRAM, 모든 교차 결합 감지증폭기와 래치	셀비 β , 그리고 읽기가 읽은 것을 부수는가
최소동작전압	DVFS 와 보존 모드	칩을 돌려도 되는 가장 낮은 전압
SNDR · ENOB	데이터시트와 고객과의 대화	고객이 경쟁사와 견줄 단 하나의 수

이 장에서 들고 나갈 쟁 결과 둘. 표본화 축전기 1 pF 은 12.1 비트짜리 부품이다 — 트랜지스터를 하나도 그리기 전에 축전기가 해상도를 정해 버린다. 그리고 6T SRAM 셀을 읽는 것은 셀비 2 에서 그 정적 여유의 45.3 % 를 치른다. 그 셀비가 존재하는 까닭이고, 논리가 아니라 배열이 칩을 돌릴 수 있는 가장 낮은 전압을 정하는 까닭이다.

다음 장은 이 장이 순간적이라고 다룬 표본화 동작에 유한한 구경과 지터와 스펙트럼을 준다 — 고속 변환기의 잡음 바닥이 실제로 어디서 오는지가 거기다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

잡음은 열역학이다. kT 에 특허를 내는 사람은 없다. 청구되는 것은 그 4 배를 되사는 모든 배치다 — 상관 이중 표본화, 초핑, 자동영점, 잡음 성형, 그리고 회로가 잡음 예산을 신호가 있는 자리에만 쓰게 해 주는 일군의 기법들. 디지털 쪽에서는 같은 모양이 SRAM 보조 회로로 나타난다: 음의 비트선, 워드선 언더드라이브, 읽고-고치고-쓰기 보조 — 전부 맨 셀이 여유를 하나도 못 갖는 전압에서 셀이 여유를 지키게 해 주는 배치다. 이 장을 이렇게 물으며 읽으라: 나는 어디서 4 배 규칙을 치르고 있고, 무엇이 있으면 그것을 안 치를 수 있는가?

T5. 표본화 — 연속 신호가 수가 되는 자리

클럭 둘이 모든 것을 정한다. 언제 표본화하는가, 그리고 그 '언제' 를 얼마나 정확히 아는가. 오늘 팔리는 모든 고속 변환기를 묶는 것은 두 번째다.

읽기 전에 알아야 하는 것

전압 · 주파수 · 위상 · 적분 · 미분 · 확률 · 표준편차 · 로그 · RC지연 · 슬루 · kTC잡음 · 열잡음 · SNR · SNDR · ENOB · 잡음대역폭

이 장에서 새로 나오는 것

표본화정리 · 엘리어싱 · 나이퀴스트 · 대역통과표본화 · 구경시간 · 구경지터 · 지터한계SNR · 추적유지 · 트래킹대역폭 · 차지인젝션 · 클럭피드스루 · 부트스트랩스위치 · 오버샘플링비 · 잡음성형 · 데시메이션

표본화는 보통 이상화로 가르친다 — 임펄스 열을 곱하고, 스펙트럼이 되풀이되는 것을 보고, 겹치지 않게 한다. 그 이야기는 맞고, 하드웨어를 묶는 것은 그 이야기가 아니다. 진짜 표본화기는 스위치를 유한한 시간 동안, 유한한 저항을 통해, 몇 피코초 안에서만 알 수 있는 순간에 닫아서, T4 의 kT/C 가 이미 크기를 정해 놓은 축전기에 엮는다. 그 네 낱말 — 유한, 유한, 순간, 축전기 — 하나하나가 해상도를 치르고, 몇백 메가헤르츠 위에서 지배하는 것은 세 번째다.

T5.1 엘리어싱은 지켜야 하는 정리가 아니라 사야 하는 필터다

유도 — 임펄스 열에서 접힌 스펙트럼까지

	이렇게 된다	왜냐하면
1	f_s 로 표본화하는 것은 신호에 $T_s = 1/f_s$ 간격의 임펄스 열을 곱하는 것이다.	이상화다. 엘리어싱에 관한 모든 것이 이 한 걸음에서 따라 나온다. 비이상성은 나중에 오고 이것을 바꾸지 않는다.
2	시간에서의 곱은 주파수에서의 합성곱이고, 임펄스 열의 스펙트럼은 그 자체가 f_s 간격의 임펄스 열이다.	그래서 입력 스펙트럼이 f_s 의 모든 배수 자리로 복사된다.
3	입력이 $f_s/2$ 보다 좁은 대역에 갇혀 있지 않으면 복사본이 겹친다.	나이퀴스트. 그것이 말하지 않는 것을 눈여겨보라. 대역이 어디에 얹는지는 아무 말도 안 하고, 얼마나 넓은지만 말한다.
4	$f_s/2$ 위의 모든 것이 아래로 접히고, 그 뒤로는 진짜 신호와 구별되지 않는다.	어떤 디지털 처리로도 되돌릴 수 없다 — 둘을 가르던 정보가 표본화기에서 이미 없어졌다. 안티엘리어스 필터가 아날로그 부품이고 소프트웨어로 옮길 수 없는 까닭이다.
5	잡음도 접힌다, 같은 규칙으로.	대역폭 5 GHz 인 전단을 200 MHz 로 표본화하면 열잡음 50 대역이 신호 위로 접힌다 — 17 dB 벌점이다. 필요 없는 대역폭은 공짜가 아니다. 값을 치르는 잡음이다.

대역통과 표본화 — 접힘을 일부러 쓰기

나이퀴스트는 대역의 폭을 묶지 그 자리를 묶지 않는다. 2.4 GHz 중심의 100 MHz 를 차지하는 신호는 250 MS/s 로 표본화할 수 있다. 대역이 나이퀴스트 구역 하나 안에 온전히 들어가고 표본화기의 입력 대역폭이 2.4 GHz 까지 닿기만 하면 된다. 그 알리아스가 곧 원하는 신호이고, 공짜로 아래로 옮겨져 있다.

값은 분명하다. 표본화 **구경**이 표본화율은 250 MHz 인데도 2.4 GHz 에 걸맞게 좋아야 한다 — 지터는 클럭 속도가 아니라 입력 주파수에 환산된다. 그 맞바꿈(믹서는 없애지만 훨씬 어려운 클럭)이 공학적 결정의 전부다.

어디에 쓰나	IF 에서 디지털화하는 RF 수신기, 소프트웨어 정의 라디오 전단, 그리고 높은 주파수에 얹은 좁은 대역을 잡아야 하는 모든 계측기.
언제 쓰나	대역이 중심 주파수에 견주어 좁고, 그러지 않으면 하향 변환 믹서와 그 LO 값을 치러야 할 때.
어떻게 쓰나	대역이 구역 하나 안에 엄격히 들어가도록 f_s 를 고르고($k \cdot f_s/2$ 에서 $(k+1) \cdot f_s/2$ 까지), 대역통과 와 안티앨리어스 필터를 단다 — 여기서 저역통과는 쓸모가 없다 — 그리고 지터를 RF 반송파에 견주어 예산 잡는다.

실제 코드는 이렇게 생겼다

```
// Zone check, written as an assertion rather than a comment.
// A band that straddles a zone boundary is a silent failure: it still samples,
// and the output is two overlapped copies.
localparam real FS = 250e6, FC = 2.4e9, BW = 100e6;
initial begin
    int k = $floor(2.0*(FC - BW/2)/FS);
    assert ($floor(2.0*(FC + BW/2)/FS) == k)
        else $fatal(1, "band straddles a Nyquist zone boundary -- pick another FS");
end
```

여기서 틀린다. 대역통과 표본화의 안티앨리어스 필터는 $f_s/2$ 위의 주파수만이 아니라 **다른 모든 구역을** 막아야 한다. 여기에 저역통과 필터를 재활용하는 기술자는 실험실에서는 되고(거기에는 원하는 신호밖에 없다) 현장에서는 안 되는 수신기를 얻는다.

T5.2 구경 지터 — 벽

구경지터 (aperture jitter) — 표본화기가 신호를 실제로 잡는 순간의 rms 불확실도. 클럭 경로(위상 잡음, 전원이 일으키는 지연 변조, 마지막 버퍼의 열잡음)와 표본화기 자신에게서 오고, 표본화율이 아니라 **입력 주파수에 환산**된다.

유도 — 신호가 빨라질수록 지터가 왜 더 비싸지나

	이렇게 된다	왜냐하면
1	사인파 $v(t) = A \cdot \sin(2\pi f_{in} t)$ 의 기울기는 $dv/dt = 2\pi f_{in} A \cdot \cos(\dots)$ 다.	기울기가 시간 오차를 전압 오차로 바꾸는 것이다 — 기구가 그것뿐이다.
2	그러므로 시간 오차 Δt 는 전압 오차 $\Delta v = (dv/dt) \cdot \Delta t$ 를 놓고, 기울기가 가장 큰 영교차에서 가장 나쁘다.	가장 큰 오차는 신호가 가장 클 때가 아니라 가장 빨리 변할 때 난다.
3	rms 지터 σ_t 와 전체 눈금 사인파에서 rms 오차는 $2\pi f_{in} \cdot (A/\sqrt{2}) \cdot \sigma_t$ 이고, 신호 rms 는 $A/\sqrt{2}$ 다.	진폭이 사라진다 — 지터가 묶는 SNR 은 신호 진폭과 무관하다 . 크기를 줄여서 거기서 물러날 수 없는 까닭이다.
4	$SNR = -20 \cdot \log_{10}(2\pi f_{in} \sigma_t)$.	식 하나, 변수 둘, 소자 매개변수는 없다. 오로지 클럭과 입력 주파수만의 성질이다.
5	$f_{in} = 1 \text{ GHz}$ 에서 지터 100 fs 면 64 dB — 10.3 비트다.	100 fs 는 이미 어려운 클럭이다 . 기가헤르츠 12 비트 변환기가 비교기와는 아무 상관 없는 방식으로 어려운 까닭이다.

표 86. 지터가 묶는 SNR 과 ENOB (서로 무관한 구경 지터만)

f_{in}	1 ps	300 fs	100 fs	30 fs
0.1 GHz	64.0 dB / 10.3 b	74.5 dB / 12.1 b	84.0 dB / 13.7 b	94.5 dB / 15.4 b
1 GHz	44.0 dB / 7.0 b	54.5 dB / 8.8 b	64.0 dB / 10.3 b	74.5 dB / 12.1 b
5 GHz	30.1 dB / 4.7 b	40.5 dB / 6.4 b	50.1 dB / 8.0 b	60.5 dB / 9.8 b

예제 14. 입력 1 GHz 12 비트 변환기에는 어떤 클럭이 필요한가?

주어진 것. 설계 목표는 입력 1 GHz 에서 실효 12 비트다. 지터가 유일한 오차원이라고 하자 — 낙관적인 가정이고, 그래서 답은 클럭이 얼마나 좋아야 하는가의 **하한**이 된다.

푸는 법. SNR 식을 뒤집는다: $\sigma_t = 1/(2\pi \cdot f_{in} \cdot 2^N)$.

답. $\sigma_t = 38.9$ fs rms. 14 비트면 9.71 fs 다.

표 87. $f_{in} = 1$ GHz 에서 필요한 rms 구경 지터

비트	필요한 σ_t
8	622 fs
10	155 fs
12	38.9 fs
14	9.71 fs
16	2.43 fs

여기서 틀린다. 그 수를 클럭 소스에 적힌 지터와 견주는 것. 중요한 규칙은 분배 경로 전체 — 버퍼, 레벨 시프터, 분주기, 그리고 디지털 쪽이 거기에 집어넣은 전원 잡음 — 를 지난 뒤 **표본화 스위치에서의** 지터다. 시끄러운 버퍼 사슬 뒤에 있는 20 fs 소스는 200 fs 표본화기이고, 소스의 데이터시트는 그것을 말해 주지 않는다.

지터와 kT/C 는 진폭이 아니라 **전력**으로 더해지고, 주파수에 대해 서로 반대로 비례한다. kT/C 는 평평한데 지터는 입력 주파수 한 옥타브마다 6 dB 씩 나빠진다. 그러므로 갈림 주파수가 늘 있고, 자신이 그 어느 쪽에 있는지를 아는 것이 어느 손잡이가 무언가를 하는지 말해 준다. 아래쪽이면 면적을 용량에 쓰라. 위쪽이면 전력을 클럭에 쓰라 — **지터가 묶는 설계에서 표본화 축전기를 키우는 것은 아무것도 사지 못하고, 재설계를 낭비하는 가장 흔한 길이다.**

T5.3 스위치 자체: 차지 인젝션, 피드스루, 그리고 부트스트랩

VDD

그림 35. 표본화 스위치. 그 온저항이 트래킹 대역폭을 정하고, 그 채널 전하는 스위치가 꺼질 때 표본화 축전기에 얹힌다.

표 88. 스위치가 더하는 오차 셋, 그리고 각각이 무엇에 달려 있나

오차	기구	무엇에 비례하나	신호 의존?
차지 인젝션	소자가 꺼질 때 채널의 반전 전하가 어디론가 가야 한다. 대략 절반이 C_s 에 얹힌다	$W \cdot L \cdot C_{ox} \cdot (V_{GS} - V_{th}) / C_s$	그렇다 — V_{GS} 가 표본화된 준위에 달려 있으므로 오프셋이 아니라 왜곡이 된다
클럭 피드스루	C_{gd} 가 떨어지는 클럭 에지를 축전기로 결합시킨다	$C_{ov} / (C_{ov} + C_s) \cdot V_{clk}$	아니다 (1 차로) — 대체로 고정 오프셋이다
R_{on} 변조	스위치의 온저항이 $V_{GS} = V_{clk} - v_{in}$ 에 달려 있다	$1/(\mu C_{ox} (W/L) (V_{GS} - V_{th}))$	그렇다 — 신호에 따라 변하는 시간상수는 신호에 따라 변하는 정착이고, 그것이 왜곡이다

부트스트랩 스위치

스위치의 게이트를 V_{DD} 가 아니라 $v_{in} + V_{DD}$ 에 붙들어 둔다. V_{DD} 로 미리 충전한 축전기를 트래킹 구간에 게이트로 뒤집어 붙여서 그렇게 한다. 그러면 V_{GS} 가 신호와 무관하게 일정하므로 R_{on} 이 일정하고 꺼질 때 주입되는 전하도 일정하다 — **신호에 따라 변하던 오차가 오프셋이 되고**, 차동 설계에서 오프셋은 해롭지 않다.

놓치기 쉬운 신뢰성 문제도 같이 풀린다. 부트스트랩이 없으면 접지 근처 신호를 지나보내는 스위치가 $V_{GS} = V_{DD}$ 를 보게 되는데, 진보한 노드에서는 그것이 소자 수명에 걸쳐 산화막의 정격을 넘을 수 있다(T14).

어디에 쓰나	10 비트를 넘는 사실상 모든 SAR 과 파이프라인 ADC 의 입력 표본화 회로망, 그리고 선형성이 높은 스위치드 커패시터 필터.
언제 쓰나	면적보다 THD 나 SFDR 이 중요할 때. 맨 NMOS 스위치는 8 비트에는 괜찮고 12 비트에는 가망이 없다.
어떻게 쓰나	부트스트랩 축전기를 그것이 몰아야 할 게이트 용량보다 훨씬 크게 잡고, 클럭 구간마다 모든 마디의 신뢰성을 확인하고, 왜곡을 단일 톤이 아니라 두 톤 과도로 검증한다 — 상호변조가 단일 톤이 감추는 잔여 신호 의존을 드러낸다.

실제 코드는 이렇게 생겼다

```
* The check that matters, and the one people skip.
* A single tone can look clean while IM3 is terrible.
Vin1 in1 0 SIN(0 0.45 100.1MEG)
Vin2 in2 0 SIN(0 0.45 103.3MEG)
.tran 10p 20u
.four 100.1MEG v(sampled)          $ harmonic distortion
* and post-process the FFT for IM3 at 2*f1-f2 = 96.9 MHz -- the number the
* customer will measure and the one that exposes Ron modulation.
```

여기서 틀린다. 부트스트랩은 게이트 전압을 전원 위로 올리므로, 부트스트랩 회로망의 모든 마디를 **빠르고, 전압이 높고, 온도가 낮은** 코너에서 산화막 한계에 견주어 확인해야 한다 — 성능을 묶는 코너와 반대쪽 코너다. 전형 코너에서만 검증한 부트스트랩 회로는 신뢰성 보고서가 기다리고 있는 것이다.

T5.4 오버샘플링: 시간으로 비트를 사기

유도 — 오버샘플링이 왜 도움이 되고, 얼마나 되나

	이렇게 된다	왜냐하면
1	양자화 잡음 전력은 표본화율과 무관하게 $LSB^2/12$ 다.	양자화기는 자기가 얼마나 빨리 클럭되는지 모른다. 표본마다의 오차는 같은 분포를 갖는다.
2	그 고정된 전력이 나이퀴스트 대역 0 에서 $f_s/2$ 에 펼쳐진다.	그래서 잡음 밀도는 $LSB^2/12$ 를 $f_s/2$ 로 나눈 것이다 — 빨리 표본화하면 뭉어진다.
3	신호 대역 B 까지 걸러 내면 그중 $2B/f_s$ 만 남는다.	오버샘플링비를 $OSR = f_s/(2B)$ 로 정의한다.
4	SNR 이 $10 \cdot \log_{10}(OSR)$ 만큼 좋아진다: 두 배마다 3 dB, 곧 반 비트.	되먹임이 없는 맨 오버샘플링이다. 이 방식으로 8 비트에서 12 비트로 가려면 $OSR = 256$ 이 필요하다 — 비싸지만 느린 센서에는 가끔 맞는 답이다.
5	$\Delta\Sigma$ 변조기는 잡음을 그저 펼치는 것이 아니라 모양 지운다 . L 차 고리면 두 배마다 $(6L+3)$ dB 다.	고리가 양자화 잡음을 대역 밖으로 밀어낸다. 2 차 변조기는 OSR 한 옥타브마다 15 dB 를 번다 — $\Delta\Sigma$ 가 오디오와 정밀 감지를 지배하는 까닭이고, 기가 헤르츠 신호 대역폭에서는 쓸모가 없는 까닭이다.

표 89. OSR 을 두 배로 할 때마다 오버샘플링이 사 주는 것

구조	OSR 2 배당 SNR 이득	OSR 2 배당 비트	어디서 이기나
맨 오버샘플링 + 데시메이션	3 dB	0.5	느린 신호에서 값싼 해상도. 안티앨리어스 필터 부담 덜기
$\Delta\Sigma$, 1 차	9 dB	1.5	간단하다. 다만 유희 톤이 있다
$\Delta\Sigma$, 2 차	15 dB	2.5	오디오, 정밀 센서
$\Delta\Sigma$, 3 차 이상 / MASH	21 dB+	3.5+	안정성 공학을 치를 값어치가 있는 자리

오버샘플링은 해상도를 사 주는 만큼이나 **안티앨리어스 필터**의 부담을 덜어 주고, 흔히 그것이 진짜 이유다. 20 kHz 오디오 대역을 44.1 kHz 로 표본화하려면 20 kHz 에서 평평하고 22.05 kHz 에서 죽은 필터가 있어야 한다 — 잔인한 아날로그 문제다. 2.8 MHz 로 표본화하면 요구가 '1.4 MHz 에서 죽어라' 로 옮겨 가고, 그것은 RC 하나다. **사라지는 그 필터가 대개 반 비트보다 값어치가 크다.**

T5.5 월요일에 무엇을 할 것인가

표 90. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
엘리어싱 · 나이키스트	f_s 와 안티엘리어스 필터 고르기	필터 차수 — 그리고 그것이 아날로그인지 오버샘플링과 나눠 질 수 있는지
구경지터	변환기를 설계하기 전의 클럭 구조	그 입력 주파수에서 목표 ENOB 에 닿는 것이 가능하거나 한가
대역통과표본화	RF 와 IF 수신기 전단	믹서를 뺄 수 있는가, 그리고 그러면 클럭에 얼마를 치러야 하는가
차지인젝션 · 부트스트랩 스위치	10 비트 이상 변환기의 표본화 회로망	THD 와 SFDR, 그리고 그에 딸린 산화막 신뢰성 검사
오버샘플링비 · 잡음성형	느리고 정밀한 신호 경로	용량 대신 시간으로 해상도를 살 수 있는가

들고 갈 수: 입력 1 GHz 에서 **실효 12 비트는 rms 구경 지터 38.9 fs 를 요구한다**. 클럭 소스가 아니라 스위치에서 낸 값으로. 고속 변환기의 모든 구조 결정 — 인터리빙, 대역통과 표본화, 클럭 버퍼가 어디에 앉는가 — 이 그 한 줄의 아래에 있다.

다음 장은 표본화된 전압을 코드로 바꾸고, 구조들이 정확도에서 다른 것보다 같은 근본 한계를 어디에 쓰는지에서 더 다르다는 것을 알아낸다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

표본화 순간은 고속 변환기에서 가장 많이 다투는 1 제곱 밀리미터이고, 특허가 그것을 보여 준다. 온저항이 신호에 안 달리도록 V_{GS} 를 일정하게 붙드는 부트스트랩 스위치. 다른 무엇과도 버퍼를 일부러 안 나누는 클럭 경로. 트리밍이 아니라 대칭으로 차지 인젝션을 상쇄하는 표본화 방식. 그리고 한 단계 위에서, 조각마다 조금씩 틀린 순간에 표본화한다는 것이 어려움의 전부인 시간 인터리브 구조. **그 하나하나가 증폭기가 아니라 구경에 대한 논의다.**

T6. 데이터 변환기 — 구조마다 같은 한계를 어디에 쓰는가

어떤 구조도 kT/C 와 지터와 불일치를 이기지 못한다. 다른 것은 무엇을 먼저 치르는가이고, 고르는 근거는 그것이 전부다.

읽기 전에 알아야 하는 것

전압 · 전류 · 전하 · 확률 · 표준편차 · 로그 · SNR · SNDR · ENOB · kT/C 잡음 · 구경지터 · 표본화정리 · 오버샘플링비 · 잡음성형 · 차지인젝션 · 정적잡음여유

이 장에서 새로 나오는 것

플래시ADC · 축차비교ADC · 파이프라인ADC · 시간인터리브 · 용량DAC · 전류구동DAC · 세그먼테이션 · INL · DNL · 미싱코드 · 단조성 · 중복성 · 디지털보정 · 글리치에너지 · 정합

변환기 구조는 목록처럼 보이지만 사실은 물음 하나다. 해상도와 표본화율이 주어졌을 때 어느 근본 한계에 먼저 부딪히고, 어느 구조가 그 한계를 내가 가진 화폐로 치르게 해 주는가. 모든 구조가 같은 kT/C , 같은 지터, 같은 불일치를 마주한다. 다른 것은 변환 한 번에 각 한계를 몇 번 치르는가다.

T6.1 지도, 그리고 경계마다 무엇이 정하나

표 91. 네 구조, 그리고 각각을 멈추는 한계

구조	표본당 비교 횟수	실용 범위	무엇이 멈추나
플래시	병렬로 $2^N - 1$	4-8 b, 수십 GS/s 까지	비트마다 면적과 입력 용량이 두 배. 비교기 오프셋 2^N 개가 각각 $\frac{1}{2}$ LSB 미만이어야 한다
SAR	직렬로 N (주기마다 하나)	8-14 b, 조각마다 ~100 MS/s 까지	DAC 정착 사슬: 표본마다 N 번 정착하고 매번 전체 정확도까지
파이프라인	단마다 ~1.5 b, 동시예	10-16 b, 50 MS/s-1 GS/s	단 사이 이득의 정확도와 그것을 대는 연산증폭기
$\Delta\Sigma$	OSR 배 속도에서 1 (또는 몇)	16-24 b, kHz-MHz 대역폭	고리 안정성과 공정이 클럭할 수 있는 OSR
시간 인터리브	위의 아무거나 M 조각	해상도가 아니라 속도를 늘린다	조각 사이 불일치 — 이득·오프셋·표본화 순간 어긋남이 f_s/M 에 스퍼로 나타난다

마지막 줄이 사람들을 놀라게 하는 줄이다. M 조각을 인터리브하면 처리량이 M 배가 되고 해상도는 나빠진다. 단일 변환기에는 없던 오차 항 셋이 새로 생기기 때문이다. 조각 사이의 타이밍 어긋남이 가장 어렵다. 그것은 **결정론적인** 지터 문제(T5)여서 잡음 바닥이 아니라 스퍼를 낳고, 예측 가능한 주파수의 스퍼야말로 고객의 스펙트럼 그림에 정확히 나타날 것이다.

T6.2 SAR 의 축전기 배열: 정합이 잡음을 이기는 자리

SAR 변환기는 용량형 DAC 로 구현한 이진 탐색이다. 그러니 그 정확도는 축전기의 비에 달려 있고, 설계 물음은 단순히 단위 축전기가 얼마나 커야 하는가다. 서로 무관한 요구 둘이 그것을 정하는데, 둘 다 계산해 보기 전에는 어느 쪽이 이기는지 분명하지 않다.

유도 — 같은 축전기에 걸린 요구 둘, 그리고 그 둘이 만나는 자리

	이렇게 된다	왜냐하면
1	요구 1 — 잡음 : 그 배열이 곧 표본화 축전기이므로 kT/C_{total} 이 양자화 잡음 아래여야 한다(T4).	전체 눈금 1 V 이면 $C \geq 12kT \cdot 2^{2N}/V_{\text{FS}}^2$ 이고, 12 비트에서 0.83 pF 다.
2	요구 2 — 정합 : 중간 눈금 전이에서의 DNL 은 가장 큰 축전기와 나머지 전체의 불일치가 지배한다.	$3\sigma \text{DNL} < \frac{1}{2} \text{LSB}$ 이려면 12 비트에서 $\sigma_u/C_u < 0.26 \%$ 가 필요하고, 펄그롬(T12)에 따라 그 상대 시그마는 면적이 고정한다.
3	그 수준으로 맞는 축전기는 요즘 MOM 축전기 공정에서 대략 1 fF 이고, 그러면 12 비트 배열이 4.1 pF — 잡음 요구의 4.91 배다.	12 비트에서 배열을 정하는 것은 잡음이 아니라 정합이다. kT/C 만으로 치수를 정하면 잡음 바닥은 훌륭하고 DNL 계단이 눈에 보이는 변환기가 나온다.
4	14 비트에서 두 요구는 16.4 pF 과 13.3 pF 다 — 수렴했다.	~14 비트 위에서는 잡음이 넘겨받고 단위 축전기는 맞기만 하면 무엇이든 된다. 그 아래에서는 정합이 지배한다. 자신이 어느 쪽에 있는지를 아는 것이 어느 손잡이가 무언가를 하는지 말해 준다.

표 92. SAR 배열의 크기를 정하는 한계 ($V_{\text{FS}} = 1 \text{ V}$, 단위 용량 1 fF)

비트	정합이 요구하는 $\sigma_u/C_u <$	정합에서 나온 배열	kT/C 에서 나온 배열	무는 것
10	0.52 %	1.02 pF	0.05 pF	정합
12	0.26 %	4.1 pF	0.83 pF	정합
14	0.13 %	16.4 pF	13.3 pF	정합

중복성 — 비트 하나를 더 써서 정착 시간을 되사기

맨 이진 탐색에는 틀린 결정에서 돌아올 길이 없다. 비교기가 판단할 때 DAC 가 아직 정착하지 않았으면 그 비트는 틀리고, 뒤의 모든 비트가 틀린 구간을 더 잘게 다듬는다. 중복성은 기수를 바꿔서 이어지는 구간이 **겹치게** 만든다 — 걸음마다 그 겹침만큼의 앞선 오차를 고칠 수 있다.

값은 주기 한둘이고, 얻는 것은 이른 주기가 더 이상 N 비트 전체 정확도까지 정착할 필요 없이 겹침 안까지만 정착하면 된다는 것이다. 정착 시간은 정확도의 로그이므로(T_2), **이른 주기를 느슨하게 하는 것이 변환 시간의 대부분이다.**

어디에 쓰나	요즘의 모든 고속 SAR, 그리고 파이프라인의 단 사이 단(거기서는 단당 1.5 비트라고 부른다).
언제 쓰나	변환이 빨라야 할 때마다. 면적으로는 거의 공짜이고, 같은 해상도에서 10 MS/s SAR 과 100 MS/s SAR 을 가르는 것이 이것이다.
어떻게 쓰나	이진보다 작은 기수를 고르고(예: 2 가 아니라 1.86), 거기 맞게 축전기 치수를 정하고, 보정 가중치로 출력을 디지털에서 다시 짓는다. 그 가중치는 보정 할 수도 있고, 그러면 축전기 불일치가 오차가 아니라 잔 상수가 된다.

실제 코드는 이렇게 생겼다

```
// Non-binary SAR reconstruction. The weights are NOT powers of two,
// and they are per-chip once calibration has run.
localparam int W [0:N] = '{2048, 1088, 576, 320, 176, 96, 56, 32, 18, 10, 6, 4, 2, 1}';
always_comb begin
    code = 0;
    for (int i = 0; i <= N; i++) if (bit_decision[i]) code += W[i];
end
// The sum of weights EXCEEDS full scale on purpose -- that surplus is the redundancy.
// A design review question worth asking: what happens if the weights are wrong by 1%?
// (Answer: nothing, until the error exceeds the overlap -- which is the point.)
```

여기서 틀린다. 중복성은 **덜 된 정착과 비교기 잡음**을 막아 주지, 단조적으로 틀린 DAC 를 막아 주지 않는다. 축전기가 겹침을 넘어서 잘못된 크기로 만들어졌으면, 중복성이 간단한 시험에서는 증상을 감추고 전체 INL 쓸기에서만 드러나는 코드 의존 오차를 남긴다.

T6.3 정적 선형성: INL, DNL, 그리고 변환기가 거짓말하는 두 가지 방식

DNL (차등 비선형성) — 코드마다의 너비가 1 LSB 에서 벗어난 정도. $DNL = -1$ LSB 는 그 코드의 너비가 0 — 미싱 코드라는 뜻이고, 거기서 변환기는 더 이상 단조가 아니다.

INL (적분 비선형성) — 전이점마다 이상적인 직선에서 벗어난 정도를 누적한 것 — DNL 의 누적합이다. 왜곡 측정이 보는 것이 이것이고, 제어 고리가 느끼는 것은 DNL 이다.

표 93. 어느 쪽을 볼 것인가, 그리고 언제

용도	중요한 쪽	왜
오디오 / RF 스펙트럼	INL	굽음이 곧 왜곡이다. 고객의 FFT 에 고조파가 보인다
페루프 제어 (예: 서보 안의 DAC)	DNL 과 단조성	단조가 아닌 코드는 그 자리에서 고리의 부호를 뒤집고 발진할 수 있다
센서 문턱 검출	DNL	미싱 코드는 시스템이 영영 보고할 수 없는 값이다
다른 블록의 디지털 보정	INL	보정은 기준 곡선만큼만 좋다

실제로 난 일

변환기가 SNDR 시험을 통과하면서 심하게 단조가 아닐 수 있다. 사인파 히스토그램 시험은 모든 코드에 걸쳐 평균을 낸다. 미싱 코드 하나는 SNDR 을 dB 의 몇 분의 일만 움직이고 눈에 안 띈다. **정적 선형성은 자기 나름의 시험이 필요하다** — 느린 램프나 코드 밀도 쓰기 — 그리고 SNDR 만 적은 IP 납품물은 그 부품을 특성화하지 않은 것이다.

T6.4 DAC 쪽: 세그멘테이션과 글리치 에너지

세그멘테이션 (segmentation) — 중요한 데는 온도계, 아닌 데는 이진

온전히 이진인 DAC 는 중간 눈금에서 가능한 가장 나쁜 글리치를 낸다. 모든 스위치가 한꺼번에 바뀌고, 그것들의 타이밍 어긋남이 면적 (글리치 에너지, $V \cdot s$)이 평균으로 안 없어지는 과도를 만든다 — 갱신율에 왜곡으로 나타난다. 온전히 온도계 부호인 DAC 에는 그런 전이가 없지만 요소가 $2^N - 1$ 개 필요하고 그것을 몰 디코더도 필요하다.

진짜 DAC 는 나눈다. 위쪽 M 비트는 온도계, 나머지는 이진. M 은 동시에 스위칭하는 가장 큰 사건이 충분히 작아지도록 고르고, 실무적인 답은 대개 4-6 이다 — 디코더 값은 2^M 으로 자라는데 글리치 개선은 포화하기 때문이다.

어디에 쓰나	송신기와 파형 발생기의 전류 구동 DAC. 파이프라인 ADC 안의 기준 DAC. 출력을 주파수 영역에서 보는 모든 DAC.
언제 쓰나	SFDR 이 규격에 있을 때마다. 이진 DAC 의 중간 눈금 글리치가 대개 어떤 정적 오차보다도 먼저 SFDR 을 정한다.
어떻게 쓰나	MSB 를 나누고, 단위 전류원을 레이아웃으로 맞추고(공통 중심, 더미), 남은 불일치가 스퍼가 아니라 잡음이 되도록 단위 요소의 선택을 무작위화하거나 돌린다(DEM).

실제 코드는 이렇게 생겼다

```
// Segmented decode: 6 thermometer MSBs + 8 binary LSBs.
// The point of the thermometer part is that consecutive codes change ONE element.
logic [62:0] therm;
always_comb begin
    therm = '0;
    for (int i = 0; i < 63; i++) therm[i] = (code[13:8] > i);
end
// Dynamic element matching: rotate the starting index so that element mismatch
// is spread across codes and appears as a raised noise floor, not as a spur.
always_ff @(posedge clk) rot <= rot + code[13:8];
assign sel = {therm, therm} >> rot;    // barrel rotate
```

여기서 틀린다. DEM 은 스퍼를 잡음으로 바꾸는 것이지 오차를 없애는 것이 아니다. 스펙트럼 요구가 잡음 바닥이면 그것은 이득이고, 총 잡음 전력이면 중립이다. DEM 을 켜 뒤 잡음 바닥은 안 적고 'SFDR 이 15 dB 좋아졌다' 고만 보고하는 것은, 고객이 자기 실험실에서 찾아내게 되는 종류의 반쪽 진실이다.

T6.5 월요일에 무엇을 할 것인가

표 94. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
플래시ADC · 축차비교ADC · 파이프라인ADC · 시간인터리브	첫 주의 구조 선택	무엇을 치르게 될 것인가
용량DAC · 세그먼테이션	트랜지스터를 하나도 그리기 전의 배열 치수	단위 축전기, 총 면적, 그리고 기준 드라이버의 전류
중복성 · 디지털보정	속도와 수율	이른 주기의 정착 정확도를 얼마나 떨어 줄 수 있나
INL · DNL · 미싱코드	특성화와 데이터시트	이 부품이 스펙트럼뿐 아니라 제어 고리에서도 쓸 만한가
글리치에너지	주파수 영역에서 보는 모든 DAC	SFDR, 그리고 온도계 비트를 몇 개나 살 것인가

이 장에서 들고 갈 수: 12 비트에서 SAR의 축전기 배열은 **잡음만 따졌을 때보다 4.91 배 크다**. 정합이 그것을 정하기 때문이다 — 그리고 14 비트에서 그 인자는 1.23로, 거기서는 잡음이 넘겨받았다. 12 비트짜리는 정합 요구에서, 14 비트짜리는 kT/C 에서 치수를 정하게 되고, 같은 배열을 서로 다른 물리로 치수 잡는 셈이다. 갈림점이 그 둘 사이에 앉아 있기 때문이다.

다음 장들은 변환기를 떠나, 이 모든 일이 언제 일어나는지를 정하는 회로 — 위상 고정 루프 — 를 다룬다. 거기서 잡음과 대역폭과 전력 사이의 같은 맞바꿈이 축전기가 아니라 되먹임 시스템 안에서 나타난다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

변환기 특허는 세 자리에 몰리고 그 모양을 보아 둘 값어치가 있다. SAR의 용량형 DAC가 쓰는 에너지를 낮추는 **스위칭 방식**(단조, 분할 용량, V_{cm} 기반 — 전부 같은 배열을 다르게 스위칭하는 것이다). 물리적 오차를 한 번 재서 디지털 영역에서 영원히 빼는 **보정**. 그리고 조각 사이의 이득·오프셋·타이밍 어긋남에 대한 **인터리빙 보정**. 셋 다 모양이 같다: 아날로그 오차를 받아들이고 그것을 더 싼 데서 없앤다. 혼성신호 설계에서 가장 확실하게 특허가 되는 생각이고, 거의 모든 블록에서 쓸 수 있다.

T7. 위상 고정 루프 — 출력이 시간인 되먹임 시스템

다른 모든 블록은 클럭을 주어진 것으로 다룬다. 이 블록은 그것을 만들고, 마주한 맞바꿈에는 좋은 쪽이 없다. 한 잡음원을 누르는 대역폭이 다른 잡음원을 들인다.

읽기 전에 알아야 하는 것

전압 · 전류 · 주파수 · 위상 · 적분 · 미분 · 로그 · 확률 · 표준편차 · 트랜스컨덕턴스 · 출력저항 · RC지연 · 열잡음 · 플리커잡음 · 구경지터 · SNR · 잡음대역폭

이 장에서 새로 나오는 것

위상검출기 · 전하펌프 · 루프필터 · 전압제어발진기 · 분주기 · 고리이득 · 감쇠비 · 자연주파수 · 루프대역폭 · 위상잡음 · 적분지터 · 기준스퍼 · 분수분주 · 델타시그마변조 · 확산스펙트럼 · 락검출 · 전원푸싱

PLL 은 이 책에서 출력이 시간인 유일한 블록이다. 지금까지의 모든 장은 클럭을 지터가 얹힌 입력으로 다뤘다. 여기서 그 지터를 만들어 내고, 설계 물음은 고리에 정반대로 반응하는 두 원천에서 그것을 어떻게 최대한 적게 만들어 낼 것인가다.

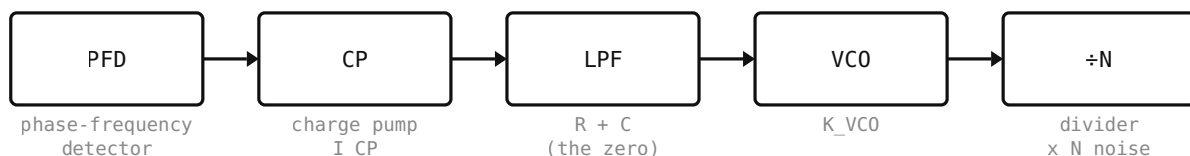


그림 36. 전하펌프 PLL. 분주기가 고리를 닫고, 동시에 고리 안의 모든 잡음원에 N 을 곱하는 것도 그것이다.

T7.1 고리가 실제로 제어하는 것

유도 — 위상 검출기에서 2 차 시스템까지

	이렇게 된다	왜냐하면
1	위상-주파수 검출기는 기준과 분주된 출력 사이의 위상 오차에 비례하는 신호를 낸다.	주파수가 아니라 위상이다 — 주파수는 그 미분이고, PLL 이 위상을 잠그는 부수 효과로 주파수를 잠그는 까닭이며 고리에 본래부터 적분기가 있는 까닭이다.
2	전하펌프가 그 오차를 i_{CP} 의 전류로 바꿔 오차 펄스가 지속되는 동안 흘린다.	평균 전류는 $i_{CP} \cdot \Delta\phi / 2\pi$ 다 — $i_{CP} / 2\pi$ A/rad 의 선형 위상-전류 이득이다.
3	필터가 그 전류를 C 에 적분하고, VCO 가 전압을 이득 K_{VCO} 로 위상에 적분한다.	고리에 적분기가 둘이다. 맨 축전기만으로는 고리가 무조건 불안정한 까닭이고, 영점(직렬 R)이 선택 사항이 아닌 까닭이다.
4	개루프 이득: $G(s) = (i_{CP}/2\pi) \cdot (1/sC) \cdot (1+sRC) \cdot (K_{VCO}/s) \cdot (1/N)$.	원점에 극 둘, $1/RC$ 에 영점 하나. 그 영점이 위상 여유를 낸다.
5	달으면 $\omega_n = \sqrt{i_{CP} K_{VCO} / (2\pi NC)}$ 과 $\zeta = (R/2) \sqrt{i_{CP} K_{VCO} C / (2\pi N)}$ 을 갖는 표준 2 차 시스템이다.	이 장의 나머지는 전부 그 두 수를 어디에 둘 것인가에 대한 논의다.
6	예로 든 설계($f_0 = 2.5$ GHz, $f_{ref} = 25$ MHz, $N = 100$, $i_{CP} = 1$ μ A, $K_{VCO} = 5$ MHz/V)에서 $\zeta = 0.9$ 로 $f_L = 1$ MHz 를 겨누면 $C = 41$ pF, $R = 12.6$ k Ω 이다.	1 % 까지 정착하는 데 대략 $4/(\zeta\omega_n) = 1.27$ μ s 걸린다 — 시스템 아키텍트가 실제로 묻는 수가 그것이다.

불쾌함이 들어오는 자리가 분주기다. 고리 안의 모든 잡음원 — 기준, PFD, 전하펌프, 분주기 — 이 출력에서 전력으로 N^2 배가 되고, $N = 100$ 이면 40 dB 다. 100 배를 곱하는 PLL의 대역 안 위상 잡음 바닥이 출발점이었던 수정보다 훨씬 나쁜 까닭이고, 아주 낮은 지터가 필요한 설계가 편한 기준이 아니라 가능한 한 높은 기준 주파수(더 작은 N)를 쓰는 까닭이다.

T7.2 좋은 쪽이 없는 맞바꿈: 루프 대역폭

고리가 누르는 잡음과 통과시키는 잡음

고리는 자기 안의 모든 것(기준, PFD, 전하펌프, 분주기)에 대해 **저역통과**이고, VCO에 대해서는 **고역통과**다. 루프 대역폭 안에서는 되먹임이 출력을 기준에 따라가게 만들므로 VCO 자신의 잡음이 고쳐진다. 그 밖에서는 고리가 너무 느려서 아무것도 못 고치고 VCO가 자유롭게 돈다.

그래서 고리를 넓히면 VCO 잡음을 더 누르고 기준 경로 잡음을 더 들인다. 좁히면 반대다. **늘 최적이고, 고리 안에서 최적인 설계 변수는 그것뿐이다.**

어디에 쓰나	모든 클럭 발생기, 모든 SerDes 송수신 PLL, 데이터 변환기의 클럭, 그리고 지터 예산을 맞춰야 하는 SoC의 칩 내부 클럭.
언제 쓰나	일찍 고정한다. 루프 대역폭이 VCO 규격과 기준 선택을 묶으므로, 나중에 바꾸면 둘 다 바뀐다.
어떻게 쓰나	모양 지운 밀도를 적분하고 쓸어 본다. 최적은 보통 대역 안 바닥과 VCO의 자유 발진 잡음이 만나는 자리이고, 그것은 적분 없이 위상 잡음 그림에서 바로 읽을 수 있다.

실제 코드는 이렇게 생겼다

```
# Phase noise measurement, and the integration that turns it into jitter.
# The instrument gives L(f) in dBc/Hz; the number the customer wants is rms jitter.
# sigma_t = sqrt( 2 * integral( 10^(L(f)/10) df ) ) / (2*pi*f0)
# THE INTEGRATION LIMITS ARE PART OF THE SPECIFICATION.
# PCIe: 10 kHz - 20 MHz (with the protocol's own filter applied)
# JESD204B: 100 Hz - 20 MHz
# Ethernet: often 12 kHz - 20 MHz
# A jitter number quoted without its limits is not comparable to any other one.
```

여기서 틀린다. 'DC'부터 적분하는 것은 뜻이 없다 — VCO의 $1/f^3$ 영역 적분은 발산하고, 그래서 모든 규격이 아래쪽 한계를 정해 둔다. 서로 다른 한계로 '서브 피코초 지터'를 말하는 두 업체는 견줄 수 있는 주장을 하고 있는 것이 아니고, 좁은 한계 쪽이 대개 나쁜 부품이다.

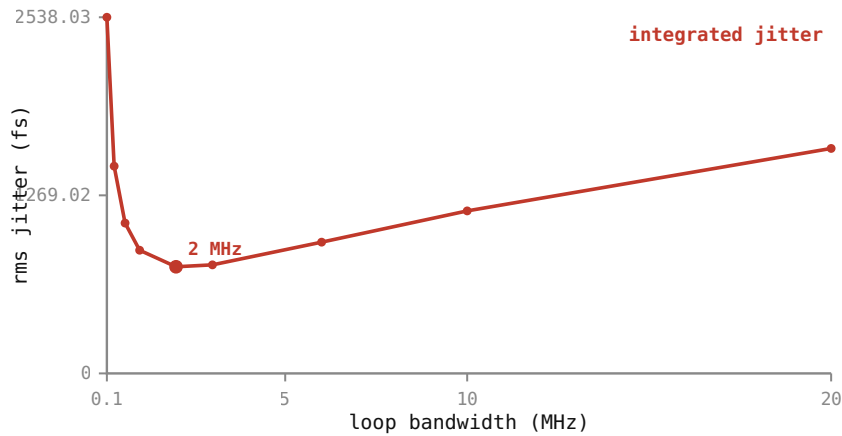


그림 37. 예로 든 설계에서 루프 대역폭에 대한 적분 지터(대역 안 바닥 ≈ -110 dBc/Hz, 1 MHz 에서 VCO ≈ -100 dBc/Hz). 최적은 2 MHz 에서 76 fs rms 이고 — **얕다**. 1 MHz 와 3 MHz 가 그것의 16 % 안에 있다. 어느 블록이 더 어렵든 그쪽을 덜어 주는 편을 고르라.

표 95. 루프 대역폭에 대한 적분 지터 (외운 것이 아니라 계산한 것)

루프 BW	rms 위상	2.5 GHz 에서 rms 지터
0.1 MHz	39,867 μ rad	2,538 fs
0.3 MHz	23,197 μ rad	1,477 fs
0.6 MHz	16,829 μ rad	1,071 fs
1 MHz	13,789 μ rad	877.8 fs
2 MHz	11,936 μ rad	759.9 fs
3 MHz	12,152 μ rad	773.6 fs
6 MHz	14,692 μ rad	935.3 fs
10 MHz	18,191 μ rad	1,158 fs
20 MHz	25,183 μ rad	1,603 fs

얕다는 것이 실무적인 결론이다. 루프 대역폭의 세 배 차이가 지터의 몇 퍼센트를 치르므로, 대역폭은 그것이 제어하는 **다른** 제약 — 정착 시간, 기준 스퍼 억제, VCO 끌림에 대한 내성 — 으로 골라야지 평평한 최적에 대고 쥐어짜서는 안 된다. 정확한 최솟값을 찾느라 일주일을 쓰는 기술자는 움직이지 않는 축 하나를 최적화한 것이다.

T7.3 스퍼: 잡음이 아닌 오차

기준스퍼 (reference spur) — 기준 주파수(와 그 고조파)만큼 떨어진 자리에 나타나는 이산 톤. 비교율로 제어 전압을 주기적으로 교란하는 무엇이든 그 원인이 된다 — 전하펌프의 up/down 불일치, 필터 축전기의 누설, 또는 기준에서 VCO 로의 결합.

표 96. 스퍼가 어디서 오나, 그리고 무엇이 실제로 없애나

원인	기구	듣는 해법	안 듣는 해법
CP 전류 불일치	올림 전류와 내림 전류가 달라서 고리가 0 이 아닌 위상 오차에 앉아야 하고, 그래서 펌프가 주기마다 동작한다	I_{up} 을 I_{dn} 에 맞추는 복제 바이어스	I_{CP} 를 낮추는 것 — 스퍼와 고리 이득이 같이 비례한다
필터 누설	축전기와 스위치의 누설이 갱신 사이에 제어 마디를 비운다	필터 마디에 두꺼운 산화막 소자. 더 큰 C	고리 이득을 더 주는 것 — 교란이 입력 쪽에 있다
전원 / 기판 결합	기준 예지가 VCO 로 곧바로 결합한다	물리적 분리, VCO 전용 레귤레이터, 차동 튜닝	고리 안의 무엇이든 — 고리는 이 경로를 아예 못 본다
분수분주 양자화	분주기가 정수 사이를 오가므로 위상 오차가 구조적으로 주기적이다	그것을 모양 지우는 $\Delta\sigma$ 변조, 그리고 충분한 고리 필터링	더 느린 기준

분수분주 (fractional-N) 와 그것이 일부러 만드는 잡음

정수 N 고리는 기준의 배수만 낼 수 있다. 평균 $N.f$ 로 나누는 것 — N 과 N+1 사이를 오가는 것 — 은 어떤 주파수든 내지만, 그 값으로 분수의 되풀이율에 몰린 스펙트럼을 갖는 주기적 위상 오차를 낸다. $\Delta\sigma$ 변조기가 그 수열을 무작위화하고 모양 지워서 오차를 고주파 잡음으로 만들고, 루프 필터가 그것을 짊는다.

맞바꿈이 분명하다. 분수분주는 주파수 해상도를 사고 그 값을 루프 대역폭 밖의 잡음으로 치르는데, 그 영역이 바로 넓은 고리가 거르지 못하는 영역이다. **분수분주 설계가 정수분주보다 좁은 고리를 쓰는 경향이 있는 까닭이고**, 그러면 다시 VCO 자신의 잡음이 더 중요해진다 — 이 장의 중심 맞바꿈이 되풀이해 나타난다.

어디에 쓰나	가진 수정의 배수가 아닌 주파수가 필요한 모든 시스템: 무선 LO, 기준 하나에서 여러 라인 속도를 내는 SerDes, 오디오 클럭, 확산 스펙트럼 클럭킹.
언제 쓰나	필요한 출력 주파수 격자가 기준보다 촘촘할 때. 시스템 구조상 수정을 고를 수 있다면 언제나 정수분주를 택하라 — 늘 더 조용하다.
어떻게 쓰나	변조기 차수를 고르고(2-3 차가 보통), 유리수 분수 자리의 유틸 톤 을 확인하고, 모양 지운 잡음이 루프 대역폭에서 충분히 밖에 있어 필터의 내리막이 그것을 잡았는지 검증한다.

실제 코드는 이렇게 생겼다

```
// Behavioural PLL for system simulation -- real-valued, event driven.
// It models what the DIGITAL side needs (frequency, lock time, jitter injection)
// and nothing else. Replacing this with a transistor model in an SoC simulation is
// how a regression that used to take 10 minutes starts taking a week.
module pll_behav #(parameter real FREF=25.0e6, parameter int N=100,
                    parameter real LOCK_US=2.0, parameter real JIT_RMS=0.8e-12)
    (input logic ref_clk, rst_n, output logic out_clk, output logic locked);
    real period = 1.0/(FREF*N);
    initial begin out_clk = 0; locked = 0; end
    always #(1s*period/2) out_clk = ~out_clk; // ideal edges
    initial begin @(posedge rst_n); #(LOCK_US*1us); locked = 1; end
endmodule
// SDC: the output is a GENERATED clock -- declaring it as a second create_clock
// makes STA treat the two domains as unrelated and silently stops checking paths.
// create_generated_clock -name pll_out -source [get_ports ref_clk] \
//     -multiply_by 100 [get_pins u_pll/out_clk]
```

여기서 틀린다. PLL 출력을 create_clock 으로 선언하고 create_generated_clock 으로 선언하지 않는 것은 흔하고 비싼 SDC 잘못이다. 그러면 도구가 기준과 출력 사이의 위상 관계를 모르게 되어 둘 사이의 모든 건넘을 비동기로 다루고, 보고서는 여전히 깨끗해 보이는 채로 **당신이 신경 쓰는 경로를 보고하기를 그만둔다.**

T7.4 실제로 무엇이 깨지나

표 97. 고리 방정식에는 안 나타나는 실패들

실패	증상	근본 원인	무엇이 찾아내나
전원 푸싱	출력 주파수가 V_{DD} 를 따라 움직인다. 지터가 디지털 활동과 상관을 갖는다	$K_{VCO, supply}$ 가 0 이 아니다 — VCO 가 의도한 것보다 여러 뜻에서 전압 제어 발진기다	시뮬레이션에서 전원을 켜서 df/dV 를 재고, PDN 의 리플(T13)에 견주어 예산 잡는다
VCO 끌림	가까이 있는 센 신호의 오프셋 자리에 스퍼가 뜬다	다른 발진기나 고출력 단에서 기판이나 전원을 통해 주입 동기가 걸린다	두 톤 벤치 측정. 플로어플랜 검토
거짓 락	고조파나 엉뚱한 에지에 잠긴다. 리셋하면 가끔 된다	주파수 능력이 없는 위상 검출기, 또는 불법 상태에서 출발하는 분주기	코너와 초기 조건에 걸친 시동 쓰기 — 공칭 한 번이 아니다
락 검출이 거짓말한다	고리가 정착하기 전에 '락' 이 뜬다	검출기가 규격이 요구하는 것보다 넓은 창 안에서 위상을 견준다	검출기의 창을 실제 정착 시간과 견준다. 시스템이 둘 다 기다리게 만든다

실제로 난 일

디지털이 무거운 SoC 를 기습하는 것이 전원 푸싱이다. 전원 민감도가 1 %/V 인 VCO 가 활동에 따라 30 mV 리플이 있는 레일 위에 앉아 있으면, 디지털 부하가 스위칭하는 그 속도로 주파수가 변조된다 — 그 속도는 상수가 아니고, 고객이 돌리는 바로 그 워크로드와 상관을 갖는다. 실험실에서는 '소프트웨어에 따라 달라지는 지터' 로 나타난다. 해법은 VCO 전용 레귤레이터이고, 그것은 플로어플랜과 전원 개수를 바꾸므로 처음부터 설계에 넣어야 한다.

T7.5 월요일에 무엇을 할 것인가

표 98. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
고리이득 · 감쇠비 · 루프 대역폭	루프 필터 설계	C 와 R, 따라서 정착 시간과 면적
위상잡음 · 적분지터	VCO 규격을 정하기 전의 지터 예산	VCO 잡음과 대역 안 바닥 중 어느 쪽에 써야 하나
기준스퍼	스펙트럼 규제 준수와 수신기 감도 저하	전하펌프 위상과 필터 마디의 누설 예산
분수분주 · 델타시그마변조	주파수 계획	수정을 골라서 분수분주를 아예 피할 수 있는가
전원푸싱	플로어플랜과 PDN	VCO 에 전용 레귤레이터가 필요한가 — 되돌리기 어려운 결정이다

들고 갈 수: 예로 든 이 설계의 루프 필터는 $C = 41 \text{ pF}$, $R = 12.6 \text{ k}\Omega$ 이고 정착은 $1.27 \mu\text{s}$ 다. 지터 최적은 루프 대역폭 2 MHz 에 앉고 **충분히 알아서 대역폭은 차라리 정착과 스퍼 요구로 골라야 한다**. 그리고 분주기는 대역 안의 모든 잡음원에 40 dB 를 곱한다 — 시스템이 댈 수 있는 가장 높은 기준 주파수를 쓰라는 가장 강한 논거다.

다음 장은 이 클럭을 일하게 한다. 직렬 링크에서, 같은 지터를 이미 눈을 거의 닫아 놓은 채널에 견주어 예산 잡아야 한다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

PLL 특허는 고리 방정식에 관한 것이 거의 아니다 — 그것은 1930년대 제어 이론이다. 특허는 **고리가 무엇을 보게 허락할 것인가**에 관한 것이다: 양자화 잡음이 중요한 대역에서 밀려나도록 모양 지운 분수 분주기. up/down 불일치를 트리밍이 아니라 복제로 상쇄하는 전하펌프. 아날로그 필터를 누산기로 갈음해서 공정을 따라 줄어드는 디지털 고리. 그리고 분주기를 변조해 복사 에너지를 규제 주파수에서 옮기는 확산 스펙트럼 방식. 전부 바뀌지 않은 고리 둘레의 배치다.

T8. 직렬 링크 —詹 것, 안 되는 자리까지

링크는 채널이 남의 문제이면서 온전히 내 책임인 유일한 블록이다.

읽기 전에 알아야 하는 것

전압 · 주파수 · 위상 · 확률 · 표준편차 · 로그 · 적분 · SNR ·
구경지터 · 위상잡음 · 적분지터 · 루프대역폭 · 엘리머싱 ·
ENOB · 열잡음

이 장에서 새로 나오는 것

심볼간간섭 · 펄스응답 · 커서 · 선행커서 · 후행커서 · 최악
왜곡 · 아이다이어그램 · CTLE · FFE · DFE · 오류전파 · 비
트오류율 · 3의법칙 · 클럭데이터복원 · 지터허용 · PAM4

직렬 링크의 모든 것은 한 가지 사실에서 나온다. 채널은 심볼률보다 대역폭이 훨씬 좁은 저역통과 필터라서, 심볼 하나하나가 이웃에 번진 채로 도착한다. 수신기가 할 일은 그 번짐을 슬라이서가 판정할 수 있을 만큼 되돌리는 것이고, 아래의 측정 — 이 저장소 자신의 채널·링크 모형으로 돌린 것 — 은 등화가 얼마나 사 주는지와, 더 쓸모 있게는 어디서 아무것도 사 주지 않는지를 둘 다 보여 준다.

T8.1 펄스 응답, 그리고 거기서 나오는 수들

커서 (cursor) — 채널의 펄스 응답을 심볼 간격으로 뽑은 표본들. **주커서**는 슬라이서가 쓰는 것이고, 그보다 앞의 것은 모두 **선행커서**, 뒤의 것은 모두 **후행커서**다. 주커서가 아닌 것들의 크기를 다 더한 것이 최악의 심볼간간섭이다.

표 99. 이 모형 채널의詹 펄스 응답 (sps = 8)

손실	주커서	선행커서 1	후행커서 1	ΣISI	최악 아이
10 dB	3.388	0.348	1.329	4.234	-1.692
20 dB	1.693	0.547	1.035	5.444	-7.502
30 dB	0.918	0.487	0.744	5.681	-9.525

마지막 칸을 꼼꼼히 읽어라. **모든** 손실에서 — 링크가 사실상 오류 없이 도는 10 dB 까지 포함해서 — 최악 아이가 음수다. 모순이 아니다. 최악(첨두왜곡) 해석은 모든 간섭원을 동시에 최대로 만드는 데이터 패턴을 가정하는데, 그런 패턴은 천문학적으로 드물다. **첨두왜곡은 한계이지 예측이 아니다**. 그것만 보고 사인오프한 설계는 과설계이고, 그것을 무시한 설계는 한계가 아예 없다. 두 수 다 납품물에 들어가야 한다.

유도 — 왜 손실이 후행커서로 나타나며, 왜 그것이 중요한가

	이렇게 된다	왜냐하면
1	손실이 있는 채널은 높은 주파수를 낮은 주파수보다 더 깎는다.	표피효과($\propto \sqrt{f}$)와 유전손실($\propto f$). 둘 다 드라이버가 아니라 매질의 물리다.
2	그래서 날카로운 에지는 시간으로 퍼진다. 펄스 응답에 긴 꼬리가 생긴다.	20 dB 에서 켄 것: 첫 후행커서가 1.035 이고 주커서가 1.693 이니 — 신호의 61.1 % 가 한 심볼 늦게 도착한다.
3	그 꼬리는 인과적이고 과거의 것이다 — 이미 판정된 심볼들에만 달려 있다.	판정귀환 등화기가 가능한 까닭이 바로 그것이다. 수신기가 그 비트들을 이미 알고 있으니, 잡음을 키우지 않고 그 몫만 빼낼 수 있다.
4	선행커서는 그렇지 않아서, 앞을 내다보는 필터로만 다룰 수 있다 — FFE 인데, 선형이라 신호와 함께 잡음도 키운다.	20 dB 에서 켄 선행커서는 0.547 로 후행커서의 3분의 1이다. 수신기가 한 종류가 아니라 두 종류의 등화기를 다 쓰는 까닭이 이 비대칭이다.

T8.2 등화기 하나하나가 실제로 사 주는 것 — 켄 것

표 100. 채널 손실과 등화에 대한 BER (SNR = 26 dB, PRBS, 씨앗 1·2·3 합침, 칸마다 420,000 비트)

손실	등화 없음	CTLE 6 dB	+ FFE 3	+ DFE 4
10 dB	2.38×10^{-6}	$< 7.14 \times 10^{-6}$ (오류 0, 3의법칙)	$< 7.14 \times 10^{-6}$ (오류 0, 3의법칙)	$< 7.14 \times 10^{-6}$ (오류 0, 3의법칙)
20 dB	0.14	0.01	0.01	0
30 dB	0.27	0.23	0.25	0.25

실제로 난 일

30 dB 줄은 결과가 아니다 — 망가진 동작점이다. 어느 구성이든 BER 이 대략 0.2 인데, 이것은 아이가 완전히 달렸을 때 슬라이서가 하는 일이다. 찍는 것이다. 거기서 등화기를 건주는 것은 동전 던지기를 건주는 것이고, FFE 가 CTLE 하나보다 나쁘다고 보이는 것(0.25 대 0.23)은 물리가 아니라 잡음이다.

이 저장소에는 바로 이것을 두고 호되게 배운 규칙이 있다. **그 위에서 켄 것을 무엇이든 보고하기 전에 동작점이 성한 지부터 본다.** 0.5 근처의 BER, 수렴하지 않은 등화기, 틀린 표본 위상 — 모두 데이터처럼 보이는 수를 낳는다.

표 101. 성한 동작점 찾기: 20 dB 손실에서 CTLE + FFE 5 + DFE 6 으로 SNR 쏘기 (줄마다 630,000 비트)

SNR	오류	BER
26 dB	2226	0
30 dB	9	1.43×10^{-5}
34 dB	0	$< 4.76 \times 10^{-6}$ (오류 0, 3의법칙)
38 dB	0	$< 4.76 \times 10^{-6}$ (오류 0, 3의법칙)

표 102. 성한 동작점에서 같은 비교 (20 dB 손실, SNR = 34 dB)

구성	오류	BER
없음	81496	0.13
CTLE	40	6.35×10^{-5}
CTLE+FFE5	0	$< 4.76 \times 10^{-6}$ (오류 0, 3의법칙)
CTLE+FFE5+DFE6	0	$< 4.76 \times 10^{-6}$ (오류 0, 3의법칙)

3의법칙 (rule of three) — 오류 0 이 무엇을 뜻해도 되는가

위의 두 줄이 **오류 0** 을 보고한다. 오류 0 은 BER 이 0 이라는 뜻이 아니다. 상한이다. n 비트에서 오류가 하나도 없을 때 오류율의 95 % 신뢰 상한은 $3/n$ 이다 — **3의법칙**이다. 630,000 비트 실행에서는 그것이 4.76×10^{-6} 이다.

그러니 이 측정이 말하는 것은 CTLE + FFE 와 CTLE + FFE + DFE 가 둘 다 4.76×10^{-6} 아래라는 것뿐이고, **둘 중 어느 쪽이 나은지는 아무것도 말하지 않는다**. DFE 가 '아무 차이도 안 났다' 고 보고하는 것은 데이터가 받쳐 주지 않는 주장이다. 정직한 진술은 이 실행이 둘을 가려내지 못했다는 것, 그리고 가려내려면 비트가 대략 100배 더 있거나 채널이 더 나빠야 한다는 것이다.

어디에 쓰나	모든 BER 측정, 모든 준수 보고서, 그리고 BER 이라는 글자가 들어가는 모든 데이터시트 줄.
언제 쓰나	오류 수가 적을 때마다 — 출하할 만한 링크라면 그것은 늘이다.
어떻게 쓰나	비율만이 아니라 오류 수와 비트 수를 보고한다. 오류 = 0 이면 $3/n$ 한계와 신뢰수준을 같이 적는다. $1e-12$ 를 주장하려면 적어도 $3e12$ 비트를 봐야 하는데, 25 Gb/s 에서 그것은 2분이 고 — $1e-15$ 에서는 하루다.

실제 코드는 이렇게 생겼다

```
# The line that belongs in every link report.
# errors = 0 in 6.30e5 bits -> BER < 4.8e-6 (95% confidence, rule of three)
# NOT "BER = 0".
def ber_bound(errors, bits, conf=0.95):
    if errors == 0:
        import math
        return -math.log(1 - conf) / bits      # 3.0/bits at 95%
    return errors / bits                      # point estimate; add a CI for small n
# And the check that catches the broken operating point before any of this matters:
assert 0.0 <= ber < 0.2, f"BER {ber} -- the eye is closed; this run measured nothing"
```

여기서 틀린다. 정확히 0.5 인 BER 은 대개 나쁜 채널이 아니라 극성이나 정렬 버그를 뜻한다 — 수신기가 뒤집혔거나 어긋난 기준과 견주고 있는 것이다. PAM4 에서 0.25 근처의 BER 은 한 층 위에서 같은 뜻이다. **등화기를 타지기 전에 이것부터 봐라.** 진짜로 닫힌 아이보다 훨씬 흔하다.

T8.3 등화기 셋, 그리고 일을 더 나쁘게 만들 수 있는 하나

표 103. 등화기 하나하나가 신호와 잡음에 하는 일

	CTLE	FFE	DFE
어디에	아날로그, 표본 뽑기 전	아날로그나 디지털, 선형 FIR	슬라이서 뒤, 되먹임
무엇을 다루나	넓은 대역의 고주파 손실	선행커서와 후행커서	후행커서만
잡음	신호와 함께 키운다	키운다 — 고주파에서의 이득이 곧 잡음 이득이다	안 키운다 — 아는 심볼을 빼는 것이다
값	작다. 소자 몇 개와 피킹 회로망	탭 × 곱셈기. 탭 수는 꼬리를 따라 늘어난다	한 심볼 주기 안의 곱셈-덧셈 — T2의 타이밍 벽
언제 진다	손실이 매끈한 기울기가 아닐 때(반사)	잡음이 이미 한계일 때	틀린 판정이 퍼질 때(오류 다발)

오류전파 (error propagation), 그런데도 DFE 가 값을 하는 까닭

DFE 는 이미 판정한 심볼들의 몫을 뺀다. 판정이 틀렸으면 틀린 것을 빼게 되고, 그러면 다음 판정도 틀릴 확률이 높아진다 — 오류가 하나씩이 아니라 다발로 온다.

그 효과는 실제로 있고, 직관이 말하는 것보다는 작다. 탭 하나짜리 DFE 에서는 다발 길이가 짧은데, 오류가 다음의 옳은 판정을 살아 넘겨야 하기 때문이다. 뒷단에서 중요한 것은 BER 이 아니라 **다발의 구조**다. 뒤따르는 FEC 가 어떤 다발 길이를 가정하고 설계됐기 때문이다. BER 은 맞추면서 가정된 다발 분포를 어기는 링크는 디코딩 뒤에 지는데, 그동안 FEC 전의 모든 측정은 멀쩡해 보인다.

어디에 쓰나	손실 ~10 dB 가 넘는 모든 수신기, 그리고 FEC 설계자와의 모든 대화.
언제 쓰나	후행커서 꼬리가 주커서에 견주어 무시할 수 없을 때마다 — 이 모형의 20 dB 에서는 첫 후행커서 하나가 주커서의 61.1 % 다.
어떻게 쓰나	BER 만이 아니라 다발 길이 분포를 재서, 인터리버 치수를 잡는 사람에게 넘긴다. 언롤링(추측)은 되먹임 곱셈을 임계경로에서 빼낸다 — T2 의 논증 — 대신 탭 k 개를 언롤하면 슬라이서가 2^k 개 나란히 든다.

실제 코드는 이렇게 생겼다

```
// Unrolled (speculative) first DFE tap: both outcomes are precomputed,
// and the previous decision only selects. The multiply leaves the loop.
always_comb begin
    d_if_prev0 = (sample > +c1) ;      // precomputed thresholds
    d_if_prev1 = (sample > -c1) ;
end
always_ff @(posedge clk) prev <= prev ? d_if_prev1 : d_if_prev0; // 2:1 mux only
// The loop now contains a mux and a flop instead of a multiply-add.
// Cost: one extra slicer per unrolled tap, and 2^k for k taps.
```

여기서 틀린다. 언롤링은 그 꼬리가 임계경로였을 때만 이득이다. 먼저 재라. 이 저장소 자신이 현실적인 동작점에서 DFE 를 낸 것에서는 되먹임 탭이 $|d| \leq 0.041$ 이었다 — **꼬리가 높고 있었다**. 그리고 진짜 임계경로는 슬라이서였다. 노는 꼬리를 다듬는 것은 가장 비싼 종류의 옳은 일이다.

T8.4 클럭 되찾기, 그리고 그게 되는지를 가르치는 지터

클럭데이터복원 (CDR) — 데이터를 기준으로 삼는 T7 의 PLL

따로 난 클럭선이 없다. 수신기는 데이터 자신의 천이에서 타이밍을 뽑아낸다. 위상 검출기(알렉산더/뱅크, 또는 선형 윌러-윌러)가 T7 과 같은 2차 고리를 몬다. 그 장의 모든 것이 그대로 들어맞되 두 가지가 다르다. '기준' 은 데이터가 천이할 때만 오고, 고리는 송신기의 확산 스펙트럼 변조를 **따라가면서** 고주파 지터는 **물리쳐야** 한다.

그것이 지터허용 마스크다. CDR 대역폭 아래에서는 수신기가 지터를 따라가므로 큰 진폭도 견뎌진다. 그 위에서는 따라갈 수 없어서 지터가 아이를 곧장 잡아먹는다.

어디에 쓰나	모든 직렬 규격의 모든 수신기 — PCIe, USB, 이더넷, MIPI, SATA, HDMI.
언제 쓰나	CDR 대역폭은 규격의 지터허용 마스크와 따라가야 하는 확산 스펙트럼 프로파일이 정한다. 마음대로 고르는 값이 아니다.
어떻게 쓰나	규격의 마스크를 가져다 허용 곡선이 그 위에 여유를 두고 앉도록 CDR 대역폭을 놓고, 송신 PLL 의 지터(T7)에 잔차를 더한 것이 예산 안에 드는지 본다.

실제 코드는 이렇게 생겼다

```
# The budget is a SUM, and each term has an owner.
# TX PLL jitter (T7)          0.30 ps rms  <- clock design
# TX driver / ISI (DJ)       0.15 UI pp   <- output stage and package
# Channel ISI residual       0.20 UI pp   <- equalisation, this chapter
# RX CDR tracking error      0.05 UI pp   <- loop bandwidth
# Sampler aperture (T5)      0.10 ps rms  <- sampling network
# Random terms add in QUADRATURE, deterministic ones LINEARLY -- mixing the two
# rules is the most common budgeting error, and it always errs optimistic.
```

여기서 틀린다. 확산 스펙트럼 클러킹은 **따라가야 하는 요구사항**이지 물리쳐야 할 교란이 아니다. 33 kHz SSC 프로파일을 따라가기에 대역폭이 모자란 CDR 은 규격을 지키는 송신기 앞에서 락을 잃는다. 깨끗한 발생기를 쓰는 벤치에서는 되는데 진짜 호스트 앞에서 지는 수신기는 대개 이것이다.

T8.5 월요일에 이걸로 무엇을 하나

표 104. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
펄스응답 · 커서 · 최악왜곡	등화기를 고르기 전	어떤 종류의 탭을 몇 개 — 그리고 데이터시트에 들어갈 한계
CTLE · FFE · DFE	수신기 구조	잡음 증폭 대 오류전파, 그리고 타이밍 벵
3의법칙	당신이 보고하는 모든 BER 수	그 측정이 그 주장을 받쳐 주기는 하는지
오류전파	FEC 와의 경계면	인터리버 깊이 — FEC 설계자가 당신에게 받아야 하는 수
클럭데이터복원 · 지터허용	준수	CDR 대역폭. 규격이 이미 당신 대신 골라 놓았다

가지고 갈 댐 결과 둘. 손실 20 dB, SNR 34 dB 에서 CTLE 하나가 링크를 0.13 에서 6.35×10^{-5} 로 데려간다 — 대략 2,037 배다 — 그리고 FFE 를 더하면 630,000 비트로 가려낼 수 있는 아래로 내려간다. 그런데 손실 30 dB 에서 같은 SNR 이면 **어느 등화기도 아무 일도 하지 않는다**. 동작점 자체가 망가져 있기 때문이다. 그 문장 없이 두 번째 표를 보고하는 것이 이 책의 규칙들이 막으려고 있는 바로 그 실패다.

다음 장은 여기서 쓴 모형 밑으로 내려가, 채널의 손실이 실제로 어디서 오는지를 묻는다 — 그리고 패키지와 비아와 커넥터가 위의 수들에 무슨 짓을 하는지를 묻는다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

링크 특허는 고리가 있는 자리에 몰린다. 되먹임 곱셈을 임계경로에서 빼내는 추측(언롤) DFE, 훈련열 없이 수렴하는 적응 방식, ISI 를 더 견디는 CDR 위상 검출기, 그리고 요즘은 등화가 전부 디지털인 ADC 기반 수신기. **이 저장소 자신의 연구가 그 첫 갈래에 있다**. 그리고 거기서의 논증은 T2 의 것이다. 발명은 자연을 어디로 옮겼는가이지 트랜지스터가 얼마나 빠른가가 아니다.

T9. 채널 — 손실이 어디서 오는가, 항 하나씩

손실은 등화할 수 있다. 반사는 못 한다. 둘 중 어느 쪽인지 아는 것이 수신기 설계와 보드 재설계를 가른다.

읽기 전에 알아야 하는 것

전압 · 주파수 · 위상 · 로그 · 적분 · 심볼간간섭 · 펄스응답 · 커서 · CTLE · FFE · DFE · 아이다이아그램

이 장에서 새로 나오는 것

특성임피던스 · 전송선 · 삽입손실 · 표피효과 · 유전손실 · 반사계수 · 리턴로스 · 노치 · 비아스터브 · 백드릴 · 근단누화 · 원단누화 · S파라미터 · 채널규격 · COM

앞 장은 채널을 손실 수 하나로 다뤘다. 그 수는 물리적으로 서로 다른 항들의 합이다. 주파수를 따라 다르게 커지고, 시스템의 다른 자리에서 오며, 그리고 결정적으로 등화에 다르게 반응한다. '채널이 20 dB 다' 만 아는 수신기 설계자는 그 설계가 어려운 것인지 불가능한 것인지 가릴 수 없다.

T9.1 네 항, 그리고 한 번의 측정에서 그것들을 가려내는 법

삽입손실 (insertion loss) 을 맞춘 합으로 보기

IEEE 802.3 채널 규격은 켄 삽입손실을 $IL(f) = a_0 + a_1\sqrt{f} + a_2f + a_4f^2$ 에 맞추고, 항마다 물리적인 임자가 있다.

- a_0 (평평) — 커넥터, 본드 와이어, 손실이 주파수를 상관하지 않는 모든 것.
- $a_1\sqrt{f}$ — 표피효과. 전류가 깊이가 $1/\sqrt{f}$ 로 가는 표면층에 몰려서 실효 저항이 $\sqrt{f}$ 로 간다. 이것이 구리다.
- a_2f — 유전손실. 보드 재료의 손실 탄젠트. 에너지가 기판을 데우는 데로 간다. 이것이 적층재 선택이다.
- a_4f^2 — 거칠기와 복사. 구리의 표면 처리, 그리고 선로가 복사해 내보내는 것.

측정을 이 꼴에 맞춰 보면 어느 발주서가 그것을 고치는지를 알 수 있다. 더 나은 구리인지, 더 나은 적층재인지, 커넥터를 줄이는 것인지, 아니면 그 어느 것도 아닌지.

어디에 쓰나	채널 규격을 읽을 때, 보드가 있기 전에 링크 예산을 잡을 때, 그리고 적층재를 두고 시스템 아키텍트와 다룰 때.
언제 쓰나	맨 처음에. 이 맞춤이 수신기에 탭이 3개 드는지 15개 드는지를 정하고, 그 결정이 수신기 전체의 치수를 잡는다.
어떻게 쓰나	켄 S21 을 dB 로 가져다 관심 대역에서 네 계수를 맞추고, 나이퀴스트에서 어느 항이 지배하는지 본다. 아래는 이 저장소의 채널 모형에서 같은 분해를 켄 것이다.

실제 코드는 이렇게 생겼다

```
# Fitting a real S21 to the standard form -- four numbers that a board
# engineer can act on. (numpy, on a Touchstone file read with scikit-rf.)
import numpy as np, skrf
ntwk = skrf.Network('channel.s4p')
f     = ntwk.f / f_nyquist           # normalise to Nyquist
IL    = 20*np.log10(np.abs(ntwk.s[:, 1, 0])) # dB, and it is NEGATIVE
A     = np.vstack([np.ones_like(f), np.sqrt(f), f, f*f]).T
a, *_ = np.linalg.lstsq(A, IL, rcond=None)
print(f"flat {a[0]:.2f} dB | skin {a[1]:.2f} | dielectric {a[2]:.2f} | rough {a[3]:.2f}")
# If |a[2]| dominates at Nyquist, the fix is laminate, not copper.
```

여기서 틀린다. 엉뚱한 대역에서 맞추면 엉뚱한 것을 알려 준다. 신호가 차지하는 대역(DC 에서 대략 나이퀴스트의 1.5 배까지)에서 맞춰라. VNA 가 어쩌다 쓸고 간 범위가 아니다 — 특히 f^2 항은 나이퀴스트 아래에서 거의 묶이지 않아서, 맞춤이 필요로 하는 것을 무엇이든 빨아들인다.

표 105. 이 모형 채널의 켄 삽입손실, 항별 (f 의 단위는 $1/U$. 나이퀴스트 = 0.5)

채널	0.125	0.25	0.5 (나이퀴스트)	1.0
도체만 (12 dB)	-6 dB	-8.5 dB	-12 dB	-17 dB
유전체만 (8 dB)	-2 dB	-4 dB	-8 dB	-16 dB
둘 다	-8 dB	-12.5 dB	-20 dB	-33 dB

유도 — 저 표에서 물리를 곧장 읽어 내기

	이렇게 된다	왜냐하면
1	표피효과 줄은 주파수가 두 배가 될 때마다 $-6.0 \rightarrow -8.5 \rightarrow -12.0 \rightarrow -17.0$ dB 로 간다.	한 걸음마다 $1.417 \approx \sqrt{2}$ 가 곱해진다 — $a_1 \sqrt{f}$ 가 예측하는 것이 그것이다. 모형은 규격의 꼴이 말하는 것을 구현하고 있고, 측정이 그것을 확인해 준다.
2	유전체 줄은 $-2.0 \rightarrow -4.0 \rightarrow -8.0 \rightarrow -16.0$ dB 로 간다.	두 배마다 정확히 2배 — $a_2 f$ 항이고, 같은 방식으로 확인된다.
3	낮은 주파수에서는 도체 항이 지배하고(-6.0 대 -2.0), 1.0/UI 에서는 둘이 엇비슷하다(-17.0 대 -16.0).	어느 손실 기구가 지배하는지가 대역을 따라 바뀐다. 그래서 '이 채널은 구리가 한계다' 는 주파수를 밝혔을 때만 참이다.
4	둘 다인 줄은 읽어 낸 자릿수 안에서 두 항을 dB 로 더한 것이다.	$-6.0 + -2.0 = -8.0$ 으로 켜고, $-12.0 + -8.0 = -20.0$ 으로 켜다. 두 항이 독립이고, 그래서 따로 맞추는 것이 정당하다.

T9.2 반사: 등화로 못 고치는 쪽

전송선 · 특성임피던스 (transmission line, Z_0) — 선로가 파장의 의미 있는 몫이 되는 주파수 위에서는 그것이 전선이기 그만두고 **전송선**이 된다. 전압 대 전류의 비가 기하와 유전체로 정해지는 진행파를 나르는 구조물이고, 단위 길이당 $Z_0 = \sqrt{L/C}$ 다. 다른 Z_0 를 만난 파동은 일부가 되돌아온다 — 이 절의 나머지가 다루는 것이 그것이고, '임피던스' 가 옴의 법칙 속의 수 하나가 아니라 보드 위의 규격인 까닭이 그것이다.

반사계수 (reflection coefficient), ρ — 임피던스가 끊기는 자리에서 들어온 파동의 $\rho = (Z_2 - Z_1)/(Z_2 + Z_1)$ 만큼 이 되돌려 보내진다. 임피던스가 10 % 틀리면 $\rho \approx 0.05$ 이고, 비아 스테르브나 커넥터 풋프린트나 패키지 볼은 훨씬 크게 줄 수 있다.

유도 — 반사가 노치를 어디에 놓으며, 얼마나 깊은가

	이렇게 된다	왜냐하면
1	반사는 왕복 지연 τ 뒤에 돌아와 곧바로 온 것에 더해진다.	메아리 하나, 지연 하나 — 가장 단순한 경우이고, 이것만으로 이미 켄 채널 대부분이 설명된다.
2	뒤집는 반사에 대해 전달함수가 $H(f) \cdot (1 - \rho e^{-j2\pi f\tau})$ 가 된다.	곧장 온 것에 늦게 온 사본을 더한 것이다. 감쇠가 아니라 간섭이다.
3	늦게 온 사본이 반대 위상인 곳에서 상쇄된다. 곧 $f = 1/(2\tau), 3/(2\tau), \dots$ 이다.	노치의 자리가 거리를 알려 준다 — 그리고 진짜 보드에서 스테르브를 짚어내는 방법이 그것이다.
4	깊이는 $20 \cdot \log_{10}(1 - \rho)$ 다.	깊이가 그 끊김의 크기를 켜다.

표 106. 켄 것과 닫힌 꼴을 견줄 — 독립 대조

ρ	자연 (UI)	노치 자리 (젠 것)	자리 (이론)	깊이 (젠 것)	깊이 (이론)
0.2	4	0.125 /UI	0.125 /UI	-1.94 dB	-1.938 dB
0.35	11	0.045 /UI	0.045 /UI	-3.75 dB	-3.742 dB
0.35	2	0.25 /UI	0.25 /UI	-3.74 dB	-3.742 dB

실제로 난 일

그 측정의 첫 시도는 이론이 0.125/UI 라고 말하는 반사에 대해 노치를 0.894/UI 라고 보고했다 — 일곱 배가 틀린 것인데, 그림은 아주 그럴듯해 보였다. 까닭은 이렇다. 코드가 **삽입손실의 최솟값**을 집었는데, 높은 주파수에서는 매끈한 손실이 어떤 노치보다도 훨씬 깊어서 최솟값이 그냥 대역의 꼭대기였다. **사소한 설명을 배제하지 않았던 것이다**. 반사 없는 채널로 먼저 나눠서 — 같은 것끼리 견줘서 — 모든 수가 이론의 0.2 % 안에 들어왔다.

왜 노치가 같은 깊이의 손실보다 나쁜가

선형 등화기는 채널을 거꾸로 뒤집는다. 매끈한 내리막을 뒤집는 값은 걸어 준 이득에 비례하는 잡음 증폭이다. 비싸지만 한계가 있다. **노치**를 뒤집으려면 한 주파수에서 엄청난 이득이 필요한데, 그 주파수에서 받은 신호는 그냥 작은 것이 아니라 상쇄돼 버린 것이다. 키울 신호가 거의 없고 잡음만 있다. **그 주파수의 정보는 사라졌다**.

그래서 평평한 손실 20 dB 와 노치 20 dB 는 전혀 다른 문제이고, 보드 검토가 선로 길이가 아니라 임피던스 굽김을 쫓는 까닭이 그것이다.

어디에 쓰나	보드와 패키지 검토, 커넥터 고르기, 그리고 수신기를 설계하기 전에 그 채널이 되기는 하는지를 정할 때.
언제 쓰나	데이터율을 확정하기 전에. 신호 대역 안의 노치는 보드로 고칠 것인지 수신기로 고칠 것이 아니다.
어떻게 쓰나	삽입손실만이 아니라 리턴로스(S11) 를 보라. IL 은 좋은데 RL 이 나쁜 채널은 에너지가 어딘가로 가고 있는 것이고, DFE 는 후행커서 메아리를 지울 수 있지만 그것이 탭이 미치는 범위 안에 떨어졌을 때만이다 — 40 UI 떨어진 메아리에는 탭이 40개 든다.

실제 코드는 이렇게 생겼다

```
# Where the echo lands decides whether ANY receiver can fix it.
# round-trip 40 mm on FR4, eps_eff ~ 4.0 -> t = 2*0.040/(3e8/2) = 533 ps
# at 25 Gbaud (40 ps/UI) -> 13 UI away
# A 13-tap DFE can reach it; a 4-tap one cannot, and no amount of CTLE helps.
# THIS calculation belongs in the datasheet's integration section, because the
# customer's board is what decides it.
```

여기서 틀린다. DFE 는 반사가 주커서보다 뒤에 있고 탭이 미치는 범위 안에 있을 때만 그것을 지운다. (송신기 가까운 굽김에서 오는) 선행커서 반사는 DFE 로는 아예 닿지 못하고, 그것을 해 보겠다는 FFE 는 노치가 지워 버린 바로 그 주파수에서 잡음을 키운다.

T9.3 누화, 그리고 실제로 아픈 쪽

표 107. 근단누화와 원단누화

	NEXT (근단)	FEXT (원단)
어디서 결합하나	같은 쪽 끝에서, 가해자의 송신기가 내 수신기로	먼 쪽 끝에서, 가해자의 송신기가 내 수신기로
채널이 깎아 주나?	아니다 — 채널을 지나온 적이 없다	그렇다 — 신호와 같은 손실 경로를 지나온다
신호에 견주면	채널에 깎인 내 신호에 맞서는 온전한 세기의 가해자	둘 다 깎여서 비가 대체로 지켜진다
무엇을 따라 커지나	결합 길이와 에지 속도	결합 길이. 그리고 주파수를 따라 커진다
무엇으로 고치나	떼어 놓기, 차폐, 그리고 TX 를 RX 옆으로 안 돌리기	간격, 유전체 선택, 그리고 수신기에서의 상쇄
어느 쪽이 지배하나	길고 손실 큰 채널 — 신호는 작는데 가해자는 안 작다	짧은 채널과 뻥뻥한 병렬 버스

저 표의 비대칭이 실무의 요점이다. 30 dB 채널에서 내 신호는 30 dB 깎여 도착하는데 NEXT 가해자는 온전한 세기로 도착한다 — 그러니 -40 dB 로 결합한 가해자가 내 신호보다 겨우 10 dB 아래다. **NEXT 는 레이아웃 검토 항목이지 수신기 항목이 아니다.** 본 적도 없는 신호를 빼낼 수 있는 등화기는 없다. 반면 FEXT 는 같은 채널을 지나왔고, 어떤 구조에서는 지울 수 있다. 가해자의 데이터가 같은 다이 위에 있기 때문이다.

T9.4 비아, 패키지, 그리고 규격이 그것을 두고 하는 일

비아 스텐브 (via stub) 와 백드릴

12층 보드에서 신호를 2층에서 3층으로 나르는 비아는 배럴의 나머지 — 4층에서 12층까지 — 를 붙은 채 안 쓰고 남긴다. 그 스텐브는 열린 전송선이고, 길이가 $\lambda/4$ 인 열린 스텐브는 공진 주파수에서 **단락**으로 보인다. $f = c/(4 \cdot L \cdot \sqrt{\epsilon_r})$ 에서 신호가 아무것도 아닌 데로 단락된다.

고침은 기계적이다. 안 쓰는 배럴을 **백드릴**로 깎아 내어, 공진이 대역 위에 있을 만큼 스텐브를 짧게 남긴다. 제조 공정이 한 걸음 더 들고, 그 결정은 채널 규격을 쓰는 사람의 몫이다 — IP 데이터시트가 맨 '25 Gb/s 를 지원한다' 가 아니라 자기가 가정하는 채널을 밝혀야 하는 까닭이 그것이다.

어디에 쓰나	~10 Gb/s 위에서 도는 층이 몇 장 넘는 모든 보드. 패키지 기판. 커넥터 풋프린트.
언제 쓰나	보드 적층 검토 — 실리콘이 있기 한참 전이고, IP 공급자의 의견이 진짜로 값을 하는 유일한 검토다.
어떻게 쓰나	스텐브 길이에서 공진을 셈해 나이퀴스트의 1.5배와 견주고, 대역 안에 떨어지면 백드릴하거나 층을 바꾼다. 가정한 채널을 — 백드릴을 가정하는지까지 — 연동 안내서에 적는다.

실제 코드는 이렇게 생겼다

```
# Stub resonance, and the line that belongs in INTEGRATION.md.
# stub 1.5 mm, eps_r 4.0 -> f = c/(4*L*sqrt(eps_r))
#                               = 3e8/(4*1.5e-3*2) = 25 GHz
# At 25 Gbaud (Nyquist 12.5 GHz) that is 2x Nyquist -- survivable.
# At 50 Gbaud (Nyquist 25 GHz) it sits ON Nyquist -- fatal, back-drill required.
#
# INTEGRATION.md:
# Channel assumed: <= 20 dB insertion loss at Nyquist, return loss <= -12 dB,
# no via stub resonance below 1.5x Nyquist (back-drilling assumed above 25 Gb/s).
```

여기서 틀린다. 공진은 $\sqrt{\epsilon_r}$ 에 달려 있는데, 비아 배럴의 실효 ϵ_r 은 적층재 데이터시트의 수가 아니다 — 스트립라인의 실효값보다 벌크 값에 가깝다. 엉뚱한 쪽을 쓰면 예측한 공진이 수십 퍼센트씩 움직이는데, 그것이 '대역 위' 와 '대역 안' 을 가른다.

표 108. 채널이 실제로 규정되는 방식, 그리고 각 항이 묶는 것

규격	무엇인가	무엇을 묶나
S 파라미터 (.s4p)	젠 4포트 회로망, 차동	전부 — 나머지 줄들은 이것의 요약이다
나이퀴스트에서의 삽입손실	S21 을 dB 로	등화기 세기. 누구나 먼저 대는 수
리턴로스 (S11)	되돌아온 에너지	손실이 매끈한지 노치가 있는지
ICR / ICN	적분 누화 비 / 잡음	레이아웃 간격과 커넥터 선택
COM (채널 동작 여부)	젠 S 파라미터에 표준화된 기준 수신기를 걸어 얻는 dB 한 수	이더넷 준수의 합불 — 그리고 그것이 고객이 돌릴 수 이 지 당신의 시뮬레이션이 아니다

COM 은 제 몫의 문장을 받을 만하다. 켄 채널을 가져다 **규정된** 기준 송신기와 수신기(등화 구조 고정, 잡음 고정, 지터 고정)를 걸고, dB 한 수를 돌려준다. 그래서 채널들이 견줄 수 있게 되고 그것이 목적이다 — 그런데 그 말은 기준보다 나은 수신기가 점수를 못 받고, 구조가 다른 수신기는 되레 깎일 수 있다는 뜻이기도 하다. **채널 도달거리를 약속하기 전에 자기 COM 을 알아라.** 고객은 그것을 셈할 것이고, 당신 수신기가 더 영리하다는 것은 상관하지 않을 것이다.

T9.5 월요일에 이걸로 무엇을 하나

표 109. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
삽입손실 · 표피효과 · 유전손실	수신기 치수를 잡기 전	탭 수, 그리고 어느 발주서가 문제를 줄여 줄지
반사계수 · 노치	보드와 패키지 검토	그 채널이 되기는 하는지 — 등화는 노치를 못 고친다
근단누화 · 원단누화	레이아웃 검토	간격 규칙, 그리고 상쇄가 가능하기는 한지
비아스터브 · 백드릴	적층, 제조 전	제조 공정 한 걸음, 그리고 연동 안내서의 한 줄
S파라미터 · COM	고객과의 대화	그들이 실제로 셈할 수 — 당신이 먼저 셈해 두었는지 확인해라

가지고 갈 켄 결과. 이 모형에서 표피효과 항은 $\sqrt{f}$ 로 오르고 (두 배를 네 번 하는 동안 6 dB → 8.5 dB → 12 dB → 17 dB) 유전체 항은 f 로 오른다 (2 dB → 4 dB → 8 dB → 16 dB) — 그러니 어느 쪽이 지배하는지는 어디를 보느냐에 달렸다. 그리고 4 UI 에 $\rho = 0.2$ 인 반사는 정확히 0.12/UI 에 -1.94 dB 노치를 놓아 달힌 꼴과 0.2 % 안에서 맞는다. **반사는 예측할 수 있고, 바로 그래서 등화할 것이 아니라 설계에서 걷어 내야 한다.**

다음 장은 칩 속으로 돌아가, 연속시간 회로는 할 수 없는 방식으로 대역폭을 정확도와 맞바꾸는 회로 갈래를 다룬다. 스위치드 커패시터다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

채널 특허는 IP 회사가 아니라 보드·커넥터 회사의 것이다 — 그런데 값을 하는 **수신기** 특허는 전부 채널의 병리에 대한 응답이다. 노치와 싸우는 대신 거기에 적응하는 등화기, 특정한 반사를 짚어내 지우는 방식, 그리고 가해자 자신의 데이터를 기준으로 쓰는 누화 상쇄. 이 장을 당신의 수신기가 풀어 달라고 요청받을 문제들의 목록으로 읽고, 그중 등화가 못 푸는 것이 어느 것인지 눈여겨보라 — 발명은 거기에 있다.

T10. 스위치드 커패시터 — 시간으로 산 정확도

값이 커패시터 비와 클럭인 저항. 그 치환 하나 덕에 정밀 아날로그가 디지털 공정으로의 이사를 살아남았다.

읽기 전에 알아야 하는 것

전압 · 전류 · 전하 · 저항 · 주파수 · 적분 · 로그 · kTC잡음 · 차지인젝션 · 부트스트랩스위치 · 표본화정리 · RC지연 · 논리적노력 · 정합 · INL · DNL

이 장에서 새로 나오는 것

등가저항 · 두상클럭 · 겹침없음 · 바닥판표본화 · SC적분기 · 되먹임계수 · 정착시간 · 유한이득오차 · 상관이중표본화 · 자동영점 · 전하보존 · 기생무감

정밀 아날로그 회로에는 정확한 계수가 필요한데, 집적 공정에서 정확한 것은 딱 하나다. 똑같이 그린 두 구조물의 **비**다. 절대 저항은 공정에 걸쳐 $\pm 20\%$ 로 흔들린다. 절대 용량도 마찬가지다. 그런데 나란히 그린 두 커패시터의 비는 1 퍼센트의 몇 분의 일까지 맞는다. 스위치드 커패시터 기법은 필요한 모든 것을 그 비로 표현하는 규율이다 — 그리고 그 값을 표본화로, 따라서 시간으로 치른다.

T10.1 커패시터와 클럭으로 만든 저항

유도 — 전하 옮김에서 등가저항으로

	이렇게 된다	왜냐하면
1	커패시터 C 를 V_1 까지 채운 다음 V_2 인 마디에 쓴다. 옮겨진 전하는 $\Delta Q = C(V_1 - V_2)$ 다.	회로 전체가 이것이다. 스위치 둘과 커패시터 하나를 겹치지 않는 클럭이 몬다.
2	f_s 로 되풀이한다. 평균 전류는 $I = f_s \cdot C \cdot (V_1 - V_2)$ 다.	주기당 전하 공급하기 초당 주기.
3	그것이 $R_{eq} = 1/(f_s C)$ 인 옴의 법칙이다.	$C = 1 \text{ pF}$ 에 $f_s = 10 \text{ MHz}$ 이면 $R_{eq} = 100 \text{ k}\Omega$ — 실리콘에서라면 어마어마했을 저항을, 그렇지 않은 커패시터로 지은 것이다.
4	이렇게 만든 시정수는 $R_{eq} C_2 = C_2/(f_s C_1)$ 이다 — 커패시터 비를 클럭 주파수로 나눈 것이다.	절댓값이 지워진다. 이제 회로의 정확도는 절대 RC의 정확도(수십 퍼센트)가 아니라 비의 정확도(퍼센트의 몇 분의 일)와 수정 발진기의 정확도(백만분의 몇)다.

표 1. 스위치드 커패시터의 등가저항

C	f _s	R _{eq}	메모
1 pF	10 MHz	100 kΩ	이만한 폴리 저항은 수백 μm^2 다
1 pF	100 MHz	10 kΩ	
0.5 pF	500 MHz	4 kΩ	이 속도에서는 아래의 정착 요구가 묶는 제약이 된다

두 상 겹침 없는 클럭 (two-phase non-overlapping clocks)

두 스위치가 **동시에 켜져서는 절대 안 된다**. 그러면 근원과 목적지가 단락되고, 옮겨진 전하가 겹친 만큼 틀리는데 — 그 겹침은 공정과 온도를 따라 흔들린다. 그래서 클럭은 일부러 죽은 시간을 두고 만들어진다. 보통은 지연 사슬이 그 틈을 정하는 교차결합 NAND 짝에서 나온다.

바닥판 표본화가 중요한 손질이다. 고정된 판에 붙은 스위치를 조금 먼저 연다. 그 스위치는 일정한 전압을 보므로 주입하는 전하가 일정하고, 그러면 신호에 따라 달라지는 왜곡이 아니라 오프셋이 된다(T5 의 차지 인젝션 문제를 회로가 아니라 순서로 푼 것이다).

어디에 쓰나	모든 스위치드 커패시터 회로. SC 필터, SAR 과 파이프라인 ADC 단, 표본유지, 기준 버퍼.
언제 쓰나	늘. 죽은 시간은 최적화가 아니다 — 상이 겹치는 설계는 어느 코너에서 지고, 눈에 띄게 부러지는 대신 느린 정확도 흐름으로 진다.
어떻게 쓰나	두 상을 한 셀 안에서 클럭 하나로 만들고, 겹침이 가장 작은 빠른 코너에서 시뮬레이션하고, 두 상을 같이 돌려서 지연 변동을 함께 겪게 한다.

실제 코드는 이렇게 생겼다

```
// Two-phase non-overlapping clock. The feedback is what guarantees
// the gap: neither output can rise until the other has fallen.
module clk_2ph (input clk, output logic p1, p1d, p2, p2d);
    wire n1 = ~(clk & p2);          // cross-coupled NAND pair
    wire n2 = ~((~clk) & p1);
    assign p1 = delay_chain(n1);     // the chain sets the dead time
    assign p2 = delay_chain(n2);
    assign p1d = delay_small(p1);    // 'delayed' phases for BOTTOM-PLATE sampling:
    assign p2d = delay_small(p2);    // the fixed-plate switch opens FIRST
endmodule
// Verify at the FAST corner -- that is where the dead time is smallest and
// where overlap, if it exists at all, appears.
```

여기서 틀린다. 상 발생기만 시뮬레이션하는 것은 아무것도 증명하지 못한다. 중요한 겹침은 배선을 마친 뒤 진짜 부하를 단 **스위치**에서의 겹침이다. 한 상은 칩을 가로질러 돌리고 다른 상은 가까이서 돌리는 레이아웃은 회로도가 주던 보장을 이미 깨뜨린 것이다.

T10.2 증폭기가 클럭 반 주기 안에 해내야 하는 일

유도 — 연산증폭기 하나에 걸린 두 요구, 그리고 왜 서로 싸우는가

	이렇게 된다	왜냐하면
1	옴김 상에서 증폭기는 출력을 $\frac{1}{2}$ LSB 안으로 정착시켜야 한다.	그보다 못하면 전하가 다 안 옮겨진 것인데 — 그 오차는 이득 같은 것이라 잡음이 아니라 INL 로 나타난다.
2	정착은 지수적이라, N 비트 정확도에 닿는 데 시정수가 $\ln(2^N)$ 개 든다.	12 비트에 8.32 개, 16 비트에 11.1 개 — 요구는 분해능을 따라 로그로만 늘어난다. 그것이 좋은 소식이다.
3	쓸 수 있는 시간은 클럭 반 주기이므로 닫힌 고리 대역폭이 적어도 $\ln(2^N)/(\pi \cdot T_S)$ 여야 하고 — 열린 고리 단위이득 주파수는 그것을 되먹임 계수 β 로 나눈 것이다.	β 는 출력에서 되먹여지는 몫인데, 이득단에서 보통 1/2 에서 1/4 다. β 가 작으면 요구가 그만큼 곱해진다.
4	따로, 정적 정확도에는 고리 이득이 든다. 남은 오차가 $1/(A\beta)$ 이므로 A 가 $2^{N+1}/\beta$ 를 넘어야 한다.	$\beta = 1$ 에서 12 비트에 78.3 dB, 16 비트에 102 dB — 그리고 이것은 분해능을 따라 지수로 늘어난다. 이것이 나쁜 소식이고, 본디 이득이 낮은 공정에서 고분해능 SC 설계가 어려운 까닭이다.
5	대역폭은 짧은 채널과 많은 전류를 원하고, 이득은 긴 채널과 캐스코드를 원하는데 캐스코드는 헤드룸을 먹는다.	어느 공정 마디에서나 두 요구가 반대로 당기고, 미세화와 함께 그 틈이 벌어졌다 — 예전 같으면 더 나은 증폭기를 썼을 설계에 요즘은 보정과 오차 상쇄가 나오는 까닭이다.

표 2. 분해능별로 증폭기가 내놓아야 하는 것

비트	정착에 드는 시정수	최소 DC 이득 ($\beta = 1$)	최소 DC 이득 ($\beta = \frac{1}{4}$)
10	6.93	66.23 dB	78.27 dB
12	8.32	78.27 dB	90.31 dB
14	9.7	90.31 dB	102.4 dB
16	11.1	102.4 dB	114.4 dB

두 칸이 늘어나는 모양을 견줘 보라. 10 비트에서 16 비트로 가면 정착 요구는 1.6 배가 되고 이득 요구는 64 배가 된다. **속도는 분해능을 따라 순하게 커지고, 정적 정확도는 그렇지 않다.** 그 비대칭 때문에 고분해능 쪽 시장이, 더 나은 증폭기를 요구하는 구조 대신 증폭기의 모자람을 재서 디지털로 바로잡는 구조로 옮겨 갔다.

상관 이중 표본화 (correlated double sampling)

한 상에서 증폭기의 오차 — 오프셋과 저주파(플리커) 잡음 — 를 표본으로 뜨고, 신호 상 동안 그 표본을 뺀다. 가까이 붙은 두 표본 사이에서 그 오차는 상관돼 있으므로, 빼기가 그 대부분을 걷어낸다.

값은 이것이다. 두 표본의 상관 없는(열) 잡음은 전력으로 더해지므로 **광대역 잡음이 $\sqrt{2}$ 만큼 나빠지고**, 오프셋과 플리커는 훨씬 좋아진다. 또 표본화율 위의 잡음을 대역 안으로 접어 넣으므로(T5), 증폭기의 대역폭을 일부러 묶어 두어야 한다.

어디에 쓰나	정밀 SC 이득단, 이미지센서 읽기(거기서는 보편적이다), 입력이 DC 근처이고 증폭기에 플리커 문제가 있는 모든 회로(T4).
언제 쓰나	오프셋과 $1/f$ 가 지배할 때 — 곧 신호 대역이 플리커 모퉁이 아래까지 닿을 때.
어떻게 쓰나	상 하나와 커패시터 하나를 더한다. 그리고 잡음 예산을 다시 본다. 설계가 이미 열잡음에 묶여 있었다면 열잡음의 $\sqrt{2}$ 벌금이 이득을 지워 버릴 수 있다.

실제 코드는 이렇게 생겼다

```
* The measurement that shows whether CDS helped -- run BOTH.
.noise v(out) VIN dec 20 1 100MEG      $ with CDS disabled
* ... then with CDS enabled, and compare the INTEGRATED noise over the signal band.
* Reporting only the 1/f improvement while the broadband floor rose by 3 dB is
* the kind of half-truth that gets found in the customer's lab.
```

여기서 틀린다. CDS 는 두 표본 사이에서 상관된 것을 걷어낸다. 표본화율에 견줄 만한 주파수의 잡음은 상관돼 있지 않아서 걸히지 않는다 — 오히려 접혀 들어온다. 오차를 쓰기 한참 전에 표본으로 뜨는 설계는 값은 다 치르고 이득은 거의 못 본다.

T10.3 잡음: kT/C 를 한 번만 치르지 않는다

표 3. 두 상 회로에서의 kT/C

C	kT/C (표본 하나)	두 상 ($\sqrt{2}$)	풀스케일 1 V 에서의 비트
0.5 pF	91 μ V	129 μ V	11.1
1 pF	64.4 μ V	91 μ V	11.6
4 pF	32.2 μ V	45.5 μ V	12.6

스위치드 커패시터 단은 한 주기에 적어도 두 번 표본을 뜬다 — 입력 커패시터에서 한 번, 되먹임 커패시터에서 한 번 — 그래서 잡음 전력이 더해진다. $\sqrt{2}$ 는 **최솟값이다**. 커패시터가 여럿이고 기준 경로가 있는 파이프라인 단은 2배에 쉽게 닿는다. T4 의 표본 하나짜리 kT/C 를 가져다 거기에 맞춰 치수를 잡는 것은 잡음 전력에서 두 배의 오차이고, 그것은 반 비트다 — 그리고 반 비트는 테이프아웃 뒤에 발견되기 딱 좋은 종류의 모자람이다.

T10.4 월요일에 이걸로 무엇을 하나

표 4. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
등가저항	디지털 공정에서의 필터와 바이어스 설계	저항이 필요하기는 한지
두상클럭 · 바닥판표본화	모든 SC 회로, 회로도 단계에서	차지 인젝션이 오프셋인지 왜곡인지
정착시간 · 유한이득오차	연산증폭기 규격	대역폭과 DC 이득 — 그리고 그 공정이 그 이득을 댈 수 있는지
상관이중표본화 · 자동 영점	DC 근처의 정밀 경로	플리커 문제를 회로에서 푸는지 구조에서 푸는지
kTC잡음	커패시터 치수	교과서 공식에 빠져 있는 $\sqrt{2}$ 라는 인자

가지고 갈 수들. 16 비트에 닿으려면 정착 시정수가 11.1 개 드는데 — 12 비트보다 1.33배 더일 뿐이다 — DC 이득은 78.27 dB 에 맞서 102.4 dB 가 들고, 그것은 16배 더다. **분해능은 시간으로는 싸고 이득으로는 비싸다.** 이 바닥의 현대 구조는 전부 그 한 문장에 대한 대답이다.

다음 장은 이 회로들이 전부 가지고 있다고 가정하는 양 — 전원이나 온도를 따라 움직이지 않는 안정한 바이어스 전류 — 을 가져다, 그것 하나를 만드는 데 무엇이 드는지를 보인다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

스위치드 커패시터 분야는 오래됐고 기본 회로들은 특허가 진작 풀렸지만, 두 갈래가 계속 청구항을 낳는다. **오차 상쇄 방식**(상관 이중 표본화, 자동 영점, 초퍼 안정화 증폭기, 그리고 한 상에서 오차를 재어 다음 상에서 빼는 모든 변종)과 **증폭기 대체물** — 링 증폭기, 동적 증폭기, 비교기 기반 스위치드 커패시터 회로 — 인데, 뒤엣것의 요점은 현대 공정이 교과서가 가정하는 고이득 연산증폭기를 못 짓는다는 것이다. 두 갈래가 같은 물음에 답한다. 연산증폭기가 모자랄 때 무엇을 하는가?

T11. 바이어스와 기준 — 움직이지 않는 수 하나 만들기

이 책의 나머지 전부가 안정한 전류나 전압이 있다고 가정한다. 하나 만드는 데는 쓰는 것보다 생각이 더 든다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 온도 · 지수함수 · 로그 · 미분 · 문턱전압 · 트랜스컨덕턴스 · 출력저항 · 정합 · 플리커잡음 · 열잡음 · 등가저항

이 장에서 새로 나오는 것

PTAT · CTAT · 밴드갭기준 · 곡률보정 · 온도계수 · 시동회로 · 퇴화동작점 · 전원제거비 · 부하조정 · 일정gm바이어스 · 기준전류 · 트리밍

칩에는 정확한 절대량이 없다. 저항은 $\pm 20\%$, 커패시터는 $\pm 10\%$, 문턱전압은 수십 밀리볼트씩 흔들리고, 모든 것이 온도를 따라 움직인다. 그런데 데이터 변환기에는 LSB의 몇 분의 일까지 안정한 기준이 필요하고, 증폭기에는 차량용 온도 범위에서 두 배가 되지 않는 바이어스 전류가 필요하다. 이 장은 그 안정성이 어디서 오는지를 다룬다 — 그리고 답은 서로 반대인 두 온도 의존성을 맞추어 지워서 지어낸다는 것인데, 그래서 그것은 결코 공짜가 아니고 결코 완전하지 않다.

T11.1 공정이 공짜로 주는 두 가지, 둘 다 온도에 달려 있다

CTAT (complementary to absolute temperature) — 온도를 따라 내려가는 양. 전류를 고정한 바이폴라 트랜지스터의 베이스-에미터 전압이 대표다. 실온에서 $V_{BE} \approx 0.7\text{ V}$ 이고 기울기가 $-2\text{ mV}/^\circ\text{C}$ 쯤인데, 선형이 아니다 — 거기서 남는 곡률이 모든 맨 밴드갭의 바닥이다.

PTAT (proportional to absolute temperature) — 절대온도에 비례해 올라가는 양. 전류 밀도 비가 N 인 두 V_{BE} 의 차가 정확히 PTAT 다. $\Delta V_{BE} = (kT/q) \cdot \ln N$ 이다. 그것은 지어진 방식상 T 에 선형이고, 공정이 아니라 비 하나와 물리 상수에만 달려 있다.

유도 — 왜 둘을 알맞은 비로 더하면 $\sim 1.2\text{ V}$ 가 나오는가

	이렇게 된다	왜냐하면
1	$V_{BE}(T)$ 는 $-2\text{ mV}/^\circ\text{C}$ 쯤으로 내려가고, $\Delta V_{BE} = (kT/q) \cdot \ln N$ 은 $+0.086 \cdot \ln N\text{ mV}/^\circ\text{C}$ 로 올라간다.	$N = 8$ 이면 $+0.179\text{ mV}/^\circ\text{C}$ 인데, 혼자서는 턱없이 작다 — 그래서 키워야 한다.
2	PTAT 항을 $R1/R3$ 으로 키워 더한다. $V_{ref} = V_{BE} + (R1/R3) \cdot \Delta V_{BE}$ 다.	그 이득이 저항 비다 — 공정이 주는 유일하게 정확한 것(T10)이고, 그래서 이 구조가 살아남았다.
3	관심 있는 온도에서 두 기울기가 지워지도록 비를 고른다.	식 하나, 미지수 하나. V_{BE} 는 굵었고 PTAT 항은 곧아서, 지워짐은 한 온도에서만 정확하다.
4	기울기가 지워지는 값은 0 K 로 외삽한 실리콘 밴드갭이다 — 1.2 V 쯤이다. 당신이 고르는 것이 아니라 그것이 스스로 정해진다.	아래에서 켜진 것: 가장 좋은 온도계수가 $V_{ref} = 1.1652\text{ V}$ 에 떨어진다. 그 수를 설계한 사람은 없다 — 지워짐이 마침 거기서 일어날 뿐이다.

T11.2 측정: 뽀족한 최적점, 그리고 그 바닥을 정하는 것

표 5. PTAT 이득에 대한 켄 온도계수 (ngspice, $-40 \cdots 125^\circ\text{C}$, $N = 8$, 상자법)

R3	V_ref	TC	
6 kΩ	1.5678 V	817.8 ppm/°C	
8 kΩ	1.3243 V	385.6 ppm/°C	
9 kΩ	1.2425 V	202.5 ppm/°C	
9.8 kΩ	1.1889 V	68.8 ppm/°C	
10 kΩ	1.1768 V	37 ppm/°C	
10.2 kΩ	1.1652 V	12.7 ppm/°C	최적
10.3 kΩ	1.1595 V	15.5 ppm/°C	
11 kΩ	1.1228 V	113.7 ppm/°C	
12 kΩ	1.0776 V	251.4 ppm/°C	
14 kΩ	1.0061 V	494.5 ppm/°C	

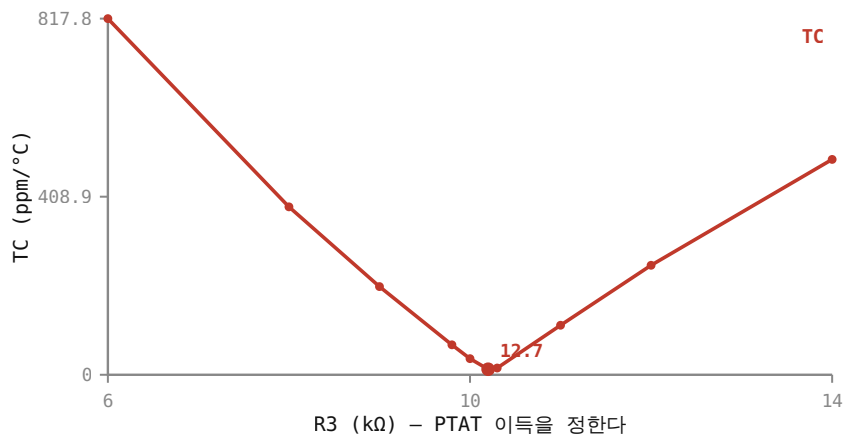


그림 1. 최적점이 T7의 PLL 고리대역폭 최적점과 달리 **뽀족하다**. 비가 2 % 틀리면 온도계수가 세 배가 된다. 그것이 지워짐이 정하는 최적점과 서로 맞서는 두 값의 합이 정하는 최적점의 차이다 — 그리고 기준은 트리밍하고 고리 필터는 안 하는 까닭이 그것이다.

예제 1. 그 비를 얼마나 정확히 맞춰야 하나?

주어진 것. 켄 최적점은 $R3 = 10.2 \text{ k}\Omega$ 에서 $12.7 \text{ ppm}/^\circ\text{C}$ 다. 이웃한 점들: $10 \text{ k}\Omega \rightarrow 37 \text{ ppm}/^\circ\text{C}$ 이고 $10.3 \text{ k}\Omega \rightarrow 15.5 \text{ ppm}/^\circ\text{C}$ 다.

푸는 법. 각 점에서 비의 상대 오차를 셈해 온도계수가 나빠진 정도와 견준다.

답. 2 %의 오차가 온도계수를 2.91배로 만든다. **비의 정확도가 1 퍼센트 아래여야 하고**, 그것은 정합 문제(T12)이며 보통은 트리밍이다.

여기서 틀린다. 시뮬레이션에서 찾은 최적점이 실리콘에서도 최적점이라고 여기는 것. 지워짐은 V_{BE} 의 실제 기울기에 달려 있는데 그것은 제 나름의 퍼짐을 갖는 공정 변수다 — 그래서 최적점이 **로트마다 움직인다**. 양산 기준을 시험에서 트리밍하는 까닭이 바로 이것이고, 트리밍할 수 없는 설계는 공칭이 아니라 최악의 최적점 어긋남을 두고 설계해야 하는 까닭도 이것이다.

표 6. 같은 손잡이를 반대쪽에서: $R3 = 10 \text{ k}\Omega$ 로 고정하고 면적비 N

N	V _{ref}	TC
4	0.9773 V	602.5 ppm/°C
8	1.1768 V	37 ppm/°C
16	1.3732 V	484.7 ppm/°C
32	1.5678 V	817.8 ppm/°C

두 표 다 같은 물리량 — PTAT 이득 — 을 조율하고, 둘 다 같은 뾰족한 최적점을 보인다. 실무에서의 차이는 값이다. N 은 $\ln N$ 으로 들어오므로 면적비를 두 배로 해도 이득은 0.69 단위밖에 못 사는데, 저항 비는 선형이고 공짜다. 그래서 실제 설계는 적당한 N (8 이 거의 보편적이다) 을 쓰고 미세 조정은 저항으로 한다. 안 그러면 저항이 거저 주는 비를 얻자고 면적이 어마어마해진다.

곡률 (curvature) — 맨 구조가 넘지 못하는 바닥

젠 최선이 12.7 ppm/°C 이고, 비를 어떻게 골라도 그보다 낮지 못하다. 까닭은 구조적이다. $V_{BE}(T)$ 에는 $T \cdot \ln T$ 항이 들어 있는데 PTAT 보정은 T 에 정확히 선형이라, 둘의 차가 모든 곳에서 0 일 수가 없다. 남는 것은 활처럼 휜 것이다 — 기준이 두 온도에서 같고 그 사이에서는 어긋난다.

곡률 보정 기준은 알맞은 곡률을 갖는 셋째 항을 더해(저항 자신의 온도계수에서, PTAT² 전류에서, 또는 전류의 온도 의존성이 다른 두 V_{BE} 의 차에서) 한 자릿수 ppm/°C 에 닿는다. 그 방식들은 하나하나가 면적과 새로운 민감도를 치르고, 그래서 수십 ppm 으로 충분한 데에는 아직도 맨 구조가 나간다.

어디에 쓰나	12 비트 이상 변환기를 먹이는 모든 기준, 그리고 차량용이나 산업용 온도 범위로 규정된 모든 시스템.
언제 쓰나	고르기 전에 동작 범위에 걸친 기준의 흐름을 변환기의 LSB 와 견준다. 12 비트에 풀스케일 1 V 면 LSB 가 244 μV 인데, 12.7 ppm/°C 기준은 165 °C 에 걸쳐 2,515 μV 를 흐른다 — 10 LSB 쯤이다 . 변환기를 한 온도에서 규정하고 기준을 트리밍하는 까닭이 그것이다.
어떻게 쓰나	TC 를 상자로 재고(국소 기울기가 아니라 범위에 걸친 최대-최소), 어느 범위에서 잤는지 적고, 트리밍한 것인지 밝힌다. 그 셋이 있어야 두 기준을 견줄 수 있다. 없으면 아무것으로도 못 견준다.

실제 코드는 이렇게 생겼다

```
* Box TC, which is what a datasheet means, and the local slope,
* which is what a naive .measure gives. They differ by 2-5x.
.dc TEMP -40 125 5
.measure dc vmax MAX v(out)
.measure dc vmin MIN v(out)
.measure dc vavg AVG v(out)
.measure dc tc_box param='(vmax-vmin)/vavg/165*1e6' $ ppm/degC -- the honest one
* Reporting a slope taken at 27 C as 'the TC' is the standard way to make a
* reference look 3x better than it is.
```

여기서 틀린다. 범위를 밝히지 않은 온도계수는 규격이 아니다. -40...125 °C 에 걸친 상자법과 27 °C 에서의 국소 기울기는 같은 회로에서 다섯 배까지 차이 날 수 있고, 낙관적인 데이터시트에 실리는 것은 두 번째 수다.

T11.3 온도와는 아무 상관 없는 실패 방식

시동 회로 (start-up) 와 퇴화 동작점

밴드갭의 고리에는 안정한 해가 둘 있다. 뜻한 해와, 모든 전류가 0 인 해다. 0 은 완벽하게 자기일관적이다 — 전류가 없으면 V_{BE} 가 없고, 그러면 전류가 없다 — 그리고 고리 안의 그 무엇도 그것을 쓸모 있는 해와 가려내지 못한다.

시동 회로는 기준이 0 근처일 때 작은 전류를 밀어 넣고 그다음 **비켜선다**. 두 쪽이 다 중요하다. 계속 흐르는 시동 회로는 제 온도 의존성을 출력에 얹고, 너무 약한 시동 회로는 느린 코너에서 일부 부품을 죽은 채로 남긴다.

어디에 쓰나	모든 밴드갭, 모든 일정 g_m 바이어스 셀, 바이어스를 제 자신에게서 얻는 모든 회로.
언제 쓰나	고를 수 있는 것이 아니고 공칭 코너 점검거리도 아니다. 0 상태에는 느린 전원 상승으로, 브라운아웃으로, 또는 설계자가 생각 못 한 어떤 순서로든 달는다.
어떻게 쓰나	동작점이 아니라 전원 상승 을 모든 코너에서, 아주 느린 것까지 여러 상승 속도로 시뮬레이션한다. 그다음 초기 조건으로 출력을 0 으로 밀어 놓고 되살아나는지 본다.

실제 코드는 이렇게 생겼다

```
* The two simulations that must both pass. The .op alone proves nothing --
* it simply finds A solution, and a degenerate circuit HAS a solution.
.tran 1u 500u                                $ 1) slow supply ramp
Vcc vcc 0 PWL(0 0 400u 1.8)                  $ 400 us ramp -- deliberately slow
.measure tran vref_final FIND v(ref) AT=480u

.ic v(ref)=0                                  $ 2) forced into the dead state
.tran 1u 100u
.measure tran vref_rec FIND v(ref) AT=90u
* Both must land at the design value. A circuit that passes .op and fails these
* is a part that works on the bench and fails in the field at cold start.
```

여기서 틀린다. 가장 흔한 시동 버그는 회로가 없는 것이 아니라 **빠른 코너에서의 세기다**. 느린 코너에 맞춰 치수를 잡은 시동 소자가 빠른 코너에서는 아직도 전류를 밀어 넣어, 공정을 따라 움직이는 방식으로 기준을 몇 밀리볼트 옮겨 놓을 수 있다. 고리가 서고 나면 시동 전류가 사실상 0 인지를 — 모든 코너에서 — 확인해라.

T11.4 기준에는 규격이 하나가 아니라 셋이다

표 7. 기준이 보장해야 하는 것

규격	무엇인가	어떻게 지나	무엇이 정하나
온도계 수	동작 범위에 걸친 흐름	주위 온도를 따라 바뀌는 이득 오차	PTAT/CTAT 균형, 곡률, 트리밍
PSRR	전원 잡음의 물리침, 주파수에 따라	디지털 쪽이 바쁠 때 변환기의 잡음 바닥이 올라간다	증폭기의 전원 제거 와 레이아웃
부하 조정	부하 전류에 따른 출력 이동	변환기가 표본을 뜰 때 기준이 움직인다 — 코드에 따라 달라지는 오차	출력 임피던스와 버퍼이지 코어가 아니다
잡음	대역 안의 적분 출력 잡음	변환기의 잡음 바닥에 곧장 더해진다(T4)	저항의 열잡음과 증폭기의 1/f

실제로 난 일

디지털 쪽 통합 담당자를 놀라게 하는 규격은 부하 조정이다. SAR 변환기의 기준은 DC 부하가 아니다. 비트 시도마다 전하 펄스를 끌어가고, 기준은 시도 사이에 변환의 온전한 정확도까지 정착해야 한다(T10의 정착 논증율, 주기마다 바뀌는 부하에 건 것이다). DC 규격을 완벽하게 맞추는 기준도 여전히 **코드에 따라 달라지는 INL** 을 갖는 변환기를 낳을 수 있다. 요구되는 전하가 시험 중인 코드에 달려 있기 때문이다. **고침은 시도당 전하에 맞춰 치수를 잡은 국소 디커플링 커패시터**이고 — 그 커패시터는 종종 변환기 자신의 어레이보다 크다.

일정 g_m 바이어스 (constant- g_m bias)

아날로그 규격의 대부분은 — 대역폭, 잡음, 이득 — 전류가 아니라 g_m 에 달려 있다. 그러니 전류를 일정하게 붙드는 바이어스는 엉뚱한 것을 붙드는 것이다. $g_m = \sqrt{2\mu C_{ox}(W/L)I}$ 는 여전히 이동도를 따라 움직이고, 이동도는 온도와 공정을 따라 움직인다.

일정 g_m 셀은 소스 퇴화 저항이 그것을 정의하게 강제해서 $g_m = 1/(R \cdot (1 - 1/\sqrt{K}))$ 로 놓는다 — 그러면 g_m 이 이동도가 아니라 **저항**을 따라간다. 의존성을 없애는 것이 아니라, 저항 종류를 골라 변동을 고를 수 있는 부품으로 옮기는 것이다.

어디에 쓰나	증폭기, 비교기, 출력 드라이버의 바이어스 — 대역폭이 코너에 걸쳐 안정해야 하는 모든 것.
언제 쓰나	대역폭이나 잡음이 공정과 온도에 걸쳐 버텨야 할 때. 혼성신호 IP의 신호 경로 바이어스에서 기본값이다.
어떻게 쓰나	저항 종류를 일부러 고르고(폴리 저항과 확산 저항은 온도계수가 아주 다르다), 이 셀에도 퇴화한 0 상태가 있다는 것을 잊지 마라 — 제 시동 회로가 필요하다.

실제 코드는 이렇게 생겼다

```
* What the bias must actually hold, checked across corners.
* Constant CURRENT is the wrong target; constant gm is the right one.
.dc TEMP -40 125 5
.measure dc gm_min MIN @m.xamp.m1[gm]
.measure dc gm_max MAX @m.xamp.m1[gm]
.measure dc gm_spread param='(gm_max-gm_min)/gm_min*100' $ percent
* A constant-current bias typically gives 30-40% here; constant-gm, under 10%.
```

여기서 틀린다. 일정 g_m 은 g_m 이 $1/R$ 을 따라가게 만들므로, 이제 회로의 대역폭이 저항의 온도계수에 달린다 — 확산 저항이면 그것이 수천 ppm/°C 일 수 있다. 저항을 고르는 것은 레이아웃 세부가 아니라 설계의 일부다.

T11.5 월요일에 이걸로 무엇을 하나

표 8. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
PTAT · CTAT · 밴드갭기준	모든 변환기나 정밀 블록	기준 구조, 그리고 트리밍이 필요한지
온도계수 · 곡률보정	온도에 걸친 시스템 규격	수십 ppm 으로 충분한지 곡률 보정이 드는지
시동회로 · 퇴화동작점	모든 자기 바이어스 셀	차가운 시동에서 부품이 살아나는지 — 모든 코너에서
전원제거비 · 부하조정	디지털 쪽과의 통합	기준에 드는 디커플링, 그리고 그 값을 누가 치르는지
일정gm바이어스	신호 경로 바이어스	대역폭이 코너에 걸쳐 버티는지, 그리고 어느 저항 종류인지

가지고 갈 켄 결과. 이 시뮬레이션에서 맨 밴드갭의 가장 좋은 온도계수는 $V_{ref} = 1.1652 \text{ V}$ 에서 $12.7 \text{ ppm/}^\circ\text{C}$ 이고, 최적점은 **뽀족하다** — 비가 2 % 틀리면 흐름이 2.91배가 된다. 이 장의 나머지는 전부 그 뽀족함에서 나온다. 기준을 트리밍하는 까닭이고, 정합(다음 장)이 한계를 긋는 규율인 까닭이며, 아무도 고르지 않은 값 — 1.2 V — 이 현장의 모든 데이터시트에 찍혀 있는 까닭이다.

다음 장은 이 장이 기대 온 가정 — 똑같이 그린 두 소자가 똑같다는 것 — 을 가져다 분포로 바꿔 놓는다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

기준은 오래된 분야인데 특허가 살아 흐르고, 세 자리에 몰린다. **곡률 보정**(맨 밴드갭이 남기는 V_{BE} 비선형성을 지우는 모든 방식), **저전압 동작**(고전적 구조가 못 하는 1.2 V 아래에서 도는 전류 모드와 저항 분압 밴드갭), 그리고 **트리밍과 보정**(한 번의 측정에서 곡률을 미루어 내는 1점 트리밍 방식). 첫째와 셋째는 같은 생각을 두 번 본 것이다. 물리적 오차를 받아들이고 더 싼 데서 걷어낸다 — T6 의 무늬다.

T12. 정합 — 똑같은 소자 둘은 똑같지 않다

칩 위에서 정확한 것은 비 하나뿐이고, 그것조차 분포다. 정밀한 것은 전부 그 폭을 아는 데 얹혀 있다.

읽기 전에 알아야 하는 것

전압 · 전류 · 확률 · 평균 · 분산 · 표준편차 · 정규분포 · 로그 · 제곱근 · 문턱전압 · 트랜스컨덕턴스 · 정합 · 최소동작 전압 · INL · DNL · PTAT · 온도계수

이 장에서 새로 나오는 것

펠그롬법칙 · 랜덤불일치 · 계통불일치 · 공통중심 · 더미소자 · 일치도면적 · 수율시그마 · 중요도표본 · 오프셋 · 트림 · 레이아웃의존효과

똑같이 그려 나란히 같은 방향으로 놓은 트랜지스터 둘도 서로 다르다. 도핑 원자는 날날이고 아무렇게나 떨어진 다. 게이트 가장자리는 완벽하게 곧지 않다. 산화막 두께는 원자 하나씩 달라진다. 그래서 생기는 변수의 차는 확률 변수이고, 정밀 아날로그 설계는 **그 표준편차를 알고 그 꼬리를 예산에 넣는** 규율이다.

T12.1 펠그롬 법칙, 그리고 왜 제곱근인가

펠그롬 법칙 (Pelgrom's law)

나란히 놓여 똑같이 그려진 두 소자에 대해, 변수 차의 표준편차는 면적의 제곱근으로 줄어든다. $\sigma(\Delta V_{th}) = A_{VT}/\sqrt{W \cdot L}$ 이고 A_{VT} 의 단위는 $mV \cdot \mu m$ 이다 — 파운드리가 재서 내놓는 공정별 상수다(몇 $mV \cdot \mu m$ 이고, 마디마다 천천히 좋아진다).

제곱근은 경험적으로 곡선을 맞춘 것이 아니다. 불일치는 서로 독립인 미시 사건(도핑 자리, 가장자리 거칠기)을 세는 통계에서 오고, 독립 사건 N 개를 센 것의 상대 요동은 $1/\sqrt{N}$ 로 줄어든다. **면적이 그 셈이다.**

어디에 쓰나	설계 안의 모든 전류 거울, 차동쌍, 비교기, 커패시터 어레이, SRAM 셀. 수율 모형에서도 쓴다 — $V_{min}(T4)$ 이 그 꼬리인 분포가 이것이다.
언제 쓰나	정밀한 것을 그리기 전에. 요구되는 σ 가 최소 면적을 정하고, 최소 면적이 종종 블록 전체의 크기와 그 기생 용량을, 따라서 속도를 정한다.
어떻게 쓰나	규격에서 거꾸로 풀어라. 3σ 에서 오프셋 5 mV 가 허용되면 $\sigma(\Delta V_{th}) \leq 1.67$ mV 이므로 $W \cdot L \geq (A_{VT}/1.67 \text{ mV})^2$ 다. $A_{VT} = 5 \text{ mV} \cdot \mu m$ 이면 $8.96 \mu m^2$ 이고 — 소자 하나당이다.

실제 코드는 이렇게 생겼다

```
* Mismatch in a PDK is a MODEL PARAMETER, not something you add by hand.
* The foundry supplies statistical models; you enable them and run Monte Carlo:
.lib 'models/pdk.lib' mc          $ NOT the 'tt' corner -- the statistical deck
.mc 200 op                        $ 200 samples
.measure op ios param='abs(i(vd2)-i(vd1))'
* AND THEN: the corners and the mismatch are DIFFERENT analyses.
*   corner = global shift (all devices move together)
*   mismatch = local spread (devices move independently)
* Running 'tt + mismatch' and calling it worst case misses the combination that
* actually fails: slow corner AND an unlucky pair.
```

여기서 틀린다. A_{VT} 는 나란히 놓인 두 소자 사이의 차에 적용된다. 그것을 소자 하나의 절댓값에 쓰거나 다이의 반대편에 있는 소자들에 쓰는 것은 양쪽으로 다 틀렸다. 절대 퍼짐은 더 크고(전역 공정 변동이 들어 있다), 다이를 가로지르는 소자 사이의 상관은 아예 다른 모형이다.

표 9. 낸 것: 소자 면적에 대한 전류 거울 비 (ngspice 몬테카를로, 80 표본, $A_{VT} = 5 \text{ mV} \cdot \mu m$, $I_{ref} = 10 \mu A$)

$W \times L$	면적	$\sigma(\Delta V_{th})$	평균 비	$\sigma(\text{비})$
1x0.2	$0.2 \mu m^2$	11.18 mV	0.9779	14.2 %
2x0.4	$0.8 \mu m^2$	5.59 mV	0.9876	7.17 %
4x0.8	$3.2 \mu m^2$	2.8 mV	0.9934	3.61 %
8x1.6	$12.8 \mu m^2$	1.4 mV	0.9966	1.81 %

유도 — 저 표에서 펄그로를 곧장 읽어 내기

	이렇게 된다	왜냐하면
1	면적 $0.2 \mu\text{m}^2 \rightarrow \sigma(\text{비}) = 14.16 \%$ 이고, 면적 $0.8 \mu\text{m}^2 \rightarrow 7.17 \%$ 다.	면적 $\times 4$ 에 $\sigma \div 1.97$ — 제곱근 법칙이고, 가정한 것이 아니라 켜진 것이다.
2	다음 두 걸음이 그것을 되풀이한다. $\div 1.99$ 와 $\div 1.99$ 다.	같은 지수를 독립으로 세 번 확인한 것이다. 이런 측정이 $\sqrt{}$ 를 안 내면 모형이나 하니스가 틀린 것이고, 그것이 불일치 설비에 쓸 수 있는 가장 빠른 점검이다.
3	평균 비가 1 이 아니다. 소자가 커지면서 0.9779 에서 0.9966 쪽으로 오른다.	대칭인 V_{th} 오차가 비대칭인 전류 오차로 옮겨진다 — 제곱 법칙이 불록해서 비의 평균이 1 아래로 끌린다. 불일치는 퍼짐만이 아니라 평균도 치우치게 한다. 그리고 그 효과는 퍼짐이 사라져야만 사라진다.

T12.2 다른 손잡이: 오버드라이브

표 10. 켜진 것: 같은 소자, 다른 바이어스 전류 ($W/L = 2/0.4$, 80 표본)

I_{ref}	$\sigma(\text{비})$	10 μA 에 견주어
1 μA	22.1 %	3.08×
10 μA	7.17 %	1×
100 μA	2.29 %	0.32×

유도 — 왜 전류가 많으면 더 잘 맞으며, 그 값이 무엇인가

	이렇게 된다	왜냐하면
1	V_{th} 오차 δ 는 트랜스컨덕턴스를 거쳐 전류 오차를 낳는다. $\delta I = g_m \cdot \delta$ 다.	소자는 제 게이트 전압이 왜 사실상 다른지를 모른다.
2	그래서 상대 전류 불일치는 $\sigma(I)/I = (g_m/I_D) \cdot \sigma(\Delta V_{th})$ 다.	g_m/I_D 가 이야기의 전부다(T1). 암페어당 g_m 이 높은 것 — 효율적이고 저전력인 동작점 — 이 가장 안 맞는 동작점이기도 하다.
3	강반전에서는 $g_m/I_D = 2/V_{ov}$ 이므로 $\sigma(I)/I = 2\sigma(\Delta V_{th})/V_{ov}$ 다.	켄 것: 1 μA 에서 22.1 % 이고 100 μA 에서 2.29 % 다 — 전류를 100배 써서 9.65배 나아진 것이고, 이것도 $\sqrt{}$ 다($V_{ov} \propto \sqrt{I}$).
4	약반전에서는 g_m/I_D 가 $\sim 1/(nV_T) \approx 25\text{-}30 \text{ V}^{-1}$ 에서 포화하고 정합이 가장 나쁘다.	그래서 서브문턱 전류 거울은 나쁜 전류 거울이고, 초저전력 아날로그가 제 효율의 값을 면적으로 치르는 까닭이 그것이다.

저 두 표가 설계 공간의 전부다. 정확도는 **면적으로** ($\sigma \propto 1/\sqrt{WL}$) 사거나 **오버드라이브로** ($\sigma \propto 1/V_{ov}$) 사는데, 둘 다 제곱근의 성격이라 둘 다 비싸다. 정밀 아날로그의 나머지 전부 — 초핑, 자동 영점, DEM, 디지털 보정 — 는 그 둘 중 어느 것도 치르지 않으려고 있다.

T12.3 아무렇게나가 아닌 불일치

계통 불일치 (systematic mismatch) — 어느 다이에서나 똑같은 차이. 웨이퍼를 가로지르는 산화막 두께의 기울기, 가까운 전력 소자에서 오는 온도 기울기, 패키지나 이웃한 웰에서 오는 응력, 또는 그저 쌍의 양쪽이 다른 이웃에 둘러싸인 것. **표본을 여럿 모아도 평균으로 지워지지 않으므로**, 몬테카를로를 아무리 돌려도 못 찾는다.

표 11. 두 가지 불일치, 그리고 저마다 무엇에 반응하나

	무작위	계통
근원	날날의 도핑 원자, 가장자리 거칠기 — 세는 통계	기울기, 응력, 다른 이웃, 열
이렇게 커진다	$1/\sqrt{W \cdot L}$	면적을 따라 줄지 않는다 — 되레 나빠질 수 있다 . 소자가 크면 기울기를 더 넓게 걸치기 때문이다
무엇이 찾아내나	몬테카를로	레이아웃 검토, 그리고 실리콘에서의 측정
무엇이 없애나	면적, 오버드라이브, 보정	레이아웃 . 공통 중심, 더미, 같은 방향과 같은 둘레
흔한 크기	$0.8 \mu\text{m}^2$ 거울에서 7.17 % (위에서 젠 것)	수백 μm 에 걸친 기울기에서 0.1-1 %

공통 중심 배치 (common-centroid layout) 와 더미

맞춰야 하는 쌍의 소자를 저마다 조각으로 쪼개 서로 엮어서, 두 소자가 같은 질량 중심을 갖게 한다. A B B A 이거나, 2차원 기울기라면 바둑판이다. 그러면 **선형** 기울기가 두 쪽에 똑같이 걸려 1차까지 지워진다.

더미 소자는 다른 문제를 푼다. 맨 바깥의 진짜 소자는 안쪽 것들과 이웃이 달라서 식각과 응력이 다르다. 더미는 모든 진짜 소자에 똑같은 돌레를 주는데, 전기적으로는 아무 일도 안 하는 면적이 그 값이다.

어디에 쓰나	중요한 모든 정합 쌍. 차동쌍, 전류 거울 어레이, 변환기의 커패시터 어레이(T6), 밴드갭 BJT 어레이(T11).
언제 쓰나	레이아웃에서. 그리고 그것은 설계자가 지정해야 한다 — 레이아웃 기술자는 넷리스트만 보고 어느 소자가 맞아야 하는지 미루어 낼 수 없다.
어떻게 쓰나	회로도에 정합 묶음을 표시하고, 더미를 붙인 공통 중심을 요구하고, 방향을 똑같이 두고(면적 아끼자고 소자를 거울로 뒤집지 마라), 두 쪽을 대칭으로 배선하고, 쌍을 패키지 응력이 가장 큰 자리 — 보통 다이 모서리 — 에서 떼어 놓는다.

실제 코드는 이렇게 생겼다

```
// The instruction that must reach the layout engineer. A netlist
// does not carry it, so it goes in the constraint file AND the schematic.
// match_group : M1 M2 common_centroid = ABBA, dummies = 2 each side
// orientation : identical (NO mirroring -- STI stress is not symmetric)
// routing : symmetric; equal parasitic on both gates
// keep_out : > 50 um from the die seal ring and from any power device
// Verified by an LVS-with-matching check, not by inspection:
// assert centroid(M1) == centroid(M2) within 0.5 um
```

여기서 틀린다. 공통 중심은 **선형** 기울기를 지운다. 2차 기울기 — 가까운 열원이 낳는 것이 그것이다 — 는 안 지워지고, 엮어 놓으면 소자를 곡률 위로 더 넓게 퍼서 되레 나빠질 수도 있다. 열 기울기에는 답이 엮기가 아니라 거리와 등온선에 대한 방향이다.

실제로 난 일

SRAM 의 V_{min} (T4) 이 설계점 문제가 아니라 수율 문제인 까닭도 불일치다. 천만 개(10 의 7 제곱)짜리 셀 어레이는 **가장 나쁜** 셀이 돌아야 하는데, 그것이 대략 -5.5σ 사건이다. 표본 천 개짜리 보통 몬테카를로는 그 꼬리를 아예 못 본다 — 10^8 정도의 표본이 들 것이다. 그래서 양산 흐름은 **중요도 표본**이나 해석적 꼬리 방법을 쓰고, 메모리를 두고 '몬테카를로 통과, 표본 1000개' 라고 하는 보고서는 자기가 받은 물음에 답하지 않은 것이다.

T12.4 월요일에 이걸로 무엇을 하나

표 12. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
펠그롬법칙 · 일치도면적	정합 소자의 치수 잡기	최소 면적 — 그것이 다시 기생과 속도를 정한다
트랜스컨덕턴스 · 랜덤불일치	바이어스 점 고르기	효율적인 동작점을 감당할 수 있는지
계통불일치 · 공통중심 · 더미소자	설계자가 쓰는 레이아웃 제약	레이아웃 기술자에게 무엇을 말해 줘야 하는지. 넷리스트가 말해주지 않으니까
수율시그마 · 중요도표본	메모리와 모든 큰 어레이	설계가 몇 σ 를 살아남아야 하는지, 그리고 어느 방법이 거기까지 볼 수 있는지
트림 · 디지털보정	면적을 감당 못 하게 될 때	실리콘으로 치를지 시험 시간으로 치를지

가지고 갈 댄 결과. 전류 거울 불일치는 면적의 제곱근으로 줄고 — 면적을 네 번 두 배로 하는 동안 14.16 % → 7.17 % → 3.61 % → 1.81 % 다 — 전류의 제곱근으로도 준다. 1 μA 에서 22.1 % 이고 100 μA 에서 2.29 % 다. **두 손잡이 다 값이 제곱이고**, 그 사실 하나가 보정이 있는 까닭이다.

다음 장은 소자를 떠나 그것들 전부를 먹이는 회로망 — 전원 공급 회로망 — 을 다룬다. 거기서 '그것은 분포다' 라는 같은 논증이 임피던스와 시간에 관한 물음으로 다시 나타난다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

정합 자체는 특허가 안 된다 — 통계이니까 — 그런데 **면적 없이 정확도를 사는** 모든 방식은 된다. 동적 소자 정합, 초퍼와 자동 영점 오프셋 상쇄, 디지털 전경·배경 보정, 그리고 기울기를 지우는 레이아웃 기법. 무늬가 또 보인다. σ 가 $1/\sqrt{\text{면적}}$ 으로밖에 안 줄므로 면적으로 정확도를 4배 사면 실리콘이 **16배** 든다 — 그래서 정말 아날로그에서 상업적으로 재미있는 생각은 거의 전부 그 값을 안 치르는 대안이다.

T13. 전원 무결성 — 나머지 전부가 올라앉은 회로망

전원은 마디가 아니라 주파수 응답을 갖는 임피던스이고, 당신 IP의 가장 나쁜 지터는 대개 그 붕우리에서 온다.

읽기 전에 알아야 하는 것

전압 · 전류 · 저항 · 주파수 · 로그 · 특성임피던스 · RC지연 · 엘모어지연 · 위상잡음 · 전원푸싱 · 전원제거비 · 구경지터

이 장에서 새로 나오는 것

목표임피던스 · PDN · ESR · ESL · 반공진 · 감쇠 · 디캡 · 전류미분 · 전압강하 · 동시스위칭잡음 · 리플버짓

이 책의 모든 블록이 전원이 있다고 가정했다. 이 장은 그 전원이 실제로 무엇인지를 다룬다. 임피던스가 주파수에 걸쳐 백 배씩 달라지고 **아무도 설계하지 않은 주파수에 붕우리가 서는**, 커패시터와 인덕턴스의 회로망이다. 당신 IP가 끌어가는 전류는 그 임피던스를 지나며 당신 자신의 전원 위에 전압이 된다 — 그리고 그것이 일으키는 실패는 지터(T7)로, 기준 흐름(T11)으로, 또는 소프트웨어와 상관되는 타이밍 실패로 나타난다.

T13.1 규격은 전압이 아니라 임피던스다

목표 임피던스 (target impedance)

$Z_{\text{target}} = V_{\text{DD}} \cdot (\text{허용 리플}) / I_{\text{max}}$ 다. 1 V 에 리플 5 %, 과도 전류 10 A 이면 그것은 **5 mΩ** 이고 — 평균이 아니라 부하가 실제로 담고 있는 모든 주파수에서 지켜져야 한다.

이것이 두루뭉술한 요구('깨끗한 전원')를 재고 시뮬레이션하고 사인오프할 수 있는 곡선으로 바꾼다. 이 개념이 있는 까닭이 통째로 그것이다.

어디에 쓰나

보드 설계, 패키지 설계, 그리고 다이 위의 격자 — 그리고 당신 IP 의 연동 안내서가 요구사항으로 적어야 하는 수다.

언제 쓰나

패키지를 고르기 전에. 패키지의 인덕턴스가 중간 주파수 임피던스를 정하고, 그것은 나중에 보드 커패시터로 못 고친다.

어떻게 쓰나

블록의 최대 과도 전류와 그 스펙트럼을 가져다, 허용 리플을 가장 예민한 회로(보통 논리가 아니라 PLL 이나 기준)에서 정하고, 임피던스 곡선을 **봉우리에서** 그것과 견준다.

실제 코드는 이렇게 생겼다

```
# What an IP deliverable should say, so the integrator can meet it.
# INTEGRATION.md:
#   Supply: 1.0 V +/- 5%, Z_pdn <= 5 mOhm from DC to 200 MHz at the block's pins.
#   Peak transient: 10 A over 2 ns at the start of a burst (di/dt = 5 A/ns).
#   On-die decoupling included in the macro: 20 nF.
#   The PLL supply (VDDA) requires a separate regulator; sharing it with the
#   digital rail has been measured to add 0.4 ps rms jitter at 1 GHz.
# Without these four lines the integrator guesses, and the guess is usually wrong
# in the direction that makes your IP look bad.
```

여기서 틀린다. 평평한 목표는 단순화다. 진짜 요구는 부하의 전류 스펙트럼 곱하기 임피던스를, 피해자 회로가 예민한 자리에서 적분한 것이다 — 1.4 MHz 의 20 mΩ 봉우리는 고리 대역폭이 1 MHz 인 PLL(T7)에는 엄청나게 중요하고 논리 블록에는 하나도 중요하지 않다.

표 13. 이 장 PDN 의 잔 임피던스 (목표 5 mΩ)

주파수	Z	목표를 맞추나?
0.01 MHz	15.5 mΩ	아니다
0.1 MHz	1.345 mΩ	그렇다
1 MHz	5.673 mΩ	아니다
10 MHz	5.232 mΩ	아니다
100 MHz	2.111 mΩ	그렇다
1, MHz	23.07 mΩ	아니다
5, MHz	115.1 mΩ	아니다

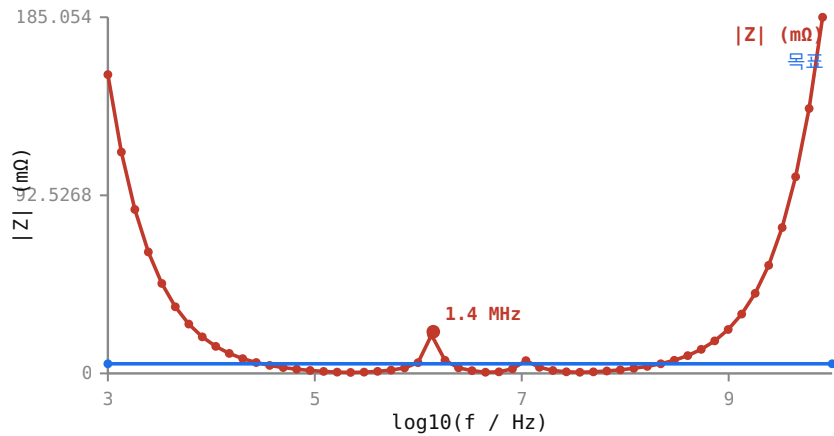


그림 2. 임피던스 윤곽. 범위의 대부분에서는 목표를 맞추고 반공진에서 어긴다 — 1.4 MHz 에서 21.66 mΩ 로 목표의 4.33배다. 평균이 아니라 봉우리가 규격이다.

T13.2 반공진: 왜 커패시터를 더하면 더 나빠질 수 있나

유도 — 봉우리가 어디서 오나

	이렇게 된다	왜냐하면
1	진짜 커패시터는 직렬 R-L-C 다. 자기공진 아래에서는 용량성이고 위에서는 유도성이다.	ESL 은 패키지와 불임새에서 온다. 100 μF 벌크 커패시터가 100 MHz 에서 아무것도 안 하는 까닭이 그것이다.
2	커패시터 둘을 병렬로 놓는다. 두 자기공진 사이에서는 하나가 이미 유도성이고 다른 하나는 아직 용량성이다.	인덕터를 커패시터와 병렬로 놓은 것은 병렬 공진 탱크이고, 병렬 탱크는 공진에서 임피던스가 높다.
3	그래서 합쳐 놓은 것은 날날이는 골이었던 자리에 봉우리를 갖는다.	이 PDN 에서 켜진 것: 1.4 MHz (21.66 mΩ) 와 11.1 MHz (6.683 mΩ) 에 봉우리가 있다.
4	봉우리의 높이는 감소 — 고리 안의 ESR — 가 정한다.	그래서 아래의 결과가 나오는데, 사람들을 놀라게 하는 것이 그것이다.

표 14. 봉우리를 낮추려고 ESR 을 키우기 (세라믹 갈래)

세라믹 ESR	봉우리 주파수	봉우리 Z	
5 mΩ	1.4 MHz	26.77 mΩ	
10 mΩ	1.4 MHz	21.66 mΩ	
30 mΩ	1.4 MHz	12.64 mΩ	
100 mΩ	1.62 MHz	7.018 mΩ	최소
300 mΩ	3.01 MHz	12.96 mΩ	

저 표를 두 번 읽어라. ESR 을 5 mΩ 에서 100 mΩ 으로 — 흔히 쓰는 품질 지표로는 커패시터를 하나하나 더 나쁘게 — 만들면 반공진 봉우리가 26.77 mΩ 에서 7.018 mΩ 으로, 3.81배 낮아진다. 최적점을 지나면 다시 오르는데, 그 갈래가 아예 안 흐르게 되기 때문이다. '가장 낮은 ESR' 이 규격이 아니라 알맞은 ESR 이 규격이다. ESR 을 다스린 커패시터가 있는 까닭이고, 잘하자고 '더 좋은' 부품으로 바꿨다가 잘 돌던 보드를 망가뜨리는 까닭이다.

T13.3 세 가지 처짐, 그리고 어느 것이 내 것인가

표 15. 전류 계단 하나가 따로 노는 처짐 셋을 낳는다

처 짐	시간 규모	무엇이 정하나	무엇으로 고치나
첫 째	1 ns 아래에서 몇 ns	다이 위 격자의 저항과 다이 위 디캡	다이 위 디캡을 더, 격자를 더 뻑뻑이 — 다이 안에서는 다른 것이 안 듣는다
둘 째	수십 ns	패키지 인덕턴스가 패키지와 다이 위 용량에 맞선 것	패키지 선택, 패키지 용량 더, 범프 더
셋 째	μ s	보드 용량과 레귤레이터의 고리 대역폭	벌크 용량, 레귤레이터 응답

di/dt, 그리고 왜 미분이 적인가

저항성 강하 I·R 은 셈하기 쉽고 좀처럼 문제가 아니다. 유도성 강하는 $L \cdot di/dt$ 인데, 현대 디지털 블록의 di/dt 는 어마어마하다. 클럭 게이팅이 풀리며 깨어나는 도메인은 나노초 안에 암페어를 움직일 수 있다.

더 나쁜 것은 di/dt 가 **주기적이고 작업부하와 상관돼 있다**는 점이다. 무거운 명령과 가벼운 명령을 번갈아 도는 고리를 돌리는 프로세서는 그 고리가 도는 속도로 전류 사각파를 낳는데 — 그것이 하필 반공진에 떨어질 수 있다. 전형적인 증상은 이렇다. 어떤 벤치마크 하나에서만 지고 나머지는 다 통과하는 기계다.

어디에 쓰나	클럭 게이팅, 전원 게이팅, 또는 몰아치는 가속기가 있는 모든 설계 — 곧 모든 SoC.
언제 쓰나	구조를 잡을 때. 넓은 데이터패스의 깨어남을 총총이 어긋내는 것은 물리 결정이 아니라 마이크로아키텍처 결정이다.
어떻게 쓰나	인에이블을 여러 주기에 걸쳐 어긋내고, 클럭 게이트 풀림을 비스듬히 올리고, 전원 도메인 전체를 한 에지에 푸는 일은 절대 하지 마라. 해석 쪽에서는 PDN 모형을 계단이 아니라 진짜 전류 프로파일로 몰아라.

실제 코드는 이렇게 생겼다

```
// The cheapest di/dt fix in RTL: stagger the ungating.
// Releasing 64 lanes on one edge is a current step; releasing them over 8 cycles
// is the same charge with 1/8 the di/dt, and usually nobody notices the latency.
always_ff @(posedge clk) begin
    if (wake) stagger <= {stagger[6:0], 1'b1};    // 8-deep shift
    else      stagger <= '0;
end
generate for (genvar i = 0; i < 64; i++)
    assign lane_en[i] = stagger[i/8];
endgenerate
// Measured effect belongs in the integration guide: peak di/dt with and without.
```

여기서 틀린다. PDN 을 전류 계단으로 시뮬레이션하면 최악의 경우가 나오고 그것은 쓸모 있지만, 공진을 가린다. 계단은 모든 주파수를 잡아드는 진폭으로 담고 있는데, 진짜 실패는 에너지를 한 주파수에 몰아 주는 주기적인 부하다. 진짜 작업부하의 전류 스펙트럼으로도 모형을 몰아라.

실제로 난 일

전원 잡음은 문 두 짝으로 당신 IP 에 닿는데, 그중 하나만이 전원이다. 첫째는 직접이다. V_{DD} 가 움직이면 VCO 의 주파수가 따라 움직인다(T7 의 푸싱). 둘째는 **기판**이다. 다이의 벌크로 돌아오는 전류가 바디 전압을 흔들고, 그것이 문턱을 흔든다(T1 의 바디 효과). 둘째 길은 어느 PDN 임피던스 그림에도 안 나타나고, 전원을 따로 조정해 주어도 물리적 거리와 가드 링이 중요한 까닭이 그것이다.

T13.4 월요일에 이걸로 무엇을 하나

표 16. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
목표임피던스	당신이 내보내는 연동 안내서	통합 담당자가 맞춰야 하는 요구 — 적어 주든지 그들의 짐작을 물려받든지
반공진 · 감쇠	보드와 패키지 커패시터 고르기	어느 커패시터인지, 그리고 '가장 낮은 ESR' 이 틀린 기준이라는 것
전류미분	RTL 과 마이크로아키텍처	깨어남을 어긋내는지 — 설계 때는 공짜이고 나중에는 비싼 고침
디커플	플로어플랜	다이 위 용량이 얼마나 들어가는지, 그리고 그 누설의 값
동시시위칭잡음	I/O 설계	출력 몇 개가 동시에 뒤집혀도 되는지

가지고 갈 셈한 결과. 이 PDN 은 대부분의 주파수에서 5 mΩ 목표를 맞추고, 어느 부품도 혼자서는 갖지 않는 1.4 MHz 반공진에서 4.33배로 놓친다. 그리고 세라믹 갈래의 ESR 을 5 mΩ 에서 100 mΩ 으로 올리면 그 봉우리가 3.81배 **낮아진다** — 부품을 더 나쁘게 만드는 것이 시스템을 더 좋게 만드는, 이 책의 유일한 자리다.

다음 장은 가장 긴 시간 규모를 다룬다. 이 회로들이 십 년을 돌고 나면 어떻게 되는가다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

전원 공급은 보드와 패키지의 규율이지만, IP 쪽 청구항도 실제로 있고 전부 같은 꼴이다. **전원을 깨끗하게 만드는 대신 블록을 둔감하게 만든다.** 예민한 블록 결의 국소 레귤레이터, 차동 및 전원 기준 신호전달, 출력이 V_{DD} 를 따라가지 않는 바이어스 방식, 그리고 전원 변조에 면역인 클럭 분배. 그 하나하나가 PDN 임피던스를 같은 배수로 낮추는 것보다 싸다.

T14. 신뢰성 — 십 년 뒤 회로는 어떤 꼴인가

이 책의 모든 변수가 흐른다. 시각 0 에서 사인오프한 설계는 엉뚱한 날짜로 사인오프한 것이다.

읽기 전에 알아야 하는 것

전압 · 전류 · 온도 · 지수함수 · 로그 · 확률 · 정규분포 · 문턱전압 · 트랜스컨덕턴스 · 슬랙 · 정착시간 · 펄그로법칙 · 오프셋 · 최소동작전압 · 전압강하

이 장에서 새로 나오는 것

BTI · HCI · TDDB · 일렉트로마이그레이션 · 아레니우스 · 가속계수 · 활성화에너지 · 블랙식 · 와이블 · 번인 · 노화인 지설계 · 수명목표

만들어진 날에 규격을 맞추는 트랜지스터는 십 년 뒤에는 못 맞춘다. 문턱은 옮겨 가고, 산화막은 무너지고, 금속 원자는 움직인다. 이 가운데 어느 것도 결함이 아니다. 재료가 늙는 보통의 일이고, 파운드리가 특성화해 모형으로 준다. 그리고 그것을 **예산에 넣는 것이 당신 몫이다** — 시각 0 의 타이밍 사인오프는 아직 출하되지도 않은 칩에 대한 사인오프이기 때문이다.

T14.1 네 가지 기구, 그리고 저마다 무엇을 먹는가

표 17. 예산에 넣어야 하는 기구들

기구	물리적으로 무슨 일이 나나	무엇을 깎나	언제 가장 나쁜가
BTI (bias temperature instability)	게이트 바이어스 아래에서 산화막 경계면에 전하가 갇힌다	$ V_{th} $ 가 올라 → 속도가 떨어지고 오프셋이 흐른다	높은 온도, 높은 V_{GS} , DC 바이어스(회복이 없음)
HCI (hot carrier injection)	기운 센 운반자가 드레인 쪽 산화막을 상하게 한다	구동 전류가 떨어지고 불일치가 커진다	뒤집히는 동안 높은 V_{DS} — 빠른 에지, 높은 활동도
TDDB (time-dependent dielectric breakdown)	게이트 산화막이 끝내 전류를 흘리고, 영영 그렇다	소자 자체 — 흐름이 아니라 딱딱한 고장 이다	높은 전계, 얇은 산화막, 큰 전체 게이트 면적
일렉트로마이그레이션	전자가 운동량을 넘겨 금속 원자를 옮긴다	배선이 끊기거나 단락된다 — 이것도 딱딱한 고장이다	높은 전류 밀도, DC(한 방향) 전류, 높은 온도

이 가운데 둘은 **흐름**이고 둘은 **고장**인데, 둘은 셈법이 다르다. 흐름은 여유로 예산 잡는다. 깎인 라이브러리로 타이밍을 사인오프하고 그 면적 값을 받아들인다. 고장은 확률로 예산 잡는다. 수명 목표 전에 지는 부품의 몫을 셈해 허용된 FIT 율과 견준다. 둘을 뒤섞는 것 — 이를테면 TDDB 를 ' V_{th} 흐름이 좀 더 있는 것' 으로 다루는 것 — 은 타이밍은 맞추고 인증에는 지는 설계를 낳는다.

T14.2 가속: 1000시간 시험이 어떻게 십 년을 예측하나

아레니우스 가속 (Arrhenius acceleration)

짧은 속도는 $\exp(-E_a/kT)$ 를 따르므로, 온도를 올리면 속도가 셈할 수 있는 배수로 곱해진다. $AF = \exp[(E_a/k)(1/T_{use} - 1/T_{stress})]$ 다.

$E_a = 0.9$ eV 로 85 °C 대신 125 °C 에서 물면 **18.7배 빠르다** — 1000시간 오븐 시험이 현장에서의 여러 해를 대신하는 방식이 그것이다. 같은 셈을 거꾸로 돌린 것이 낯익은 어림이다. 10 °C 를 식힐 때마다 수명이 대략 2.08배다.

어디에 쓰나 인증 계획, 그리고 접합부 온도를 두고 열 담당자와 하는 다툼.

언제 쓰나 수명을 주장할 때마다. 온도 없는 수명은 규격이 아니다.

어떻게 쓰나 E_a 를 그 기구에 대한 파운드리 모형에서 가져오고 — **두루뭉술한 0.7 eV 가 아니다** — 결과 옆에 같이 적는다. 답이 거기에 세게 달려 있기 때문이다.

실제 코드는 이렇게 생겼다

```
# The same 1000-hour test, read with three different activation energies.
# This is why Ea must be quoted with any lifetime claim.
Ea = {0.7: 9.8, 0.9: 18.7, 1.1: 35.9}    # AF from 85 C to 125 C
# 1000 h at 125 C therefore stands for 9800 h / 18700 h / 35900 h at 85 C
# -> 1.1 years / 2.1 years / 4.1 years.
# The SAME DATA supports "passes 1 year" or "passes 4 years" depending on a
# number the report often does not state.
```

여기서 틀린다. 활성화 에너지는 기구마다 다르고, 뒤섞인 고장 모집단에 수 하나를 쓰는 것은 수명을 3.68배 틀리게 하는 표준적인 방법이다. 인증 보고서가 어느 기구이고 어느 E_a 인지 말하지 않으면, 그것은 당신이 확인할 수 있는 주장을 한 것이 아니다.

표 18. 85 °C 사용에서 125 °C 스트레스로의 가속 계수

E_a	AF (125 °C 대 85 °C)	10 °C 식히면	1000 h 스트레스가 대신하는 것
0.7 eV	9.76×	1.77×	1.11 년
0.9 eV	18.7×	2.08×	2.14 년
1.1 eV	35.9×	2.44×	4.1 년

예제 2. 1000시간 BTI 스트레스: 그것이 십 년에 대해 무엇을 말하나?

주어진 것. BTI의 문턱 이동은 시간에 대해 거듭제곱 법칙으로 자란다. $\Delta V_{th} \propto t^n$ 이고 n 은 보통 0.15-0.25다. 부품을 1000시간 몰아 어떤 이동을 보였다.

푸는 법. 십 년은 87,600 시간이고 스트레스 시간의 87.6배다. 이동은 그 비의 n 제곱만큼 자란다.

답. $n = 0.15 \rightarrow 1.96$ 배, $n = 0.2 \rightarrow 2.45$ 배, $n = 0.25 \rightarrow 3.06$ 배다. **같은 측정이, 이 시험에서 아무도 재지 않은 지수에 따라 1.56배나 다른 이동을 예측한다.**

표 19. 같은 까닭으로, 전압 가속

γ	100 mV 과전압이 높히는 배수
8 /V	2.23×
12 /V	3.32×
16 /V	4.95×

여기서 틀린다. 시간 점 하나에서 거듭제곱 법칙을 외삽하는 것. 스트레스 길이가 하나뿐이면 n 은 측정이 아니라 가정인데 — 답은 거기에 대략 선형이다. 믿을 만한 BTI 특성화는 **여러 길이로** 몰아 n 을 맞춘다. 그 밖의 것은 밝히지 않은 배수가 붙은 수다.

전압 가속은 설계를 공칭에서만 특성화해서는 절대 안 되는 까닭이다. $\gamma = 12$ /V — 흔한 값이다 — 에서 공칭보다 100 mV 위로 돌리면 소자가 3.32배 빨리 늙는다. 그러니 DVFS 전환 동안 넘실대는 레일은 몇 초에 여러 해의 예산을 먹어 치울 수 있다. **DVFS에 대한 신뢰성 물음은 평균 전압이 아니라 과도 현상이다.**

T14.3 일렉트로마이그레이션, 그리고 디지털 설계자를 놀라게 하는 규칙

블랙 식 (Black's equation) 과 왜 AC 가 DC 보다 수월한가

$MTF = A \cdot J^{-n} \cdot \exp(E_a/kT)$ 이고, 혼한 영역에서 $n \approx 2$ 다. 수명은 전류 밀도의 **제곱**으로 줄고 온도가 내려가면 지수로 는다. $E_a = 0.9$ eV 에서 배선을 110 °C 에서 85 °C 로 식히면 수명이 6.7배가 된다.

디지털 설계자를 놀라게 하는 대목은 이것이다. **양방향(AC) 전류는 한방향(DC) 전류보다 훨씬 덜 상하게 한다.** 한쪽으로 밀린 원자가 일부 되밀리기 때문이다. 그래서 EM 규칙은 신호 넷에 전원 넷보다 훨씬 높은 한계를 준다 — 그리고 지는 넷은 거의 언제나 전원이거나 클럭 드라이버이거나 듀티가 50 % 에서 한참 먼 신호다.

어디에 쓰나	전원 격자 치수, 클럭 트리 드라이버 출력, 그리고 정적 전류가 흐르는 모든 드라이버의 출력단.
언제 쓰나	사인오프에서 EM/IR 도구가 자동으로 본다 — 그런데 격자의 폭이 플로어플랜 결정이므로 설계는 그보다 한참 전에 한다.
어떻게 쓰나	전류 밀도를 최대 접합부 온도에서 파운드리 한계 아래로 두고, 그 한계가 듀티와 방향에 달려 있다는 것을 잊지 마라. 묶는 제약은 보통 배선이 아니라 비아다. 비아 하나는 그것이 잇는 배선보다 훨씬 적게 나른다.

실제 코드는 이렇게 생겼다

```
# What the EM report actually flags, in order of frequency:
# 1. VIAS on power straps -- a single via where four were needed
# 2. Clock driver outputs -- high activity, high capacitance, one direction of
#    net current per edge
# 3. Long thin nets from a pad to a block at the far side of the die
# The fix for (1) and (2) is always the same: more parallel vias, wider metal,
# and it is FREE if done at floorplan time and expensive after routing.
```

여기서 틀린다. EM 한계는 어떤 온도에서 규정된 것이다. 85 °C 에서 점검하고 110 °C 에서 도는 설계는 보고서가 말하는 것보다 여유가 6.7배 적다 — 그리고 막 잠에서 깬 블록의 접합부 온도는 누가 시뮬레이션한 평균 온도가 아니다.

T14.4 첫해가 아니라 열째 해를 두고 설계하기

표 20. 기구마다 설계 흐름에 어떻게 들어오나

기구	무엇으로 들어오나	누가 임자인가
BTI / HCI	짧은 타이밍 라이브러리 — 사인오프를 시각 0 과 수명 끝에서 두 번 돌린다	타이밍 사인오프. 그 여유는 설계의 몫이다
TDDB	주어진 전계에서의 게이트 면적 한계, 그리고 FIT 예산	파운드리 규칙. 설계자는 산화막 종류를 고른다
일렉트로마이그레이션	모든 넷에 대한 전류 밀도 점검	물리 설계. 그런데 격자는 플로어플랜에서 정해졌다
노화 불일치	$\sigma(\Delta V_{th})$ 가 수명에 걸쳐 커져서 오프셋과 V_{min} 이 나빠진다	아날로그 설계자 — 그리고 자주 잊힌다

실제로 난 일

마지막 줄이 아날로그 설계를 걸어 넘어뜨리는 것이다. BTI 는 평균 이동만이 아니다. **퍼짐**이 있고, 그 퍼짐이 T12 의 불일치에 더해진다. 시험에서 오프셋을 트리밍한 비교기는 다시 흘러간다. 두 쪽이 다르게 늙기 때문이고, 다르게 늙는 까닭은 바로 두 쪽이 다른 듀티를 지고 있기 때문이다. **트리밍한 오프셋은 트리밍한 그날에 대해 트리밍된 것이다.** 오프셋을 십 년 동안 붙들어야 하는 설계는 현장에서 다시 트리밍하거나 (자동 영점, T10) 지어진 방식상 두 쪽이 똑같이 늙게 만든다.

노화 인지 설계 (ageing-aware design)

설계가 열째 해를 살아남게 하는 것이 셋인데, 어느 것도 여유 하나만은 아니다.

- **스트레스를 고르게 한다.** 차동쌍의 두 쪽이 다른 듀티를 보면 갈라진다. 역할을 주기적으로 바꾸면(초핑, T10) 같이 늙는다.
- **늙은 소자들의 비를 쓴다.** 똑같이 늙는 두 소자로 지은 기준은 절대량으로 지은 것보다 훨씬 덜 흐른다.
- **회복을 허락한다.** BTI 는 바이어스를 걸으면 일부 회복한다. 십 년 동안 DC 로 바이어스된 회로는 최악의 경우를 받고, 듀티를 나눠 쓰는 회로는 안 그렇다.

어디에 쓰나	십 년이나 차량용 수명 목표가 있는 모든 IP, 그리고 정확도를 트리밍하는 모든 아날로그 블록.
언제 쓰나	구조를 고를 때. 이 가운데 어느 것도 나중에 끼워 넣을 수 없다.
어떻게 쓰나	PDK 가 주는 노화 시뮬레이션(썩인 모형 텍)을 진짜 듀티로 — 전부 DC 로 두지 말고 — 돌리고, 타이밍만이 아니라 수명 끝의 오프셋과 V_{min} 을 확인한다.

실제 코드는 이렇게 생겼다

```
* Ageing simulation: the same netlist, twice, with the PDK's aged models.
.lib 'models/pdk.lib' tt
.option reliability_analysis          $ vendor-specific; every PDK has one
.param age_years = 10
.param duty_cycle = 0.5              $ THIS is the parameter people get wrong --
                                      $ a DC-biased device is duty_cycle = 1.0

.op
.measure op vth_shift param='v(m1.vth_aged) - v(m1.vth_fresh)'
```

* Sign off BOTH decks. A design that only passes fresh is a design that only
* passes on the day it is made.

여기서 틀린다. 노화 모형은 통계적이고 보수적이라, 최악의 코너 와 최악의 듀티 와 최악의 온도를 한꺼번에 걸고 돌리면 지을 수 없는 설계가 나온다. 숨씨는 어느 조합이 물리적으로 가능한지를 고르는 데 있다 — 최고 온도에서 DC 로 바이어스된 소자는 가능하지만, 최고 온도에 최대 활동도에 최저 전압은 대개 가능하지 않다.

T14.5 월요일에 이걸로 무엇을 하나

표 21. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
BTI · HCI	두 번 돌리는 타이밍 사인오프	수명 끝의 여유, 따라서 면적
아레니우스 · 가속계수	인증 계획과 데이터시트의 주장	1000시간 시험이 무엇을 말해도 되는지
블랙식 · 일렉트로마이크레이션	플로어플랜에서의 전원 격자와 드라이브 치수	배선 폭과 비아 수 — 일찍 하면 공짜, 늦으면 비싸다
노화인지설계	트리밍하는 모든 것의 구조 고르기	그 트리밍이 열째 해까지 사는지
수명목표	데이터시트	수명, 그 온도와 듀티와 함께 — 아니면 아무 말도 안한 것이다

가지고 갈 수들. $E_a = 0.9 \text{ eV}$ 에서 125°C 스트레스는 85°C 사용보다 18.7배 빠르므로 1000시간이 2.14 년 을 대신한다 — 그리고 E_a 가 다르면 같은 데이터가 1.11 년 이라고도 4.1 년 이라고도 한다. E_a 와 온도와 듀티가 없는 수명 주장은 주장이 아니다. 그리고 과전압 100 mV 는 소자를 3.32배 빨리 늙히는데, DVFS 방식의 과도 현상이 그 평균보다 중요한 까닭이 그것이다.

다음 장은 이 모든 것이 기하가 되는 자리다. 레이아웃이다. 어느 모로 보나 옳은 회로도가 그래도 질 수 있는 곳이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

신뢰성 물리는 파운드리 것이다. IP 쪽 청구항은 **그것을 살아남는 것에** 관한 것이다. 제 퇴화를 스스로 감지해 보상하는 회로(적응 바디 바이어스, 다이 위 노화 감시기가 모는 적응 전압 조정), 결정적인 변수가 절대량이 아니라 똑같이 늙은 두 소자의 비인 구조, 그리고 소자가 회복하게 두는 듀티 나눔 방식. 마지막 것은 눈여겨볼 만하다 — **BTI 는 스트레스를 견으면 일부 회복하는데**, 그것이 물리 상수를 설계 변수로 바꿔 놓는다.

T15. 레이아웃 — 좋은 회로도가 그래도 질 수 있는 곳

회로도는 무엇이 이어졌는지를 말한다. 무엇을 하는지는 레이아웃이 정한다.

읽기 전에 알아야 하는 것

저항 · 전압 · 전류 · 주파수 · RC지연 · 엘모어지연 · 배선지연 · 리피터 · 정합 · 계통불일치 · 공통중심 · 더미소자 · 문턱전압 · 일렉트로마이그레이션 · 전압강하 · 목표임피던스 · 레이아웃의존효과

이 장에서 새로 나오는 것

기생 · 웰근접효과 · STI응력 · 밀도규칙 · 안테나규칙 · 추출 · LVS · DRC · 플로어플랜 · 핀배치 · 추상화뷰 · GDS · 하드매크로 · 소프트매크로

이 책이 섰한 모든 것이 소자들을 이상적인 배선이 있다고 가정했다. 레이아웃은 그 가정을 한꺼번에 네 가지로 걷어낸다. 배선에는 저항과 용량이 있고(T2), 소자는 곁에 무엇이 그려졌느냐에 따라 거동이 바뀌고, 전원 격자는 T13의 임피던스가 되고, 기하 자체가 회로와는 아무 상관 없는 제조 규칙 수백 가지를 지켜야 한다. 어느 모로 보나 좋은 회로도가 그 넷을 다 지는 수가 있다.

T15.1 소자는 회로도가 말하는 그것이 아니다

레이아웃 의존 효과 (LDE, layout-dependent effects) — 소자의 변수가 제 치수가 아니라 둘레 때문에 계통적으로 바뀌는 것. 같은 W/L 이라도 다르게 그리면 문턱이 다르고 이동도가 다르고 정합이 다르다 — 그리고 그 차이는 되풀이되므로 불일치(T12)가 아니다. 예측할 수 있고 반드시 추출해야 하는 이동이다.

표 22. 가장 중요한 셋, 그리고 저마다 무슨 일을 하나

효과	원인	흔한 크기	어떻게 피하나
웰 근접 (WPE)	주입할 때 이온이 웰 마스크 가장자리에 튕겨서 그 근처의 V_{th} 를 올린다	웰 가장자리에서 ~1 μm 안쪽으로 수십 mV	정합 소자를 웰 경계에서 멀리 — 그리고 똑같이 멀리 — 떼어 놓는다
STI 응력 (LOD)	얇은 트렌치 산화막이 채널을 누른다. 게이트에서 트렌치까지의 거리에 따라 이동도가 바뀐다	구동 전류의 몇 퍼센트	정합 묶음의 모든 소자에 똑같은 확산 연장 — 더미 소자가 주는 것이 그것이다
폴리 간격 / OD 간격	리소와 식각이 어레이 가장자리에서는 다른 환경을 본다	몇 퍼센트	모든 행 끝에 더미 게이트

셋 다 실무에서는 같은 결과를 낳고, 그것을 대놓고 적어 둘 만하다. **어레이 가장자리의 소자는 가운데의 소자와 같은 소자가 아니다.** 더미가 있는 까닭이고, 정합 쌍이 그저 치수만 같은 것이 아니라 둘레까지 같아야 하는 까닭이며, '레이아웃 기술자가 알아서 하겠지'가 설계 방법론이 아닌 까닭이다 — 그 제약은 전기적인 것이고 어느 소자가 그것을 지는지는 설계자만 안다.

추출 (extraction) — 뜻이 있는 유일한 넷리스트

레이아웃 뒤 추출은 레이아웃이 만들어 낸 모든 기생 저항과 용량이 담긴 넷리스트를 낳는다. 실리콘에 대응하는 넷리스트가 그것이다. 회로도 넷리스트는 의도의 진술이다.

둘 사이의 틈은 작지 않다. 아날로그 블록에서 높은 임피던스 마디의 추출된 용량은 설계자가 가정한 부하를 예사로 두 배로 만들고, 전류 거울의 소스 연결에 있는 배선 저항은 T12의 불일치 예산보다 큰 계통 오프셋을 더할 수 있다. **추출 뒤 시뮬레이션을 거치기 전에는 설계가 검증된 것이 아니다.** 레이아웃 전에 중요했던 모든 코너에서 그렇게 해야 한다.

어디에 쓰나

테이프아웃 전 모든 블록. 아날로그에서는 주된 검증 단계이고, 디지털에서는 STA 흐름 안에 있다(T2의 SPEF).

언제 쓰나

첫 레이아웃이 나오자마자 — 끝에서가 아니다. 결정적인 마디에서 용량이 2배라는 것을 드러내는 추출은 재설계이고, 그것은 일찍 아는 편이 싸다.

어떻게 쓰나

결합 용량까지 넣어(접지 용량만 말고) 추출하고, 코너 묶음을 다시 돌린다. 앞뒤 수를 **대놓고** 견준다 — 볼 것은 그 비이고, 거기서 놀랄 일이 있으면 대개 넷 하나를 가리킨다.

실제 코드는 이렇게 생겼다

```
# The comparison that belongs in every block's review, as a table:
#   node          C_schematic  C_extracted  ratio
#   vout_amp       42 fF        96 fF         2.3x  <- investigate THIS
#   vbias          15 fF        31 fF         2.1x
#   vin            80 fF        94 fF         1.2x
# A 2x on a high-impedance node is a bandwidth halving. Finding it here costs a
# day; finding it in silicon costs a spin.
```

여기서 틀린다. 추출 도구에는 **모드**가 있다 — 접지만, 결합까지, 인덕턴스를 넣거나 빼거나 — 그리고 싼 모드로 추출한 블록은 다 통과하고도 누화로 실리콘에서 질 수 있다(T9의 교훈을 한 층 아래에서). 어느 모드를 썼는지를 결과와 함께 적어 뒀다.

T15.2 당신 회로에 관한 것이 아닌 규칙들

표 23. 설계자가 그저 지키는 것이 아니라 미리 헤아려야 하는 제조 규칙

규칙	왜 있나	무시하면 무엇을 치르나
밀도 (금속과 OD 채움)	CMP 는 울퉁불퉁한 표면을 고르지 않게 닦는다	채움이 자동으로 들어간다 — 당신의 정합 아날로그 영역 안으로 . 그러면 공들여 맞춘 용량이 바뀐다. 채움 제외 영역을 일부러 잡아 뒤라
안테나	게이트에 붙은 긴 금속 토막이 플라스마 식각 동안 전하를 모아 산화막을 뚫는다	도구가 다이오드나 금속 우회를 끼워 넣는다 — 대개 괜찮지만, 높은 임피던스의 아날로그 마디에서는 그 다이오드의 누설이 안 괜찮다
최소 면적 / 둘러쌈 / 간격	리소와 식각의 한계	DRC 실패. 고치는 것은 기계적이다 — 값은 기능이 아니라 일정이다
래치업 규칙 (가드 링, 웰 타이)	모든 CMOS 구조가 이루는 기생 사이리스터	I/O 를 레일 너머로 몰면 스스로를 태워 먹는 칩 — 그리고 그 전에 모든 기능 시험을 통과한다

실제로 난 일

아날로그 설계자를 놀라게 하는 것은 밀도 채움이다. 도구가 밀도 규칙을 맞추려고 떠 있는 금속 모양들을 더하는데, 그것들이 당신이 손으로 간 정합 커패시터 곁에 앉는다 — **맞춰지지도 않았고 초기 실행에서 추출되지도 않은 용량을 더하면서**. 프로다운 버릇은 아날로그 영역 안의 채움을 스스로, 자기가 다스리고 추출기가 보는 무늬로 놓는 것이지, 당신이 눈을 떴 뒤엔 도구가 하게 두는 것이 아니다.

T15.3 IP 가 실제로 넘기는 것

표 24. 블록을 내주는 두 방식, 그리고 저마다 무엇을 걸게 하나

	소프트 매크로 (RTL)	하드 매크로 (GDS)
무엇이 나가나	RTL, 제약, 테스트벤치, 문서	GDS, LEF 추상화, Liberty 타이밍, 그리고 같은 문서
공정	아무거나 — 통합 담당자가 합성한다	하나 . 새 마디는 새 구현이다
통합 담당자의 자유	온전하다. 라이브러리와 플로어플랜을 그들이 고른다	매크로 안에서는 없다. 직사각형 하나를 놓을 뿐이다
무엇을 보장해야 하나	그들의 라이브러리에서 합성되고 밝힌 주파수에서 타이밍을 맞춘다는 것	타이밍, 전력, EM, 안테나, 밀도, 핀 접근 — 전부
위험이 어디 있나	통합 담당자에게. 그들이 틀릴 수 있다	당신에게. 그리고 테이프아웃에서 굳는다
보통 무엇이	컨트롤러, 프로토콜 논리, 디지털인 모든 것	PHY, SerDes, PLL, 메모리, 아날로그가 든 모든 것

추상화 뷰 (abstract / LEF) 와 핀 접근 문제

통합 담당자는 당신의 GDS 를 놓지 않는다. 도구가 놓는 것은 당신의 **추상화**다 — 윤곽과 핀 모양과 막힘만 담은 간추린 뷰다. 추상화가 어떤 핀이 매크로 가장자리 한가운데 메탈 3 에 있다고 말하면, 배선기는 쓸 수 있는 층으로 거기에 닿을 수 있어야 한다.

핀 접근은 하드 매크로에서 가장 흔한 통합 실패이고, 온전히 피할 수 있는 것이다. 핀을 성근 격자 위에, 통합 담당자의 흐름이 배선하는 층에, 둘레에 넉넉한 공간을 두고 놓고, 한쪽에만 몰아 놓지 않는다. 설계할 때는 아무 값도 안 들고 나중에는 못 고친다.

어디에 쓰나	모든 하드 매크로. 그리고 플로어플랜이 정해질 때 같이 정해진다.
언제 쓰나	내부 레이아웃을 끝내기 전에 — 핀 계획이 내부 배선을 묶는 것이지 그 반대가 아니다.
어떻게 쓰나	통합 담당자에게 배선 층과 트랙 간격을 묻고, 그 간격에 핀을 놓고, 배선이 되는 시험 배치를 같이 건넨다. 그리고 LEF 와 함께 그것을 보여 주는 작은 예제 통합을 내보낸다.

실제 코드는 이렇게 생겼다

```
# The deliverable list for a hard macro. Missing any one of these turns
# into a support call, which is the thing a one-person house cannot afford (Z12).
# design.gds          the layout
# design.lef          abstract: outline, pins, blockages    <- pin access lives here
# design.lib          timing/power at each corner          <- one per corner
# design.cdl          netlist for LVS
# design_antenna.rpt  antenna and density status of the macro
# design.sdc          the constraints the integrator must apply
# INTEGRATION.md      supply, decoupling (T13), keep-out, temperature, lifetime
# example/            a placement that routes, so they can diff against it
```

여기서 틀린다. 혼자서는 타이밍을 맞추는 매크로가 통합 담당자의 맥락에서는 질 수 있다. 클럭이 **그들의** 트리를 거쳐 오기 때문이다. .lib 은 입력 클럭의 가정된 천이를 밝혀야 하고, SDC 는 당신이 가정한 삽입 지연을 밝혀야 한다. 이상적인 클럭으로 만든 .lib 을 내보내는 것은 T2 에서 이상적인 클럭의 F_{max} 를 대는 것과 같은 잘못이다.

T15.4 월요일에 이걸로 무엇을 하나

표 25. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
레이아웃의존효과 · 웰근접효과 · STI응력	정합되거나 정밀한 모든 소자	떼어 놓을 거리와 더미 무늬. 제약으로 적는다
추출	블록을 끝났다고 말하기 전에	시뮬레이션한 설계가 당신이 그린 그것인지
밀도규칙 · 안테나규칙	아날로그 영역, 일찍	당신이 눈을 떴 뒤 도구가 당신의 정합 구조를 고치는지
추상화뷰 · 핀배치	하드 매크로 플로어플랜	통합 담당자가 당신에게 배선할 수 있기는 한지
하드매크로 · 소프트매크로	사업 결정	어느 공정 위험을 당신이 지는지, 그리고 무엇에 값을 매길 수 있는지

레이아웃은 IP 사업이 무엇을 파는지 정하는 자리다. 소프트 매크로는 물리적 위험을 고객에게 넘기고 어느 공정에도 팔 수 있다. 하드 매크로는 그 위험을 그대로 지고 더 비싸게 받는데, 고객이 사는 것이 **확실성**이기 때문이다. 이 장의 공학이 그 확실성을 내줄 수 있게 만드는 것이고 — 위의 문서 목록이 그것을 팔 수 있게 만드는 것이다.

다음 장은 칩에 전원이 들어오기도 전에 나는 실패를 다룬다. 사람 손가락에서 수백 볼트가 핀에 닿는 일이다. 그다음부터 이 책은 소자를 떠나, 넷리스트를 기하로 바꾸는 흐름으로 옮겨 간다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

레이아웃 기법은 대개 특허가 아니라 업계의 관행이지만, 이웃한 두 영역은 다르다. 아날로그의 **자동 레이아웃 생성** (회로도 와 제약 묶음에서 정합되고 DRC 가 깨끗한 어레이를 뽑아내는 생성기)과 **레이아웃 인지 합성**(배치가 치수 잡기로 되먹여지는 것)이다. 둘 다 같은 약점을 친다 — 가장 중요한 제약들(이것들을 맞춰라, 이것을 저것에서 떼어 놓아라, 이 넷은 DC 를 나른다)이 설계자의 머릿속에 있고 넷리스트에는 없다는 것이다.

T16. ESD — 칩에 전원이 들어오기도 전에 나는 실패

모든 핀이 손가락에서 오는 몇 암페어를 견뎌야 하는데, 그것을 견디는 회로가 하필 그것 때문에 느려지면 안 되는 회로의 신호 경로 위에 있다.

읽기 전에 알아야 하는 것

전압 · 전류 · 전하 · 저항 · 주파수 · 적분 · RC지연 · 게이트
용량 · 문턱전압 · 반사계수 · 기생 · 레이아웃의존효과 ·
TDD · 일렉트로마이그레이션 · 특성임피던스

이 장에서 새로 나오는 것

정전기방전 · 인체모델 · 기계모델 · 충전소자모델 · 클램프
· 스냅백 · 설계창 · 유지전압 · 트리거전압 · 전류경로 · 패
드용량 · 이차항복 · 래치업

카펫 위를 걸은 사람은 몇 킬로볼트로 충전된다. 그가 핀을 건드리면 그 전하가 백 나노초 만에 칩 안으로 흘러드는 데, 안쪽의 게이트 산화막은 하나하나가 몇 나노미터 두께다. 그러니 보호는 고를 수 있는 것도, 자잘한 것도 아니다. 그것은 당신 신호와 직렬로 놓인 회로이고, 그 용량은 당신 대역폭에서 곧장 빠져나간다.

T16.1 세 모형, 전혀 다른 세 사건

인체·기계·충전소자 모델 (HBM, MM, CDM)

HBM 은 충전된 사람을 본뜬다. 1 pF 을 2 kV 로 채워 1.5 kΩ 을 거쳐 방전한다. 그러면 첨두 전류가 1.33 A 이고 시정수가 15 ns, 에너지가 2 μJ 다 — **셈한 것이고, 한 번은 손으로 해 볼 만하다**. 수가 사람들이 생각하는 것보다 덜 극적이기 때문이다.

CDM 은 칩 자신이 충전됐다가 핀 하나로 방전되는 것을 본뜬다. 훨씬 빠르고(상승 시간이 1 나노초를 한참 밑돈다) 첨두 전류가 몇 암페어지만 총 에너지는 작다. 500 V 에 대해 발표된 CDM 파형은 첨두가 대략 5-10 A 인데 — 규격과 관행에서 온 수치이고 **여기서 셈한 것이 아니다**. 첨두를 정하는 것이 저항이 아니라 패키지의 인덕턴스이기 때문이다.

MM(기계 모델)은 둘 사이에 있고 대체로 물러났다.

어디에 쓰나	모든 제품의 모든 핀에 대한 인증. 목표는 보통 HBM 2 kV 와 CDM 500 V 이고, 고객이 더 요구하기도 한다.
언제 쓰나	I/O 설계의 맨 처음에 — 보호의 용량은 대역폭 예산의 한 항이고 나중에 더할 수 없다.
어떻게 쓰나	HBM 은 패드에서 클램프와 다이오드로, CDM 은 그것이 지키는 게이트 가까이에 2차 보호를 두고 작은 직렬 저항을 붙여 설계한다. 시간 규모가 다르므로 둘은 다른 구조가 필요하다.

실제 코드는 이렇게 생겼다

```
* TLP (transmission-line pulse) is how protection is characterised:
* rectangular 100 ns pulses of increasing current, measuring V at each step.
* The plot that comes out IS the design window check.
* I_t2 : the current at which the structure fails (second breakdown)
* V_h : holding voltage after snapback -- must be ABOVE the supply
* V_t1 : trigger voltage -- must be BELOW the oxide limit
* A 2 kV HBM target needs I_t2 >= 1.33 A with margin -- the number computed above.
```

여기서 틀린다. HBM 과 CDM 은 서로 다른 구조를 무너뜨린다. HBM 4 kV 를 통과하는 설계가 CDM 250 V 에 질 수 있다. CDM 의 전류는 다이 안에서 나타나 피코초 만에 치솟고, 패드 클램프가 켜지기 전에 가장 가까운 얇은 산화막을 찾아내기 때문이다. HBM 만 보고하는 것은 쉬운 쪽 절반만 보고하는 것이다.

표 26. 세 사건을 나란히

	HBM	MM	CDM
무엇이 충전되나	사람	공구나 고정구	칩 자신
흔한 목표	2 kV	200 V (대체로 물러남)	500 V
첨두 전류	1.33 A (셈한 것)	몇 A	5-10 A (관행에서)
상승 시간	~10 ns	~ns	< 250 ps
길이	~15 ns	~ns	~1 ns
에너지	2 μJ	~μJ	~1 μJ
무엇이 지나	패드 구조물, 클램프	HBM 과 비슷	안쪽의 얇은 산화막 — 패드에서 먼 데

T16.2 설계창, 그리고 왜 창인가

유도 — 한꺼번에 지켜져야 하는 부등식 넷

	이렇게 된다	왜냐하면
1	클램프는 정상 동작에서 흘러서는 안 된다. 넘실댐까지 넣어 $V_{\text{trigger}} > V_{\text{DD,max}}$ 다.	전원 과도에 켜지는 클램프는 전원 레일을 가로지르는 단락이다 — 칩이 잘 돌다가 스스로를 태운다.
2	다른 무엇이 무너지기 전에 켜져야 한다. $V_{\text{trigger}} < V_{\text{oxide breakdown}}$ 이다.	있는 목적이 그것이다. 이 줄과 앞 줄이 창을 정하는데, 산화막이 얇고 전원이 거기에 가까운 첨단 마디에서는 창이 겨우 1~2 볼트 폭이다.
3	스냅백 뒤에는 전원 위에서 버텨야 한다. $V_{\text{hold}} > V_{\text{DD}}$ 다.	아니면 사건으로 한번 켜진 클램프가 정상 전원이 돌아와도 켜진 채로 있는다 — 래치업 이고, ESD 시험기가 이미 합격을 낸 뒤 작업대 위에서 부품이 망가진다.
4	지지 않고 전류를 날라야 한다. I_{t2} 가 목표의 침투보다 커야 하니, HBM 2 kV 면 1.33 A 위로 여유를 두고다.	2차 항복은 열적인 것이고 되돌릴 수 없다.

둘째와 셋째 부등식은 공정이 미세해질수록 반대로 움직인다. 산화막은 얇아지고(천장이 내려간다) 전원은 그보다 느리게 내려간다(바닥이 올라간다). **설계창은 마디마다 좁아진다**. ESD 설계가 전문 분야가 된 까닭이고, 새 마디의 보호가 옛것을 그대로 옮긴 것인 경우가 드문 까닭이다.

전류 경로 (the current path) — 사람들이 잊는 대목

보호는 핀에 붙이는 부품이 아니라 **모든 핀 쌍 사이의 경로**다. 다른 어떤 I/O 를 기준으로 입력에 걸린 펄스는 임피던스가 낮은 길을 찾아야 한다. 보통은 다이오드를 타고 국소 전원으로 올라가, 전원 버스를 따라가, 레일 클램프를 거쳐 접지로 내려가, 다른 다이오드로 돌아오는 길이다.

그 길을 끊는 것이 셋이고 셋 다 회로도가 아니라 레이아웃이다. **버스 저항**(핀과 클램프 사이의 전원 버스 — 몇 암페어에서는 수백 밀리옴이 몇 볼트다), 서로 다른 전원 도메인 사이의 **맞대어 놓은 다이오드가 없는 것**, 그리고 **너무 멀리 있는 클램프**다. 회로도는 보호를 보여 준다. 경로를 보여 주는 것은 추출된 레이아웃뿐이다.

어디에 쓰나

모든 I/O, 특히 제 전원 도메인에 앉아 있는 모든 핀 — PHY 의 아날로그 전원, 따로 조정되는 PLL 레일(T13).

언제 쓰나

플로어플랜에서. 답이 '모든 패드 가까이에 클램프' 이고 그것이 다이 가장자리의 면적을 먹기 때문이다.

어떻게 쓰나

핀 쌍을 (I/O 대 I/O, I/O 대 모든 전원, 전원 대 전원) 다 적어 놓고 저마다 설계된 경로가 있는지 확인한다. 그다음 ESD 경로 점검 도구로 추출된 레이아웃 위에서 확인한다 — 중요한 수는 클램프 자신의 전압이 아니라 목표 전류에서의 **경로 전체에 걸린 전압**이다.

실제 코드는 이렇게 생겼다

```
# What an ESD path check reports, and how to read it.
# pad_rx -> VSS : clamp V 4.1 V + bus IR 1.3 V = 5.4 V @ 1.33 A <- FAIL (oxide 5.0)
# pad_rx -> VDDA : diode 1.1 V + bus 0.4 V = 1.5 V <- ok
# VDDA -> VDD : NO PATH <- FAIL
# Two failures, and NEITHER is the clamp's fault:
# the first is a BUS WIDTH problem; the second is a MISSING back-to-back
# diode pair between two supply domains. Both are floorplan decisions.
```

여기서 틀린다. 따로 난 전원 도메인이 고전적인 구멍이다. 다이 위에서 한 번도 이어지지 않는 두 레일도 시험기 위에서는 이어지고, 둘 사이에 설계된 경로가 없으면 전류는 경계를 건너는 아무 신호로나 길을 찾아낸다 — 보통은 레벨 시프터의 얇은 산화막 게이트다.

T16.3 신호가 치르는 값

표 27. 패드 용량에 대한 대역폭 ($f_{3dB} = 1/(2\pi RC)$). 25 Ω 은 양쪽에서 종단한 50 Ω 선로다)

패드 + 보호 C	25 Ω 에서 f_{3dB}	50 Ω 에서 f_{3dB}	어디에 쓸 수 있나
100 fF	63.66 GHz	31.83 GHz	112G PAM4 급
200 fF	31.83 GHz	15.92 GHz	25-56 G
300 fF	21.22 GHz	10.61 GHz	25-56 G
500 fF	12.73 GHz	6.366 GHz	10 G 이하
1, fF	6.366 GHz	3.183 GHz	저속 / 제어

유도 — 왜 이 표가 SerDes 의 진짜 ESD 제약인가

	이렇게 된다	왜냐하면
1	암페어를 나를 만큼 큰 다이오드에는 접합 면적이 있고, 접합에는 용량이 있다.	보호 용량과 I_{t2} 는 대체로 비례한다 — 하나 없이 다른 하나를 가질 수 없고, 그것이 문제의 전부다.
2	그 용량은 선로와 병렬로 패드에 얹는다.	300 fF 에 25 Ω 이면 극이 21.22 GHz 인데 — 56 Gb/s 링크의 나이퀴스트 아래다.
3	그러니 보호가 채널 여유를 곤장 걸어 간다 — T8 의 등화기들이 싸워서 얻으려던 바로 그 여유다.	ESD 와 링크 예산은 같은 협상이고 , 데이터율을 약속하는 자리에 ESD 담당자는 대개 없다.
4	빠져나갈 길은 구조적이다. 보호를 유도성 회로망(T 코일)을 따라 흘러 놓아 용량이 전송선 안으로 빨려 들어가게 하거나, 1차 보호를 작게 하고 빠른 2차 보호를 안쪽에 둔다.	이 바닥에 특허가 뿔뿔한 까닭이 둘 다다. 어느 것도 공짜가 아니다 — T 코일은 면적과 제 기생을 치른다.

실제로 난 일

ESD 는 한 사람자리 IP 회사가 제 한계를 분명히 말해야 하는 자리다. 보호를 설계하고 **인증하는** 데는 TLP 측정 장비와 특성화할 실리콘이 드는데, 혼자 하는 디지털 흐름에는 둘 다 없다. 그러니 정직한 납품물은 패드에서 멈추는 소프트웨어 매크로, 가정한 보호와 그 용량 예산을 밝힌 연동 안내서, 그리고 이 저장소의 규칙이 요구하는 그 문장이다. **확인하지 않은 것을 확인하지 않았다고 적는 것**. 한 적도 없는 ESD 인증을 암시하는 데이터시트는 작은 공급자를 끝장내는 종류의 주장이다.

T16.4 월요일에 이걸로 무엇을 하나

표 28. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
인체모델 · 충전소자모델	인증 계획과 데이터시트	목표, 그리고 HBM 뿐 아니라 CDM 도 보고한다는 것
설계창 · 유지전압 · 트리거전압	그 마디에 맞는 클램프 고르기	이 전원과 이 산화막에서 보호가 존재할 수 있기는 한지
전류경로	플로어플랜과 전원 도메인 계획	클램프 자리, 버스 폭, 그리고 도메인 사이의 다이오드
패드용량	SerDes 설계자가 함께 얹은 링크 예산	데이터율 — 등화기들이 싸워서 얻으려는 바로 그 여유다
래치업	어디에나 가드 링과 웰 타이	제 I/O 가 범위 밖으로 물렸을 때 부품이 살아남는지

가지고 갈 셈한 수들. HBM 2 kV 사건은 1.33 A 가 대략 15 ns 동안 2 μ 를 나르는 것이다 — 살아남을 만큼 작고 얇은 산화막을 태울 만큼 크다. 그리고 패드의 300 fF 보호는 극을 21.22 GHz 에 놓는데, 그것이 이 장을 T8 에 잇는 문장이다. **ESD 는 링크 예산과 따로 노는 분야가 아니라 그 안의 한 줄이다.**

이것으로 이 권의 소자 절반이 닫힌다. 동작 영역 넷을 가진 소자 하나로 시작해 패드에서 끝났고, 그 사이에 지연, 준안정, 잡음, 표본화, 변환, 클러킹, 링크, 채널, 스위치드 커패시터, 기준, 정합, 전원, 노화, 기하를 지나왔다. 꿰뚫는 실은 규율 하나였다. 모든 수는 셈하거나 잴고, 수를 보고하기 전에 사소한 설명을 죽였고, 세우지 못한 것은 세우지 못했다고 말했다.

다음에 오는 것이 나머지 절반이다. 검증된 넷리스트를 제조되고 시험된 실리콘으로 바꾸는 흐름이다. 다른 종류의 공학이다 — 재는 양이 볼트와 피코초가 아니라 실행 시간, 트랙 수, 코너 목록, 시험 초, 백만 개당 불량 개수다 — 그런데 규율은 같은 것이고, 여섯 장이 그것을 여섯 단계에 적용한다. 흐름과 되돌이의 값(T17), 플로어플랜과 그 전원 격자(T18), 배치·클러킹·배선(T19), 사인오프(T20), 먼저 왔어야 했던 검증 환경(T21), 그리고 출하되는 모든 부품이 지나는 시험기(T22)다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

ESD 는 I/O 설계에서 특허가 가장 활발한 영역 가운데 하나이고, 까닭은 제약 그 자체다. 보호가 **신호 경로 위에** 앉으므로, 모든 청구항은 펄스패러다임 보호를 더 얻는 방법이다. T 코일이나 인덕터로 피킹한 회로망을 따라 흘러 놓은 보호, 진짜 사건 동안에만 흐르는 능동 클램프, 드라이버 자신의 소자를 다시 쓰는 보호, 그리고 패드가 아니라 게이트에 놓는 CDM 전용 2차 보호. 고속 인터페이스를 내보낸다면, 당신의 차별점이 사는 곳이 이 장이다.

T17. 구현 흐름 — 단계마다 무엇을 정하나

단계마다 자유도 하나가 사라진다. 잘못된 값은 그 자유도를 도로 넣는 값이다.

읽기 전에 알아야 하는 것

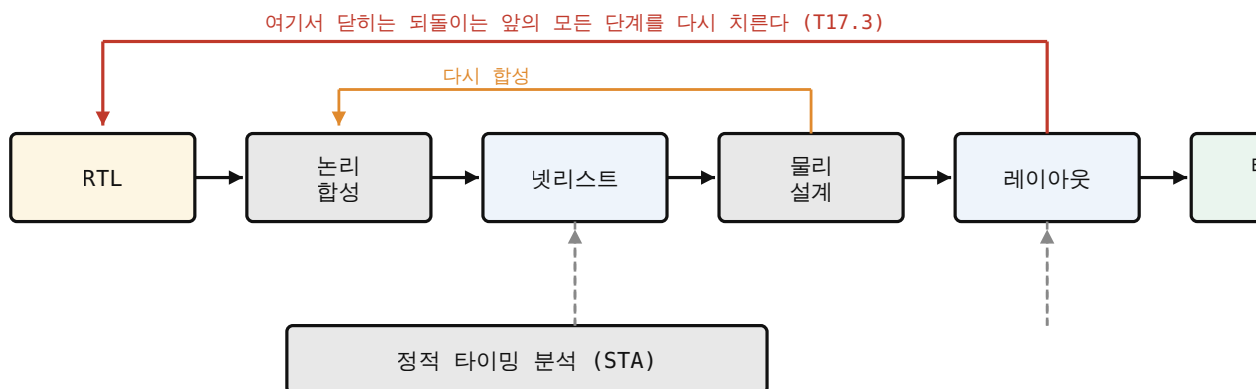
RC지연 · 배선지연 · 슬루 · 슬랙 · 도착시각 · 요구시각 ·
OCV · 플로어플랜 · 추상화뷰 · 추출 · GDS · DRC · LVS ·
리셋동기화기 · 리커버리

이 장에서 새로 나오는 것

구현흐름 · 합성 · 넷리스트 · 표준셀라이브러리 · 타이밍라이브러리 · LEF · DEF · SDC · SPEF · SDF · 정적타이밍분석 · 타이밍수렴 · ECO · 배선부하모델 · 물리인지합성 · 사인오프

설계는 한 걸음에 침이 되지 않는다. 도구의 사슬을 지나는데, 도구마다 어떤 면에서 자유로운 기술을 받아 그렇지 않은 기술을 돌려준다. 합성은 어떤 게이트가 존재하는지를 굳히고, 배치는 그것들이 어디 있는지를 굳히고, CTS는 저마다 언제 클럭을 받는지를 굳히고, 배선은 어느 금속이 어느 넷을 나르는지를 굳힌다. **단계마다 자유도 하나가 사라지고, 뒤의 어느 단계도 그것을 싸게 되돌려 놓지 못한다.** 그 문장 하나가 흐름 전체의 꼴을 설명한다 — 왜 힘이 앞쪽에 실리는지, 왜 파일 형식이 저렇게 생겼는지, 그리고 프로젝트 관리자가 실제로 좇는 양이 진척이 아니라 되돌이 횟수인 까닭을.

T17.1 사슬, 그리고 경계마다 무엇이 건너가나



단계마다 자유도가 하나씩 사라진다. 각 단계가 다음 단계에 대해 알아야 하는 것은 아직 없다 — 그래서 단계마다 다음을 어렵하고, 그 어려움이 빛나간 만큼이 몇 바퀴를 도는지를 정한다.

그림 3. 같은 사슬을 흐름이 실제로 도는 대로 그린 것. 회색 화살표가 STA 인데, 설계를 두 번 본다 — 한 번은 배선이 어렵인 넷리스트에서, 한 번은 배선이 진짜인 레이아웃에서. 색깔 있는 두 되돌이가 이 장이 값을 매기는 것이다. 주황은 합성을 다시 돌리고, 빨강은 전부를 다시 돌린다.

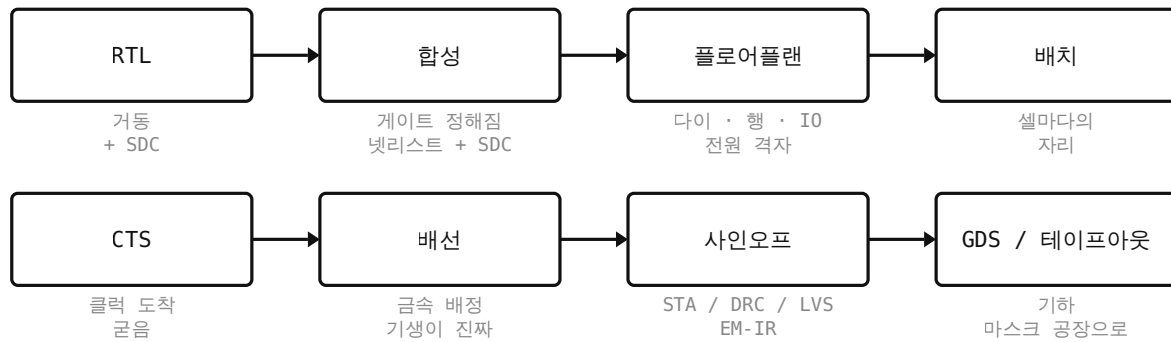


그림 4. 디지털 구현 흐름. 화살표 하나하나가 파일이고, T17.2의 파일 형식들이 그 화살표를 가능하게 하는 것이다.

구현 흐름 (implementation flow) — RTL 기술과 제약 묶음을 제조할 수 있는 기하로 바꾸는, 차례가 정해진 도구의 사슬. 차례가 정해진 까닭은 한 단계의 출력이 다음 단계의 입력이기 때문이고 — 그리고 **각 단계가 그 뒤 단계들에 대해 알아야 하는 것이 아직 없기 때문이다**. 이 장의 모든 어려움이 그 마지막 구절에서 나온다.

흐름은 사슬인데 **정보**는 거꾸로 흐른다. 합성은 알 수 없는 지연을 맞추려고 게이트 크기를 골라야 한다. 그 지연이 배치가 아직 고르지 않은 배선 길이에 달려 있기 때문이다. 배치는 알 수 없는 지연을 맞추려고 자리를 골라야 한다. 배선이 아직 안 땀기 때문이다. 그러니 단계마다 **제 다음 단계를 어렵고** 있고, 최적화의 질이 아니라 그 어림의 질이 몇 바퀴를 도는지를 정한다.

표 29. 경계마다 무엇이 건너가며, 그것이 무엇을 위한 것인가

파일	누가 쓰나	무엇을 나르나	틀렸을 때 나는 실패
.v / .sv (넷리스트)	합성	어떤 셀이 있고 어떻게 이어지는지	RTL이 기술하지 않은 레지스터를 뒤의 무엇도 더해 주지 못한다
.lib (Liberty)	파운드리 / 라이브러리 공급자	모든 셀의 지연·천이·전력·제약 표. 코너마다 파일 하나	실리콘이 결코 보지 않는 코너에서 사인오프한 타이밍
.lef	라이브러리 공급자 + 매크로 임자	추상화된 기하. 셀 윤곽, 핀 모양, 막힘, 배선 층	배선기가 핀에 못 닿는다(T15.3) — 배선 뒤에 찾고 플로어플랜에서 고친다
.def	배치·배선	실제 배치와 배선. 인스턴스마다의 자리, 넷마다의 금속	이것이 설계다. 전원 연결을 잃은 DEF는 소리 없이 아무 데도 안 이어진 것이다
.sdc	설계자	클럭, IO 타이밍, 거짓 경로와 다주기 경로, 디레이트	이 목록에서 도구가 못 만드는 유일한 파일이고 — '타이밍을 맞춘다'가 무슨 뜻인지를 정하는 파일이다
.spef	추출	진짜 기하에서 나온 넷마다의 R과 C	배선 전 어림이 그 자리를 대신하는데, 그 어림은 낙관적이다
.sdf	STA	게이트 수준 시뮬레이션을 위해 되돌려 적은 지연	사인오프가 동의한 적 없는 지연 위에서 시뮬레이션이 통과한다
.gds / .oas	흐름의 마지막 걸음	모든 마스크 층의 모든 다각형	제조되는 것이 이것이다

실제로 난 일

어떤 블록이 모든 코너에서 타이밍을 맞추고 실리콘에서 저온에 졌다. SDC에 `set_false_path`가 리셋에 걸려 있었는데, 그 리셋은 걸릴 때는 정말로 비동기였고 **풀릴 때는 동기였다**(T3, X42). 거짓 경로가 리커버리 위반을 잡았을

그 하나의 검사를 건너낸 것이다. 흐름에는 아무 잘못이 없었다. 흐름이 무엇을 검사해야 하는지 말하는 파일이 틀렸고, 그것이 아무도 자동으로 만들어 주지 않는 그 파일이다.

T17.2 단계마다 제 다음 단계에 대해 하는 어림

배선 부하 모델 (wire-load model) 과 왜 그것을 버렸나

고전적인 합성은 넷의 용량을 **팬아웃**에서 어림했다. 라이브러리와 함께 오는 표를 써서, 이만한 크기의 블록에서 팬아웃 4 인 넷에는 통계적으로 이만큼의 배선이 붙어 있다고 보는 것이다. 배선 지연이 경로에서 작은 몫 일 때는 버틸 만한 어림이다.

그것이 버틸 수 없게 된 까닭은 X14 에서 셈한 그것이다. 배선 지연은 트랜지스터를 따라 줄지 않는다. 배선이 경로의 큰 몫이 되고 나면 통계적인 배선 길이는 쓸모가 없다. 지는 경로란 바로 배선이 전형적이지 **않은** 경로 이기 때문이다. 어림은 평균에서 맞고 중요한 데서 틀린다.

그 자리를 대신한 것이 **물리 인지 합성**이다. 합성이 안에서 성근 배치를 돌려, 배선 길이가 표가 아니라 자리에서 나오게 한다. 어림이 정확해지는 것은 아니다 — 다음 단계와 같은 방향으로 틀리게 만드는 것이고, 쓸 만한 어림이란 그런 것이어야 한다.

어디에 쓰나	가장 긴 배선이 다이의 의미 있는 몫을 가로지를 만큼 큰 블록의 모든 합성 실행.
언제 쓰나	이제는 늘. 배선 부하 흐름은 아주 작은 블록과 교재에만 남아 있다.
어떻게 쓰나	합성이 최적화하기 전에 — 뒤가 아니라 — 플로어플랜(다이 윤곽, 매크로 자리, IO 자리)을 준다. 그다음 같은 경로들에 대해 합성이 예측한 슬랙과 배선 뒤 슬랙을 견준다. 그 차이가 당신 흐름이 수렴하는지를 말해 주는 수다.

실제 코드는 이렇게 생겼다

```
# The correlation check that belongs in every block's log. Not
# "did it meet timing" -- how far the estimate moved between stages.
#
#   path                synth slack   place slack   route slack   drift
#   u_alu/.../sum[31]    +120 ps       +85 ps       -40 ps       -160 ps
#   u_ctl/.../state[2]   +95 ps       +90 ps       +70 ps       -25 ps
#
# A drift column that is large and negative means the flow is not converging:
# you are not closing timing, you are discovering it.
```

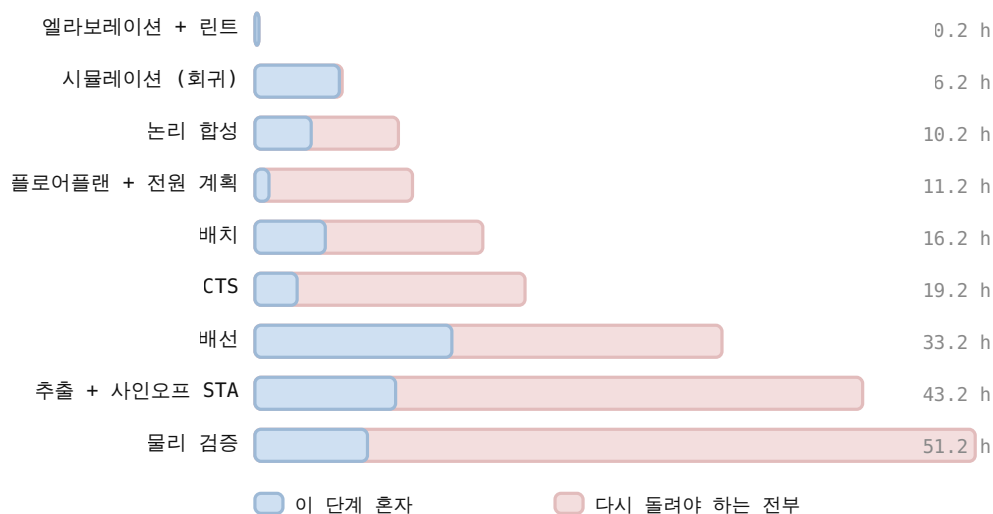
여기서 틀린다. 물리 인지 흐름에서 합성이 큰 양의 슬랙을 보고한다고 해서 그 자체로 좋은 소식은 아니다 — 그것이 쓴 플로어플랜이 배치·배선이 쓸 플로어플랜인지 확인해라. 낡은 플로어플랜에 대고 한 낙관적인 어림은 어림이 아예 없는 것보다 나쁘다. 믿어지기 때문이다.

T17.3 되돌이의 값, 잔 것

흐름의 경제는 도구의 질에 관한 것이 아니다. 사슬의 어디에서 잘못을 찾았느냐에 관한 것이다. 단계 k 에서 발견된 문제는 1 부터 k 까지를 다시 돌려야 한다. 단계마다 그 앞 단계의 출력을 먹어 치웠기 때문이다. 아래 표는 서버한 대에서 백만 인스턴스짜리 블록에 대해 **가정한** 실행 시간을 쓴다 — 가정한 것이고, 그렇다고 표시해 두었다. 진

짜 수는 도구와 기계와 설계에 달렸기 때문이다. 거기에 달리지 않은 것은 끝이다. 되돌이의 값은 그것이 늦게 닫힐 수록 선형보다 빠르게 커진다. 비싼 단계들이 끝에 있기 때문이다.

되돌이의 값 - 어디서 닫히느냐에 따라



마지막 단계에서 찾은 문제는 린트에서 찾았을 때의 256배가 든다.

모든 검사를 앞으로 당기라는 논거는 꼼꼼함에 대한 취향이 아니라 그 배수다.

그림 5. 파란 막대는 그 단계가 혼자 치르는 값이고, 뒤의 분홍 막대는 그 단계에 다시 닿으려고 다시 돌려야 하는 전부다. 비싼 단계가 끝에 있어서 둘이 벌어진다.

표 30. 단계마다의 실행 시간(가정), 그리고 거기서 닫히는 되돌이의 값

단계	도는 시간	여기서 닫히는 되돌이의 값	린트 되돌이에 견주어
엘라보레이션 + 린트	0.2 h	0.2 h	1 ×
시뮬레이션 (회귀)	6 h	6.2 h	31 ×
논리 합성	4 h	10.2 h	51 ×
플로어플랜 + 전원 계획	1 h	11.2 h	56 ×
배치	5 h	16.2 h	81 ×
CTS	3 h	19.2 h	96 ×
배선	14 h	33.2 h	166 ×
추출 + 사인오프 STA	10 h	43.2 h	216 ×
물리 검증	8 h	51.2 h	256 ×

마지막 줄을 읽어라. 물리 검증에서 찾은 문제는 그 자리에 다시 닿는 데 계산 시간 51.2 h 가 드는데, 린트가 찾은 같은 문제는 0.2 h 다 — 256 × 배다. 그리고 그것은 기계 시간뿐이다. 기술자의 시간, 달력의 날들, 그리고 이제 그 고침을 이미 닫힌 모든 것을 건드리지 않고 해내야 한다는 사실이 그 위에 얹힌다.

앞쪽에 힘을 싣는 논거가 통째로 이것이다. 꼼꼼한 RTL 에 대한 문화적 취향이 아니라 산수다. 단계는싼 것에서 비싼 것 순으로 놓여 있으므로, 앞으로 당길 수 있는 검사는 무엇이든 위의 비만큼 곱해진다. 린트와 CDC 해석과 형식 등

가성과 진짜 플로어플랜을 쓴 합성을 쉬지 않고 돌리는 까닭이고, '백엔드에서 고치자'가 같은 고침에 수백 배를 치르겠다는 결정인 까닭이다.

T17.4 왜 타이밍 수렴이 때로 안 달하나

위의 되돌이 값은 한 바퀴만 돈다고 가정한다. 더 어려운 물음은 몇 바퀴를 도느냐이고, 거기서 흐름은 처음 만나는 사람을 놀라게 하는 성질을 갖는다. 수렴은 발산할 수 있다. 위반 하나를 고치면 설계가 바뀌고, 바뀐 설계는 전에 없던 위반을 낳을 수 있다. 셋업 경로를 고치려고 셀을 키우면 그 드라이버에 부하가 걸린다. 버퍼를 끼워 넣으면 배치 공간을 먹어 이웃을 민다. 홀드 고침은 셋업 경로가 없기를 기대고 있던 지연을 더한다.

유도 — 고침 하나가 또 하나를 낳을 수 있을 때 기대되는 고침 수

	이렇게 된다	왜냐하면
1	위반 하나에서 시작한다. 그것을 고치는 것이 일 한 단위다.	우리가 세는 일이 그것이다 — ECO 하나를 걸고 다시 해석하는 것.
2	고침마다 확률 p 로, 서로 독립으로, 새 위반을 낳는다.	이것이 모형의 가정이고, 따져 볼 대목이 이것이다. 고침의 결과지가 그 앞에 고침이 몇 개 있었는지에 안 달렸다는 말이다.
3	그러면 이것은 평균 자손이 p 인 분기 과정이다.	아버지 하나에 평균 p 개의 자식 — 표준적인 골턴-왓슨 꼴이다.
4	세대 n 의 기대 크기는 p^n 이다.	독립인 세대들에 걸쳐 기댓값은 곱해진다.
5	기대되는 총 일은 $p < 1$ 일 때 $1 + p + p^2 + \dots = 1/(1-p)$ 다.	등비급수다. 그 합은 총 고침 수이지 도구를 돌린 바퀴 수가 아니다.
6	$p \geq 1$ 이면 급수가 발산한다. 수렴이 아예 안 된다.	고침마다 평균 하나 이상을 더 낳으므로 대기열이 영영 안 빈다. 그 설계에 필요한 것은 더 나은 기술자가 아니라 플로어플랜이나 제약이나 목표 주파수를 바꾸는 것이다.

표 31. 기대되는 총 고침 수, 서로 독립인 두 길로

p (고침이 또 하나를 낳음)	급수 $1+p+p^2+\dots$	분기 과정 시뮬	여극남
0.1	1.111	1.111	0 %
0.3	1.429	1.441	0.9 %
0.5	2	2.012	0.6 %
0.7	3.333	3.361	0.8 %
0.9	10	9.976	0.2 %
0.9	20	20.06	0.3 %

두 칸은 **다른 방법**으로 셈한 것이다 — 더한 급수와, 급수를 한 번도 안 쓰는 시뮬 대기열 — 그리고 보인 범위에 걸쳐 0.9 % 안에서 맞는다. 그 맞음이 주장을 허락하는 것이다. 셈 하나만으로는 안 되는데, 깨끗해 보이는 곡선에 대한 사소한 설명이 같은 산수를 두 번 찍은 것이기 때문이다.

예제 3. 이 블록은 닫히겠는가?

주어진 것. ECO 1 바퀴가 위반 400 개를 고쳤고 다음 STA 실행이 전에 없던 260 개를 새로 보고했다. 2 바퀴가 그 260 개를 고치고 180 개를 새로 넣었다.

푸는 법. 관측된 비에서 p 를 어림한다. $260/400 = 0.65$ 이고 $180/260 \approx 0.69$ 다. $p \approx 0.67$ 로 잡는다. 지금 열려 있는 180 개에서 기대되는 남은 일은 $180/(1-0.67)$ 이다.

답. 545 개쯤 더 — 같은 크기로 세 바퀴쯤 더다. **p 가 0.67 에 머문다면** 느리게나마 수렴한다.

여기서 중요한 독립 대조는 수학이 아니다. p 를 믿기 전에, 새 위반이 정말로 새 경로인지 아니면 제약을 바꾼 뒤 같은 경로가 다시 보고된 것인지 확인해라 — 다시 보고하는 도구가 겹보기 p 에 대한 사소한 설명이고, 그것은 수를 들여다봐서가 아니라 끝점 목록을 건주어서 잔다.

여기서 틀린다. p 는 상수가 아니다. 점유율이 오르고 남은 위반이 어려워질수록 오으므로, 쉬운 고침에서 잔 p 는 꼬리를 과소평가한다. 비가 오르는 바퀴가 둘 이어지면 ECO 를 그만두고 구조적인 무언가를 바꾸라는 신호다 — 위의 산수는 p 가 1 에 닿으면 ECO 를 아무리 해도 끝나지 않는다고 말한다.

T17.5 월요일에 이걸로 무엇을 하나

표 32. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
구현흐름 · 사인오프	블록 하나를 계획할 때	어떤 검사를 얼마나 자주 돌릴 여유가 있는지
배선부하모델 · 물리인지합성	합성 설비를 갖출 때	초기 슬랙 수가 뜻이 있기는 한지
LEF · DEF · SDC · SPEF · SDF	모든 넘김	다음 단계에 무엇을 빚졌고 무엇을 받아야 하는지
타이밍수렴 · ECO	백엔드가 수렴을 멈출 때	계속 고칠지, 플로어플랜이나 주파수를 바꿀지

다음 세 장은 사슬을 차례대로 걸으며 단계마다 실제로 무엇을 정하는지를 셈한다. 나머지 전부가 올라왔는 플로어플랜과 전원 격자(T18), 배치와 클럭 트리와 배선(T19), 그리고 그 기하를 제조해도 되는지를 정하는 사인오프 검사들(T20)이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

흐름 자체는 특허가 안 되지만, 그 안의 두 가지는 되고 둘 다 활발히 다투어진다. **기하가 있기 전의 타이밍 예측**(RTL 이나 넷리스트의 특징 묶음에서 배선 뒤 지연을 어림해, 되돌이가 며칠이 아니라 몇 분에 닫히게 하는 것)과 **ECO 배치** — 이미 타이밍을 맞춘 것을 건드리지 않고 배선된 설계에 고침을 끼워 넣는 것이다. 둘 다 이 장이 셈한 같은 경계를 친다. 흐름의 값은 그것을 몇 바퀴 도느냐가 지배한다.

T18. 플로어플랜과 전원 계획 — 나머지 전부가 올라앉는 기하

배치도 클럭도 배선도 누군가가 그린 상자 안에서 최적화한다. 이 장은 그 상자를 셈한다.

읽기 전에 알아야 하는 것

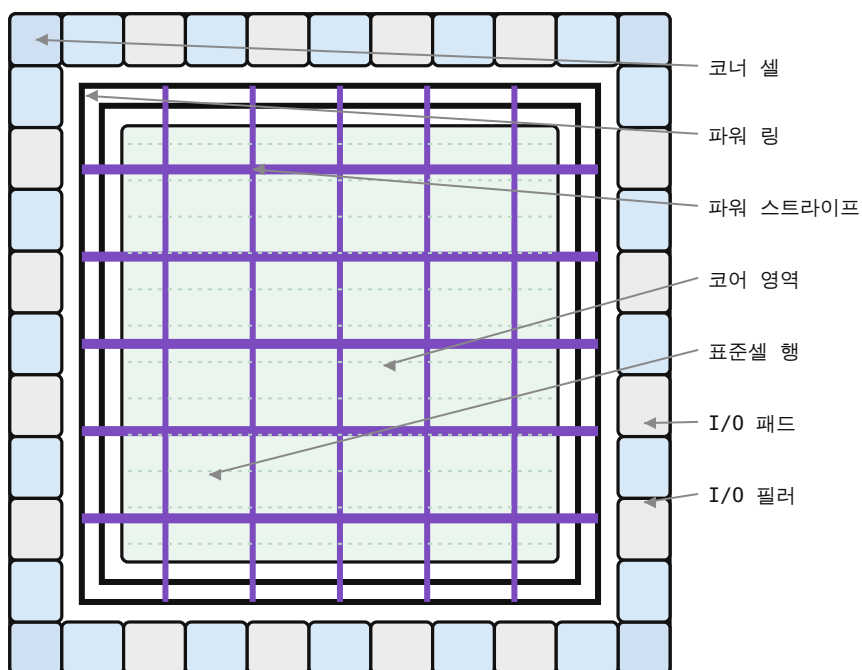
플로어플랜 · 구현흐름 · LEF · DEF · 사인오프 · 전압강하 · 목표임피던스 · PDN · 디캡 · 동시스위칭잡음 · 일렉트로마 이그레이션 · 블랙박스 · 하드매크로 · 핀배치 · 패드용량 · 클 램프

이 장에서 새로 나오는 것

다이 · 코어영역 · 점유율 · 종횡비 · 표준셀행 · SITE · 전원 그리드 · 파워링 · 파워스트라이프 · 스페셜라우트 · IO패드 링 · 코너셀 · IO필러 · 필러셀 · 탭셀 · 킵아웃 · 패드제한다이 · 코어제한다이

뒤의 모든 단계가 이 단계가 그린 경계 안에서 최적화한다. 배치는 플로어플랜이 떼어 주지 않은 면적을 못 쓴다. 클럭 트리는 플로어플랜이 떼어 놓은 플롭들 사이의 거리보다 짧아질 수 없다. 배선은 전원 격자가 먹어 버린 트랙을 못 쓴다. **플로어플랜은 그 잘못을 뒷단에서 고칠 수 없는 유일한 단계다** — 그래서 맨 먼저 그리고, 그래서 거기서 자동인 것이 거의 없다.

T18.1 게이트 수에서 직사각형으로



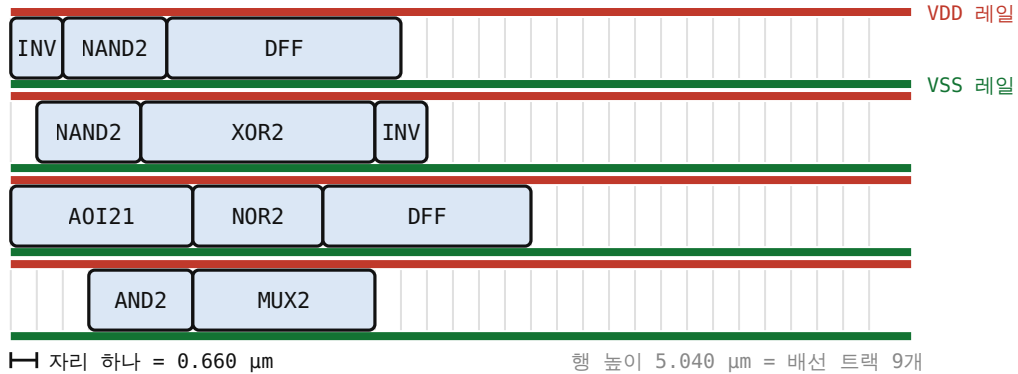
스페셜 라우트는 신호를 배선하기 전에 링과 스트라이프를 짓는다 — 그것들이 먹은 트랙은 사라지고, 배선기는 남은 것에 맞춰 들어간다.

그림 6. 플로어플랜을 바깥에서 안으로: 코너 셀과 필러가 붙은 패드 링, 파워 링 둘, 코어를 가로지르는 스트라이프, 그리고 셀들이 앉을 행. 이 그림의 모든 요소가 이 장의 어떤 셈으로 치수가 잡힌다.

자리 (SITE) 와 표준셀 행 (standard-cell row) — 디지털 라이브러리는 셀을 아무 데나 놓지 않는다. **자리** — 배치의 가장 작은 단위 — 를 정의하고, 셀은 그것의 정수 배다. 행은 자리 하나 높이고 코어의 너비만큼 뻗는다. 셀은

행 안에서 자리 경계에 물린다. 디지털 면적 산수의 모든 것이 여기서 나온다. 레이아웃은 자유 배치가 아니라 **타일 깔기**다.

SITE coreSite CLASS CORE ; SIZE 0.660 BY 5.040 ;



디지털 레이아웃은 자유 배치가 아니라 타일 깔기다. 합법화는 이 격자 위로 도로 투영하는 것이다.

그림 7. 행과 자리, 그리고 모든 행의 길이를 따라 뻗은 두 전원 레일. 셀은 자리를 정수 개 차지하고 자리 경계에 물린다. 디지털 면적 산수를 타일 깔기 문제로 만드는 것이 그것이다.

표 33. 진짜 SITE 정의, 그리고 수마다 무슨 뜻인가

LEF	여기서의 값	무엇을 정하나
SITE coreSite	이름	이 셀이 어느 행에 앉아도 되는지. 라이브러리는 둘 이상을 정의할 수 있다(예를 들어 두 배 높이 행)
CLASS CORE	코어	I/O 링의 PAD 자리와 달리 코어 셀이라는 뜻
SIZE 0.660 BY 5.040	0.66 μm × 5.04 μm	행 높이(라이브러리가 굳혀 놓은 것이고, 셀 면적이 양자화되는 까닭)와 x 방향의 배치 격자

유도 — 인스턴스 수에서 코어 면적으로

	이렇게 된다	왜냐하면
1	자리 하나는 0.66 μm × 5.04 μm = 3.326 μm^2 다.	위의 LEF 에서 곧장 읽은 것이다.
2	평균 셀을 자리 4개로 잡으면 인스턴스당 13.31 μm^2 다.	가정이다. 진짜 수치는 합성 면적 보고서에서 나오고 설계마다 다르다 — 데이터패스가 무거운 블록은 더 넓고 제어 블록은 더 좁다.
3	그러니 인스턴스 100,000 개는 셀 면적으로 1.331 mm^2 를 차지한다.	곱셈이다. 이것은 셀 면적뿐이다 — 배선 공간도, 전원 격자도, 틈도 없다.
4	점유율 70 % 면 코어는 1.901 mm^2 다.	점유율은 셀 면적 ÷ 코어 면적이다. 나머지는 배선 기가 갈 데가 있도록, 그리고 ECO 단계가 버퍼를 놓을 데가 있도록 일부러 비워 둔 것이다.
5	정사각형 코어면 한 변이 1,379 μm 이고, 행이 273.6 개다.	정사각형은 주어진 면적에서 가장 긴 배선을 최소로 하므로 기본 출발점이다 — 요구사항이 아니다.

점유율 (utilisation) — 면적을 나머지 전부와 맞바꾸는 수

점유율은 코어에서 셀이 차지한 몫이다. 플로어플랜에서 가장 결과가 큰 손잡이 하나이고, 품질 지표가 **아니다**. 점유율이 높은 것은 좋은 공학이 아니라 내기다.

올리면 다이가 작아지고 따라서 부품당 값이 싸진다. 동시에 배선과 클럭 버퍼와 홀드 고침 버퍼와 배선 뒤 ECO 가 다 필요로 하는 공간을 없앤다. 70 % 에서 배선되고 타이밍이 닫히는 블록이 80 % 에서는 배선이 안 될 수 있다 — 조금씩이 아니라 절벽처럼이다. 혼잡은 문턱 현상이기 때문이다(X14).

정직하게 고르는 길은 **재는** 것이다. 여러 값으로 흐름을 돌려 배선 시간과 DRC 위반 수가 선형보다 빠르게 오르기 시작하는 자리를 본다. 그 무릎이 그 설계의 한계이고, 설계를 따라 움직인다.

어디에 쓰나	배치가 있기 전에 플로어플랜에서 고른다.
언제 쓰나	설계가 자랄 때마다 다시 본다 — 게이트 수가 10 % 바뀌는 RTL 변경은 점유율을 바꾸는데, 그것 때문에 플로어플랜을 다시 돌리는 사람은 없다.
어떻게 쓰나	흐름이 일단 수렴하도록 첫 바뀌는 낮게(60-70 %) 잡고, 타이밍이 닫히면 올리되 점유율 수치가 아니라 혼잡 지도를 보면서 올린다.

실제 코드는 이렇게 생겼다

```
# The floorplan sweep that should exist in every block's history.
# util  route time  DRC after route  worst slack  die
# 0.60   9 h      0             +60 ps    1.90 mm^2
# 0.70   14 h     0             +40 ps    1.63 mm^2
# 0.75   22 h    41            -15 ps    1.52 mm^2  <- knee
# 0.80   49 h   890           -120 ps    1.43 mm^2
# The knee, not the smallest die, is the answer.
```

여기서 틀린다. 도구가 셈하는 점유율은 보통 매크로 면적과 막힘을 뺀다. '70 %' 를 대는 두 흐름이 서로 다른 뜻일 수 있다. 블록끼리 건준 것을 믿기 전에 **셀 면적 ÷ (코어 면적 - 매크로 면적)** 을 대놓고 건줘라.

T18.2 패드의 고리, 그리고 실제로 무엇이 치수를 묶나

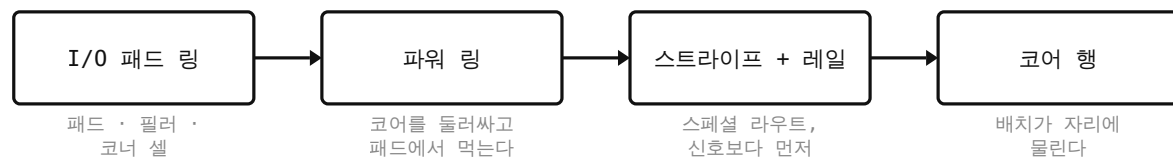


그림 8. 플로어플랜을 바깥에서 안으로. 층마다 다음을 먹인다. 패드가 링을 먹이고, 링이 스트라이프를 먹이고, 스트라이프가 행의 레일을 먹이고, 레일이 셀을 먹인다.

표 34. I/O 링에 있는, I/O 가 아닌 셀들

셀	무엇을 위한 것인가	없으면 어떻게 되나
코너 셀	패드 링의 전원과 신호 버스를 90° 모퉁이로 돌려 나른다	링의 전원 버스가 끊긴 네 토막이 된다. 한쪽 패드들에 돌아올 길이 없다
I/O 필러	패드 사이의 틈을 메워 링의 버스를 이어 주고 주입층과 웰층이 규칙에 맞게 한다	끊긴 전원 버스, 그리고 틈마다 DRC 의 숲
필러 셀 (코어)	코어 안에서 같은 생각. 놓인 셀들 사이에서 n 웰과 전원 레일을 이어 준다	떠 있는 웰 영역과 끊긴 레일 — LVS 와 DRC 실패이고 늦게 발견된다
탭 셀	정해진 간격 안에서 웰과 기판을 전원에 묶는다	래치업(T16). 그 간격은 취향이 아니라 파운드리 규칙이다

패드 제한 다이 (pad-limited) 대 코어 제한 다이 (core-limited)

다이 크기를 정하는 것이 서로 독립으로 둘 있다. 논리가 필요로 하는 면적과 패드가 필요로 하는 둘레다. 논리가 크기를 정하고 패드가 그 둘레에 넉넉히 들어가면 **코어 제한**이고, 패드 수가 논리가 요구하는 것보다 큰 둘레를 강요해 코어가 얼마간 비면 **패드 제한**이다.

그 구별이 무엇을 다듬을 값이 있는지를 정한다. 코어 제한 다이에서는 게이트를 하나 덜어 낼 때마다 돈이다. 패드 제한 다이에서는 게이트를 덜어 내는 것이 **값도 없고 득도 없다** — 다이가 안 줄어든다 — 그리고 손잡이는 패드 간격, 엇갈린 패드나 면적 배열 패드, 그리고 핀 수 줄이기뿐이다.

어디에 쓰나	RTL 이 굳기도 전, 맨 처음의 플로어플랜 어림.
언제 쓰나	핀 목록이 나오자마자. 곱셈 하나인데, 앞으로 반년 동안 팀이 무엇에 시간을 써야 하는지를 바꾼다.
어떻게 쓰나	패드가 필요로 하는 둘레(패드 수 × 간격, 코너 셀 넷을 셈에 넣고)와 논리가 필요로 하는 정사각 코어의 한 변을 셈한다. 큰 쪽이 다이를 정한다.

실제 코드는 이렇게 생겼다

```
# One line of arithmetic that reorders a project's priorities:
# pads = 120, pad pitch = 50 um -> perimeter = 6000 um -> side = 1500 um
# logic needs a core side of 1278 um
# => PAD-LIMITED. Shrinking the netlist buys nothing. Reduce pins,
# or stagger the pads, or accept the empty core and fill it with decap.
```

여기서 틀린다. 패드 제한 다이에서 빈 코어는 쓰기만 하면 낭비가 아니다. 당신이 얻을 수 있는 가장 싼 디커플링 용량이고(T13), 드는 값은 필러 셀뿐이다.

예제 4. 이 다이는 패드 제한인가?

주어진 것. 간격 50 μm 의 패드 120 개, 그리고 코어 한 번으로 1,379 μm 를 요구하는 T18.1 의 논리.

푸는 법. 패드가 필요로 하는 둘레는 $120 \times 50 \mu\text{m} = 6,000 \mu\text{m}$ 이니 한 번이 1,5 μm 다. 코어 번과 견준다.

답. 패드 제한이다 — 패드가 한 번에 1,5 μm 를 요구하는데 논리는 1,379 μm 이니, 패드가 다이를 정하고 논리가 안 쓰는 코어가 0.35 mm^2 남는다.

같은 비교가 **면적 배열**(플립칩) 패드 배치를 쓸지도 정한다. 그것은 둘레 제약을 통째로 없애는 대신 더 비싼 패키지와 다른 ESD 전략(T16)을 치른다.

여기서 틀린다. 패드 간격은 마음대로 고르는 것이 아니다. 패키지의 본드 와이어나 범프 능력이 정하고, 패키지가 조립할 수 없는 간격은 플로어플랜이 아니라 바람이다. 산수를 한 뒤가 아니라 하기 전에 패키지 회사에 그 수를 물어라.

T18.3 전원 격자 치수 잡기 — 그리고 실제로 무엇이 그것을 뻑뻑하게 만드나

코어는 위에서 가정한 35 mW 를 쓰려고 1.8 V 에서 194.4 mA 를 끌어간다. 그 전류는 금속을 거쳐 와야 하고, 금속은 서로 아무 상관 없는 별개의 제약 둘을 지켜야 한다. **전압**이 예산보다 더 처지면 안 되고(T13), **전류 밀도**가 요구되는 수명에 대해 T14 의 블랙 식이 정하는 일렉트로마이그레이션 한계를 넘으면 안 된다. 격자의 치수를 잡는다는 것은 둘 다 지키는 것이고, 둘 중 어느 쪽이 묶고 있는지를 아는 것이다.

유도 — 양쪽 끝에서 먹이는 스트라이프의 IR 강하

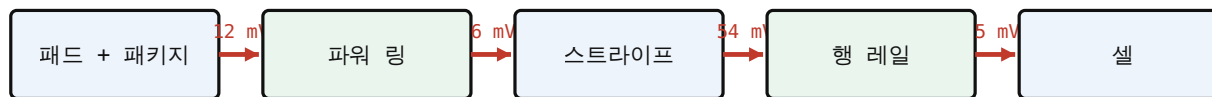
	이렇게 된다	왜냐하면
1	길이 L, 폭 w 의 스트라이프는 저항이 $R = R_s L/w$ 다.	면저항 곱하기 정사각형 개수. R_s 는 금속 층마다의 공정 상수다.
2	부하는 끝에 있지 않고 스트라이프를 따라 흩어져 있다.	전류를 끌어가는 셀들이 스트라이프가 가로지르는 행들에 퍼져 있다.
3	양쪽 끝에서 먹이고, 단위 길이당 전류를 $i = I/L$ 이라 하자.	양쪽 끝이 파워 링이 주는 것이다 — 링이 있는 까닭이 그것이다.
4	한쪽 끝에서 거리 x 인 데의 전류는 $I/2 - i \cdot x$ 다.	전체의 절반이 끝마다 들어와 가는 동안 소비된다.
5	가운데까지의 강하는 $\int_0^{L/2} (I/2 - i \cdot x)(R_s/w) dx = R \cdot I/8$ 이다.	기초적인 적분이다. L^2 항들이 모여 $R = R_s L/w$ 가 된다.
6	그러니 스트라이프에서 가장 나쁜 강하는 가운데의 $R \cdot I/8$ 이다.	끝에 점부하를 단 R·I 와 견줘 보라. 부하를 흘리고 양쪽에서 먹이는 것이 8배다. 격자가 격자인 까닭이 통째로 그 배수다.

표 35. IR 예산 3 % (54 mV) 와 EM 한계 2 mA/ μm 에 필요한 스트라이프 폭

스트라이프	IR 을 위한 폭	EM 을 위한 폭	어느 쪽이 묶나	총 금속 폭	코어 폭의
2	12.4 μm	24.3 μm	EM	48.61 μm	3.53 %
4	6.21 μm	12.2 μm	EM	48.61 μm	3.53 %
8	3.1 μm	6.08 μm	EM	48.61 μm	3.53 %
16	1.55 μm	3.04 μm	EM	48.61 μm	3.53 %
32	0.78 μm	1.52 μm	EM	48.61 μm	3.53 %

마지막 두 칸을 보라. 총 금속 폭이 스트라이프 수를 따라 안 바뀐다. 두 제약 다 스트라이프마다 1/N 로 줄므로, 폭 w/N 짜리 N 개가 폭 w 짜리 하나와 똑같이 든다. 순진한 결론 — "스트라이프 수는 공짜이니 아무거나 골라라" — 은 틀렸고, 왜 틀렸는지를 찾아내는 것이 이 절 나머지의 요점이다. 위의 고른 부하 모형에는 스트라이프에서 셀까지의 경로가 빠져 있다.

전원 전압이 어디로 가나



모두 77 mV. 예산은 토막으로 쓰이고, 지배하는 토막이 늘 당신이 치수 잡은 그것은 아니다.
정적 IR 해석은 시간과 공간을 둘 다 평균한다 — 뜨거운 매크로는 두 평균을 한꺼번에 깨뜨린다(T18.3).

그림 9. 전원 예산은 토막으로 쓰이고, 이 절의 산수는 어느 토막이 지배하느냐에 관한 것이다. 상자 아래의 계단은 같은 수들을 거리에 대한 전압으로 그린 것이다.

유도 — 스트라이프 수가 실제로 사 주는 것

	이렇게 된다	왜냐하면
1	셀은 스트라이프에 붙지 않는다. 제 행의 레일에 붙는다.	레일은 라이브러리의 행 간격으로 행마다 가로로 뻗고, 스트라이프는 그 위를 세로로 지나며 비아로 내려온다.
2	그러니 전류는 가장 가까운 스트라이프에서 레일을 따라, 스트라이프 간격 $p = L/N$ 의 절반까지 간다.	레일은 양옆의 스트라이프에서 먹으므로, 앞과 같은 양쪽 끝 기하가 더 작은 규모로 되풀이된다.
3	두 스트라이프 사이의 레일 토막은 제가 맡은 넓이만큼의 전류를 나른다. $I \cdot (\text{행 높이} \times p) / L^2$ 다.	코어에 고른 전류 밀도를 그 토막의 넓이에 곱한 것이다.
4	그 강하는 $(R_{s,M1} \cdot p / w_{\text{rail}}) \cdot I_{\text{seg}} / 8$ 이고, p^2 에 비례한다.	저항에서 p 가 한 번, 더 긴 토막이 모으는 전류에서 p 가 한 번. N 이 나타나는 자리가 여기 다.
5	$N = 4$ 에서 그것은 5.1 mV 이고, $N = 1$ 에서는 81.7 mV 다.	이 장 첫머리의 수들로 셈한 것이다. 그래도 작다 — 정직한 결과이고 대부분의 사람이 기대하는 결과는 아니다.

표 36. 강하를 쪼갠 것 — 고른 부하

스트라이프	스트라이프에서	행 레일에서	합	예산
1	54 mV	81.7 mV	135.7 mV	54 mV
2	54 mV	20.4 mV	74.42 mV	54 mV
4	54 mV	5.1 mV	59.1 mV	54 mV
8	54 mV	1.28 mV	55.28 mV	54 mV
16	54 mV	0.32 mV	54.32 mV	54 mV

독립 대조. 위의 스트라이프 강하는 유도의 적분에서 나온 것이다. 그 대수를 믿는 대신 확인하려고, 같은 스트라이프를 양쪽 끝을 접지하고 전류를 고르게 뿜는 400 토막짜리 저항 사다리로도 풀었다 — 키르히호프 법칙과 선형 풀이 뿐, 공식은 하나도 안 썼다. 그 값이 54 mV 이고 닫힌 끝은 54 mV 로, 어긋남이 1.0×10^{-11} % 다. 그 대조가 확인해 주는 것은 산수다. 모형을 확인해 주지는 않는다. 둘 다 고른 부하와 이상적인 링을 가정하고 있고, 다음 문단은 그 가정이 깨질 때 무슨 일이 나는데에 관한 것이다.

실제로 난 일

정적 IR 강하 54 mV 로 사인오프한 블록이 제 속도에서 졌다. 정적 해석은 옳았고 상관없었다. 전류가 고르지 않았던 것이다. 한쪽 모퉁이의 하드 매크로가 면적의 10분의 1에서 전체의 3분의 1을 끌어갔고, 스위칭이 상관돼 있었다 — 데이터패스 전체가 한꺼번에 클럭을 받았다. 정적 IR 해석은 시간과 공간을 평균하는데, **두 평균이 같은 방향으로 틀렸다.**

표 37. 뻥뻥한 격자가 제 트랙 값을 하는 자리: 면적의 10 % 에서 전류의 30 %

스트라이프	간격	레일 강하, 고른 부하	레일 강하, 열점	비
2	689.3 μm	20.4 mV	24.5 mV	1.2 $\times$
4	344.7 μm	5.1 mV	15.3 mV	3 $\times$
8	172.3 μm	1.28 mV	3.83 mV	3 $\times$
16	86.17 μm	0.32 mV	0.96 mV	3 $\times$
32	43.08 μm	0.08 mV	0.24 mV	3 $\times$

사소한 읽기부터 죽인다. 비 칸은 3 $\times$ 이고 거기 머무는데, 그것은 발견이 아니다. 국소 전력 밀도 비 0.30/0.10 바로 그것이다. 미묘한 일은 아무것도 안 났다. 쓸모 있는 것은 그 결과다. 레일 강하가 p' 로 가므로, 밀도가 3배인 데서 같은 강하를 지키려면 간격을 $\sqrt{3} = 1.732$ 만큼 줄여야 한다. 곧 나머지 영역보다 열점 위에 스트라이프를 73.2 % 쪼름 더 놓아야 한다. 그것은 행동으로 옮길 수 있는 수이고, 실무자들이 유도 없이 말하는 규칙의 정량적인 꼴이다. **격자 밀도는 평균이 아니라 가장 나쁜 국소 전력 밀도가 정한다.** 일정에 관한 따름정리도 나온다 — 전력이 어디 있는지 알기 전에는 격자 치수를 잡을 수 없으므로, 플로어플랜과 전원 계획은 진짜 전력 지도가 생긴 뒤에 함께 고쳐지는 것이지 처음에 한 번 그려 놓는 것이 아니다.

스페셜 라우트 (special route) — 왜 전원을 신호보다 먼저 배선하나

전원 격자는 신호 배선기가 배선하지 않는다. 별도의 단계가 먼저 짓고, 신호 배선기는 그것을 장애물로 다룬다. 까닭은 전원 배선이 신호 배선에 없는 성질을 갖기 때문이다. 모양이 넓고 규칙적이며 층이 고정돼 있고 모든 것에 이어져 있으며, 그 폭이 연결 관계가 아니라 해석으로 정해진다.

그 차례가 우선순위로 새겨 넣는다. 격자가 먹은 트랙은 사라지고, 배선기는 남은 것에 설계를 맞춰 넣어야 한다. **반대로 하면 신호가 어쩌다 남긴 공간에 맞춰 전원 격자의 치수를 잡는 꼴인데, 그것은 정확히 거꾸로다.** 격자의 폭은 흥정하지 않는 IR 과 EM 한계에서 나온다.

어디에 쓰나	플로어플랜 바로 뒤, 배치가 합법화되기 전이고 신호 배선보다 한참 전.
언제 쓰나	전력 지도가 크게 바뀔 때마다 다시 돌린다 — 새 매크로, 주파수 변경, 다른 동작 전압.
어떻게 쓰나	코어 둘레와 매크로마다 링을 두르고, 열점 해석이 요구하는 간격으로 스트라이프를 놓고, 그 다음 레일 연결, 그다음 모든 교차점의 비아 배열. EM 은 금속에서만 아니라 비아 에서 확인한다 — 전류의 병목은 대개 비아 배열이다.

실제 코드는 이렇게 생겼다

```
# The order, and why each step is where it is:
# 1 add_io_ring      pads, corner cells, fillers      <- perimeter fixed
# 2 add_power_ring   core ring, macro rings           <- both-end feed exists
# 3 add_stripes      pitch from the hot-spot map      <- tracks consumed here
# 4 connect_rails    stripes down to every row rail
# 5 add_via_arrays   every crossing; check EM per via
# 6 ...only now: place, cts, route
```

여기서 틀린다. 비아 일렉트로마이그레이션이 대개 묶는 제약인데 가장 자주 놓친다. 금속 폭은 레이아웃에서 보이고 비아 개수는 안 보이기 때문이다. EM 에 맞게 제대로 치수를 잡은 스트라이프도 비아 둘로 레일에 내려오면 그 비아에서 진다.

T18.4 월요일에 이걸로 무엇을 하나

표 38. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
다이 · 코어영역 · 점유율 · 총횡비	RTL 을 얼리기 전의 첫 플로어플랜	그 부품을 감당할 수 있는지, 그리고 어느 치수를 다듬어야 하는지
패드제한다이 · 코어제한다이 · IO패드링 · 코너셀 · IO필러	핀 목록이 나오자마자	논리를 줄이는 것이 무엇이라도 사 주기는 하는지
파워링 · 파워스트라이프 · 전원그리드 · 스페셜라우트	플로어플랜 바로 뒤	IR 강하, EM 수명, 그리고 배선 트랙이 얼마나 남는지
필러셀 · 탭셀 · 킵아웃	물리 검증 전	그 기하가 규칙에 맞기는 한지(T20)

이 장에서 가지고 나갈 수들은 특정한 값이라기보다 구조적인 것이다. 격자 금속의 총량은 총 전류가 정하고 그것을 어떻게 나누는지에 안 달렸다. 격자 **간격**은 가장 나쁜 국소 전력 밀도가 정하고, 고른 해석에는 보이지 않는다 —

위의 예에서 열점은 같은 스트라이프 수로 레일 강하를 $3 \times$ 로 만든다. 그리고 양쪽 끝 분산 부하 결과의 8배가, 전원 분배가 트리가 아니라 링과 격자인 정량적인 까닭이다.

상자와 격자와 행 묶음이 생기고 나서야 도구들이 드디어 무언가를 움직이기 시작할 수 있다. 다음 장은 그것들로 무엇을 하는가다. 배치, 클럭 트리, 그리고 배선이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

전원 격자 합성 — 설계자의 어림이 아니라 전력 지도에서 스트라이프 간격과 폭을 자동으로 정하는 것 — 이 활발한 영역이고, 모형이 매크로 배치를 제안하는 **기계학습 플로어플래닝**도 그렇다. 둘 다 흐름에서 아직 대체로 판단에 맡겨진 유일한 단계를 자동화하려는 시도이고, 둘 다 정확히 이 장이 셈한 수들로 평가된다. IR 강하, 혼잡, 면적이다.

T19. 배치, 클럭 트리, 배선

세 단계, 세 결정. 셀이 어디 있는가, 언제 클럭을 받는가, 어느 금속이 어느 넷을 나르는가.

읽기 전에 알아야 하는 것

구현흐름 · 넷리스트 · 표준셀행 · SITE · 점유율 · 코어영역 · 전원그리드 · 파워스트라이프 · 정적타이밍분석 · 슬랙 · OCV · 배선지연 · RC지연 · 슬루 · 엘모어지연 · 리피터 · 논리적노력 · SDC · DEF · ECO · 타이밍수렴 · MTBF · 동기화기깊이 · 안테나규칙 · 필러셀 · 디캡

이 장에서 새로 나오는 것

배치 · 합법화 · HPWL · 타이밍주도배치 · 혼잡도 · 트랙 · gcell · 렌트법칙 · 클럭트리합성 · 삽입지연 · 클럭스큐 · 유용스큐 · 클럭메시 · H트리 · 클럭게이팅 · CGIC · 전역배선 · 상세배선 · 탐색수리 · 안테나위반

T18의 플로어플랜이 상자 하나, 전원 격자, 행 묶음을 내놓았다. 상자 안에 있는 것은 아직 넷리스트다 — 셀과 연결의 목록일 뿐, 자리가 없다. 이 장의 세 단계가 거기에 기하를 준다. 그리고 저마다 다음 단계가 바꿀 수 없는 것을 굳힌다. 배치는 거리를 굳히고, 클럭 트리 합성은 도착 시각을 굳히고, 배선은 진짜 기생을 굳힌다. 배선이 끝나면 설계에는 ECO라 불리는 작고 국소적이고 비싼 자유도 말고는 남는 것이 없다.

T19.1 배치: 이산 제약이 붙은 최적화

합법화 (legalisation) — 최적화기가 원하던 아무 데서나 가장 가까운 **합법인** 자리로 셀을 옮기는 단계. 행 위에, 자리 경계에 물려서(T18), 다른 셀과 겹치지 않고, 라이브러리가 지고 있는 모든 배치 규칙을 지켜서다. 전역 배치는 연속 문제를 풀고, 합법화는 그 답을 실제로 존재하는 이산 자리 집합 위로 도로 투영한다.

그 두 걸음 구조 — 느슨하게 만든 연속 문제를 풀고 투영한다 — 가 배치 품질 보고서에 합법화에서 계단이 생기는 까닭이다. 연속 해는 아무도 이룰 수 없는 하한이다. **그 계단이 크면 원인은 거의 언제나 밀도다.** 최적화기가 자리도 없는 데에 셀을 쌓았고, 합법화가 그것을 멀리 옮겨야 했던 것이다. 그것은 배치기의 문제가 아니라 배치기가 보고한 플로어플랜 또는 점유율의 문제다.

표 39. 배치 목적함수마다 실제로 무엇을 최소로 하나

목적	그 양	언제 그것이 옳은가
배선길이 (HPWL)	넷마다 그 넷의 경계 상자 둘레의 절반을 다 더한 것	기본값. 값매기기가 싸고 최적화할 만큼 미분 가능하다. 지연도 혼잡도 결정적이지 않을 때 둘 다에 대한 좋은 대리값이다
타이밍 주도	가중 배선길이. 넷의 가중치는 그 경로가 얼마나 결정적인지에 따라 오른다	타이밍이 가까워졌을 때. 총 배선길이를 내주고 $F_{\max}$ 를 정하는 몇 안 되는 경로를 산다
혼잡 주도	영역마다 예측된 배선 수요를 퍼 놓는 것	타이밍이 아니라 배선이 질 때. 설계가 배선되길라도 하게 하려고 일부러 배선길이를 늘린다

실제로 난 일

아슬아슬한 STA 실행 뒤에 타이밍 가중치를 세게 걸어 블록을 다시 배치했다. 최악 슬랙이 40 ps 좋아졌고 설계는 배선이 안 되게 됐다. 배치기가 결정적인 데이터패스를 뽁뽁한 덩어리로 끌어당겼고, 그 영역의 배선 수요가 공급을 넘어섰다. 위의 세 목적은 서로 맞서 있고, 하나를 다듬으면서 나머지를 재지 않는 것이 수렴하던 흐름이 수렴을 멈추는 방식이다.

T19.2 혼잡: 공급과 수요, 그리고 왜 평균이 거짓말을 하나

트랙 (track) · gcell · 혼잡도 (congestion) — 트랙은 금속 층 하나 위에서 그 층의 간격으로 난 배선 가능한 선 하나다. 배선기는 코어를 **gcell**(전역 배선 칸. 트랙 수십 개 폭)로 나누고, gcell 의 변마다 그것을 건너야 하는 배선 수를 쓸 수 있는 트랙 수와 견준다. **혼잡도**가 그 비다. 1 을 넘으면 어떤 넷은 거기로 못 지나가 돌아가야 한다 — 그리고 둘레가 다 1 을 넘으면 아예 못 지나간다.

유도 — T18 블록의 트랙 공급 — 순전히 기하다

	이렇게 된다	왜냐하면
1	코어는 한 변이 1,379 μm 다(T18).	두 장이 같은 칩을 말하도록 가져온 값이다.
2	배선 간격 0.56 μm 이면 층 하나가 코어를 가로질러 2,462 트랙을 준다.	코어 너비를 간격으로 나눈 것. 간격은 가정인데 , 행 높이 5.04 μm 가 트랙 아홉 개라는 것과 아귀가 맞는다.
3	트랙마다 1,379 μm 를 다 뺀다.	막힘을 무시한 트랙당 쓸 수 있는 길이 — 상한이고, 상한이라는 것을 기억해 둘 만하다.
4	쓸 수 있는 신호 층이 4 개면 공급은 배선 13.58 m 다.	전원 격자가 안 가져간 층들이다. 그 빠기가 T18 과 이 장이 만나는 자리다. 스트라이프가 트랙을 먹는다.
5	수요는 넷 100,000 개 $\times$ 평균 30 μm = 3 m 다.	평균 넷 길이가 이 어림의 약한 고리이고 가정한 값이다. 그리고 여기서 진짜 흐름이 예측이 아니라 재는 유일한 양이기도 하다.
6	전역 비: 0.22.	1 보다 넉넉히 아래다 — 그리고 다음 문단이 그것이 거의 아무것도 말해 주지 않는 까닭을 설명한다.

표 40. 결론이 가정한 수 하나에 얼마나 달려 있나

가정한 평균 넷 길이	수요	수요 $\div$ 공급	읽기
20 μm	2 m	0.15	평균으로는 배선 가능
30 μm	3 m	0.22	평균으로는 배선 가능
45 μm	4.5 m	0.33	평균으로는 배선 가능
60 μm	6 m	0.44	평균으로는 배선 가능
80 μm	8 m	0.59	평균으로는 배선 가능

저 표에 대한 정직한 진술 둘. 첫째, 이 어림은 **가정 하나가 지배한다**. 평균 넷 길이가 네 배면 답이 넉넉함에서 불가능으로 옮겨 가고, 기하를 아무리 조심해도 그것은 안 바뀐다. 둘째, 그리고 더 중요하게는 **전역 비가 엉뚱한 양이다**. 배선은 국소적으로 진다. 전역이 0.22 인 설계도 백 마이크로미터 남짓한 영역에서 gcell 이 1.0 을 넘을 수 있고, 배선기가 끝내는지를 정하는 것이 바로 그 gcell 들이다. T18 의 고른 전력 지도와 같은 구조적 잘못이다 — 어디서나 맞고 아무 데서도 쓸모없는 평균이다.

렌트 법칙 (Rent's rule) — 왜 배선 수요가 면적보다 빨리 느냐

경험적으로, 진짜 설계에서 아무렇게나 고른 이어진 셀 N 개 둘레에 경계를 그으면 그 경계를 건너는 연결 수가 $T = k \cdot N^p$ 이고, 논리에서 p 는 보통 0.5 와 0.75 사이다. 그 지수는 설계가 저 자신과 얼마나 이야기하는지를 재는 값이다.

배선에 대한 결과는 곧바르다. 넓이 A 인 영역은 $N \propto A$ 개의 셀을 담고 제 경계로 $T \propto N^p$ 개의 연결을 날라야 하는데, 경계 자체가 주는 트랙 용량은 $\propto \sqrt{A} \propto N^{0.5}$ 다. $p > 0.5$ 이면 — 곧 사실상 모든 진짜 논리에서 — 블록이 커질수록 수요가 공급을 앞지른다. **그래서 자라는 설계에 금속 층을 더해도 배선이 쉬워지지 않고, 그래서 계층이 있다.**

정리가 아니라 경험 법칙이고, 그렇게 써야 한다. 아무렇게나 고른 분할에서 건너는 수를 세어 p 를 당신 넷리스트에서 직접 재고, 남이 대는 값은 의심해라.

어디에 쓰나	블록을 쪼개야 하는지 어려울 때, 그리고 배치가 있기 전에 혼잡을 예측할 때.
언제 쓰나	블록 크기를 아직 흥정할 수 있는 구조 단계에서 — 그때가 답을 싸게 행동으로 옮길 수 있는 유일한 때다(T17).
어떻게 쓰나	넷리스트를 여러 크기로 쪼개 경계 넷을 세고, $\log T$ 를 $\log N$ 에 맞춘다. p 가 0.7 쯤을 넘으면 그 블록은 혼잡에 묶일 것이니 쪼개거나 금속을 더 줘야 한다.

실제 코드는 이렇게 생겼다

```
# Measuring p on your own netlist instead of quoting a textbook value:
# N      T (crossings)    log-log fit -> p
# 100      68
# 1000     271
# 10000    1090          p ~ 0.60, k ~ 4.3
# Then: a 100k-instance block implies ~ 4.3 * 100000^0.60 crossings at the
# top level -- compare with the tracks its perimeter actually offers.
```

여기서 틀린다. 렌트 법칙은 평균 연결성을 기술한다. 방송 버스나 큰 크로스바가 있는 설계는 하필 질 바로 그 자리에서 국소적으로 그것을 어기므로, 전역 p 가 좋다고 배선 가능성이 보장되지 않는다 — 그 법칙이 기술하지 않는 구조물들을 들여다보라는 신호다.

T19.3 클럭 트리 합성: 왜 삽입 지연이 스큐를 치르게 하나

삽입지연 (insertion delay) · 클럭스큐 (clock skew) — 삽입 지연(또는 레이턴시)은 클럭이 들어오는 자리에서 플롭의 클럭 핀까지의 시간이다. **스큐**는 두 플롭 사이의 삽입 지연 차다. 셋업과 홀드 해석이 신경 쓰는 것은 스큐다. 레이턴시는 그 자체로는 발사-포획 비교에서 지워진다 — 그래서 교과서의 답이 '레이턴시는 상관없다' 인 것이다. 그 답은 실리콘에서는 틀렸고, 까닭은 다음에 셈한다.

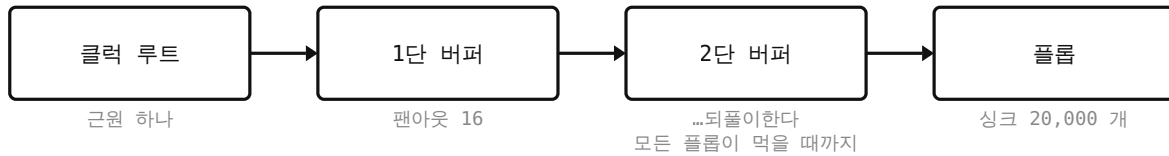


그림 10. 버퍼를 단 클럭 트리. 팬아웃 16 에 싱크 20,000 개면 깊이가 4 단이고 버퍼가 1,335 개쯤 든다 — 그 하나하나가 아래에서 썬하는 변동의 근원이다.

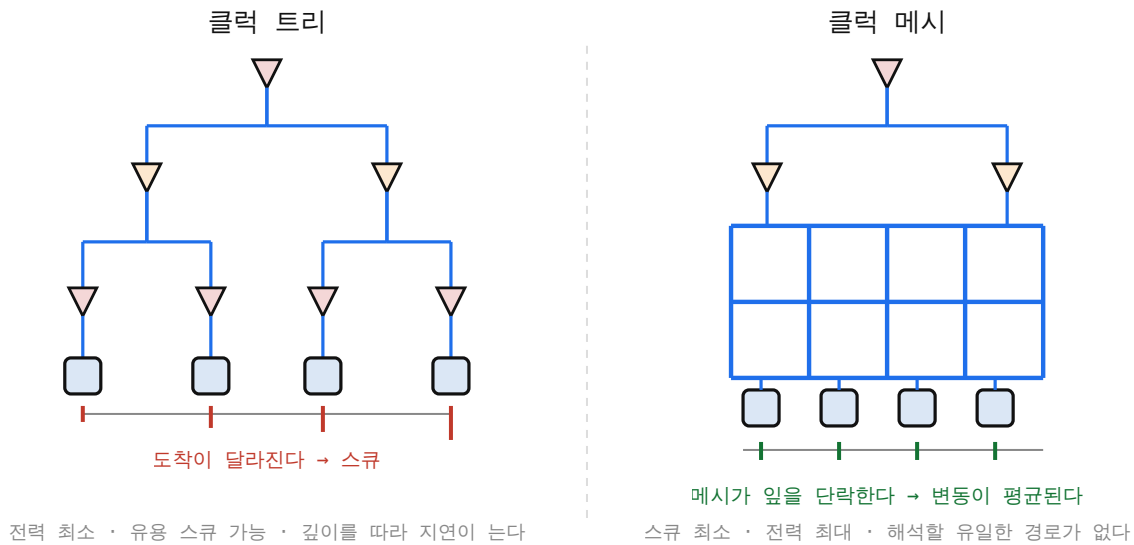


그림 11. 두 구조, 그리고 둘을 가르는 성질. 트리에서는 앞마다 제 경로가 있어 국소 변동이 그 경로를 따라 쌓이고, 메시에서는 앞들이 서로 단락돼 있어 변동이 대신 평균된다.

유도 — 칩 위의 변동이 어떻게 레이턴시를 스큐로 바꾸나

	이렇게 된다	왜냐하면
1	두 플롭은 트리가 갈라지는 자리까지 클럭 경로를 함께 쓴다.	지어진 방식이 그렇다 — 트리는 트리다. 함께 쓰는 뭉을 공통 경로 라 하자. 여기서는 레이턴시의 98 % 다(가정).
2	OCV 해석은 발사 경로를 늦게, 포획 경로를 이르게 디레이트 한다.	다이 위 두 점 사이의 공정·환경 변동에서 가장 나쁜 조합을 가정해야 하기 때문이다. STA 는 실리콘이 어느 조합을 내놓을지 알 길이 없다.
3	디레이트가 $\pm 7\%$ 면 늦은 경로는 $\times 1.07$, 이른 경로는 $\times 0.93$ 이다.	디레이트 값은 가정 이다. 진짜 값은 파운드리에서 오고 거리와 경로의 단 수에 달렸다.
4	실효 스큐 = $1.00 \times 1.07 - 0.98 \times 0.93 =$ 삽입 지연의 15.86 % 다.	공통 부분이 안 지워진다. 거기에 걸린 두 디레이트가 다르기 때문이다. 그 뭉을 돌려주는 것은 공통 경로 비관 제거 뿐이고, 그것도 정말로 함께 쓰는 경로에 대해서만이다.
5	그래서 진짜 스큐가 레이턴시의 2 % 인 트리가 15.9 % 인 것처럼 해석된다.	이것이 '왜 레이턴시가 중요한가'에 대한 답이다. 레이턴시에 디레이트 차가 곱해져 스큐로 다시 나타난다 — 그러니 깊은 트리는 완벽히 균형 잡혀 있어도 주파수를 치른다.



실효 스큐

$$= 1.00 \times 1.07 - 0.98 \times 0.93$$

$$= 1.070 - 0.9114$$

$$= 15.86 \% \text{ of the insertion delay}$$

공통 부분은 지워지지 않는다.

거기 걸린 두 디레이트가 다르기 때문이다.

그래서 완벽히 균형 잡힌 트리도

제 지연의 15.9 % 를 스큐로 쓴다 -

그래서 CTS 는 지연도 목표로 삼는다.

그림 12. 산수를 그림 하나로. 공통 경로가 한쪽에서는 늦게, 다른 쪽에서는 이르게 디레이트되어 지워지지 않는다 — 그리고 안 지워진 것이 삽입 지연의 한 몫이다.

표 41. 삽입 지연이 치르는 값, 5 MHz (주기 2, ps) 에서

삽입 지연	진짜 스큐 (2 %)	OCV 디레이트 뒤 스큐	클럭 주기의
200 ps	4 ps	31.72 ps	1.59 %
400 ps	8 ps	63.44 ps	3.17 %
600 ps	12 ps	95.16 ps	4.76 %
900 ps	18 ps	142.7 ps	7.14 %
1,2 ps	24 ps	190.3 ps	9.52 %

마지막 칸을 읽어라. 삽입 지연이 1,2 ps 이면 게이트 하나가 뒤집히기도 전에 OCV 만으로 주기의 9.52 % 가 사라진다. 그래서 CTS 의 공학적 목표는 스큐 하나가 아니라 레이턴시와 스큐를 함께이고, 가장 잘 듣는 고침은 대개 구조적이다. 단 수를 줄이거나, 루트에서의 경로를 짧게 하거나, 분배 구조를 바꾸는 것이다.

표 42. 클럭 분배 구조 셋, 그리고 저마다 무엇을 사 주나

구조	스큐	전력과 면적	언제 쓰나
버퍼 트리 (CTS 기본)	좋고 싱크마다 다스릴 수 있다. 레이턴시는 깊이를 따라 늘다	가장 낮다 — 팬아웃이 요구하는 버퍼만	거의 늘. 유용 스큐와 게이팅을 받쳐주는 유일한 구조다
H 트리	지어진 방식상 훌륭하다 — 모든 잎이 루트에서 같은 거리다	더 높다. 싱크가 실제로 어디 있는지를 무시한다	싱크가 정말로 격자 위에 있는 규칙적인 배열. ADC 의 비교기들, 똑같은 타일의 메시
클럭 메시	가장 낮다. 메시가 잎들을 단락해서 국소 변동이 쌓이는 대신 평균된다	가장 높다 — 늘 뒤집히는 큰 용량이고, 메시에는 유일한 경로가 없어 해석이 어렵다	스큐가 묶는 제약이고 그 전력을 치를 값이 있는 고주파 코어

유용 스큐 (useful skew) — 클럭을 일부러 쓰기

스큐가 오차이기만 한 것은 아니다. A 에서 B 로 가는 경로가 빠듯하면 B 의 클럭을 늦게 주어 그 경로에 시간을 더 준다 — 그리고 같은 양을 B 자신이 내보내는 경로에서 가져간다. CTS 도구는 이것을 일부러 한다. 도착을 고르게 만드는 대신, 설계 전체에서 최악 슬랙을 가장 크게 만드는 도착 시각 묶음을 푼다.

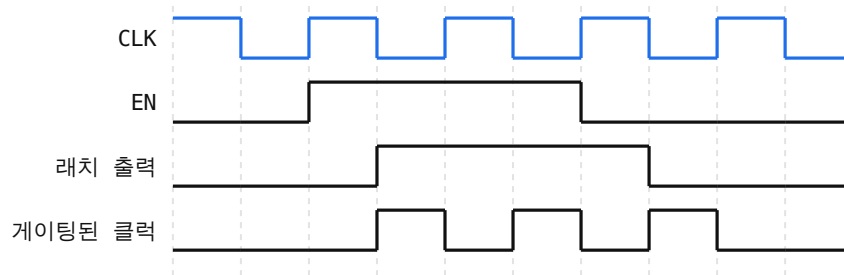
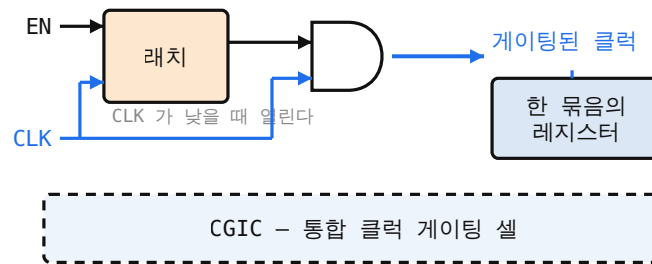
정말로 공짜인 주파수이고, 정말로 값이 있다. 이제 설계가 클럭 트리가 해석이 가정한 그것이라는 데 달리게 된다. 스큐 0 으로 균형 잡은 트리는 버퍼가 모형보다 느려도 점잖게 나빠지지만, 결정적인 경로에 유용 스큐 80 ps 를 실은 트리는 홀드 위반으로 무너지는데 그것은 클럭을 늦춰서 고칠 수 없다.

어디에 쓰나	CTS 안에서, 그리고 CTS 뒤 최적화에서 다시.
언제 쓰나	몇 안 되는 경로가 주파수를 막고 나머지는 여유가 있을 때 — 고전적인 경우다.
어떻게 쓰나	묶어 뒤라. 유용 스큐를 주기의 어느 뭉까지만 허락하고, 그것을 쓰는 경로에는 홀드 여유를 대놓고 더 크게 두고, 모든 코너에서 다시 확인한다. 스큐와 경로 지연은 코너에 걸쳐 서로 따라가지 않는다.

실제 코드는 이렇게 생겼다

```
# The report to read after CTS, per critical endpoint:
# endpoint      latency    useful skew  setup slack  hold slack
# u_alu/acc[7]   412 ps      +80 ps      +12 ps      +9 ps    <- thin
# u_ctl/st[1]    398 ps      0 ps        +95 ps      +60 ps
# An endpoint living on useful skew with 9 ps of hold margin is a silicon
# failure waiting for a slow corner. Bound the skew, not just the slack.
```

여기서 틀린다. 게이팅 셀이 스큐를 진 경로 위에 앉으면 유용 스큐와 클럭 게이팅이 나쁘게 얹힌다. 켜고 끄는 것이 그 가지의 부하를 바꾸고, 따라서 그 가지가 주는 스큐를 바꾼다. 게이팅 셀은 유용 스큐가 걸리는 자리보다 위에 두어라.



래치가 없으면 CLK 가 높을 때 EN 이 바뀌며 글리치가 난다.

플롭이 볼 만큼 길고 최소 펄스 폭을 어길 만큼 짧은 반쪽 펄스다. 래치는 EN 의 변화를 낮은 상으로 가둔다.

그림 13. 통합 클럭 게이팅 셀, 그리고 왜 래치가 빠도 되는 것이 아닌가. 파형이 논증의 전부다. 래치는 클럭이 낮을 때만 투명하므로 인에이블이 높은 상 내내 안정하고, 게이팅된 클럭은 늘 온전한 펄스이거나 펄스가 없거나다.

클럭 게이팅 (clock gating) 과 그것을 합법으로 만드는 래치

클럭 회로망은 그것이 먹이는 플롭에 할 일이 있든 없든 주기마다 뒤집힌다. 그것을 꺼 두는 것이 디지털 설계에서 가장 큰 동적 전력 손잡이 하나다. 그 일을 하는 회로 — **통합 클럭 게이팅 셀 (CGIC)** — 은 래치 뒤에 AND 를 단 것인데, 래치는 빠도 되는 것이 아니다.

왜 래치인가. 인에이블을 클럭과 곧바로 AND 하면 클럭이 높은 동안 인에이블이 바뀔 때마다 글리치가 난다. 플롭이 볼 만큼 길고 최소 펄스 폭을 어길 만큼 짧은 반쪽 펄스다. 래치는 클럭이 낮을 때 투명하므로 인에이블은 낮은 상에서만 바뀔 수 있고 높은 상 내내 붙들려 있다. 그러면 게이팅된 클럭은 늘 온전한 펄스이거나 펄스가 없거나다.

여기서 가정한 블록의 클럭 회로망은 플롭 클럭 핀 20,000 개에 버퍼 1,335 개쯤이다 — 용량으로 84,005 fF 쯤이 5 MHz 로 뒤집히니 136.1 mW 다. 그것을 60 % 의 시간 동안 꺼 두면 81.65 mW 를 아낀다.

그것을 저 자신이 아니라 블록에 견주어 확인해라. T18 은 블록 전체가 350 mW 를 끌어간다고 가정했다. 여기서 썸한 클럭 회로망은 그것의 38.9 % 다 — 높고, 높아야 한다. 이것은 플롭 20,000 개를 먹이는 게이팅 안 된 트리이고, 게이팅을 하게 만드는 바로 그 설계다. 그중 60 % 를 게이팅하면 블록 전체 전력의 23.3 % 를 돌려받는다. 그 뒤편이 5 % 나 90 % 로 나왔다면, 옳은 대응은 아낀 값을 보고하는 것이 아니라 용량 가정을 의심하는 것이었을 것이다.

어디에 쓰나

RTL 에 인에이블이 있는 자리마다 합성이 끼워 넣고, RTL 이 그것을 드러내지 않는 자리에는 손으로 다시 넣는다.

언제 쓰나

어디서나. 물음은 게이팅을 할지가 아니라 어느 알갱이로 할지다 — 플롭 8 개당 CGIC 하나가 흔하고, 플롭마다 하나는 아끼는 것보다 더 든다.

어떻게 쓰나

인에이블이 보이도록 RTL 을 쓰고(되돌리는 먹스가 아니라 `if (en) q <= d;`), 그다음 실제로 게이팅된 플롭의 비율을 보고서에서 확인한다. 저전력 설계에서 게이팅 안 된 플롭은 대개 도구의 문제가 아니라 RTL 문제의 문제다.

실제 코드는 이렇게 생겼다

```
// The RTL that lets the tool insert a CGIC:
always_ff @(posedge clk) if (en) q <= d;           // gateable

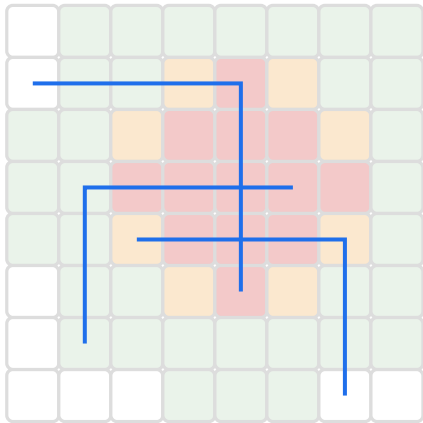
// The RTL that prevents it -- the clock still toggles every cycle:
always_ff @(posedge clk) q <= en ? d : q;          // recirculating mux

// Both simulate identically. Only the first one saves power, and the
// difference does not appear anywhere in functional verification.
```

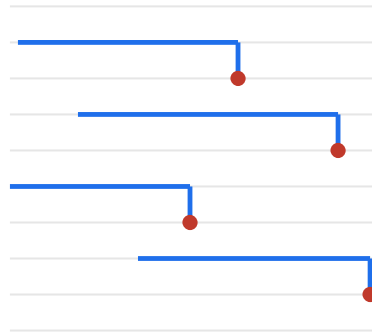
여기서 틀린다. 게이팅된 클럭은 CDC 와 STA 가 보기에 새 클럭이다. 제 제약이 필요하고 인에이블 자체도 타이밍을 봐야 한다 — 래치의 셋업을 어길 만큼 늦게 오는 인에이블은 래치가 막으려던 바로 그 글리치를 낳는다.

T19.4 배선, 두 바퀴와 수리 되돌이

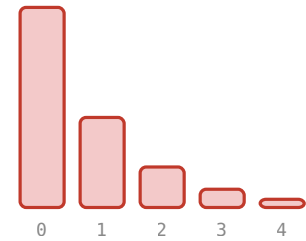
1. 전역 배선



2. 상세 배선



3. 탐색 수리



바퀴마다의 위반 수
비가 1로 가면 그 되돌이가 위반을 고치는 것이 아니라 옮기고 있다는 뜻이다.

전역 배선은 가장 큰 그래프 위의 흐름 문제를 풀기 위한 것인데, 상세 배선은 그 뒤를 이어서, 모든 세 배선 단계 뒤지기에는 너무 큰 기하 문제다.

그림 14. 서로 다른 세 문제를 서로 다른 세 방법으로 푼 것. 첫 칸의 빨간 칸은 수요가 공급을 넘은 gcell 이고, 둘째 칸의 빨간 네모는 상세 배선이 피하지 못한 DRC 위반이며, 셋째 칸은 수리 되돌이가 그것들을 가지고 하는 일이다.

표 43. 왜 배선이 알고리즘 하나가 아닌가

바퀴	무엇 위에서	무엇을 정하나	무엇을 일부러 무시하나
전역 배선	gcell 격자	넷마다 어느 gcell 들을 어느 층으로 지나는지	정확한 배선 자리, 대부분의 설계 규칙 — 기하 문제가 아니라 용량 문제다
상세 배선	gcell 안의 진짜 트랙과 비아	진짜 금속. 모든 토막, 모든 비아, 모든 우회	없다 — 이 바퀴는 DRC 가 깨끗한 기하를 내놓아야 한다
탐색 수리	남은 위반들	걸리는 배선을 뜯어내고 그 위반을 벌점으로 삼아 다시 배선한다	최적성. 국소 수리 되돌이이고, 수리마다 새 위반을 낳을 수 있어서 되풀이한다

둘로 나뉜 까닭은 두 문제의 꼴이 다르기 때문이다. 전역 배선은 마디 수천 개짜리 그래프 위의 흐름 문제라 거의 최적으로 풀린다. 상세 배선은 규칙 수백 개가 걸린 기하 문제이고 공간이 뒤지기에는 너무 커서, 넷 하나씩 욕심껏 푼다 — 그리고 그 욕심이 바로 뒤의 탐색 수리가 치우는 것이다.

표 44. 수렴하는 수리 되돌이: 바퀴마다 남은 것의 55 % 를 걷어낸다

되풀이	열린 위반	이번 바퀴에 걷어낸 것
0	4,200	0
1	1,890	0
2	850	0
3	383	0
4	172	0
5	78	0
6	35	0
7	16	0

예제 5. 이 수리 되돌이는 끝나겠는가?

주어진 것. 상세 배선 뒤 어떤 블록이 DRC 위반 4,200 개를 보고한다. 잇따른 탐색 수리 바퀴가 4,200 → 2,310 → 1,290 → 980 → 910 → 890 을 보고한다.

푸는 법. 처음 두 비는 0.55 와 0.56 쯤 — 깨끗한 등비 감쇠다. 마지막 셋은 0.76, 0.93, 0.98 이다. 감쇠가 멈췄다. 끝남을 정하는 것이 꼬리이므로 머리가 아니라 꼬리를 맞춘다.

답. 안 끝난다. 비가 1 로 가는 것은 남은 위반이 수리되는 것이 아니라 옮겨지고 있다는 뜻이다 — 같은 혼잡한 영역을 서로 주고받는 것이다. 890 쯤의 찌꺼기는 구조적이다. 그 넷들이 가야 하는 데에 트랙 공급이 모자란 것이다.

고침은 결코 수리를 한 바퀴 더 도는 것이 아니다. 위쪽에 있다. 그 영역의 점유율을 낮추거나, 배치를 퍼거나, 배선 층을 더하거나, 핀 접근을 바꾸는 것이다. T17 의 되돌이 값 산수가 실무에 닿은 것이다 — 고침은 원인이 있는 데서 싸고 증상이 있는 데서 비싸다.

여기서 틀린다. 구조적이라고 결론 내리기 전에 사소한 설명을 죽여야 한다. 그 위반들이 바퀴마다 새로 생기는 묶음이 아니라 같은 자리의 **같은** 것인지 확인해라. 바퀴 사이에 위반 좌표를 견줘 보라. 움직이면 되돌이가 전역 부족 위에서 헛돌고 있는 것이고, 가만히 있으면 국소 원인 하나다 — 흔히 닿을 수 없는 매크로 핀이다(T15.3).

안테나 위반 (antenna violation) — 회로가 아니라 제조에 관한 규칙

식각하는 동안, 게이트에 붙어 있으면서 아직 어느 확산에도 안 붙은 긴 금속 토막이 전하 수집기 노릇을 한다. 모인 전하가 얇은 산화막으로 방전되며 그것을 상하게 한다. 이 규칙은 붙은 금속 면적과 게이트 면적의 비를 **제조의 모든 중간 단계에서** 묶는다. 다 만들어진 칩에서가 아니다.

헛갈리게 만드는 것이 그 마지막 구절이다. 다 만들면 안테나가 깨끗한 레이아웃이 메탈 2 에서는 규칙을 어길 수 있다. 그 토막을 다이오드에 이어 주었을 메탈 3 이, 공정의 그 시점에는 아직 없기 때문이다.

고침은 둘이다. 긴 토막을 더 높은 층으로 뛰어 끊거나(그러면 어느 한 층에서도 수집기가 커지지 않는다), 방전 길을 주는 작은 다이오드를 더한다. 배선기가 둘 다 자동으로 끼워 넣는다. 그래도 이해해 둘 까닭은, 그 고침들이 보통 이미 결정적인 넷에서 면적과 지연을 치르기 때문이다.

어디에 쓰나	상세 배선 뒤, 물리 검증의 한 부분으로(T20).
언제 쓰나	늘. 고를 수 있는 것도 아니고 파운드리와 흥정할 수 있는 것도 아니다.
어떻게 쓰나	배선기가 고치게 두고, 그다음 어디서 고쳐야 했는지를 보라. 안테나 고침이 몰려 있는 자리는 길고 한 층에 있으며 게이트에 닿는 넷을 가리킨다 — 흔히 클럭이나 리셋인데, 더해진 다이오드 용량이 가장 반감지 않은 넷이 하필 그것이다.

여기서 틀린다. 클럭 앞에 붙은 안테나 다이오드는 CTS 가 이미 균형 잡아 놓은 가지에 용량을 더하는데, 그것이 CTS 뒤에 더해진다. 트리가 그것을 살아남았으려니 하지 말고 안테나 고침 뒤에 스큐를 다시 확인해라.

T19.5 월요일에 이걸로 무엇을 하나

표 45. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
배치 · 합법화 · HPWL · 타이밍주도배치	모든 배치 실행	합법화에서의 계단이 플로어플랜이 너무 뻥뻥하다고 말하는지
혼잡도 · 트랙 · gcell · 렌트법칙	배선 전, 그리고 조개기 전	이 블록이 이 크기로 배선되기는 하는지
클럭트리합성 · 삽입지연 · 클럭스큐 · 유용스큐 · 클럭메시 · H트리	배치 뒤	논리가 돌기도 전에 클럭 회로망이 주기를 얼마나 먹는지
클럭게이팅 · CGIC	RTL, 그다음 게이팅 보고서	가장 큰 동적 전력 손잡이 하나, 그리고 당신의 RTL 문체가 그것을 허락하는지
전역배선 · 상세배선 · 탐색수리 · 안테나위반	CTS 뒤	수리 되돌이가 수렴하는지, 안 하면 위쪽에서 무엇을 바꿔야 하는지

가지고 갈 수는 OCV의 곱셈이다. 공통 경로가 98 % 이고 디레이트가 ± 7 % 이면 삽입 지연의 15.86 % 가 해석된 스큐로 다시 나타난다. 클럭 트리를 균형만이 아니라 레이턴시까지 두고 설계하는 까닭이고, 클럭 메시가 그 전력에도 불구하고 있는 까닭이며, 클럭 트리를 얇게 두라는 조언의 정량적인 알맹이다.

이제 기하가 있고, 배선기가 보기에는 끝났다. 그것을 제조해도 되는지는 다른 물음이고, 다른 도구 묶음이 답한다. 그것이 다음 장이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

여기서 특허가 뻥뻥한 두 영역은 다시 발명하기 전에 알아 둘 만하다. **클럭 트리 구조** — 메시, 다중 근원 CTS, 그리고 위는 메시 아래는 트리인 혼합 방식 — 은 출원이 많다. 스큐가 주파수 값을 하고 주파수가 돈값을 하기 때문이다. **기계학습 혼잡 예측을 쓰는 상세 배선**도 그렇다. T17 에서 셈한 바로 그 되돌이 값 때문에 혼잡 정보를 앞으로 당기려는 것이다.

T20. 사인오프 — 지어도 되는지를 정하는 검사들

사인오프 검사는 저마다 좁은 물음 하나에 답하고, 실리콘까지 가는 실패는 그 사이의 틈에 산다.

읽기 전에 알아야 하는 것

사인오프 · 구현흐름 · DRC · LVS · 안테나위반 · 밀도규칙 · 추출 · SPEF · SDF · SDC · 정적타이밍분석 · OCV · 슬랙 · ECO · GDS · 전압강하 · 일렉트로마이그레이션 · 블랙식 · 상세배선 · 클럭스큐 · 문턱전압 · 누설전류 · 래치업

이 장에서 새로 나오는 것

물리검증 · ERC · 금속채움 · CMP · 코너 · MMMC · 온도 반전 · 정적IR · 동적IR · 스위칭벡터 · 기능ECO · 금속ECO · 스페어셀 · 테이프아웃 · 마스크

배선된 설계는 다각형 묶음 하나와 주장 하나다. 이 다각형들을 이 공정으로 만들면 RTL 이 말한 대로 거동할 것이라는 주장이다. 사인오프는 누군가 마스크 값을 치르기 전에 그 주장을 시험하는 독립된 검사들의 묶음이다. 검사마다 다른 도구이고 설계에 대한 모형이 다르니, 어느 검사가 어느 물음에 답하는지를 또렷이 해 둘 값이 있다 — **실리콘까지 가는 실패는 거의 언제나 검사 안이 아니라 두 검사 사이의 공간에 있기 때문이다.**

T20.1 검사마다 실제로 무엇을 주장하나

표 48. 사인오프 묶음을 주장으로 적은 것

검사	세우는 주장	못 보는 것
DRC	기하가 파운드리가 이 공정과 이 금속 적층에 대해 내놓은 모든 제조 규칙을 지킨다	그 기하가 설계를 구현하는지. DRC 가 깨끗한 빈 다이도 통과한다
LVS	레이아웃에서 추출한 소자와 연결이 넷리스트와 소자마다 넷마다 맞는다	넷리스트가 옳은지. LVS 는 레이아웃을 회로도와 견준다. 회로도가 틀렸으면 LVS 는 그 잘못을 확인해 준다
ERC	떠 있는 게이트 없음, 안 이어진 웰 없음, 흐르면 안 되는 소자를 거쳐 단락된 전원 없음	동적인 것은 무엇이든 — 위상 검사다
안테나 (T19)	식각 동안 게이트 산화막을 상하게 할 만큼 전하를 모으는 금속 토막이 없다	다 만들어진 회로에 대해서는 아무것도
밀도 / 채움	층마다의 국소 금속 밀도가 CMP 가 요구하는 창 안에 있다	채움이 더하는 용량 — 그것은 추출의 몫이고, 채움 뒤에 다시 돌려야 한다
EM / IR	요구되는 수명에 대해 전류 밀도 한계를 넘는 배선이 없고(T14), 예산 아래로 처지는 마디가 없다	그것을 몬 스위칭 활동이 진짜 작업부하를 닳았는지 — T20.3 을 보라
STA	모든 타이밍 경로가 목록의 모든 코너에서, 목록의 모든 모드에서 셋업과 홀드를 맞춘다	SDC 가 빼놓은 경로들, 그리고 아무도 적지 않은 모드들

검사마다 건전하고 좁으며 다음 검사에 눈이 멀어 있다

DRC	기하가 규칙을 지킨다	무엇을 구현하는지는 모른다
LVS	레이아웃이 넷리스트와 맞는다	넷리스트가 옳은지는 모른다
STA	목록에 든 모든 경로가 타이밍을 맞춘다	SDC 가 빼놓은 경로들
EM / IR	한계를 넘는 배선 없음	안 돌린 벡터들
안테나	식각 중 산화막 손상 없음	다 만들어진 회로에 대해서는 아무것도
밀도	CMP 창을 맞춘다	채움이 방금 더한 용량

왼쪽 칸이 사인오프가 세우는 것이다. 오른쪽 칸이 실리콘 실패가 사는 곳이다 -

검사 안이 아니라 두 검사 사이의 공간이다. 그 공간이 공학적 판단이다.

그림 15. 같은 표를 그림으로. 꼴이 곧 전할 말이기 때문이다. 검사마다 좁은 것을 세우고, 오른쪽의 빨간 칸이 실리콘까지 가는 실패가 실제로 사는 곳이다.

셋째 칸을 실리콘 실패 뒤에 어디를 들여다볼지의 목록으로 읽어라. 검사마다 건전하고, 저마다 좁고, 그 좁음은 일부러다. 모든 것을 세우려는 검사는 유한한 시간 안에 못 돌 것이기 때문이다. 사인오프의 공학적 판단은 검사를 돌리는 데 있지 않다 - 그 검사들의 합집합이 무엇을 빼놓는지를 아는 데 있다.

금속 채움 (metal fill) 과 그것이 무효로 만드는 추출

화학 기계 연마는 국소 무늬 밀도에 달린 속도로 재료를 깎아 낸다. 금속이 적은 영역은 빨리 깎여 더 얇아지는데, 그것이 저항과 그 위 유전체 두께를 둘 다 바꾼다. 그래서 파운드리들은 층마다의 밀도가 창 안에 들 것을 요구하고, 흐름은 빈 자리에 떠 있거나 접지된 금속 모양 — **채움** — 을 더해 그것을 맞춘다.

채움은 배선 뒤에 더해진다. 그 말은 타이밍 사인오프가 쓴 추출 **뒤에** 더해진다는 뜻이다. 그 모양들은 신호 배선 결의 금속이고 용량을 더한다. 접지된 채움은 예측할 수 있게 더하고, 떠 있는 채움은 더 적지만 모형으로 잡기 더 어려운 양을 더한다.

그러니 차례는 이렇다. 배선하고, 채우고, **다시 추출하고**, STA 를 다시 돌린다. 채움 전 추출로 타이밍을 사인오프한 흐름은 제조되지 않을 설계를 사인오프한 것이다.

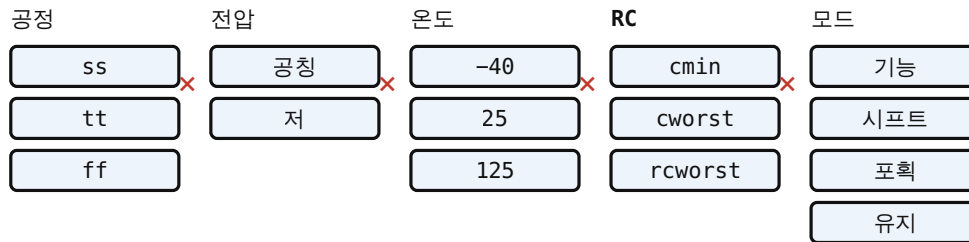
어디에 쓰나	상세 배선과 최종 타이밍 사인오프 사이.
언제 쓰나	늘. 다른 점은 채움을 P&R 도구가 넣느냐 물리 검증 도구가 넣느냐뿐인데, 둘의 결과가 똑같지는 않다.
어떻게 쓰나	채움을 넣고, 채움이 있는 채로 다시 추출하고, 코너 묶음을 다시 돌리고, 채움 전 실행과 슬랙을 견준다 . 작고 고른 악화는 예상된 것이고, 몇몇 넷에서 큰 변화는 길고 결정적이며 돌레가 텅 비어 있던 배선 결에 채움이 떨어졌다는 뜻이다.

실제 코드는 이렇게 생겼다

```
# The diff that must exist before tape-out:
#   path          slack pre-fill  slack post-fill  delta
#   u_dsp/.../mac_out[17]      +45 ps      +31 ps      -14 ps
#   u_if/.../clk_gate_en      +88 ps      +12 ps      -76 ps  <- look
# A -76 ps outlier is a wire that was alone in an empty region and is now
# surrounded by fill. It is real, it is predictable, and it is missed by any
# flow that signs off before filling.
```

여기서 틀린다. 채움은 고속 신호의 인덕턴스와 되돌아오는 길도 바꾸는데, 그것은 논리보다 T8 의 링크에 훨씬 중요하다. 그런 신호에는 채움 제외 구역을 요구해라 — 그리고 제외 구역은 돌레가 벌충해 주지 않으면 밀도 위반이라는 것을 잊지 마라.

T20.2 코너는 곱해지고, 대부분은 지배당한다



$3 \times 2 \times 3 \times 3 \times 4 =$ 코너·모드 조합 **216** 가지

$\times$ STA 2 h 씩 = 계산 **18** 일

그리고 ECO 하나가 그것을 통째로 버린다. T17.3 의 되돌이 값이 가장 비싼 꼴로 나타난 것이다.

그림 16. 왜 사인오프가 서버 농장에서 도는가. 축들이 서로 독립이라 더해지는 것이 아니라 곱해진다 — 그래서 코너 목록은 가정하는 것이 아니라 흥정하는 것이다.

유도 — 사인오프 실행 시간이 어디서 오나

	이렇게 된다	왜냐하면
1	공정: 3 · 전압: 2 · 온도: 3 · RC(배선): 3 · 모드: 4.	축마다 서로 독립이다 — 그래서 더해지지 않고 곱해진다. 개수는 가정한 것이고 흔한 값이다.
2	그러니 $3 \times 2 \times 3 \times 3 \times 4 = 216$ 가지 코너·모드 조합이다.	곱셈이다. 미묘한 일은 아무것도 안 난다. 그것이 요점이다.
3	STA 실행마다 2 h 면 온전한 묶음이 계산 432 h 다.	기계 한 대로 18 일이다 — 사인오프가 농장에서 도는 까닭이고, 코너 목록을 가정하지 않고 흥정하는 까닭이다.
4	ECO 하나가 그것을 통째로 무효로 만든다.	T17 의 되돌이 값 산수가 가장 비싼 꼴로 나타난 것이다. 사슬의 마지막 단계가 가장 크다.

표 49. 축들, 그리고 저마다 왜 거기 있나

축	값	왜 뺄 수 없나
공정	3	느림 / 전형 / 빠름 — 그리고 혼성신호 부품이면 그 위에 slow-n/fast-p 코너까지
전압	2	공칭과 레귤레이터 허용 범위의 아래 끝
온도	3	-40 °C, 25 °C, 125 °C
RC (배선)	3	cmin / cworst / rcworst — 추출 코너는 소자 코너와 독립이다
모드	4	기능, 스캔 시프트, 스캔 포획, 저전력 유지

MMMC — 모드와 코너를 하나씩이 아니라 함께 해석하기

현대 흐름에서 위의 216 가지 조합은 따로 노는 216 번의 해석이 아니다. 다중 모드 다중 코너 해석은 여러 모드 제약과 여러 코너 라이브러리를 한 세션에 올려 한 번만 전파하고, 그 사이에 공통인 일을 나눠 쓴다.

아끼는 것은 진짜지만 그것이 쓰는 주된 까닭은 아니다. 주된 까닭은 **최적화가 모든 코너를 한꺼번에 봐야 한다**는 것이다. 느린 코너의 셋업 위반을 구동력을 키워 고치면 빠른 코너에서 홀드 위반을 만들 수 있다. 코너를 하나씩 보는 도구는 둘 사이를 오가고, 그 진동이 바로 $p \geq 1$ 인 T17 의 비수렴이다.

어디에 쓰나	CTS 뒤 최적화부터, 그리고 사인오프 전부.
언제 쓰나	코너에 따라 달라지는 맞바꿈을 할 수 있는 첫 최적화부터 — 실무에서는 홀드 고침이 시작되 자마자다.
어떻게 쓰나	해석 뷰를 대놓고 정의하고(뷰 하나는 모드 하나 더하기 지연 코너 하나 더하기 RC 코너 하나 다), 어느 것이 셋업용이고 어느 것이 홀드용인지 표시하고, 그 목록이 지난 프로젝트에서 물려받은 것이 아니라 이 제품의 진짜 동작 범위를 덮는지 확인한다.

실제 코드는 이렇게 생겼다

```
# An analysis-view list is a specification, not a configuration:
# view      mode      delay corner  rc corner  checks
# func_ss_cw functional ss_0.9v_125c cworst     setup
# func_ff_cb functional ff_1.1v_m40c cbest      hold
# shift_ss_cw scan_shift ss_0.9v_125c cworst     setup
# shift_ff_cb scan_shift ff_1.1v_m40c cbest      hold
# The question to ask of this table is not "is it right" but
# "which operating condition of the product is NOT in it".
```

여기서 틀린다. 스캔 시프트 모드가 가장 자주 잊히고 가장 자주 깨진다. 클럭 구조가 기능 모드와 다르고 홀드 요구가 가혹하기 때문이다. 시험기에서만 지는 부품은 대개 결함이 아니라 시프트 모드 타이밍 문제다.

코너 목록이 긴 까닭의 일부는 사람들이 기대하는 지배 관계가 성립하지 않기 때문이다. 직관은 느린 코너가 셋업을 묶고 빠른 코너가 홀드를 묶으니 코너 둘이면 되겠다는 것이다. 그 직관은 지연이 온도를 따라 단조롭게 커지는 영역에서 온 것인데 — 현대의 저전압 동작은 그 영역에 있지 않다.

유도 — 온도 반전을, T1 의 소자 모형에서

	이렇게 된다	왜냐하면
1	지연 $\propto C \cdot V / I_d$ 이고 $I_d \propto \mu(T) \cdot (V - V_{th}(T))^\alpha$ 다.	T1 의 α 제곱 꼴이다. 단채널 소자에서 $\alpha \approx 1.3$ 이다.
2	이동도는 온도를 따라 떨어진다. $\mu \propto T^{-1.5}$ 다. 그래서 뜨거우면 느려진다.	포논 산란. 누구나 아는 효과다.
3	문턱도 온도를 따라 떨어진다. $dV_{th}/dT \approx -0.8 \text{ mV/K}$ 다. 그래서 뜨거우면 빨라진다.	페르미 준위가 온도를 따라 움직인다. 잊히는 효과가 이것이다.
4	어느 쪽이 이기는지는 오버드라이브 $V - V_{th}$ 에 달렸다. 문턱 이동이 작은 오버드라이브에서는 큰 몫이고 큰 오버드라이브에서는 작은 몫이기 때문이다.	두 효과가 곱으로 들어오므로, 견줄 것은 둘의 로그 미분이다.
5	$d(\ln \text{지연})/dT = 1.5/T - \alpha\kappa/(V - V_{th})$ 다. 0 으로 두면 $V - V_{th} = \alpha\kappa T/1.5$ 가 나온다.	미분해서 푼 것이다. 결과는 두 온도계수와 α 말고는 아무것에도 안 달렸다.
6	300 K 에서 그것은 오버드라이브 0.208 V, 곧 $V \approx 0.658$ V 다.	그 전역 아래에서는 부품이 차가울 때 가장 느리고, 코너 목록에 셋업을 위한 두 온도 끝이 다 들어 있어야 한다.

표 50. 온도에 대한 지연, 25 °C 로 정규화

전원	-40 °C	25 °C	125 °C	어느 쪽이 느린 코너인가
1.8 V	0.728 ×	1 ×	1.432 ×	보통
1.2 V	0.759 ×	1 ×	1.352 ×	보통
0.9 V	0.812 ×	1 ×	1.247 ×	보통
0.75 V	0.887 ×	1 ×	1.133 ×	보통
0.65 V	1.026 ×	1 ×	0.994 ×	반전
0.6 V	1.211 ×	1 ×	0.881 ×	반전

이 표가 0.6 V 에서 멈추는 데는 적어 둘 만한 까닭이 있다. -40 °C 에서 이 모형의 문턱이 0.503 V 이므로, 전원이 0.5 V 쯤 아래로 내려가면 차가울 때 소자에 오버드라이브가 아예 없고 지연은 그저 긴 것이 아니라 정의되지 않는다. 인공물이 아니다 — 문턱 근처 영역이고, 저전압 유지 모드가 차가운 코너에서 최소 전원을 규정해 두는 까닭이다.

독립 대조. 교차 전원을 두 번 셸했다. 한 번은 닫힌 꼴 $V = V_{th0} + \alpha\kappa T/1.5$ 로 0.658 V 가 나왔고, 한 번은 그 대수를 하나도 안 쓰고 수식으로 미분한 지연의 부호를 이분법으로 좁혀 0.658 V 가 나왔다 — 어긋남이 $2.53 \times 10^{-8} \text{ mV}$ 다. 그리고 사소한 설명은 말로 물리치는 대신 표로 죽인다. 이 효과가 정규화의 인공물이라면 '반전' 판정이 전원을 따라 움직이지 않을 텐데, 움직인다.

실제로 난 일

125 °C 와 전형 전압에서 사인오프한 설계가 차가운 시동 시험에서 -40 °C 에 졌다. 저전압 유지 모드였다. 그 전원에서 부품은 차가울 때 가장 느렸는데, 직관이 성립하던 1.8 V 프로젝트에서 물려받은 코너 목록에는 셋업용으로 뜨

거운 코너만 들어 있었다. 흐름에는 아무것도 안 부서져 있었다. 무엇을 검사할지의 목록을, 더는 참이 아니게 된 규칙을 적용하는 사람이 썼던 것이다.

T20.3 EM 과 IR 사인오프에는 설계에 없는 것이 필요하다

정적 IR (static IR) · 동적 IR (dynamic IR) · 스위칭 벡터 — 정적 IR 해석은 평균 전류를 격자에 흩어 놓고 저항 회로망을 푼다 — T18 의 셈이다. 동적 IR 해석은 전원의 인덕턴스와 디커플링까지 넣어(T13) 격자를 푸는데, 어떤 스위칭 무늬가 낳는 전류 파형이 그것을 본다. 그 무늬가 **스위칭 벡터**이고, 그것은 설계의 일부가 아니다. 누가 대줘야 한다.

이것은 답이 넷리스트에도 제약에도 없는 입력에 달려 있는 유일한 사인오프 검사다. 조용한 벡터를 고르면 해석이 통과하고, 가장 나쁜 것을 고르면 안 통과할 수 있다 — 그리고 흐름의 무엇도 당신이 어느 쪽을 골랐는지 말해 주지 않는다. **정직한 절차는 사인오프 보고서에 어느 벡터를 썼고 왜 그것이 진짜 작업부하를 한계 짓는다고 보는지를 적는 것이고, 그 진술이 없는 동적 IR 통과는 결과가 아니라 확인되지 않은 주장으로 다루는 것이다.**

표 51. 벡터를 얻는 세 길, 그리고 저마다 얼마나 값을 하나

출처	얼마나 대표적인가	무엇을 놓치나
진짜 작업부하의 기능 시뮬레이션	쓸 수 있는 것 중 가장 낫다 — 실제 스위칭이다	시뮬레이션한 작업부하만 덮고, 긴 시뮬레이션은 어마어마한 활동 파일을 낳는다
벡터 없이 (토글을 몇 개에서 도구가 확률을 전파)	첫 바퀴와 큰 격자 잘못 찾기에는 충분하다	상관. 블록들이 서로 독립으로 뒤집힌다고 가정하는데, 실패는 전부가 한꺼번에 뒤집힐 때 온다
스캔 포획 패턴	많은 설계에서 최악의 경우인데 좀처럼 해석하지 않는다	없다 — 흔히 그것이 진짜 답이다. 스캔 포획은 어느 기능 모드보다도 다이의 훨씬 많은 부분을 동시에 뒤집기 때문이다

예제 8. 어느 모드가 침투 전류를 정하나?

주어진 것. T18/T19 의 블록은 기능적으로 350 mW 를 끌어간다. 스캔 시프트 모드에서는 모든 플롭이 주기마다 뒤집히고, 시프트 클럭은 기능 주파수의 4분의 1로 돈다.

푸는 법. 논리의 기능 스위칭 활동도는 보통 주기당 마디의 10-20 % 이고, 시프트 활동도는 주기당 플롭의 50 % 에 다가간다. 클럭 회로망은 고정으로 두고(어느 쪽이든 뒤집힌다) 플롭 출력 스위칭을 건준다. 대략 0.15 → 0.5 이니 그 성분에서 3.3 배쯤이고, 느려진 클럭에서 0.25 배다.

답. 시프트 모드는 그 성분에서 기능 스위칭 전력의 대략 **0.8배** 를 끌어간다 — 엇비슷하고, 눈에 띄게 작지 않다. 침투는 높은 활동도에서 클럭이 가장 빠르게 도는 모드가 정하는데, 많은 부품에서 그것은 시프트가 아니라 스캔 포획이다.

시험 담당자가 시프트 주파수를 낮추고 동시 토글을 묶는 저전력 ATPG 를 쓰는 까닭도 이것이다 — 시험 품질 때문이 아니라 부품이 그 전류를 댈 수 없기 때문이다. 그 제약은 T18 의 플로어플랜 논의에 속하는데, 대개 그 뒤에 도착한다.

여기서 틀린다. 위의 수들은 측정이 아니라 가정들의 비이고, 결과로 대서는 안 된다. 살아남는 결론은 구조적인 것이다. **시험 모드는 저전력 모드가 아니다.** 기능 벡터로만 사인오프한 격자는, 출하되는 모든 부품에서 시험기 위에서 가장 먼저 돌 그 모드에 대해서는 사인오프되지 않은 것이다.

T20.4 사인오프한 설계를 바꾸기

표 52. 두 가지 ECO, 그리고 저마다 무엇을 치르나

종류	무엇이 바뀌나	어느 사인오프를 다시 해야 하나
금속만 바꾸는 ECO (스페어 셀을 쓴다)	배선만. 기능은 이미 놓여 있고 이미 마스크에 있는 셀들에서 나오고, 금속 층에서 다시 이어질 뿐이다	배선 뒤의 전부. 추출, STA, EM/IR, DRC, LVS. 그런데 바뀐 마스크만 다시 만든다 — 그것이 요점 전부다
기능 ECO (새 셀)	배치와 배선. 새 인스턴스가 틈으로 들어가며 거기 있던 것을 밀어낸다	위의 전부, 그리고 마스크가 전부 새것이다

스페어 셀 (spare cells) — 나중에 바꿀 권리를 지금 사 두기

스페어 셀은 안 쓰는 게이트다. 보통 NAND 와 NOR 와 인버터와 플롭을 섞어 배치 곳곳에 흩어 놓고 입력을 묶어 둔다. 면적과 누설을 먹으면서 아무 일도 안 한다. 그것이 사 주는 것은 **금속만 다시 이어** 논리 변경을 구현할 수 있는 능력이다. 그 변경에 필요한 트랜지스터가 이미 확산 마스크에 있기 때문이다.

경제는 곧바른 옵션 가격이다. 첨단 마디의 온전한 마스크 묶음은 수백만 달러이고, 금속만 바꾸는 개정은 그 얼마이며 몇 달이 아니라 몇 주가 걸린다. 변경이 필요할 확률이 무시할 만하지만 **않**다면 면적의 1-2 % 를 스페어에 쓰는 것은 싼 보험이다 — 그리고 첫 실리콘에서는 결코 무시할 만하지 않다.

어디에 쓰나	배치에서 끼워 넣는다. 뭉쳐 놓지 말고 흩어서.
언제 쓰나	첫 실리콘이 있을 모든 설계에서. 검증된 부품의 잘 아는 파생품에서만 뺀다.
어떻게 쓰나	어느 영역에서도 손닿는 데에 몇 개는 있도록 흩는다 — 고칠 자리에서 500 μm 떨어진 스페어 셀은 못 쓴다. 거기까지 가는 배선이 그 고침이 맞추려던 타이밍을 깨기 때문이다. 플롭도 넣어라. 예측 못 하는 변경은 대개 순차 논리다.

실제 코드는 이렇게 생겼다

```
# A spare-cell plan is a distribution, not a count:
# region      spares: inv  nand2  nor2  dff      nearest-spare distance
# core NW      24    16    16    8          < 60 um
# core NE      24    16    16    8          < 60 um
# near macro A   6     4     4     2          < 90 um  <- thin
# 'We have 400 spare cells' says nothing. 'No point in the core is more
# than 90 um from a spare flop' is the statement that makes an ECO possible.
```

여기서 틀린다. 스페어 입력을 제대로 묶어 두고 ERC 가 묶인 것으로 보는지 확인해라. 안 묶인 스페어 게이트는 떠 있는 입력이다 — 반쯤 켜진 인버터로 지나친 전류가 흐르는 실패 방식인데, DRC 도 LVS 도 STA 도 그것을 찾고 있지 않아서 다 통과한다.

테이프아웃 (tape-out) — GDS 를 마스크 공장에 내보내는 시점. 기술적인 사건이라기보다 상업적인 사건이다. 그 뒤로는 설계를 바꾸는 데 마스크 묶음 하나가 든다. 이 장이 기술하는 모든 것은 그 내보냄이 안전하다는 주장을 세우려고 있다 — 그리고 그 주장의 정직한 끝에는 늘 **확인하지 않은 것**의 목록이 함께 있다.

T20.5 월요일에 이걸로 무엇을 하나

표 53. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
물리검증 · ERC · 금속채움 · CMP	모든 테이프아웃 전	그 기하를 제조할 수 있는지, 그리고 실제로 만들어질 그 기하 위에서 타이밍을 사인오프했는지
코너 · MMMC · 온도반전	해석 뷰 목록을 쓸 때	코너 목록이 이 제품을 덮는지 지난 프로젝트를 덮는지
정적IR · 동적IR · 스위칭벡터	전원 무결성 사인오프	그 벡터가 진짜 작업부하를 — 시험 모드까지 — 한계 짓는지
기능ECO · 금속ECO · 스페어셀	필요해지기 몇 달 전, 배치에서	실리콘 뒤의 고침이 금속 개정으로 끝나는지 온전한 마스크 묶음이 드는지
테이프아웃 · 마스크	내보냄 결정	당신이 무엇을 주장하고 있고 무엇을 주장하지 않는지

가지고 갈 수들. 코너·모드 조합 216 가지에 2 h 썩이면 계산 18 일이고 ECO 하나가 그것을 버린다 — 일찍 고치라는, 이 책에서 가장 센 정량적 논거다. 그리고 교차 오버드라이브 0.208 V 가 "느린 것은 뜨거운 것" 이 안전한 규칙이기를 그만둔 까닭이다. 이 모형에서 전원이 0.66 V 쯤 아래로 내려가면 느린 코너는 차가운 쪽이다.

이제 부품을 지을 수 있다. 낱알의 다이가 도는지는 또 다른 물음이고, 도구가 아니라 다이를 물고 있는 기계가 답한다 — 그것이 이 갈래의 마지막 장이다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

사인오프 자체는 흐름에서 특허가 가장 덜 걸린 자리다 — 규칙은 파운드리 것이고 검사는 범용품이다. 출원이 뻥뻥한 것은 **ECO 자동화**다. 어느 스페어 셀을 쓸지 고르고 흔들림을 가장 적게 하며 다시 배선해서, 이미 값을 치른 사인오프를 고침이 무효로 만들지 않게 하는 것이다. T17 에서 셈한 값을 곧바로 치는 것이다.

T21. 검증 환경 — 무엇을 증명하나

커버리지는 해 봤다고 말한다. 됐다고 말하는 것은 검사기뿐이다. 둘을 헛갈리는 것이 검증에서 가장 비싼 잘못이다.

읽기 전에 알아야 하는 것

구현흐름 · 넷리스트 · CDC검증 · 준안정 · 동기화기깊이 · 비동기FIFO · 핸드셰이크 · 그레이코드 · 사인오프 · ECO · 정적타이밍분석 · 타이밍수렴

이 장에서 새로 나오는 것

검증환경 · 테스트벤치 · 에이전트 · 시퀀스 · 시퀀서 · 드라이버 · 모니터 · 스코어보드 · 참조모델 · 가상인터페이스 · 기능커버리지 · 커버그룹 · 커버포인트 · 크로스커버리지 · 제약랜덤 · 어서션 · 커버리지닫기 · 지시시험 · 회귀

흐름에서 이 장의 단계 뒤에 오는 모든 것이 RTL 이 옳다고 가정한다. 합성은 버그를 충실히 구현할 것이고, 배치·배선은 그것 위에서 타이밍을 달을 것이며, 사인오프는 그것을 제조할 수 있다고 증명해 줄 것이다. **설계가 하기로 한 일을 하는지에 답할 수 있는 단계는 검증뿐이고**, 거의 모든 프로젝트에서 공학적 노력을 가장 많이 먹는 단계가 그것이다. 이 장은 그 노력이 갖는 구조에 관한 것이고, 그 노력이 값을 했는지를 정하는 구별 하나에 관한 것이다.

T21.1 구조, 그리고 왜 그렇게 나뉘어 있나

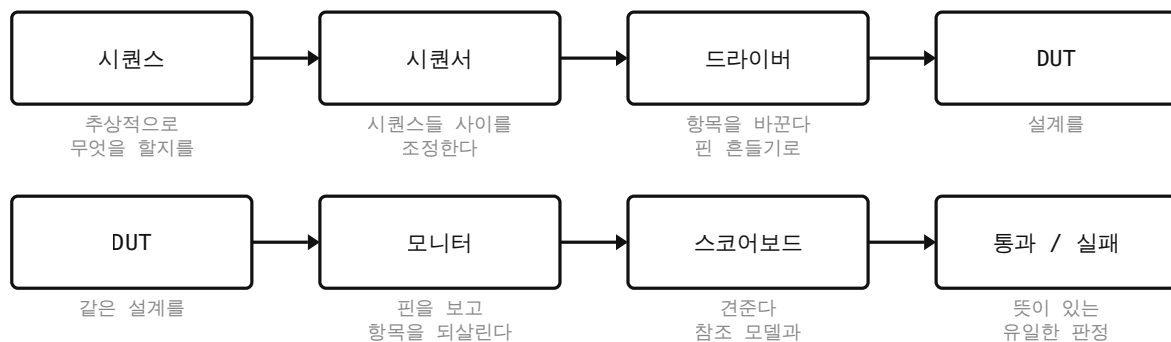
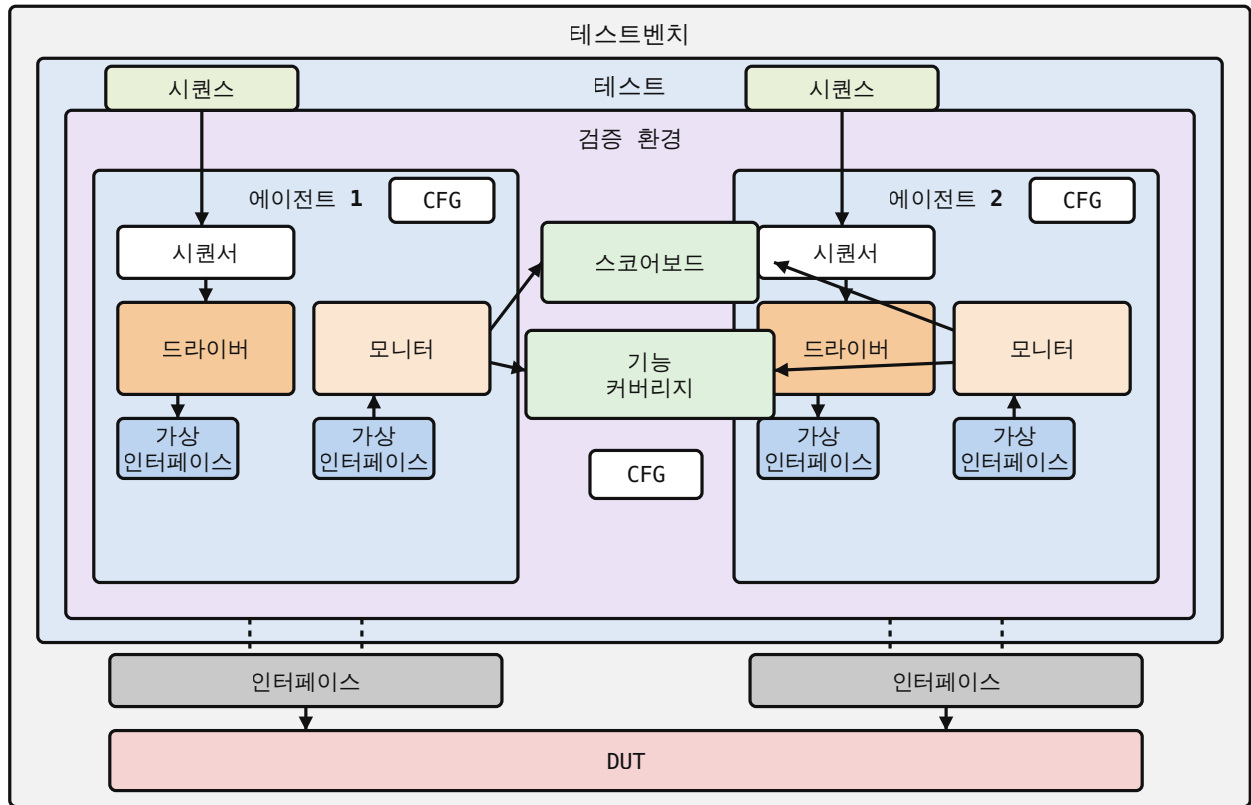


그림 17. 에이전트의 두 쪽. 위쪽 길은 물고, 아래쪽 길은 보고 판정한다. 둘은 일부러 서로 안 이어져 있다 — 모니터가 핀에서 무슨 일이 났는지를 되살리므로, 드라이버 버그가 스코어보드에 제 의도를 일러 주며 제 몸을 숨길 수 없다.



위쪽 길은 물고 아래쪽 길은 본다. 둘은 일부러 서로 안 이어져 있다 –
 드라이버의 의도를 믿는 모니터는 엉뚱한 것을 본 드라이버를 못 잡아낸다.
 커버리지는 해 봤다고 말한다. 됐다고 말하는 것은 스코어보드뿐이다.

그림 18. 환경 전체. DUT에서만 만나는 두 쪽으로 읽어라. 시퀀스가 에이전트마다 왼쪽으로 내려가며 물고, 모니터가 오른쪽으로 올라오며 보고, 가운데의 스코어보드가 설계가 틀렸다고 말하는 유일한 것이다.

표 54. 구성요소마다, 그리고 그것이 답하는 물음 하나

구성요소	무엇에 답하나	왜 이웃과 떨어져 있나
시퀀스	어떤 거래가 일어나야 하나?	시험마다 쓰이고 핀에 대해서는 아무것도 모른다 — 그래서 같은 시퀀스가 다른 버스 프로토콜에서도 돈다
시퀀서	여러 시퀀스가 다룰 때 어느 차례로?	조정은 정책이고, 정책은 시험마다 바뀌는데 드라이버는 안 바뀐다
드라이버	거래 하나를 선 위에서 어떻게 나타내나?	프로토콜의 타이밍을 아는 유일한 구성요소다. 그 위의 모든 것은 프로토콜을 모른다
모니터	선 위에서 실제로 무슨 일이 났나?	드라이버와 독립이어야 한다. 드라이버의 의도를 믿는 모니터는 엉뚱한 것을 본 드라이버를 잡아낼 수 없다
스코어보드	옳았나?	참조 모델을 지고 있다. 그것이 지는 것만이 DUT가 틀렸다는 뜻인 유일한 구성요소다
커버리지 수집기	무엇을 해 봤나?	판정하면 안 된다. 커버리지와 검사를 섞으면 아무것도 아닌 것의 100%를 보고하는 테스트벤치가 나온다
설정 객체	위의 것 가운데 어느 것이 있고, 어떻게 차려지나?	환경 하나가 블록과 서브시스템과 칩을 다 섬기게 하려고. 해당 없는 에이전트는 지우는 것이 아니라 꺼 둔다

가상 인터페이스 (virtual interface) — 소프트웨어에 대응물이 없는 유일한 구성물

테스트벤치의 드라이버와 모니터는 **클래스 객체**다. 실행 시간에 만들어지고, 쓰레기 수집되고, 핸들로 건네진다. 그것들이 흔들어야 하는 핀은 **모듈 인스턴스**의 것인데, 그것은 엘라보레이션에서 만들어져 시뮬레이션 내내 존재한다. 클래스는 정적으로 엘라보레이션된 객체를 곧바로 가리킬 수 없다. 클래스를 쓰는 시점에 그 인스턴스가 아직 없기 때문이다.

가상 인터페이스가 그 다리다. 클래스가 쥐고 있는 핸들이, 엘라보레이션 된 인터페이스 인스턴스를 가리킨다. 테스트벤치의 설정 단계가 그것을 채워 주고, 그다음부터 드라이버는 동적인 코드에서 진짜 핀을 몰 수 있다.

새 환경에서 가장 흔한 실패는 널인 가상 인터페이스다 — 아무것도 그것을 채워 주기 전에 드라이버가 지어진 것이다. 미묘한 버그는 아닌데 낯설다. 보통의 소프트웨어 언어에는 두 객체 수명이 이렇게 갈라지는 일이 없기 때문이다.

어디에 쓰나	DUT 에 닿는 모든 에이전트.
언제 쓰나	어떤 시퀀스가 돌기 전, build/connect 단계에서 채운다.
어떻게 쓰나	최상위 모듈에서 설정 데이터베이스로 넣고, 드라이버와 모니터마다 build 에서 꺼내 오게 한다 — 그다음 천 주기 뒤에 첫 핀 접근이 널 핸들 오류로 죽게 두지 말고 널이 아님을 단언해라 .

실제 코드는 이렇게 생겼다

```
// Top level, where both worlds meet:
my_if dut_if (.clk(clk));           // elaborated: exists for the run
my_dut u_dut (.p(dut_if));

initial begin
    // Hand the elaborated thing to the dynamic world, by name:
    uvm_config_db#(virtual my_if)::set(null, "*", "vif", dut_if);
    run_test();
end

// In the driver's build phase -- and check it, do not assume it:
if (!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
    `uvm_fatal("NOVIF", "virtual interface was never set")
```

여기서 틀린다. 여기서는 클러킹 블록이 중요하다. 그것 없이 테스트벤치에서 DUT 입력을 물면 클럭 에지에서 DUT 자신의 평가와 경합이 나고, 그 경합은 시뮬레이터마다 다르게 풀린다. 한 시뮬레이터에서 통과하고 다른 데서 지는 테스트벤치는 거의 언제나 도구 버그가 아니라 빠뜨린 클러킹 블록이다.

T21.2 커버리지는 자극의 척도이지 옳음의 척도가 아니다

기능 커버리지 (functional coverage) · 커버포인트 · 크로스 — **커버포인트**는 값을 표본으로 떠 칸에 갈라 넣는다. **커버그룹**은 커버포인트를 모아 정해진 사건에서 표본을 뜬다. **크로스**는 커버포인트 둘 이상의 칸 조합 묶음이다. 커버리지는 적어도 한 번 맞은 칸의 묶이다. 그 정의에 **없는** 것을 눈여겨보라. 그 칸이 맞았을 때 설계가 옳게 거동했는지에 대한 언급이 하나도 없다.

한 번도 안 터지는 스코어보드를 달고 얻은 기능 커버리지 100 % 는 아무것도 아닌 것의 100 % 짜리 증거다. 커버리지는 '자극이 이 상황에 닿았나' 에 답한다. 옳음은 스코어보드와 어서션만이 답한다. 둘은 나란히 보고되고, 비슷해 보이고, 전혀 다른 것을 낸다 — 그리고 초록인 커버리지 보고서는 버그를 내보내는 가장 설득력 있는 방법이다. 수이고, 높고, 그것이 무엇을 썼는지 아무도 안 묻기 때문이다.

실제로 난 일

모든 커버그룹이 100 % 인 채로 블록이 나갔다. 스코어보드의 비교 메서드에 디버깅하다 넣은 이른 `return` 이 있었고 안 지워졌다. 버스트마다 첫 박만 검사하고 나머지는 소리 없이 통과시켰다. 모든 칸이 맞았다. 아무것도 견주지 않았다. 그 새어 나감은 고객이 찾았고, 그것을 놓친 테스트벤치는 내내 완벽한 커버리지 보고서를 갖고 있었다. 그것을 잡았을 검사는 커버리지를 더 하는 것이 아니다 — 자해 검사다. DUT 출력의 비트 하나를 바꾸고 스코어보드가 빨개지는지 확인하는 것이다.

자해 검사 (mutation testing) 를 테스트벤치에

테스트벤치는 설계가 틀렸을 때 져야 하는 프로그램이다. 보통의 회귀에서는 그 성질을 한 번도 부려 보지 않는다 — 실행마다 (그때쯤엔) 옳은 설계를 상대하므로 실패 경로를 한 번도 안 탄다.

고침은 설계를 일부러 망가뜨리는 것이다. 응답의 비트 하나를 뒤집고, 거래 하나를 떨어뜨리고, 핸드셰이크를 한 주기 늦추고, 버스트 가운데 바이트 하나를 망가뜨린다. 그다음 회귀를 돌려 빨개지는 것을 확인하고, 어느 검사가 잡았는지를 확인한다. 아무것도 못 잡는 변이는 검사의 구멍이고, 그 자리가 정확히 쑤인다.

이 책이 제 수들에 적용하는 것과 같은 규율이다. 틀리게 나올 수 없는 측정은 아무것도 잰 것이 아니다.

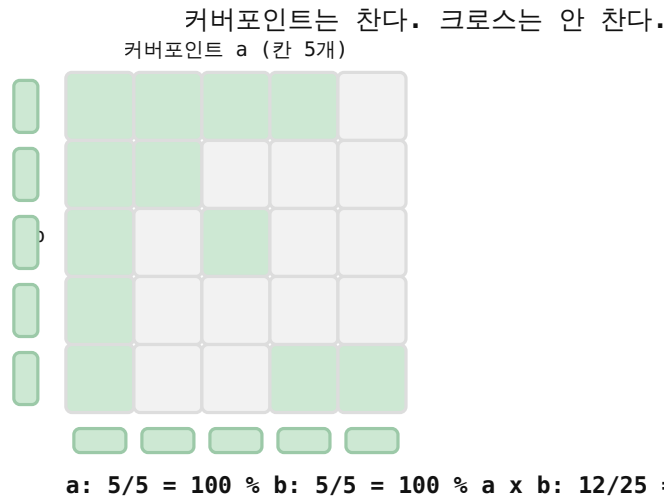
어디에 쓰나	모든 환경에서, 큰 블록마다 적어도 한 번씩. 실행 시간을 감당할 수 있으면 CI 에서도.
언제 쓰나	커버리지 보고서를 믿기 전에. 변이 작업은 그 커버리지 수가 검사기에 붙어 있기는 한지에 대한 유일한 증거다.
어떻게 쓰나	변이 목록을 파일로 두고, 저마다 그것을 잡아야 하는 검사를 적어 둔다. 돌려 본다. 어느 검사도 못 잡았거나 시간 초과로만 잡힌 변이는 발견이다.

실제 코드는 이렇게 생겼다

```
# mutations.txt -- and the expected catcher, which is the real content
# id mutation expected catcher
# m1 invert resp[0] scoreboard data compare
# m2 drop every 8th write scoreboard transaction count
# m3 delay ready by 1 cycle protocol assertion a_ready_stable
# m4 return wrong burst length scoreboard length compare
# m5 corrupt byte 3 of burst scoreboard data compare
#
# Result column after the run. Any row whose catcher is "none" or
# "test timeout" is a hole -- a timeout means nothing noticed, it just
# stopped.
```

여기서 틀린다. 모든 검사가 다 잡는 변이도 좋은 결과가 아니다 — 그 변이가 너무 거칠어서 아무 자리도 짚어 주지 못했다는 뜻이다. 쓸모 있는 변이는 정확히 검사 하나가 잡는 것들이다. 그것들이 검사와 그 검사가 책임지는 실패를 맞대어 주기 때문이다.

T21.3 왜 크로스가 터지나: 무작위 닫기의 산수



고른 무작위 자극으로 칸 K 개를 채우는 데 시험이 $K \cdot \ln K$ 번쯤 든다. 100 칸짜리 둘을 크로스하면 $K = 10,000$ 이다.
시험 519 번이 97,876 번이 된다. 크로스는 모형에 더하는 것이 아니라 닫는 노력에 곱하는 것이다.

그림 19. 축마다 온전히 덮였는데 크로스는 안 덮였다. 가장자리의 막대가 커버포인트 보고서가 보이는 것이고, 격자가 닫기가 실제로 요구하는 것이다.

유도 — 무작위 자극이 칸 K 개를 채우는 데 얼마나 걸리나

	이렇게 된다	왜냐하면
1	시험마다 K 개의 칸 중 하나에 고르게, 서로 독립으로 떨어진다고 하자.	따져 볼 가정이 이것이다. 진짜 제약 랜덤 자극은 고르지도 독립이지도 않다 — 그런데 이 경우가 낙관적인 쪽이므로, 여기서 셈하는 것은 드는 노력의 한이다.
2	이미 j 개가 맞았을 때 시험 하나가 새 칸을 맞출 확률은 $(K-j)/K$ 다.	세기다. 남은 칸이 K 개 중 $K-j$ 개다.
3	그러니 j 에서 j+1 로 가는 데 드는 기대 시험 수는 $K/(K-j)$ 다.	기하 대기 시간의 평균은 $1/p$ 다.
4	$j = 0 \cdots K-1$ 에 걸쳐 더하면 $K \cdot (1 + 1/2 + \cdots + 1/K) = K \cdot H_K$ 다.	기다림들이 서로의 결과와 독립이 아니어도 기댓값은 더해진다.
5	$H_K \approx \ln K + 0.577$ 이므로 값은 $K \cdot \ln K$ 쯤이다.	칸 수에 대해 선형보다 아주 조금 나뉘 뿐이고, 그것은 괜찮다. 문제는 이 인자가 아니라 크로스할 때 K 자체가 곱이라는 것이다.

표 55. 모든 칸을 채우는 데 드는 기대 시험 수, 서로 독립인 두 길로

칸 K	$K \cdot H_K$	찰 때까지 뽑아 보는 시능	어긋남
10	29.29	29.564	0.94 %
100	518.74	519.18	0.08 %
1,000	7,485.5	7,356	1.73 %
10,000	97,876.1	시능 안 함	요점이 그것이다. 이미 10^5 번쯤이다

두 칸은 서로 다른 수단으로 섰한 것이다 — 더한 급수와, 그것을 하나도 안 쓰는 진짜 되풀이 뽑기 — 그리고 몇 퍼센트 안에서 맞는데, 남는 것은 가장 큰 경우에서 시행 300 번의 몬테카를로 표본 오차다. 그것이 마지막 줄을 허락하는 것이고, 중요한 것이 그 줄이다. **100 칸짜리 커버포인트 둘을 크로스하면 $K = 10,000$ 이 되고, 시험 518.7 번짜리 일이 97,876.1 번짜리가 된다.** 크로스는 커버리지 모형에 조금 보태는 것이 아니라 달는 노력에 곱하는 것이다.

표 56. 그리고 그것이 낙관적인 경우였다 — 드문 칸 하나가 전부를 지배한다

가장 드문 칸의 확률	한 번 맞히는 데 드는 기대 시험 수	실무에서 무슨 뜻인가
0.01	100	넉넉하다
0.001	1,000	밤새 도는 회귀
1.000×10^{-4}	10,	밤새 도는 회귀
1.000×10^{-5}	100,	밤새 도는 회귀
1.000×10^{-6}	1.0000×10^6	영영 안 난다 — 지시 시험을 써라

고른 경우의 섰은 문제를 한참 과소평가한다. 무작위 닫기를 다스리는 것이 평균이 아니라 **가장 드문** 칸이기 때문이다. 마지막 줄들이 제약 랜덤 흐름에서도 지시 시험이 여전히 있는 정량적인 까닭이다. 그 제약 아래에서 확률이 10^{-6} 인 구석에는 기계를 더 붙여서 달지 못한다. 거기에는 시험 하나를 위해 제약을 바꾸어 달거나 그 시험을 손으로 써서 달는다 — 그리고 어느 쪽이든 누군가 그것이 중요하다고 결정해서 달는 것인데, 그것은 검증 결정이 아니라 설계 결정이다.

예제 9. 이 크로스를 더해야 하나?

주어진 것. '전송 크기'(칸 8개)와 '주소 정렬'(칸 4개)과 '미해결 거래 수'(칸 16개)를 크로스하자는 제안.

푸는 법. $K = 8 \times 4 \times 16 = 512$ 다. 고른 무작위 자극 아래에서 채우는 데 시험이 3,490.1 번쯤이다. 그런데 합법성부터 확인해라. 1 바이트 전송은 8 바이트 어긋남일 수 없으니, 512 칸의 꽤 많은 뭉치 지어진 방식상 달을 수 없다.

답. 더하되, 불법 조합을 비워 두지 말고 `illegal_bins` 로 선언해라. 닫기를 재는 기준은 달을 수 있는 수다. 분모로 512 를 남겨 두면 보고서가 영영 100 % 에 못 달고, 모두가 그것을 무시하도록 길들인다.

불법 칸을 선언하는 데는 첫째보다 값이 더한 둘째 이득이 있다. 터지는 `illegal_bins` 는 즉각 오류다. 무엇이 일어날 수 없다는 진술을, 그것이 일어나지 않았다는 검사로 바꾼다.

여기서 틀린다. 함정은 뻔한 것의 반대다. 위험은 크로스가 너무 비싸다는 것이 아니라, 가지치기 안 한 크로스가 제 목표에 영영 못 달는 커버리지 수를 낳는다는 것이다. 그리고 못 달는 줄 아는 목표는 목표이기를 그만둔다. **달을 수 있는 칸에 대한 정직한 100 % 가 아무도 안 믿는 분모에 대한 정직한 87 % 보다 값이 있다.**

T21.4 언제 멈출지 알기

커버리지 닫기는 멈추기 위한 필요조건이지 충분조건이 아니다. 프로젝트가 실제로 답해야 하는 물음은 다르다. 아직 몇 개나 남아 있나? 그것은 곧바로 썰 수 없지만 버그가 발견되는 속도는 썰 수 있고, 잣아드는 속도에는 정보가 실려 있다.

유도 — 아직 못 찾은 버그를 어렵하기

	이렇게 된다	왜냐하면
1	주마다 새로 찾는 버그 수가 일정한 비 q 로 준다고 하자.	많은 프로젝트에서의 경험적 관찰이고, 이 어려움이 얹혀 있는 가정이다. 법칙이 아니다.
2	그러면 앞으로 나올 버그는 $n \cdot q + n \cdot q^2 + \dots = n \cdot q / (1 - q)$ 다. n 은 이번 주의 수다.	등비급수. T17 의 ECO 수렴과 같은 것이고 — q 가 1 에 다가가면 같은 방식으로 진다.
3	이번 주에 $n = 4$ 를 찾았고 $q = 0.7$ 이면 9.33 개 더다.	대입이다. 그 수는 기운 날 만큼 작고 위험할 만큼 불확실한데, 그래서 다음 걸음이 중요한 것이다.
4	이제 사소한 설명을 죽인다. 속도는 버그가 떨어져서 줄 수도, 노력이 줄어서 줄 수도, 시험이 안 바뀌게 돼서 줄 수도 있다.	원인 셋에 관측은 하나다. 가려내는 것은 고를 수 있는 것이 아니다 — 셋 중 둘이면 그 어렵은 뜻이 없다.
5	버그 수 옆에 노력과 새 자극의 속도를 같이 재라.	시뮬레이션 시간이 일정하고 커버리지가 오르는데 버그 속도가 주는 것은 증거다. 회귀 수가 줄면서 버그 속도가 주는 것은 설계가 아니라 팀 일정에 대한 측정 이다.

표 57. 이번 주에 버그 넷: 감쇠율이 뜻하는 것

q (주 대비 비)	기대되는 남은 버그	읽기
0.5	4	수렴 중
0.7	9.333	수렴 중
0.85	22.67	느리다
0.95	76	정보 없음 — 이걸 근거로 내보내지 마라

마지막 줄이 정직한 줄이다. $q = 0.95$ 에서 어렵은 76 개 더인데, 모형의 오차 막대가 그 수보다 훨씬 넓다 — 등비 가정이 일을 다 하고 있다. **값이 확인되지 않은 가정에 지배되는 어렵은 어렵이 아니라 가정으로 보고해야 한다.** 이 책이 제 안의 모든 수에 적용하는 그 규칙이다.

T21.5 월요일에 이걸로 무엇을 하나

표 58. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
검증환경 · 테스트벤치 · 에이전트 · 시퀀스 · 시퀀서 · 드라이버 · 모니터	환경을 지을 때	한 층 위에서 다시 쓸 수 있는지
스코어보드 · 참조모델 · 어서션	모든 환경에서, 맨 먼저	설계가 틀렸다고 말할 수 있는 유일한 구성 요소들
기능커버리지 · 커버그룹 · 커버포인트 · 크로스 커버리지	커버리지 모형을 계획할 때	자극이 어디까지 닿았는지 — 그리고 그것 뿐
제약랜덤 · 지시시험 · 커버리지닫기 · 회귀	시뮬레이션 시간을 어디 쓸지 정할 때	무작위가 영영 못 닿을 칸이 어느 것인지, 그래서 손으로 쓸 것
가상인터페이스	환경을 DUT 에 잇기	보통 소프트웨어에 대응물이 없는 유일한 구성물

가지고 갈 수들. 고른 무작위 자극으로 칸 K 개를 채우는 데 시험이 $K \cdot \ln K$ 번쯤 드니, 100 칸짜리 둘의 크로스는 518.7 이 아니라 97,876.1 이 든다 — 크로스는 닫는 노력에 곱해지고, 그래서 불법 조합을 쳐내는 것은 깔끔함이 아니라 공학이다. 그리고 확률 10^{-6} 인 칸에 한 번 닿는 데 시험이 1.00000×10^6 번 드는데, 그 산수가 지시 시험을 살려 두는 것이다.

검증은 설계가 옳다는 것을 세운다. 낱알의 실리콘 조각이 제대로 제조됐는지에 대해서는 아무 말도 안 한다 — 그 물음은 다이를 물고 있는 기계가 답하고, 그것에 싸게 답하는 것이 마지막 장의 주제다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

검증 방법론은 특허가 아니라 공개된다 — 방법론 라이브러리가 오픈 소스인 것이 그 요점이다. 출원되는 것은 **커버리지 주도 시험 생성** 둘레다. 커버리지 결과로 제약 풀이기를 아직 빈 칸 쪽으로 몰아가는 것인데, 이 장에서 셈한 쿼폰 수집가 산수를 곧바로 치는 것이다.

T22. DFT 에서 테스터까지 — 시간, 값, 그리고 새어 나가는 것

시험은 다이가 좋다고 증명하지 않는다. 나쁜 것이 얼마나 자주 나가는지를 한계 짓고, 그 한계는 셈할 수 있다.

읽기 전에 알아야 하는 것

구현흐름 · 검증환경 · 스코어보드 · 사인오프 · 테이프아웃 · 정적타이밍분석 · 클럭게이팅 · 동적IR · 스위칭백터 · 수율시그마 · 패드용량 · MMMC · 코너 · 번인 · 와이블

이 장에서 새로 나오는 것

스캔 · 스캔체인 · ATPG · 고착고장 · 천이지연고장 · 결함 커버리지 · 스캔압축 · 해제기 · 압축기 · LBIST · MBIST · ATE · 핀일렉트로닉스 · 패턴메모리 · 테스트프로그램 · 시험시간 · 병렬시험 · 다이수율 · 결함수준 · DPPM · 웨이퍼 시험 · 양품다이

검증된 설계(T21)를 사인오프한 흐름(T20)으로 제조해도 안 도는 다이는 나온다. 제조가 결함이 있는 물리적 과정이기 때문이다. 시험은 그것들을 가려내는 작업이고, 흐름 전체에서 값을 한 번이 아니라 **부품마다** 치르는 유일한 단계다. 그 사실 하나가 그것이 어떻게 설계되는지를 통째로 정한다.

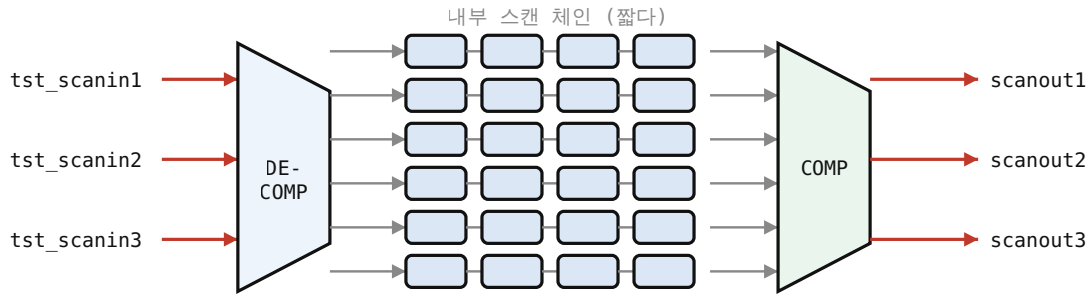
T22.1 스캔, 그리고 관측 가능성의 값

스캔 (scan) · 스캔 체인 (scan chain) — 모든 플롭을 입력에 먹서를 단 것으로 바꿔, 기능 D 와 스캔 입력 중에서 고르게 한다. 시험 모드에서 플롭들은 시프트 레지스터 — **체인** — 로 이어진다. 그러면 모든 플롭이 제어 가능해지고(값을 밀어 넣는다) 관측 가능해져서(결과를 밀어 낸다), 순차 논리를 시험하는 풀 수 없는 문제가 두 레지스터 경계 사이의 조합 논리를 시험하는 풀 수 있는 문제로 바뀐다.

표 59. 스캔이 치르는 값, 그리고 사 주는 것

값	크기	무엇을 사나
면적	플롭마다의 먹서. 설계의 대략 5-10 %	모든 플롭이 시험 지점이 된다
타이밍	먹서가 D 경로에 있다 — 모든 레지스터 입력에 걸리는 진짜 지연	곧바로는 아무것도. 순수한 값이고, 스캔 먹서가 어느 라이브러리에서나 가장 공들여 다듬어진 셀 가운데 하나인 까닭이다
전력	시프트는 기능 동작보다 훨씬 많은 마디를 뒤집는다(T20.3)	아무것도. 시프트 주파수를 묶고, 그것이 시험 시간을 치르게 한다
배선	체인은 다이를 가로지르는 긴 넷이다	아무것도. 그리고 배치 뒤 체인 차례 잡기가 그 값을 붙드는 방법이다

스캔 압축: 테스터 채널 몇 개, 내부 체인 여럿



시프트 길이 = 플롭 / (체인 × 압축). 테스터는 채널 3 개를 보고, 다이에는 체인이 6 개 있다.

시험 시간과 패턴 메모리가 같은 비로 함께 준다 - 그리고 묶는 쪽은 흔히 메모리다.

lab: python3 lab/run.py --stage dft (진짜 넷리스트에서 둘 다 켜다)

그림 20. 압축을 그림 하나로. 해제기가 테스터 채널 몇 개를 짧은 내부 체인 여럿으로 퍼 주고, 압축기가 응답을 도로 짜 낸다. 시험 시간과 패턴 양이 같은 배수로 함께 줄어든다.

표 60. 고장 모형들, 그리고 저마다 무엇을 대리값인가

모형	무엇을 가정하나	무엇을 놓치나
고착	마디가 영영 0 이거나 1 이다	타이밍에 달린 것은 무엇이든. 위상적인 모형이고 큰 결함은 잘 잡는다
천이 / 동작 속도	마디가 너무 느리게 바뀌어 다음 포획을 못 맞춘다	기능 속도로 패턴 둘이 필요하다 - 비싸고, 그러는 동안 부품이 끌어갈 수 있는 전력에 묶인다
브리징	두 마디가 단락됐다	어느 둘인지 - 현실적인 목록은 넷리스트가 아니라 레이아웃 추출에서 나온다
IDDQ	결함이 전류를 흘리면 정지 전류가 올라간다	미묘한 것은 없는데, 누설이 큰 마디에서 안 들게 됐다. 좋은 다이의 누설이 결함 신호와 엇비슷해진 것이다

T22.2 시험 시간이 곧 설계 결정이다

유도 — 다이 하나가 테스터에서 얼마나 걸리나

	이렇게 된다	왜냐하면
1	T19의 블록에는 플롭이 20,000 개 있다.	이 장이 앞의 네 장과 같은 칩을 다루도록 가져온 값이다.
2	체인이 하나면 패턴 하나를 거는 데 20,000 주기를 시프트해야 한다.	체인이 그 길이의 시프트 레지스터다. 패턴마다 끝까지 밀어 넣고 결과를 끝까지 밀어 내야 한다.
3	ATPG는 이만한 블록에 패턴을 5,000 개쯤 넣는다.	가정이고, 크기보다 논리 구조에 따라 더 달라진다.
4	시프트 클럭이 50 MHz면 다이당 2 s다.	곱셈이다. 시프트 클럭이 느린 까닭은 플롭이 더 빨리 못 돌아서가 아니라 T20.3의 전력 문제 때문이다.
5	체인을 N 개로 쪼개면 시프트 길이가 N 으로 나뉜다.	체인들이 나란히 시프트한다. 테스터가 입력 N 개를 몰고 출력 N 개를 본다. 한계는 설계가 아니라 테스터 채널이다.
6	체인 100 개면 0.02 s. 체인 100 개에 50배 압축이면 8.000×10^{-4} s다.	압축은 테스터 채널 몇 개를 내부 체인 여럿으로 퍼주는 해제기를 칩 위에 두는 것이라, 채널을 더 쓰지 않고도 테스터가 보는 체인 수가 늘어난다.

표 61. 시험 시간, 패턴 양, 다이당 값 (테스터 2 M\$, 5년, 가동률 70 %)

설계	체인	압축	시프트 길이	다이당 시간	패턴 메모리	다이당 테스터 값
블록 (플롭 20k)	1	1×	20,	2 s	25 MB	0.036 \$
블록 (플롭 20k)	10	1×	2,000	0.2 s	25 MB	0.004 \$
블록 (플롭 20k)	100	1×	200	0.02 s	25 MB	3.696×10^{-4} \$
SoC (플롭 2M)	1	1×	2.0000×10^6	1,6 s	20, MB	28.99 \$
SoC (플롭 2M)	100	1×	20,	16 s	20, MB	0.29 \$
SoC (플롭 2M)	100	50×	400	0.323 s	400 MB	0.006 \$
SoC (플롭 2M)	100	200×	100	0.083 s	100 MB	0.002 \$

블록에는 압축이 필요 없고, SoC는 그것 없이는 안 된다. 그 대비가 둘을 한 표에 넣은 까닭이고, 쓸모 있는 교정이다. 플롭 20,000 개에서는 체인 100 개로 쪼개는 것만으로 시험이 0.02 s가 되고, 그 위에 압축을 더해 봐야 신경 쓸 값어치가 없는 수가 나온다. 플롭 2,000,000 개에서는 같은 한 체인 시험이 1,6 s다 — 다이당 26.7 분이고, 그것은 시험 계획이 아니라 그 제품이 존재하지 않는 까닭이다 — 그리고 체인 100 개에 50배 압축이면 0.323 s, 곧 부품당 테스터 시간 0.006 \$가 된다.

패턴 메모리 칸은 따로 눈여겨볼 만하다. 묶는 제약이 시간이 아니라 그쪽인 경우가 흔하기 때문이다. 테스터의 패턴 메모리는 유한하고, 안 들어가는 패턴 묶음은 여러 번에 나눠 실어야 하는데 그때마다 시험 시간 위에 핸들러와 다시 실행하는 시간이 얹힌다. 50배 압축에서는 메모리에 들어가고 10배에서는 안 들어가는 설계는 시험 값에 불연속이 있고, 시프트 주파수를 아무리 다듬어도 그것은 매끄러워지지 않는다.

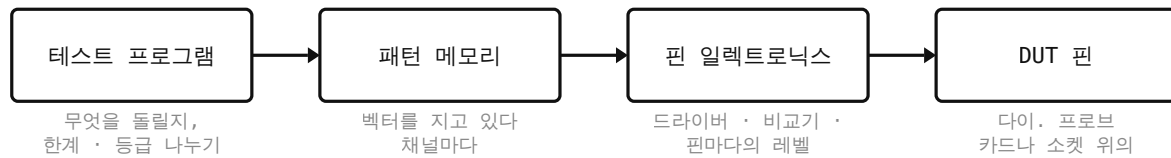
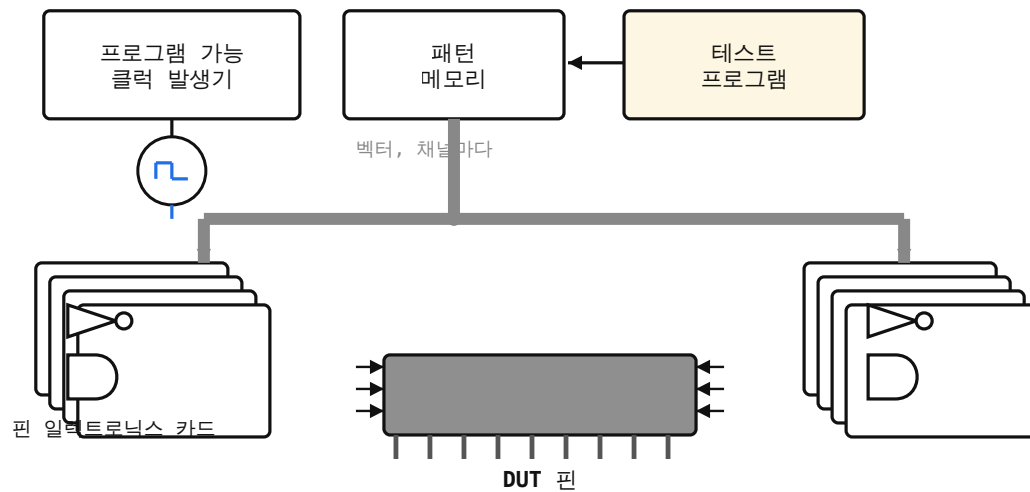


그림 21. ATE 가 기능적으로 무엇인가. 아주 정밀하고 아주 비싼 패턴 재생기에, 프로그램 가능한 타이밍 발생기와 핀마다 드라이버·비교기 한 쌍이 달린 것이다. 앞 표의 모든 것이 이 상자들 가운데 하나에 대한 진술이다.



다이당 값 = (시험 시간) × (테스터 \$/초 ÷ 자리 수). DFT 가 하는 모든 것이 그 셋 중 하나를 치는 것이다.

그림 22. 저 표의 마지막 칸이 빌리고 있는 기계. 패턴 메모리가 벡터를 지고, 핀 일렉트로닉스가 핀 하나씩 물고 견주며, 테스트 프로그램이 무엇을 돌리고 무엇을 통과로 칠지 정한다.

병렬 시험 (multi-site test) — 값을 나누는 다른 길

압축은 다이 하나가 걸리는 시간을 줄인다. 병렬 시험은 같은 테스터에서 다이 여럿을 저마다 다른 채널로 한꺼번에 시험해, 그 시간의 값을 줄인다. 테스터의 초당 값이 고정이고 핸들러의 부대 비용을 나눠 쓰므로, 다이당 값이 자리 수로 거의 정확히 나뉜다.

한계는 채널과 전력이다. 4 자리 설비에는 채널이 네 배 드니, 다이당 핀 수가 자리가 몇 개 들어가는지를 정한다 — 그것은 누군가 스캔 채널을 몇 개 빼낼지 고르던 몇 해 전의 DFT 결정이다.

위의 블록이라면 한 자리에서 1.450×10^{-5} \$가 네 자리에서 3.624×10^{-6} \$가 된다. 시험 내용도 품질도 같은 채로다.

어디에 쓰나	양산 시험 계획, 그리고 결정적으로는 플로어플랜 때의 핀 예산에서.
언제 쓰나	패키지 핀 목록을 이야기하자마자 함께 따진다. 스캔 채널을 나중에 더하는 것은 마스크 변경이다.
어떻게 쓰나	세어 보라. (테스터 채널) ÷ (다이당 채널) = 자리 수다. 그다음 전력을 확인해라. 테스터의 DUT 전원이 모든 자리의 시프트 전류를 한꺼번에 대 줘야 하는데, 4 자리 설비가 실제로 지는 자리가 거기다.

여기서 틀린다. 병렬 시험은 아슬아슬한 시험을 키운다. 어느 자리의 접촉 저항이 조금 다르거나 드라이버에서 오는 길이 더 길면, 그 자리만 다른 자리들이 통과시키는 부품을 떨어뜨린다 — 그리고 수율 보고서는 그것을 공정 문제로 보여 줄 것이다. 수율이 튀면 조사하기 전에 늘 자리별 수율부터 확인해라.

T22.3 BIST: 테스터를 다이 위로 옮기기

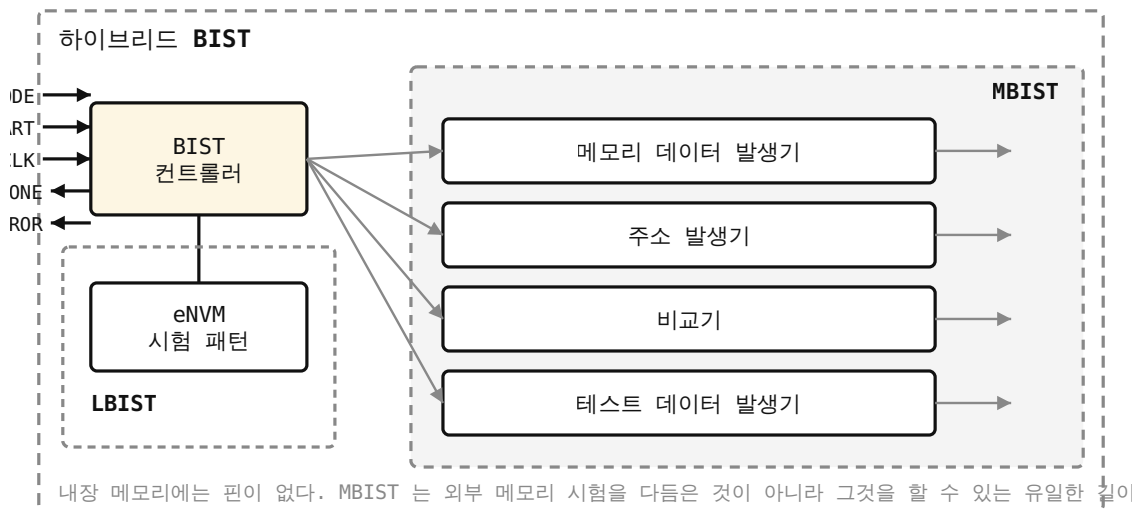


그림 23. BIST 는 패턴의 근원을 다이 위로 옮긴다. 메모리 쪽은 데이터를 알고리즘으로 바꾸고, 논리 쪽은 그것을 LFSR 과 서명으로 바꾼다. 컨트롤러가 둘 중 어느 것이 도는지를 정한다.

표 62. BIST 가 이기는 자리와 그렇지 않은 자리

종류	무엇을 시험하나	거기서 왜 외부 패턴보다 나은가
MBIST (메모리)	내장 RAM 을 알고리즘 패턴(마치 시험)으로	메모리는 규칙적이라 패턴이 데이터가 아니라 알고리즘이다 — 작은 상태 기계가 메가바이트짜리 벡터를 대신한다. 그리고 내장 메모리의 핀은 바깥에서 아예 닿을 수가 없다
LBIST (논리)	무작위 논리를, 칩 위의 LFSR 이 체인을 먹이고 MISR 이 결과를 짜 내며	빠른 테스터 없이 기능 속도로 돌고, 현장에서 다시 돌릴 수 있다. 같은 시간에 대한 커버리지는 ATPG 보다 낮다. 패턴이 겨냥한 것이 아니라 무작위이기 때문이다
하이브리드	덩어리는 LBIST 로, 무작위가 놓친 것은 없는 ATPG 패턴으로	그 조합이 패턴 양의 얼마만으로 ATPG 수준의 커버리지에 닿는다 — 둘은 대안이 아니라 서로를 채워 준다

MBIST 의 재미있는 성질은 시험 시간을 아낀다는 것이 아니다. **내장 메모리에는 핀이 없다는 것이다.** 둘레의 논리를 거치지 않고는 바깥에서 거기에 패턴을 걸 길이 아예 없는데, 그 길은 느리고 커버리지가 낮고 패턴이 어마어마하게 든다. MBIST 는 외부 메모리 시험을 다듬은 것이 아니라 그것을 실제로 할 수 있는 유일한 길이다. 그리고 그 하드웨어가 일단 있으면 고칠 수도 있다 — 퓨즈를 끊어 여분 행을 갈아 끼우는 것인데, 그것은 지는 다이를 좋은 다리로 바꾸고 아낀 시험 시간보다 값을 더 한다.

T22.4 시험이 실제로 값하는 것: 수율, 커버리지, 새어 나감

유도 — 다이 수율, 그리고 왜 시험 커버리지에 값이 붙나

	이렇게 된다	왜냐하면
1	T18 의 코어는 1.901 mm^2 이고, 위에서 쓴 SoC 는 100 mm^2 다.	둘 다 가져온 값이다. 블록 쪽 수치는 패드 링을 무시하므로 상한이고, 아래 산수가 쓰는 것은 SoC 쪽이다. 수율은 면적 이야기이고 작은 블록은 거기에 대해 재미있게 할 말이 없다.
2	결함 밀도가 $0.1 / \text{cm}^2$ 이면 $Y = \exp(-A \cdot D)$ 가 블록에서 99.81 %, SoC 에서 90.48 % 다.	결함 밀도는 가정 이고, 푸아송 모형 자체가 낙관적인 쪽이다. 진짜 결함은 뭉치고, 그러면 같은 평균 밀도에서 수율이 이보다 올라간다.
3	지는 다이 가운데 시험이 잡는 뭉치 T 다 — 고장 커버리지다.	커버리지는 ATPG 가 고장 목록에 대고 재는 것인데, 그 목록 자체가 모형이다(T22.1). 잡힌 결함의 뭉치 아니고, 그렇게 여기는 것이 여기서의 표준적인 잘못이다.
4	다이가 나쁜 채로 나가려면 결함이 적어도 하나 있고 그 결함이 모두 놓쳐져야 한다.	중요한 사건이 그것이다. 두 가지의 교집합이고, 그래서 답이 그냥 $1 - T$ 가 아니다.
5	윌리엄스와 브라운: 나간 부품 가운데 결함 수준은 $DL = 1 - Y^{(1-T)}$ 다.	고전적인 결과다. 그 꼴을 눈여겨보라. $T = 1$ 에서 0 이고 $T = 0$ 에서 $1 - Y$ 곧 낱것의 불량률이다 — 두 극한이 다 맞고, 어느 공식이나 먼저 통과해야 하는 검사가 그것이다.

표 63. SoC 의 $Y = 90.48 \%$ 에서 나가는 결함 수준, 서로 독립인 두 길로

고장 커버리지 T	윌리엄스-브라운	다이 200,000 개 웨이퍼 시능	어긋남
90 %	9,950.2 DPPM	10,107 DPPM	1.58 %
95 %	4,987.5 DPPM	5,087.5 DPPM	2.01 %
99 %	999.5 DPPM	1,143 DPPM	14.4 %
99.9 %	99.995 DPPM	66.332 DPPM	33.7 %

어긋남 칸을 정직하게 읽어라. 시능은 다이를 하나씩 지어 푸아송 결함을 흠어 넣고, 결함마다 확률 T 로 잡히게 하고, 결함 있는 다이가 몇 개나 빠져나갔는지를 센다 — 공식은 하나도 안 쓴다. 결함 수준이 높은 데서는 잘 맞고 낮은 데서는 나빠지는데, 그것이 예상된 것이다. 마지막 줄에서는 시행 200,000 번에 새어 나감이 겨우 수십 번이라 몬테카를로 표본 오차가 수십 퍼센트인데 달한 꼴에는 그것이 없다. 사건이 드물어질수록 커지는 어긋남은 공식이 틀렸다는 표시가 아니라 표본 잡음의 표시다 — 그리고 그 둘을 가려내는 것이 독립 대조를 돌릴 값이 있게 만드는 바로 그 검사다.

예제 10. 커버리지 99 % 면 충분인가?

주어진 것. 위의 SoC, $Y = 90.48\%$, 그리고 100 DPPM 아래를 규정하는 차량용 고객에게 납품.

푸는 법. $T = 99\%$ 에서 결함 수준은 999.5 DPPM 이다. $T = 99.9\%$ 에서는 100 DPPM 이다. 100 DPPM 에 달하는 커버리지를 풀면 $T = 1 - \ln(1 - DL) / \ln Y$ 다.

답. T 가 적어도 99.9 % 여야 한다. 99 % 위로 커버리지 0.1 퍼센트마다 패턴이 들고, 패턴은 시험 시간과 패턴 메모리를 치른다 — T22.2 의 표다.

그리고 모형 자신의 한계도 답에 들어가야 한다. 고장 커버리지는 고장 모형의 커버리지이지 결함의 커버리지가 아니다. 그 모형이 기술하지 않는 결함 갈래 — 뜨거운 때만 지는 저항성 비아, 아슬아슬한 타이밍 경로 — 는 T 의 어디에도 안 나타난 채 DPPM 에 보탠다. 번인(T14)과 동작속도 패턴이 있는 까닭이 그것이고, 차량용 흐름이 고착 커버리지를 99.99 % 로 밀어붙이는 대신 둘 다 쓰는 까닭이다.

여기서 틀린다. 커버리지가 유일한 손잡이도 아니고, 그것이 비싼 쪽이다. 수율을 올려도 같은 수가 움직인다. Y 가 90.5 % 가 아니라 95 % 이면 같은 $T = 99\%$ 가 999.5 DPPM 대신 512.8 DPPM 을 낸다. 팍에서 오는 결함 밀도 개선이 고장 커버리지의 마지막 1 퍼센트보다 값이 더하고, 시험 시간도 안 든다.

실제로 난 일

어떤 부품이 웨이퍼 정렬에서는 DPPM 목표를 맞추고 고객에게서 못 맞췄다. 새어 나간 것들이 전부 한 클럭 도메인의 아슬아슬한 동작속도 실패였다. 동작속도 패턴은 있었는데, 시프트 전력 때문에 포획 클럭을 낮췄고 '동작속도' 시험이 기능 주파수보다 30 % 낮게 돌고 있었다. **커버리지 수는 진짜였고, 그것이 기술하는 시험이 실제로 돈 시험이 아니었다.** 시험 시간과 전력 제약이 패턴의 뜻을 바꿔 놓는데, 커버리지 보고서는 그것을 모른다.

T22.5 월요일에 이걸로 무엇을 하나

표 64. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
스캔 · 스캔체인 · ATPG · 고착고장 · 천이지연고장 · 결함커버리지	RTL 을 열리기 전의 DFT 계획	그 부품을 시험할 수 있기는 한지, 그리고 어느 커버리지로
스캔압축 · 해제기 · 압축기 · 시험시간 · 패턴메모리	체인 수와 압축비 고르기	부품당 시험 값. 그것은 출하되는 모든 부품에서 치른다
ATE · 핀일렉트로닉스 · 테스트프로그램 · 병렬시험	양산 계획, 그리고 몇 해 전의 핀 예산에서	테스터 1 초를 다이 몇 개가 나눠 쓰는지
LBIST · MBIST	모든 내장 메모리. 현장에서 스스로를 시험하는 모든 부품	핀 없는 블록을 시험할 수 있는 유일한 길
다이수율 · 결함수준 · DPPM · 웨이퍼시험 · 양품다이	고객과 품질 목표를 합의할 때	그 목표가 실제로 요구하는 커버리지 — 그리고 수율이 더 싼 손잡이인지

가지고 갈 수들. 플롭 2 M 짜리 SoC 에서 체인 하나는 다이당 26.67 분이고 체인 100 개에 50배 압축은 0.323 s 다 — 출하되는 모든 부품에서 치르는 값에서 4,951 × 배다. 그리고 수율 90.48 % 에서 고장 커버리지 99 % 는 999.5 DPPM 을 내보낸다 — RTL 을 열리기 전에 내린 DFT 결정을 고객의 품질 규격 속 수에 잇는 문장이 그것이다.

이것으로 구현 갈래가 닫힌다. 넷리스트와 제약 묶음으로 시작해(T17), 상자와 전원 격자를 그리고(T18), 그 안의 것을 배치하고 클럭을 주고 배선하고(T19), 그 기하를 제조해도 된다는 것을 세우고(T20), 애초에 설계가 옳았다는 것을 세우고(T21), 출하되는 모든 낱알의 부품에 묻는 유일한 물음으로 끝났다. 이것은 도는가? 단계마다 관례가 아니라 셈한 양이 다스렸고, 단계마다 중요한 양은 그 단계 안에서만은 보이지 않는 양이었다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

시험 압축은 EDA 에서 특허가 가장 뻥뻥한 영역 가운데 하나다 — 해제기 구조(링 발생기, 방송 회로망, 순차 선행 확장기)가 제품이고, 그것들이 이루는 비가 경쟁의 주장이다. **저전력 ATPG** 도 그렇다. T20.3 에서 셈한 까닭으로 동시 스위칭을 묶으려고 패턴을 제약하는 것이다. 둘 다 이 장이 셈하는 수들을 움직이려고 있다.

T23. 흐름을 끝에서 끝까지 돌리기 — 블록 하나, 아홉 단계, 진짜 수

이 장의 모든 수는 이 저장소에서 흐름을 돌려 나온 것이다. 어느 것도 남의 것을 옮겨 적지 않았다.

읽기 전에 알아야 하는 것

구현흐름 · 합성 · 넷리스트 · 정적타이밍분석 · 슬랙 · 표준 셀행 · SITE · 점유율 · 코어영역 · 파워스트라이프 · 배치 · 합법화 · HPWL · 클럭트리합성 · 삽입지연 · 클럭스큐 · 전역배선 · 상세배선 · 탐색수리 · 혼잡도 · 사인오프 · 물리검증 · 코너 · 스캔 · 스캔체인 · ATPG · 고착고장 · 결함커버리지 · 결함수준 · DPPM · 시험시간 · 스캔압축 · 검증환경 · 스코어보드 · 기능커버리지 · 크로스커버리지 · 제약랜덤 · 지시시험 · 일렉트로마이그레이션 · 전압강하

이 장에서 새로 나오는 것

실습흐름 · 회귀기준

앞의 다섯 장은 흐름의 양들을 모형에서 셈했다. 이 장은 그것을 돌린다. 설계는 제어 FSM 과 두 플롭 동기화와 게이팅된 누산기를 단 작은 MAC 이다. T2 의 RC 모형에서 만든 셀 라이브러리에 대고 **yosys** 가 합성하고, 이 기계에 상용 배치·배선 도구가 없어서 나머지 여덟 단계는 이 저장소 안에 구현했다. 그 제약이 되레 이점이었다. **모든 중간 양이 보이고 다시 잴 수 있다**는 것인데, 그것이 바로 상용 도구가 주지 않는 것이다.

이 장의 모든 것을 명령 하나로 재현한다. `python3 lab/run.py` . 8 초쯤 걸린다. 단계 하나씩은 `python3 lab/run.py --stage sta` (또는 `fp place cts route signoff dft dv`) 다. 아래의 수들은 그 명령이 쓰는 `lab/기준.json` 에서 읽은 것이고, `tests/test_lab.py` 가 흐름을 다시 돌려 그 수들이 여전히 맞는지 확인한다 — 그러니 **이 장이 낡으면 검사가 빨개진다**.

T23.1 아홉 단계, 그리고 저마다 무엇을 했나

표 65. 한 번의 실행, 끝에서 끝까지

단계	무엇을 했나	중요한 수
01 합성 (yosys)	RTL → 셀 538 개, 플롭 29 개	셀 면적 10,794.2 μm^2 — 뒤의 모든 면적 결정에 들어가는 입력
02 STA (배선 전)	52 단짜리 임계경로	$F_{\text{max}} = 158.04 \text{ MHz}$, 그리고 서로 독립인 두 방법이 0 ps 까지 맞는다
03 플로어플랜	점유율 70 % 로 25 행 × 185 자리	코어 122.1 μm × 126 μm . 스트라이프 폭은 최소폭 이 정한다
04 배치	힘 주도, 그다음 두 번 쏘아 합법화	HPWL 16,640.8 μm → 22,951 μm , 겹침 0
05 CTS	2 단 트리, 버퍼 5 개	삽입 832.14 ps, 스큐 122.7 ps
06 배선	21×21 gcell 격자에서 전역, 그다음 뜯어내고 다시 배선	배선 27,942 μm — HPWL 에 대한 우회 계수 1.217 ×
07 사인오프	다시 추출해 진짜 RC 로 STA 를 다시 돌림	F_{max} 158.04 MHz → 99.31 MHz
08 DFT	고착 고장 1112 개, 병렬 패턴 고장 시뮬레이션	무작위 패턴 384 개로 커버리지 99.19 %
09 DV	UVM 꼴 환경으로 거래 32 건	불일치 0. 번이 5 개 중 새어 나간 것 0

python3 lab/run.py — 아홉 단계, 8 초쯤



노란 것이 바깥 도구(yosys)를 부르는 유일한 단계다. 나머지 여덟은 이 저장소 안에 있고, 그래서 위의 중간 양을 전부 다시 짤 수 있다 — 그리고 tests/test_lab.py 가 실제로 켜다.

그림 24. 이 장이 보고하는 실행을 단계별로. 명령 하나로 재현된다. 상자 하나하나가 검사가 다시 재는 수다.

T23.2 따져 볼 값이 있는 네 가지 결과

T23.2 따져 볼 값이 있는 네 가지 결과

유도 — 1. 배선 전 타이밍은 조금 낙관적인 것이 아니다

	이렇게 된다	왜냐하면
1	합성과 배선 전 STA 가 슬랙 3,672.4 ps 로 158.04 MHz 를 보고한다.	배선 RC 가 아예 없다 — T17.2 의 낙관적 한계다.
2	배선 뒤에 같은 끝점들이 99.31 MHz 와 -69.14 ps 를 보고 한다.	같은 넷리스트, 같은 라이브러리, 같은 제약이다. 새 로 들어온 정보는 배선이 실제로 어디로 갔는가뿐이 다.
3	$F_{\max}$ 의 37.2 % 이고, 끝점이 평균 -1,155.5 ps 움직였 다.	최악이 아니라 평균 이다. 넷마다 없던 저항과 용량 이 붙었으므로 모든 끝점이 나빠졌다.
4	이제 설계가 진다 . 목표 주기에서 끝점 1 개가 위반한다.	물리 인지 합성 단계 없이 흐름을 돌린 것의 정직한 결말이고 — 그것을 두어야 한다는 정량적 논거다.

사소한 설명을 죽인다. 37.2 % 가 떨어진 것은 그저 배선 모형이 너무 비관적이라는 뜻일 수도 있다. 두 가지가 그 령지 않다고 말한다. 우회 계수가 HPWL 에 대해 $1.217 \times$ 인데 그것은 진짜 배선기가 내는 범위 안에 있고, 배선 지 연이 전원 격자 치수를 잡는 데 쓴 것과 같은 마이크로미터당 R 과 C 에서 나오는데 거기서 나온 IR 강하가 멀쩡했다. 둘 다 증명은 아니다. 증명이 될 것은 두 번째 추출 방법인데 이 실습에는 그것이 없다 — **그래서 그 한계를 붙여서 수 를 보고한다.**

유도 — 2. 스큐는 주어지는 것이 아니라 사는 것이다

	이렇게 된다	왜냐하면
1	기하만으로 지은 트리는 삽입 지연 832.14 ps 에 스큐가 122.7 ps — 14.74 % 다.	자리로 묶는 것은 배선을 최소로 하지 도착 시각을 최소로 하지 않는다. 짓는 과정의 무엇도 아무것을 균형 잡지 않는다.
2	그것을 2 % 쯤으로 균형 잡는 데 버퍼 25 개가 더 든다.	이른 가지에 늦은 가지와 맞을 때까지 지연을 더한 다. CTS 도구가 트리를 짓고 나서 하는 일이 그것이 다.
3	그 버퍼들이 클럭 회로망 전력을 21.09 % 올린다.	클럭 위의 다른 모든 것처럼 주기마다 뒤집힌다. 맞 바꿈은 스큐 대 전력이고 정량적이다.
4	결과: 19.6 ps, 곧 삽입 지연의 2.35 % 다.	업계가 대는 수치가 그것인데, 이제 값표가 붙었다.

표 66. 3. 수리 되돌이가 수렴하지 않는다 — 켜 것

바퀴	열린 위반	앞 바퀴에 대한 비
0	2	—
1	4	2
2	15	3.75
3	16	1.067
4	13	0.812
5	7	0.538
6	10	1.429
7	7	0.7
8	9	1.286

수열은 [2, 4, 15, 16, 13, 7, 10, 7, 9] 이고 비가 되풀이해서 1 을 넘으므로, 이 되돌이는 위반을 수리하는 것이 아니라 **옳기고** 있다 — T19.4 가 기술하는 바로 그 표시다. 그것 때문에 실습에 두 가지를 넣어 두었다. 배선기는 마지막 것이 아니라 지금까지 본 가장 좋은 해를 붙든다 (가장 좋은 것 2, 마지막 것 9). 진짜 배선기가 하는 일이 그것이고, "되풀이를 더 하면 더 나빠진다" 를 결론에서 인공물로 바꾸는 것이 그것이다. 그리고 수렴하지 않는 되돌이에 대한 옳은 대응은 한 바퀴를 더 도는 것이 아니라 흐름의 위로 올라가는 것이다 — 점유율을 낮추고, 배치를 펴고, 층을 더한다.

표 67. 4. 변이 작업이 찾아낸 구멍은 자극이 아니라 검사에 있었다

변이	무엇을 망가뜨리나	무엇이 잡았나
m1_cnt	RUN 이 한 주기 짧다 — 누산 하나를 잃는다	acc 비교
m2_load	LOAD 가 공급 안 싣는다	acc 비교
m3_pulse	펄스 대신 레벨 — start 가 다시 걸린다	done 개수
m4_done	done 이 한 번도 안 뜬다	done 없음
m5_prod	곱 대신 합	acc 비교

실제로 난 일

m3_pulse 변이 — 한 주기 펄스를 레벨로 바꾼 것 — 를 첫 회귀가 **못 잡았고**, start 를 스무 주기 동안 높게 붙드는 지시 시험을 더한 뒤에도 못 잡았다. 까닭이 배울 만하다. 레벨이면 FSM 이 유희로 돌아오자마자 다시 걸리는데, 첫 완료의 누산기 값은 여전히 옳다. 값 비교로는 그것을 볼 수 없다. **구멍은 자극이 아니라 검사에 있었고**, 스코어보드가 거래마다 done 이 정확히 하나 난다고 단언하기 시작하고서야 달렸다. 그 프로토콜 검사 하나가 작업을 새어 나감 하나에서 0 으로 데려간다.

T23.3 이 실습이 하지 않는 것

표 68. 정직한 한계 — 위의 수를 대기 전에 이것부터 읽어라

안 한 것	왜 중요한가	무엇이 고칠까
상용 P&R 없음	여기의 배치기·배선기·CTS 는 가르치기 위한 구현이다. 절대적인 품질은 양산 도구와 견줄 것이 못 된다	OpenROAD 가 이 같은 넷리스트를 돌릴 수 있다. 흐름은 그것을 갈아 끼울 수 있도록 일부러 짜 두었다
추출 모형 하나	배선 R 과 C 가 마이크로미터당 상수 하나에서 나온다. 그래서 $F_{\max}$ 가 37.2 % 떨어진 것에 독립 대조가 없다	두 번째 추출(장 풀이거나 다른 모형)과 둘의 비교
DRC 없음, LVS 없음	배선이 기하가 아니라 트랙 수를 내놓으므로, 여기의 무엇도 제조 가능성을 주장할 수 없다 (T20.1)	진짜 레이아웃 데이터베이스와 규칙 덩어리 — 이 기계에는 둘 다 없다
고착 고장만	99.19 % 커버리지는 고착 커버리지다. 천이와 브리징 고장은 시뮬레이션하지 않는다	동작속도 패턴과 레이아웃에서 뽑은 브리징 목록 (T22.1)
무작위 ATPG 만	고장 9 개가 남아 있고 무작위 패턴은 거기에 못 닿는다 — T21.3 의 드문 칸 문제가 시험의 꼴로 나타난 것이다	남은 것에 대한 결정적 ATPG(D 알고리즘이나 SAT 호출)
전력은 재지 않고 가정한다	플로어플랜의 전류가 가정한 W/mm^2 에서 나오므로, IR 과 EM 결론이 그 가정을 물려받는다	DV 실행에서 나온 토글률을 전력 어림으로 되먹이는 것 — 두 단계가 같은 저장소에 있으면서 아직 안 이어져 있다

마지막 줄은 곱셈을 만한다. 여러 해를 살아남는 종류의 틈이기 때문이다. 검증 실행은 마디마다 얼마나 자주 뒤집히는지를 정확히 안다. 플로어플랜에 필요한 것이 바로 그 수다. 둘은 같은 프로그램 안에, 같은 언어로, 8 초 떨어져 있다 — 그런데 안 이어져 있어서, 그것을 대신할 정보가 바로 옆 디렉터리에 앉아 있는 동안 전력 수는 가정으로 남는다. 진짜 흐름에서 해석이 빠진 것처럼 보이는 것의 대부분이 이것이다. 서로가 필요로 하는 것을 가진 두 단계와, 그 사이에 없는 길.

T23.4 월요일에 이걸로 무엇을 하나

표 69. 이 장의 개념이 실제로 쓰이는 자리

개념	쓰이는 블록	무엇을 정하나
실습흐름	흐름을 배우거나 그것에 가한 변경을 시험할 때	읽고 망가뜨리고 8 초 만에 다시 썰 수 있는 아홉 단계
회귀기준	lab/ 이나 이 장에 가하는 모든 변경	책의 수들이 진짜 실행과 아직 맞는지

돌려 보고, 그다음 일부러 망가뜨려 보라. `lab/se/floorplan.py` 에서 합법화가 질 때까지 점유율을 올려 보라. 합법화가 배치를 망가뜨릴 때까지 퍼는 힘을 약하게 해 보라. `lab/se/dv.py` 에서 done 개수 검사를 지우고 변이가 새어나가는 것을 보라. 그 하나하나가 앞 장의 교훈이 당신이 낸 실패로 도착하는 것이고, 이 재료가 사실의 목록이기를 그만두는 유일한 길이 그것이다. 명령 하나는 `python3 lab/run.py` 이고, 흐름을 정직하게 붙들어 두는 파일 하나는 `tests/test_lab.py` 다.

특허로 가는 길 — 이 주제에서 발명이 일어나는 자리

여기에는 특허 낼 것이 하나도 없고, 그것이 요점이다. 흐름은 범용품이고, 값어치는 수마다 무슨 뜻인지, 그리고 그 중 어느 것이 당신에게 거짓말하고 있는지를 아는 데 있다.